

The Siemens logo is displayed in a white rectangular box in the top left corner. The word "SIEMENS" is written in a bold, teal, sans-serif font. The background of the entire page is a low-angle photograph of a modern glass skyscraper reaching towards a blue sky with scattered white clouds.

SIEMENS

Teamcenter 12.3

Getting Started with Product Structure

PLM00006 - 12.3

Contents

Overview of product structure

Overview of product structure	1-1
Structure-related applications	1-1
Structure-related applications	1-1
Structure Manager	1-1
Teamcenter Integration for NX	1-2
Systems Engineering	1-2
Multi-Structure Manager	1-3
Design Context	1-3
Manufacturing Process Planner	1-4
Teamcenter Mechatronics Process Management	1-4
Multi-Site Collaboration	1-4
Platform Designer	1-4
Workflow	1-5
Change Manager	1-5
As-Built Manager	1-5
Service Manager	1-6
Prerequisites	1-8
Basic concepts	1-8
Basic concepts	1-8
Understanding structure objects	1-9
Working with items	1-12
Managing alternate and alias identifiers	1-13
Navigating the structure	1-25
Configuring the structure	1-27
Creating occurrences	1-35
Understanding properties	1-37
Creating multiple views	1-40
Using BOM view revisions	1-41
Using representations	1-41
Overview of configuration methodologies	1-43
Integrating with CAD systems	1-44
Integrating with ERP systems	1-45

Part I: Basic principles

Building and maintaining product structure

Building and maintaining product structure	2-1
Building product structure	2-1
Cloning the product structure	2-2
Editing the product structure	2-3
Creating precise and imprecise assemblies	2-4
Creating substitute components	2-5

Importing and exporting product structure	2-5
Importing a product structure with Excel	2-6
Packing and unpacking product structure	2-6
Grading	2-7
Copying data between applications	2-9
Making mass updates to the structure	2-10
Using the embedded viewer	2-14
Analyzing clearances	2-15
Clearance analysis principles	2-15
Analyzing clearances	2-16
Understanding the clearance database	2-17
Best practices for managing clearance analysis	2-17

Comparing product structures

Comparing product structures	3-1
Lowest level comparison mode	3-1
Using the lowest level comparison mode	3-1
Example of lowest level comparison mode	3-2
Multilevel comparison mode	3-3
Comparing product structures in multilevel mode	3-3
Example of multilevel mode comparison	3-3
Single-level comparison mode	3-4
Comparing product structures in single-level mode	3-4
Example of single-level comparison mode	3-5
Using supersedures	3-5
Making graphical comparisons	3-6

Finding items in the product structure

Searching for items in the product structure	4-1
Where-used searches	4-1
Creating where-used searches	4-1
Report types	4-2

Part II: Configuration

Configuring product structure with revisions

Revision configuration principles	5-1
Introduction to revision control	5-1
Revision control principles	5-1
Understanding revisioning terminology	5-2
Defining revision rules	5-3
Revision rule criteria	5-3
Working entry	5-4
Status entry	5-4
Override entry	5-5
Precise entry	5-5

Latest entry	5-6
Date entry	5-6
Unit number entry	5-6
Grouping entries	5-7
Managing user privileges	5-7
Configuring precise and imprecise structures	5-7
What are precise and imprecise structures?	5-7
Changing from precise to imprecise	5-9
Applying revision rules	5-10
Creating baselines	5-10
Creating snapshots	5-11
Creating intermediate data captures (IDCs)	5-11
Best practices for revision configuration	5-13

Configuring product structure with effectivity

Configuring product structure with effectivity	6-1
What effectivity schemes are available?	6-1
Understanding revision effectivity	6-1
Using structure (occurrence) effectivity	6-4
Using nested effectivity	6-6
Best practices when applying effectivity	6-9

Managing incremental changes

Managing incremental changes	7-1
Incremental changes principles	7-1
Tracking changes with incremental change	7-1
Configuring incremental changes	7-1
Configuring with incremental change	7-3
Configuring attachments	7-5
Configuring by intents	7-7
Configuring against a background configuration	7-9
Using incremental change with Change Manager	7-9
Using incremental change with revision configuration	7-9
Tracking changes to occurrence data	7-10
Baselining structures with active incremental changes	7-12
Best practices when using incremental change	7-13

Configuring product structure with variants

Configuring product structure with variants	8-1
Configuring product structure with variants	8-1
Choosing a variants technique	8-1
Using classic variants	8-3
Using modular variants	8-13
Best practices when designing for modularity	8-26

Configuring structure with branch definitions

Using branch definitions to configure product structures	9-1
Managing single branches	9-4
Managing branch chains	9-5
Create a branch	9-6

Part III: Architecture

Using occurrences and absolute occurrences

Understanding occurrences and absolute occurrences	10-1
Understanding occurrence terminology	10-1
Using occurrences	10-2
Using occurrence groups	10-2
Using absolute occurrences	10-3
Understanding absolute occurrences	10-3
Tracking changes to absolute occurrences	10-8
Best practices when using absolute occurrences	10-10
Using arrangements	10-10

Using Platform Designer

What is Platform Designer?	11-1
Purpose of Platform Designer	11-1
Platform Designer principles	11-1
Scenarios for using Platform Designer	11-2
Platform Designer terms	11-2
Set up Platform Designer	11-5
Understanding data storage	11-9
Storing Platform Designer data	11-9
Storing product family data	11-10
Storing product-specific data	11-11
Applying the variability and variance	11-15
Analyzing variability and variance	11-15
Using shared or unshared NVEs	11-15
Creating a full or partial breakdown	11-16
Preventing overlapping NVEs	11-17
Using hierarchical variability	11-17
Choosing variability settings	11-18

Aligning structure

Alignment methods	12-1
Aligning the structure internally	12-1
Aligning structure items internally	12-1
Aligning parts and designs	12-2
Aligning with allocations	12-8
Making alignments in Multi-Structure Manager	12-8
Aligning the structure externally	12-9

Using collaboration contexts

Using collaboration contexts	13-1
Purpose of Multi-Structure Manager	13-1
Understanding the collaboration context data model	13-3
Managing occurrences and absolute occurrences	13-4
Using occurrences in collaboration contexts	13-4
Using occurrences	13-4
Using absolute occurrences	13-4
Working with representations	13-5
What are representations?	13-5
Modeling with occurrence groups	13-5
Modeling with structure contexts	13-8
Modeling with compositions	13-8
Building structure contexts	13-9
Structure context principles	13-9
Applying variant rules and selected option sets	13-9
Using absolute occurrences in a structure context	13-10
Managing configuration changes	13-10
Creating compositions	13-11
Transferring data to external applications	13-12

Manufacturing product structures

Manufacturing product structures	14-1
Using Multi-Structure Manager	14-1
Using Manufacturing Process Planner	14-2

Using allocations

Understanding allocations	15-1
Understanding allocations	15-1
Choosing allocations or trace links	15-4
Implementing allocations in Teamcenter	15-5
Understanding allocations in Teamcenter	15-5
Making allocations between configured structures	15-5
Making conditional allocations	15-7
Creating allocations	15-8
Revising allocations	15-9
Configuring allocations	15-10
Releasing allocations	15-12
When can I release allocations?	15-12
Releasing an allocation	15-13
Releasing an item with an allocation to a physical component	15-13
Releasing allocation maps	15-13

Index — Index-1

1. Overview of product structure

Overview of product structure	1-1
Structure-related applications	1-1
Structure-related applications	1-1
Structure Manager	1-1
Teamcenter Integration for NX	1-2
Systems Engineering	1-2
Multi-Structure Manager	1-3
Design Context	1-3
Manufacturing Process Planner	1-4
Teamcenter Mechatronics Process Management	1-4
Multi-Site Collaboration	1-4
Platform Designer	1-4
Workflow	1-5
Change Manager	1-5
As-Built Manager	1-5
Service Manager	1-6
Prerequisites	1-8
Basic concepts	1-8
Basic concepts	1-8
Understanding structure objects	1-9
Working with items	1-12
Managing alternate and alias identifiers	1-13
Navigating the structure	1-25
Configuring the structure	1-27
Creating occurrences	1-35
Understanding properties	1-37
Creating multiple views	1-40
Using BOM view revisions	1-41
Using representations	1-41
Overview of configuration methodologies	1-43
Integrating with CAD systems	1-44
Integrating with ERP systems	1-45

1. Overview of product structure

Overview of product structure

This guide is for persons responsible for creating, configuring, or managing complex product structures. It describes the basic concepts behind Teamcenter product structure and includes information on how to use and administer it. It provides information on best practices when configuring and managing product structures.

Structure-related applications

Structure-related applications

Teamcenter provides editing and viewing applications to allow manipulation of the structures that you can manage.

Note:

Getting Started with Product Structure does not provide details about how to complete a particular task with a specific editor. For this information, see the documentation for the relevant editor.

Structure Manager

Use Structure Manager to create, view, and modify product structures. You can also use Structure Manager to manage product structures that were created in an MCAD program such as NX.

You can also use Structure Manager to view and manage electromechanical (Mechatronics) product structures that were created in an ECAD program such as Mentor Graphics Capital Harness. However, Siemens Digital Industries Software does not recommend that you use Structure Manager as your primary method of creating product structures of this type.

Structure Manager allows you to create generic bills of materials to show different configurations of components. You specify the configuration to load by setting values in a rule. You can configure the product to answer many of the questions that arise during the design process, for example:

- What revision of items?
- What components of an assembly are effective?
- What variant of a product?

You can specify global alternates of a component, one of which is designated as the *preferred* global alternate. Global alternates may be used in any product and are not defined in the context of a particular

assembly. The global alternate selection is independent of the current revision. A user cannot delete an item if it is designated the global alternate of another item.

You can also specify substitutes, which are components that can be used interchangeably within an occurrence, typically for manufacturing purposes. The preferred substitute is displayed in the structure. In contrast with global alternates, substitutes may only be used in the context of a particular assembly.

Teamcenter Integration for NX

Teamcenter Integration for NX provides a tight integration between Teamcenter and NX. It allows you to create items and structures in NX, then store them in Teamcenter as BVRs with the master data held in Teamcenter. It also allows you to synchronize item attributes, including IRM attributes, owning user, group, and release status.

You can generate visualization files that may be displayed in the embedded viewer. These files can be used as the basis of spatial or proximity searches, allowing you to specify background parts.

The Teamcenter Integration for NX supports variants and revision rules as load options.

Systems Engineering

Systems Engineering allows you to:

- Develop requirement structures and create functional and logical models for the product design.
- Define interfaces between design elements in functional and logical models, and understand the relationships between them.
- Create traceability between requirements, models, and physical architectures.
- Support formal systems engineering processes and standards such as IEEE 1220, ISO 15288, EIA 632, and INCOSE.
- Support ad hoc systems engineering and product development activities that do not conform to a formal standard.

As you develop the requirements, you draft a list of features or functions that need to be built into the product. You may also discuss alternative ways of satisfying the requirements. As this list matures, you can create an initial functional model (decomposition or breakdown) of the system, followed by a logical model. You can then allocate functions to logical blocks. Later, when these models are sufficiently mature, you can create a physical model that you can map to the product structure. This physical decomposition precedes the creation of CAD files, software codes, and other components. It is the final step before the requirements engineer hands off the emerging design to the engineers responsible for the detailed design.

Multi-Structure Manager

The Multi-Structure Manager application includes most of the capabilities of a primary editor such as Structure Manager. Use the Multi-Structure Manager application to view product structure that is located in a collaboration context object so that the data they contain can be shared with an external application or other users. You can create subsets of the data in a large structure or create a composition of elements of different product structures. You can store data in a collaboration context object and apply any applicable revision rules or variant rules.

You can export data to or import data from third-party or other external applications in PLM XML format. If you implement incremental change, only changed data can be imported or exported.

Multi-Structure Manager also allows you to create allocations and to align your structure, both internally and with external systems, such as variant management programs.

It allows you to collect lines from one or more source structures into a composition structure. A composition always references the sources, so that updates to the source structure are inherited by the target composition. The configuration of the source controls the configuration of the target.

Conversely, you can create a simple, static structure context to persist a configuration determined by revision or variant rules. If you use a static structure context to reorganize source lines, the structure context is not automatically updated. Thus, for example, if you reorganize an EBOM into an MBOM, you must manually update the target MBOM if you reposition components.

It also allows you to collect occurrence groups from the product structure to create different views. You can create *derived* views from a base view to collect a set of associated parts from the base product structure. You can create occurrence groups to collect parts for assembly planning, group a set of parts for a design study, or collect parts in spatial proximity. Multi-Structure Manager allows you to identify underconsumed or overconsumed occurrences when reconciling the base views with derived views.

Note:

The Multi-Structure Manager application also allows you to view and manage manufacturing structures.

Design Context

Design Context provides fast searches for background parts within a certain proximity that is defined by filters. It supports variants and quick searches. It allows you to quickly focus on a particular work part in a Repeatable Digital Validation (RDV) environment and any other parts affected within the context of a change to that part. Use this application to:

- Select a product item.
- Configure revisions of components.

- Configure variants of the assembly.
- Review the set of components.
- Retrieve the components.
- Initialize the display of components in CAD or a Lifecycle Visualization application.
- Save the current session to a context object for subsequent retrieval in Design Context, NX, or Lifecycle Visualization.

Manufacturing Process Planner

Manufacturing Process Planner allows you to define the manufacturing processes for your product. You can open the product structure that was created in Structure Manager and create the corresponding process and plant structures (view types). Manufacturing Process Planner supports revision configuration and classic variant configuration. You can also create and manage data in collaboration contexts, in a similar way as in the Multi-Structure Manager application.

Use relationships such as **MEConsumed** to link nodes in the product structure to the process structure, allowing configurations in the product structure to also configure the process structure. For example, a part in the product structure with an applied variant condition may be consumed in an operation in the process structure. If a variant rule causes the part in the product structure to be unconfigured, the operation is unconfigured in the process structure.

Teamcenter Mechatronics Process Management

If you are designing electromechanical (Mechatronics) structures, you can define item elements or GDEs (general design elements) and use them to model ports for electrical and hydraulic circuits. Teamcenter also supports connections and signals that can connect the ports.

When you design schematics, you may find it useful to create a functional breakdown before creating the physical schematic which includes the physical parts. Create **allocations** to link nodes in the functional structure to the corresponding instances of physical parts in the physical structure.

Multi-Site Collaboration

Multi-Site Collaboration allows teams of designers to work on the same product at different locations. A product structure that is changed at one site can be exported to the other sites.

Platform Designer

Platform Designer allows you to create and manage product architecture—generic structures that are configured by named variant expressions (NVEs). It allows a designer to associate a solution with a node in the context of one or more NVEs.

Platform Designer is useful when designers are working with highly configurable variant structures that include huge amounts of variant data that must be kept consistent with a master in other business systems. It provides a variant authoring wizard that guides the user through the application of the correct variant data (variant conditions) when adding their new design solution to the CAD product structure. You work within a hierarchy of logical architecture nodes, rather than the actual physical assemblies that you manage in Structure Manager.

As a designer, you can assign options and variant expressions to a design item, even though you do not own the expression definitions. Configuration experts in your organization establish the architectural breakdown, the associated variability and assemblies in the CAD product structure. The expression definitions are owned by the configuration experts and are associated with the platform architecture. This configuration guides the designer and limits the choices to the appropriate expressions (variant conditions).

Workflow

Use Workflow to apply release status to the structure in a controlled manner..

Change Manager

Use the Change Manager application to create, manage, and view engineering changes (ECs) to the product structure.

As-Built Manager

As-Built Manager allows you to manage the as-built configuration of an asset realized from a product definition structure, such as a design or manufacturing structure. It permits you to:

- Create an as-built physical product instantiation of any configured product structure.
- Store and manage additional information about the as-built physical part structure that may not be tracked in the product definition structure, such as serial numbers and lot numbers.
- Reconcile the as-built structure with a configured product definition structure to identify parts that are missing, alternates, or deviations.
- Compare the as-built structure with the product definition structure or with another as-built structure.
- View the as-designed product structure and the as-built physical structure configuration side-by-side in the embedded viewer in the rich client.
- Import an as-built structure from an external manufacturing execution system (MES) or an enterprise resource planning (ERP) system in PLM XML format.
- Export an as-built structure to an external system in PLM XML format.

- Export an as-built structure to a Teamcenter Enterprise Service Lifecycle Management (SLM) system using Data Exchange.
- Update the as-built structure with Excel Live.
- Show untraced structures.
- Renumber physical parts, that is, edit the part numbers and serial numbers.
- Rebuild the as-built structure.
- Copy as-built physical structures.
- List parts allowed for installation and replacement.
- Set a default name for structure contexts.

Service Manager

Service Manager supports as-maintained structure management and service event management capabilities within Teamcenter. It allows you to manage the as-maintained configuration of an asset over its lifetime. As-maintained structures are generated from, and validated against, product definition structures, such as design or planning structures. You can use the Service Manager as-maintained management to:

- Create the as-maintained physical product instantiation of any configured product definition structure.
- Store and manage additional information about the as-maintained physical part structure that is not tracked on the product definition structure, such as serial numbers and lot numbers.
- Reconcile the as-maintained structure with a configured product definition structure to identify parts that are missing, alternates, substitutes, or deviations.
- Track part utilization, part usage, and disposition history.
- Manage physical locations and the assets in those locations.
- Manually update the as-maintained structure with changes that occur over the lifetime of the physical asset.
- Compare the as-maintained structure to the product definition structure or to another as-maintained structure.
- View the as-designed product structure and the as-maintained physical structure configuration side-by-side or by using the embedded viewer available in the rich client.

- Configure a deviation request as a valid deviation authority.
- Configure default values for the part logistics form.
- Show untraced structures.
- Renumber physical parts, that is, edit part numbers and serial numbers.
- Copy as-maintained physical structures.
- List parts allowed for installation and replacement.
- Set a default name for structure contexts.
- Create, calculate, and manage derived characteristics definitions.
- View log book entries for a physical asset.
- Create display filters for the physical location **Contains** view.
- Import an as-maintained structure from an external manufacturing execution system (MES) or an enterprise resource planning (ERP) system in PLM XML format.
- Import log books, log entries, and disposition types in PLM XML format.
- Export an as-maintained structure to an external system in PLM XML format.
- Update the as-maintained structure with Excel Live.
- Manage as-maintained structures in a Multi-Site Collaboration environment.

Service Event Management is a separately licensed extension of Service Manager that allows you to manage service events that affect the as-maintained configuration. You can use Service Manager with the Service Event Management extension to:

- Define service group structures to efficiently identify maintenance events that have occurred on an asset.
- Record service events, such as an inspection, overhaul, repair, or other type of service.
- Maintain service history by recording service event data such as work scope, approval, resource information, maintenance times, and cost.
- Record and track part discrepancies and failures and the associated corrective action service events.

- Record part movement transactions such as install, uninstall, and replace for both tracked and nontracked parts.
- Record utilization information (life, date, and observation characteristic values) on service events and apply these values to the as-maintained structure upon approval of these service events.
- Apply configuration updates to the as-maintained structure driven by approved service events.

Prerequisites

Some of the applications and features described in this guide are not included with the standard Teamcenter product, but must be licensed separately. For more information, contact your Siemens Digital Industries Software representative.

Basic concepts

Basic concepts

In Structure Manager, you create a product structure, sometimes loosely called a bill of materials (BOM). Teamcenter allows you to view an indented listing of the assemblies and piece parts that comprise the product structure; the indenting allows you to interpret the relationship between the assemblies and piece parts. An assembly has structure of its own; a piece part has no structure.

Because the assembly or part may have a different life cycle to the associated CAD design, Teamcenter permits you to manage and release the part and design separately. One part may be represented by several CAD designs and, likewise, a CAD design may apply to more than one part. Alternatively, a site may maintain fully synchronized, identical part and design structures.

Part structures are typically optimized for engineering change control and part reuse. They may also be linked to ERP (enterprise resource planning) or other business systems. This contrasts with the CAD structures created by design engineers that are optimized for part positioning and reuse.

In My Teamcenter, you can attach CAD design files, specification documents or other information to the top-level assembly (the product itself), or to any of the assemblies or piece parts.

You can also attach item elements to items in the product structure. They represent *features* that are not parts of the physical structure but are associated with it or implemented by it, for example, weld points or ports for connectors. They are stored as occurrence relations.

You can build, view, and manipulate the product structure in Teamcenter, or you can import it from CAD systems such as NX and Mentor Graphics.

You can include 3D visualization data as **DirectModel** dataset objects within an item revision. Users can then view these images in the embedded viewer in Structure Manager.

Understanding structure objects

Understanding structure objects

Teamcenter allows you to manage the structure of your product, including the assemblies and component parts of the product, CAD designs, and any associated structure information.

In their designs, engineers group parts together in assemblies to allow reuse of the same assemblies elsewhere in the product or in other products. An assembly can contain components that are piece parts or other assemblies. In this way, you can model a complete product structure as a hierarchy of single-level assemblies. From the Teamcenter perspective, piece parts and assemblies are both represented by *items*, and each item has at least one revision.

Each item revision may include an optional additional level of identification called a *sequence*. Sequences represent complete iterations within a revision of the item. The use of sequences is optional and must be configured by the Teamcenter administrator.

You can create assemblies that are *precise* and reference a specific revision of each component; you can also create *dynamic* assemblies in which current revision rule determines the configuration of the assembly. Dynamic assemblies are sometimes referred to as *imprecise* assemblies. The hierarchical structure relationship between the immediate parent assembly and its child component item or item revision in a precise assembly is represented by an *occurrence* (sometimes called a relative occurrence).

You can define sophisticated revision rules that allow you to configure the structure in different ways. This allows you to create a single structure and reuse it many times, for example, for different versions of the product. Revision configuration depends on the release status of an item revision, and its related effective date, effective unit number, or the release date. It allows you to reproduce a configuration that was effective at a certain date in the past or recreate the configuration of a specific unit.

You can also define and configure *variant* products that allow variations of a single product structure.

Understanding structure terminology

A structure can be any hierarchical composition of the physical components in a product, not necessarily the entire product. Different groups in your business may have different organizations or *views* of the same physical components, for example, design views may differ from manufacturing views. The structure may also include associated items that are not part of the product, for example, lifting fixtures, tooling or instrumentation.

Certain commonly used terms have specific meanings in Teamcenter, and these may differ from the meanings in other products or in your general business practices. The terms *component*, *line*, and *occurrence* all have the same basic meaning—a single-level link in the product structure between an assembly and a component. Information may typically be associated with this link, including find numbers and notes for assembly (for example, torque setting and lubrication notes). You can also store configuration data on an occurrence, for example, variant conditions.

Term	Meaning
Absolute occurrence	A relationship between a parent assembly and an item one or more levels lower in the structure. The parent assembly is the <i>context</i> in which the absolute occurrence exists. You can define data on the absolute occurrence that overrides the data stored on the item when you select the context assembly and view the structure. Both relative occurrence data (notes and transforms) and attachments can be overridden with data on absolute occurrences. Each absolute occurrence can have one or more unique attribute values that distinguish it from the other absolute occurrence derived from the same single occurrence.
Assembly	A single-level assembly with no hierarchy, as distinct from a multilevel product structure. Assembly data is stored in a BOM view revision.
BOM	Bill of materials. A flat list that defines the parts that are manufactured or procured to build the product structure. The BOM does not include geometric relationships. The use of the BOM is outside the scope of this guide.
BOM view revision (BVR)	A workspace object that stores the single-level assembly structure of an item revision. Access may be controlled on the structure (BOM view revision) independently of other data. BOM view revisions are only meaningful in the context of the item revisions for which they are created.
Composition	A collection of nodes from different structures and other data that are collected to facilitate reviews and interfaces into other applications.
Component	A node in a structure. A child in a single level parent-child relationship. When you add components, you create an occurrence for each component added. Contrast with <i>instance</i> (a child in a multilevel parent-child relationship). Note: Do not confuse a component in Teamcenter with a component in NX. In Teamcenter, a component is a single-level assembly-child relationship (a <i>relative occurrence</i>).
Configuration context	An aggregation of all the configuration rules applicable to a structure.
Default view type	Teamcenter applies a default global view type where possible, for example, when opening an item revision. Use of a default view type avoids the need for a user to choose between multiple views.
Design	The CAD design solution that implements a business part. Each part may be implemented by one or more CAD designs. Likewise, a CAD

Term	Meaning
	design may implement more than one part. Certain parts do not require a design solution, for example, paint and glue. A design may have one or more business owners.
DirectModel dataset	A dataset containing a JT (visualization) file.
Effectivity	The point at which an object becomes effective or valid, tracked by date or unit number. You can specify a closed or open-ended effectivity. Effectivity may also be specific to a given end item.
Find number	A number that identifies individual occurrences (or groups of occurrences) within a single-level assembly. Components are ordered by find number within an assembly.
Global alternate	A global alternate is interchangeable with another item, regardless of where the item is used in the product structure. A global alternate applies to any revision of the item and is independent of views. One is the <i>preferred</i> global alternate. If these global alternates are intended as <i>manufacturing</i> alternatives, you can manufacture an assembly using any of the global alternates.
	Note: Do not confuse global alternates with configurable variants. Likewise, do not confuse global alternates with substitutes.
Instance	A specific node in the product structure relative to the top level. In a multilevel structure, a complete path to the node is associated with the instance.
Line	A line in the structure and all the attributes associated with it. A line represents a single occurrence in the structure. When you click a line, you select all the attributes associated with the line, including both occurrence and item attributes.
Node	A single unique point in the product structure. Sometimes called a <i>BOM line</i> .
Occurrence note	Assembly related textual data that is associated with an occurrence.
Occurrence note type	The administrator can create different note types, for example, to specify a torque value or pressure. Users can then specify values for the different note types when required.
Occurrence	(Sometimes called <i>relative occurrence</i> .) A hierarchical structure relationship between the immediate parent assembly and its child component item (in an imprecise assembly) or item revision (in a precise assembly). You can use a find number to identify an

Term	Meaning
	occurrence, but this number may not be unique. Data can be stored on the occurrence, including occurrence notes and transforms.
Occurrence group	A collection of nodes relative to the top-level item from any level in the base product structure. An occurrence group is used to create a tightly-linked derived view. See also <i>view type</i> .
Part	A business object that is represented by an item in the product structure. Each part may have one or more <i>CAD designs</i> associated with it. The part is managed by the company's part releasing system; it is typically revised and released separately from the associated design. A part may have one or more business owners.
Piece part	A part with no structure (no associated BOM view revision). This is sometimes referred to as a detail part.
Product structure	The whole multilevel product structure, as distinct from a single-level assembly (which is sometimes referred to as a BOM view revision or BVR). The product structure is sometimes called the bill of materials (BOM).
Sequence	A subrevision of an item revision. A sequence represents a complete iteration within a given item revision. It does not track incremental changes and only one sequence may be active at any given time.
Substitute	A part that is interchangeable with a particular component in an assembly. Substitutes are often defined for manufacturing purposes, allowing use of the substitute if the preferred part is unavailable, for example, due to a stock shortage. You define a substitute for a single occurrence in an assembly and not for an item.
View type	An attribute of a BOM view revision that specifies its intended usage, for example, design or manufacture. The administrator may define any number of view types.

Working with items

Items are the fundamental objects used to manage information in Teamcenter. They represent parts and other objects that you want to manage with a life cycle. Items generally represent data that is configuration controlled by revisioning. Items collect a variety of different types of business data, for example, CAD design files for parts, document files such as specifications, reports, and forms for metadata. All such data is related to the item by appropriate relationships.

Each item has at least one item revision and a label containing two pieces of information:

- **Item ID**
An identifier for the item. No two items can have the same item ID. An item ID may be the part number or the document number of the object it represents.

In a standard environment, each item has a single, unique identifier. However, in a multifield keys environment, the administrator may combine a domain name (business object type) and one or more of the object's properties to construct the identifier. For example, in the **Part**{*item_id*} key, **Part** is the domain. In this environment, the item ID may not be unique and the system may present you with a list of possible items if you specify a nonunique item ID.

- **Item name**
A short description that is usually a logical name such as **Bolt**, **Bracket**, or the title of a document.

The term *item* generically describes all types of items that exist in Teamcenter. To effectively manage many types of items, you may create specific subtypes of items appropriate to your business. Many subtypes of items are provided with the default Teamcenter data model, for example **Part** and **Design**.

You should also distinguish between the item and its associated item revisions, as follows:

- **Item**
An item commonly represents manufactured product such as parts, assemblies, end items, and tools. It is an abstract container that holds item revisions and general documents that apply to the product, rather than to a particular revision. You cannot build or test an item.
- **Item revision**
An item revision represents a physical entity and is a unique, specific revision of a previously created item. It may have associated CAD models, drawings or specifications that are applicable only to this revision. You can release an item revision with a workflow or through change management. This action applies a **Released** status to the item revision, preventing further edits and allowing Teamcenter to maintain product history.

Users can search for an item if they do not know the applicable item revision; all available item revisions are grouped below the item. When the relevant item revision is identified, you can retrieve CAD design files and other file representations of part data including 2D or 3D images of drawings or models. If you search for an assembly, you can also view the structure of the product.

Managing alternate and alias identifiers

Overview of alternate and alias identifiers

Items and item revisions can be identified in a number of ways to match the business practices of your enterprise and those of your business partners. Initial identifier attributes, alternate identifiers, and alias identifiers are all used to communicate information about items and item revisions.

- An *alias ID* is the identifier of a part that is similar to the current part.
You can use it as a substitute part in the product structure.
- An *alternate ID* is a different identifier for the current part, but it represents the same part.
Different organizations and suppliers can have their own part numbers. With alternate IDs, you are able to find the part you are looking for using your naming scheme.

You enable display of alternate and alias identifiers by using shown item relations. Separator and context length associated with the identifiers can be configured.

Note:

If you set the **TC_publishable_<business-object-name>_contexts** Multi-Site preference for an item or item type, you can use the **Search** view to find items by specifying the alternate identifier. To search based on **Alternate ID** and **Context** criteria to find items that have been published to the Object Directory Service (ODS), choose **Change Search**→**System Defined Searches**→**Remote**. Also, your system administrator can add the identifier and context to **Item** and **Item Revision** saved queries.

For information about publishing alternate ID information to the ODS, see the *Environment Variables Reference*.

To search alternate identifier values in Structure Manager, use an **Item** or **Item Revision** search. To display alternate identifier values in Structure Manager, you must edit a preference to add the column to the BOM line display table; an administrator can use the Business Modeler IDE to add a title for the new column.

For information about making product structure searches, see Structure Manager.

Warning:

The alias identifier feature (accessed from the **Revise** and **New Item** dialog boxes or using the **New**→**ID** command on the **File** menu) is a replacement for the alias object feature accessed by choosing **File**→**New**→**Alias**. These features must not be used together in the same database.

Initial identifier attributes

When an item or item revision is created, an initial identifier (ID) attribute is assigned to the object.

Initial identifier attributes have the following characteristics:

- They are required.
- They are not case-sensitive.
- They are unique within the Teamcenter database.
- The identifier attribute cannot be modified if any revision of the item has been released.

Alias and alternate identifiers

Identifier creation prerequisites

You administrator must perform several tasks before you can work with identifiers:

- Define alias and alternate identifier contexts.
- Create identifier types in the database.
- Create **IdContext** rules to define valid combinations of the **IdContext**, **Identifier** type, and **Identifiable** type.
- If you want to use custom attributes for identifiers, your administrator must create two new classes, one for the identifier and one for the identifier revision. These new classes should be based on the identifier class.

For information about defining contexts, creating types, identifier classes and defining identifier context rules, see *Configure your business data model in BMIDE*.

For more information about alternate identifier rule characteristics, see *Configure your business data model in BMIDE*.

Alias identifiers

Alias identifiers store part numbers and other attribute information for similar parts, and they can be associated with many items or item revisions.

Alias IDs let you store information about external entities.

For example, alias IDs can be used to do any of the following:

- Store parts according to internal naming conventions and also according to the naming conventions of other companies, such as suppliers.
- Maintain a cross reference of the relationships between other manufacturer's part numbers and the part numbers used by your organization.
- Distinguish between parts tracked in other systems.
For example, you can use the alias ID to indicate that a Teamcenter item has information stored in an external database of part attributes. Based on the alias ID, you know whether the Teamcenter item has information stored in the separate database. This information can also be useful in workflows.

Alternate identifiers

Alternate identifiers store information about part numbers and attributes of the same part from different perspectives. They allow different user communities to identify and display an item or item revision according to their own rules rather than according to the rules of the user who created the object.

Assigning alternate identifiers to a part at different stages of development and production allows you to maintain a history of the lifecycle of the part.

Both alias and alternate identifiers are created within a context. The context is used to denote a specific organizational focus, such as a supplier or a department in your organization. You can also use identifiers to store custom information, such as a supplier's name and address or cost data.

Alternate identifiers have the following characteristics:

- An alternate ID identifies only one item or item revision in the database.
- Once created, the context and owning item revision cannot be modified.
- The identifier cannot be modified if any revision of the alternate has been released.
- The item alternate cannot be deleted if any of the revision alternates cannot be deleted.
- The last revision alternate of an item alternate cannot be deleted.

Alternate IDs let you define additional identifiers for an item that are then useful for setting up appropriate display contexts. For example, the design department can use item IDs, but other departments or other companies may have other IDs. A single item can be assigned any number of IDs, each unique within its context and controlled and assigned by its own naming rules.

The following example shows possible alternated IDs for an item:

```
MyItem123
    123456789@dept01
    K999999999@company01
    0000-9999999@company02
```

- A user in department 01 can set the display context to see all items with their **dept01** number.
- A manager that deals primarily with company 01 can set the context to show all items with the **company01** ID values.
- A designer can switch between display contexts, depending on the current situation.

Create identifiers for items and item revisions

You can create alternate and alias identifiers for an item or item revision, based on business rules established by your administrator, as follows:

1. Select an item or item revision in the tree or **Details** table.
2. Choose **File→New→ID**.
The system displays the **New ID** dialog box.
3. Select to create an alternate ID or an alias ID, then click **Next**.

4. Select the context for the new ID from the **Select context** shortcut menu.
5. Select the identifier type from the **Select type** list.
Only types that are valid for the selected context are displayed in this list.
6. Manually enter ID and revision values, or click **Assign** to automatically generate the ID and revision values.

Note:

The revision text box and the **Assign** button are not available when you create an alias identifier. The **Assign** button is available when you create alternate identifiers, but it is functional only when naming rules and automatic generation are set for the identifier type and identifier revision type.

7. Type a name for the new ID in the **Name** box.
8. (Optional) Type a description of the new ID.
9. Click **Next**.
If attributes are defined for the ID type, the system displays the **Enter Additional ID Information** pane. Mandatory attributes are indicated by a red asterisk in the upper-right corner of the box.
10. Fill in the attribute values, as required.
11. Click **Next**.
If attributes are defined for the ID revision type, the system displays the **Enter Additional Rev Information** pane.
12. Fill in the attribute values, as required.
13. Click **Next**.
The system displays the **Define Display Options** pane.

Note:

Display options are not available when creating alias IDs.

14. (Optional) Select one or both of the **Use item identifier as default display** and **Use revision identifier as default display** options to set the new identifier as the default display for the item or item revision.
15. Click **Finish**.
The new identifier is created and displayed according to your display rules and default settings.

Note:

The new identifier may not yet be displayed under the appropriate item or item revision.

Define alias ID and alternate ID as a shown relation

Alias and alternate identifiers must be defined as shown item relations to be displayed in Teamcenter. Perform the following steps to define shown item relations:

1. Choose **Edit→Options**.
The system displays the **Options** dialog box.
2. Expand the **General** folder (in the left pane) and select the **Item** node.
The system displays the item options in the right pane of the dialog box.
3. Click the **General** options tab.
4. Select the **Alias IDs** and **Alternate IDs** relations in the **Available Relations** list and click **Add**.
The system displays the relations in the **Shown Relations** list.
5. Click **Apply**.
Alias ID and alternate ID objects are displayed in My Teamcenter.

Note:

Use the same procedure to display the identifier under the item revision node.

Configure the format of alias ID and alternate ID objects

The default string displayed for an alias ID or alternate ID object consists of the first characters of the context name, a separator, and the alias or alternate ID.

1. Choose **Edit→Options**.
The system displays the **Options** dialog box.
2. Expand the **General** folder (in the left pane) and highlight the **Identifier** node.
The system displays the identifier options in the right pane of the dialog box.
3. Type a value for the length of the context.
4. Type a character to separate the identifier from the context.
5. Click **Apply** to save the change and retain the dialog box, or click **OK** to save the change and exit the dialog box.

Define the default display identifier for an item or item revision

1. Right-click the alternate identifier object that will be the default display identifier.
The system displays the **Item Object** shortcut menu.
2. Choose **Copy** from the shortcut menu.
The system copies the alternate identifier object to the clipboard.
3. Right-click the item or item revision for which the alternate identifier will become the default.
The system displays the **Item Object** shortcut menu.
4. Choose **Properties** from the shortcut menu.
The system displays the **Properties** dialog box.
5. Locate the **id_dispdefault** (Display Default ID) property.

Note:

If the property is not visible, click the **More** link in the lower-left corner of the dialog box.

6. If necessary, click **Check Out and Edit** to enable the arrows for the **Paste** command.
7. Click the arrow next to the value of the **id_dispdefault** property and choose the **Paste** option.
8. Click **Save** and **Check In**.

The alternate ID is now the default display for the item or item revision and is displayed according to your identifier display rule settings.

Assign an alternate identifier to a new item or item revision

1. Click the **Enter Identifier Basic Information** link in the left pane of the dialog box.
2. Select the context for the identifier.

Note:

The **Select Context** options are derived from rules set by your administrator.

3. Select the identifier type.

Note:

Only identifier types that are valid for the selected context appear in the list.

4. Enter an item ID, revision, and name for the alternate ID, or click **Assign** to automatically generate the item ID and revision identifiers.

Tip:

It may be necessary to resize the dialog box to view the **Assign** button.

Note:

The **Assign** button is available only if naming rules and automatic generation have been implemented for the selected object type.

5. Click **Next** to move to the next step and further define the item or click **Finish** to create the item or item revision immediately.

Specify additional attribute information for alternate IDs

1. Click the **Enter Additional ID Information** or **Enter Additional Rev Information** link in the left pane of the dialog box.
The system displays input boxes in the right pane of the dialog box. Mandatory attributes are indicated by a red asterisk in the upper-right corner of the box.
2. Fill in the attribute values as needed or required.
3. To further define the item, click **Next** or click **Finish** to create the new item or item revision.

Note:

This feature is available only if attributes have been defined for the selected alternate ID type.

Setting identifier display rules

What are identifier display rules?

Identifier display rules let you select the context in which items and item revisions are displayed in your workspace. If no rule is applied, the initial identifier attribute is displayed. You can create display rules or use those belonging to other users.

Note:

Display rules apply to alternate identifiers, but do not apply to alias identifiers.

Display rules can be associated with multiple contexts. The selected contexts are evaluated by the system in the order in which they appear in the **Selected Contexts** list, from top to bottom. If an item or item revision alternate identifier exists that corresponds to the first context in the list, that identifier is displayed in your workspace. If none match the first context, the next context is evaluated and if an

alternate identifier exists, it is displayed. This continues until a context is found that matches one of the alternate ID contexts defined for the item or item revision.

For more information about ID context rules, see *Configure your business data model in BMIDE*.

If the item or item revision does not have alternate identifiers corresponding to any of the contexts in the display rule, the **Use Default** option, found on the **Id Display Rules** dialog box, lets you specify that the default identifier specified for the item or item revision should be displayed. If no default identifier is specified, the initial identifier attribute of the item or item revision is displayed as specified by your default display identifier settings.

Note:

Default identifiers are specified when an alternate item or item revision identifier is created, or they can be defined as a property of the item or item revision using the **Properties** dialog box.

View and set the current display rule

This option allows you to view the current display rule or select a new rule from your rule list. You cannot create rules or select rules owned by other users with this option.

1. Choose **Tools**→**ID Display Rule**→**View/Set Current**.
The system displays the **Id Display Rules** dialog box. Your current rule is shown in the area above the rule list.
2. To change the rule, select a different rule from the list.
The system displays the context of the selected rule in the right pane of the dialog box.
3. Click **OK**.
The system displays a confirmation dialog box stating that the rule will be changed and the currently displayed data will be affected by the new rule.
4. Click **OK**.
The system closes the confirmation dialog box and the **Id Display Rules** dialog box.

Create a display rule

1. Choose **Tools**→**ID Display Rule**→**Create/Edit**.
The system displays the **Id Display Rules** dialog box. Your current rule is shown in the area above the rule list.
2. Click **Create**.
The system displays the **Create ID Display Rule** dialog box.
3. Type a name for the display rule in the **Rule Name** box.
This is the only requirement for creating a rule. However, if you do not select a context, the initial ID attributes of the items and item revisions are displayed.

4. Select one or more contexts from the **Available Contexts** list and click **Add (+)**.
The system displays the context, or contexts, in the **Selected Contexts** lists.
5. (Optional) Change the order of the contexts using the up-arrow and down-arrow buttons.
The contexts in the **Selected Contexts** list are evaluated in order, from top to bottom, to determine which identifier is displayed.
6. (Optional) Select the **Use Default** check box to display either the default or initial identifier attributes for objects that do not fit any of the selected contexts.
7. Click **OK**.
The system displays the new rule in the **Rule List** pane.
8. To set this rule as your current display rule, select the rule and click **Set Current**.
The system displays a confirmation dialog box stating that the rule will be changed and the currently displayed data will be affected by the new rule.
9. Click **OK**.
The system closes the confirmation dialog box.
10. Click **Close** to close the **Id Display Rules** dialog box.

Add rules created by other users to your display rule list

1. Choose **ToolsID→Display Rule→Create/Edit**.
The system displays the **Id Display Rules** dialog box. Your current rule is shown in the area above the rule list.
2. Click **Add**.
The system displays the **Add Id Rule** dialog box.
3. Select the owner of the rules you want to add from the **Select User** list.
4. Select the rules that you want to add to your list. You can select a single rule or multiple rules.
5. Click **OK**.
The dialog box closes and the rules are displayed in your rule list.

Modify the current display rule

Note:

This action is allowed only when there is a display rule currently set.

1. Choose **Tools→ID Display Rule→Modify Current**.
The **Modify ID Display Rule** dialog box shows the current settings of the display rule.

2. Edit the display rule as required.
3. Click **OK**.
The dialog box closes and the rules are displayed in your rule list.

Modifying identifiers

Methods of modifying identifiers

You can use the **Viewer** tab or **Properties** dialog box to modify the properties of alternate and alias identifier objects.

Once assigned, the context of the object cannot be modified. If any of the alternate revision IDs are released, the alternate ID cannot be modified. In addition, you cannot convert an alias identifier to an alternate identifier.

Modify identifier properties in the viewer pane

1. Select the identifier in the My Teamcenter tree.
2. Click the **Viewer** tab.
The system displays the properties of the identifier object in the **Viewer** pane.
3. Modify the properties, as required.
4. Click **Apply**.

Modify identifier properties in the Properties dialog box

1. Select the identifier in the My Teamcenter tree or **Details** table.
2. Right-click and choose **Properties**, or choose **View→Properties**.
The system displays the properties of the identifier object in the **Properties** dialog box.
3. Modify the values of the properties, as required.
4. Click **Apply** to modify the properties and retain the dialog box, or click **OK** to modify the properties and exit the dialog box.

Delete alias, alternate, or alternate revision identifiers

1. Select the alternate ID or alternate revision ID in the My Teamcenter tree.
2. Click **Delete** on the toolbar or choose **Edit→Delete**.
The system displays the **Delete** dialog box. You are prompted to confirm the deletion.

3. (Optional) Select related components to delete, as follows:

a. Click **Explore Selected Components** .

The system displays the component structure of the selected object in the **Explore** dialog box along with a pane for defining rules that determine which related objects are included.

b. Select the related objects, using one of the following methods:

- By individual selection
Select the check box corresponding to the component in the tree.
- By selecting all components
Click **Select All Components** located beneath the tree.
- According to user-defined rules
The right pane of the **Explore** dialog box lists type and relation combinations that can be used to select components, as defined by your preference settings. Both the **Type** and **Relation** lists include the **Any** option, which allows you to select all instances of a specific object type, regardless of relation, or to select all instances of a specific relation, regardless of object type.
Apply rule filters, as follows:

- A. Click **Add a Rule (+)** to add a rule to the table.
- B. Choose a type and relation combination by double-clicking the boxes and selecting a value from the **Type** and **Relation** lists.
- C. Click **Update the selection in the tree based on rules** to update the selections in the tree.

Note:

Rules can be removed from the table by selecting the row and clicking **Remove selected rules**.

- D. Click **OK** to apply the filters to the objects in the **Explore** tree.
The system closes the **Explore** dialog box and displays the pane related to the original operation.

Note:

The selection rules are saved as a user preference.

- E. Click **OK** to accept the related objects and return to the original operation.

4. Click **Yes** to delete the alternate ID/alternate revision ID. Click **No** to cancel the delete operation.

Note:

You cannot delete the last alternate revision ID of an alternate ID, you must delete the entire alternate ID.

Use an alternate ID as an alias ID for an item or item revision

1. Select the alternate ID object that you want to use as an alias ID.
2. Right-click and choose **Copy** from the shortcut menu, or choose **Edit→Copy**.
3. Select the item or item revision for the new alias ID.
4. Choose **Edit→Paste Special**.
The system displays the **Paste** dialog box.
5. Select **Alias ID** in the **Add As** list.
6. Click **OK**.
The system pastes the identifier object to the item or item revision with an alias ID relationship.

Navigating the structure

Nonspatial searches

You can make fast nonspatial searches for items using item attributes, Classification parameters, or occurrence notes. It is not necessary to define positional data for nonspatial searches.

Where-used

A *where-used* search allows you to find all parent assemblies in which a part is used, navigating up the structure. You can set the search depth as follows:

- One level
Reports immediate parent assemblies only.
- All levels
Reports all assemblies up to the top-level product.
- Top level
Reports final products only.

The search results may be displayed or stored as a report.

Compare

The compare command allows you to view and reconcile the differences between structures. The comparison is performed on the *as-expanded* structure and you can choose the areas of the structure to compare. You can set the revision rule and variant rule separately for each structure, allowing different configurations to be compared. The comparison identifies quantity and revision differences in three modes:

- **Single level**
Compares only the first levels of the product structures.
- **Multilevel**
Performs a single-level comparison at the top level, and then invokes further single level comparisons on any subassemblies that match between the two product structures. This process is repeated successively down the product structure.
- **Lowest level only**
Compares only the lowest level items of the product structures, ignoring all intermediate assemblies. This is useful for checking that piece parts are consistent.

The results are displayed in a dialog box that steps you through each difference. You can also print the results to a report.

You can also perform the comparison graphically in the embedded viewer, which is a useful tool for designers. In this case, the components present in one structure but not the other are highlighted in separate colors and can be isolated from the other common parts, allowing easy visualization of the changes. You can also perform this comparison on different revisions of piece parts to highlight geometry changes.

Several applications including Multi-Structure Manager allow you to compare two views of the same product structure to identify differences. For example, you may want to compare two revisions of the same structure or the design and manufacturing views of the same structure. Any anomalies found during the comparison are reported, although an imbalance between the two structures may not necessarily indicate a problem with either of the structures. The following anomalies are reported:

- Lines in the source structure that are not consumed in the target.
- Each time a line is consumed more than once.

You can run an analysis manually or in batch mode. You can also run it with or without variant options applied.

Find in BOM

The **find in BOM** command allows you to search for components in the structure by any of their properties, for example, item ID, name, attributes on the item revision master form, and occurrence

note attributes. You can build a query with as many expressions as necessary, giving a flexible way to find components in large structures. This search works on the expanded state of the structure.

Fast spatial searches

Teamcenter allows you to search the product structure to locate items within a specified area. For example, you can search for all items within a given proximity of a design assembly to identify possible clearance issues. You can also search within a 3D box.

You can initiate a fast spatial search in one of two ways:

- Entering coordinate values in the search dialog box.
- Dragging a bounding box on geometry shown in the embedded viewer.

The search returns a list of items found in the specified area. Spatial searches can be combined with attribute value searches, Classification searches, occurrence note searches, and saved queries to refine the results.

Configuring the structure

Configuration principles

There are two optional methods of configuring structure—revision configuration and occurrence configuration.

Revision configuration

Revision configuration principles

For each BOM line in the structure, this configuration method determines the revision of the corresponding item to configure according to attributes of the revision, for example, its release status, effectivity, or owning user. When you open the structure in Structure Manager, a revision rule is always active and consists of a number of parameters that Teamcenter uses to make the evaluation.

The revision rule returns one of the available revisions of each item in the structure, according to the criteria set in the revision rule. If it finds no revisions, Structure Manager displays ??? for the affected line and no further expansion of the structure is possible below that point. (If the item is an assembly, each revision may have different components and, if a specific revision is not configured, further expansion may not be sufficiently defined.)

Note:

Structure Manager can only display a specific configuration. It cannot display all revisions of the same item.

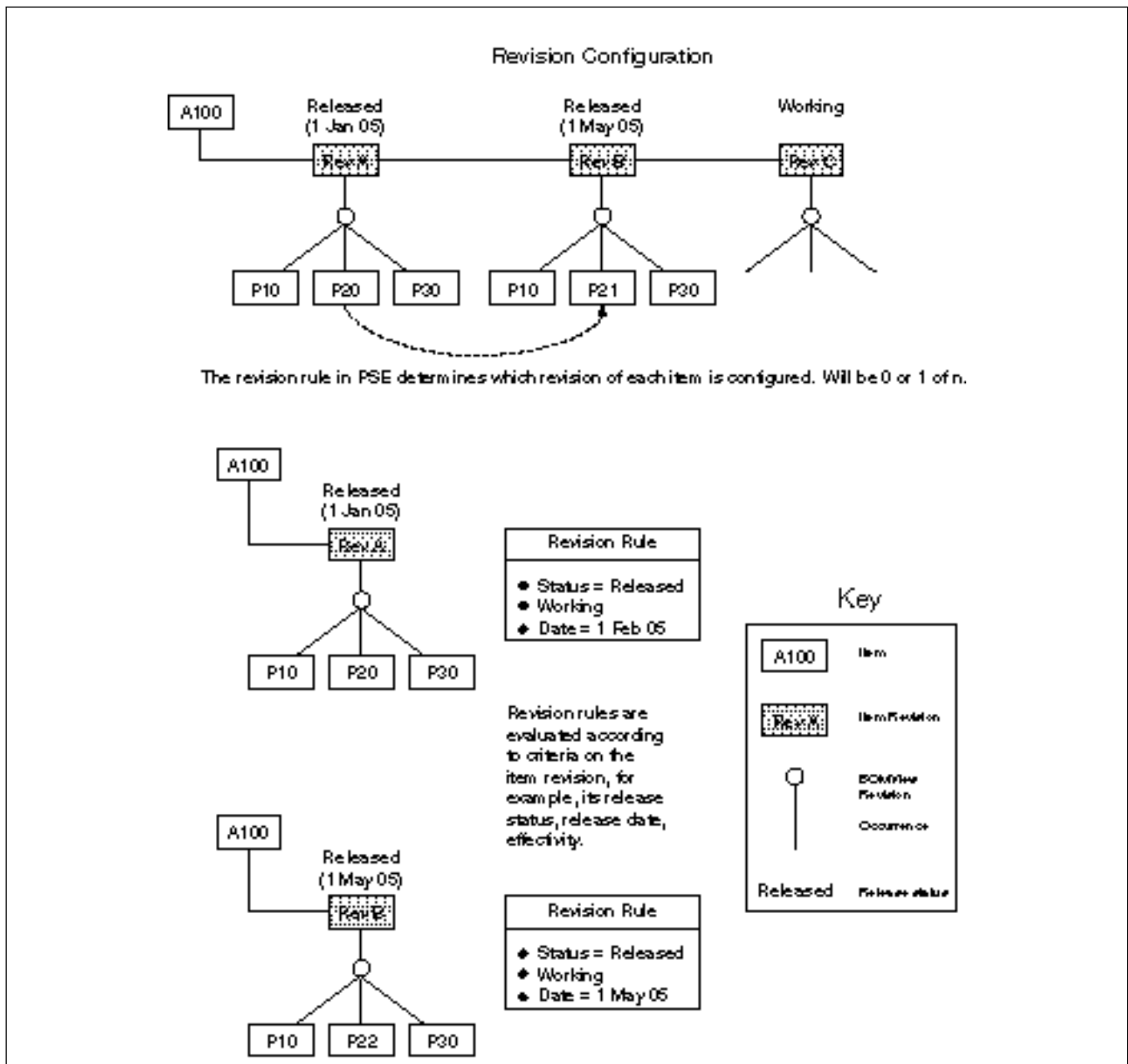
Depending on your business practices, it may not be appropriate to apply the revision rule to all levels of the product structure. You may want to configure the higher level subassemblies with revision effectivity, but not the lower level piece parts. Piece parts may have a date effectivity, for example, if they have a limited shelf life. They may also have a status, for example, **InWork** that changes to **Released** when the product is released.

Engineering change management

You can use the Change Manager application to apply engineering changes (ECs) to the structure. ECs are workspace objects that collect all information about an engineering change. In particular, they contain the new **Affected** item revisions to release, the old **Problem** item revisions to replace, and the new **Solution** item revisions created for the change and consumed in the new **Affected** item. Change management computes the BOM changes for the new assembly and lists them in the Change Manager application, making it easier for you to see what changed. These changes can also be viewed graphically.

An EC is typically released with all its **Affected** and **Solution** item revisions. The item revisions then share the same release date or effectivity, ensuring the complete change is configured. In Structure Manager, you apply the revision configuration to the **Affected** and **Solution** item revisions, rather than the EC object itself. However, as the objects all share the same effectivity, if the effectivity is changed on the EC, it is also on the related item revisions to ensure consistency.

You can define a **supersedure** that defines the part or parts that replaced a component, creating a revision history that can be referenced in the future.



Using revision configuration to configure structure

Occurrence configuration

The occurrence configuration method applies a filter to each occurrence in the structure. Teamcenter configures occurrences according to data defined on the occurrence in combination with the configuration rule. The configured occurrences are visible in the structure and unconfigured occurrences can be hidden.

Teamcenter supports the following kinds of occurrence configuration:

- Incremental change

A revision rule determines the incremental changes that are configured. Occurrences are added or removed from a structure with reference to an incremental change, and are thereby configured by the revision rule.

- Occurrence effectivity
A revision rule sets a date or unit for the configuration. Effectivity is defined on the occurrence and thereby configures the occurrences.
- Variant configuration
You define option values and set them against a variant rule. Teamcenter evaluates the variant conditions and configures the structure accordingly.

Incremental change and occurrence effectivity are alternative methods. Incremental change is preferred in most cases, as it provides a mechanism to link the add and remove when a component is replaced. You can use either method in combination with variant configuration.

There are some limitations if you use occurrence data with a CAD system. The CAD system typically has a design file for a specific configuration and the design file is stored in a dataset attached to the item revision, for example, a **UGMASTER** dataset for NX geometry. In this case, you can configure any permutation of occurrences (CAD components) and this constraint has implications for CAD usage, for example, how you define mating conditions.

Incremental change configuration

Traditionally change management has focused on revisioning of items. An item revision defines the state of an item at a particular point in its life cycle. When the item moves to a new life cycle stage with a changed state, a new item revision is created. Changes may include the addition or removal of (product structure) components, changes to encapsulated data (datasets) and changes to the metadata that describes the item (for example, metadata on forms). This approach allows you to create incremental changes (ICs) for specific configurations of manufacturing items. It also allows CAD expansions to be driven by incremental change elements (ICEs).

However, there are situations where this traditional method of change management is not ideal and the incremental change functionality is designed to address those situations. A typical situation is that of very large, flattened bills of materials for manufacturing process structures or product structures. Here, several independent structure changes may be made concurrently, adding and removing unrelated components; significantly, those changes can be implemented in any sequence. In this case, revisioning is not practical as a separate revision would be needed for every permutation of change, resulting in a huge amount of redundant duplicated data.

An incremental change is modeled in the same way as engineering changes in the Change Manager application. It collects changes to components or attachments (data) to the component added or removed. Effectivity is typically applied to ICs and thereby configures the associated changes. ICs are configured by revision rules in the same way as revisions to items in the product structure.

Note:

CAD integrations do not support data tracked by incremental changes. For data associated with these integrations, an item revision does not directly correspond to the CAD dataset representing a fixed set of components.

In summary, consider using IC configuration for:

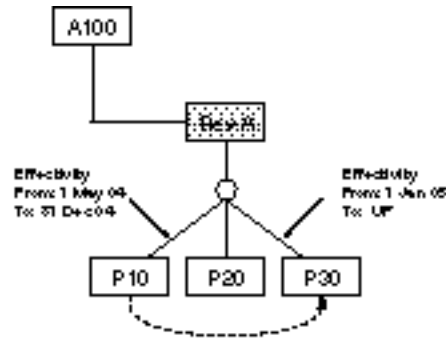
- Large flat structures.
- Structures with independent changes.
- Structures with changes applied in any sequence and configured by unit or date effectivity.
- Supporting *intent* (alternative solution) configuration.

Incremental change is available in the following applications:

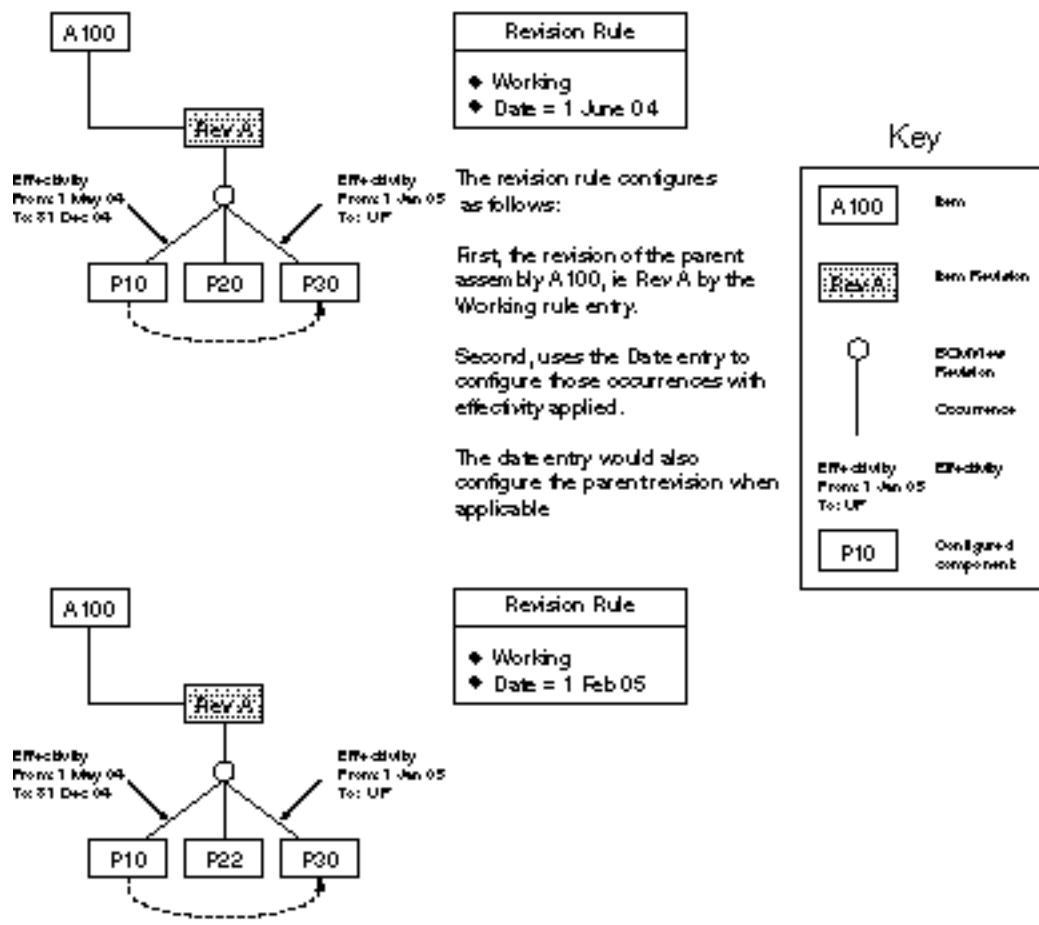
- Structure Manager
- Manufacturing Process Management – Manufacturing Process Planner and Part Planner
Process structures are typically flat and may change frequently while they are developed. Frequent changes may be made to the setup data attached to operations. Incremental change is an appropriate way to control and configure these changes.
- Multi-Structure Manager
You can set IC mode when exporting or importing PLM XML data. This allows the export of only changed data, rather than the complete set of data. Similarly, only the changes are imported and used to update the existing data.

Occurrence effectivity

The following figure shows how to use occurrence configuration to configure a structure.



A revision rule in Structure Manager determines which occurrences of each BOM View revision are configured. In Structure Manager, the unconfigured occurrences can be hidden.



Using occurrence configuration

Teamcenter supports both date and unit number effectivity if you implement occurrence effectivity.

Variant configuration

Variant configuration

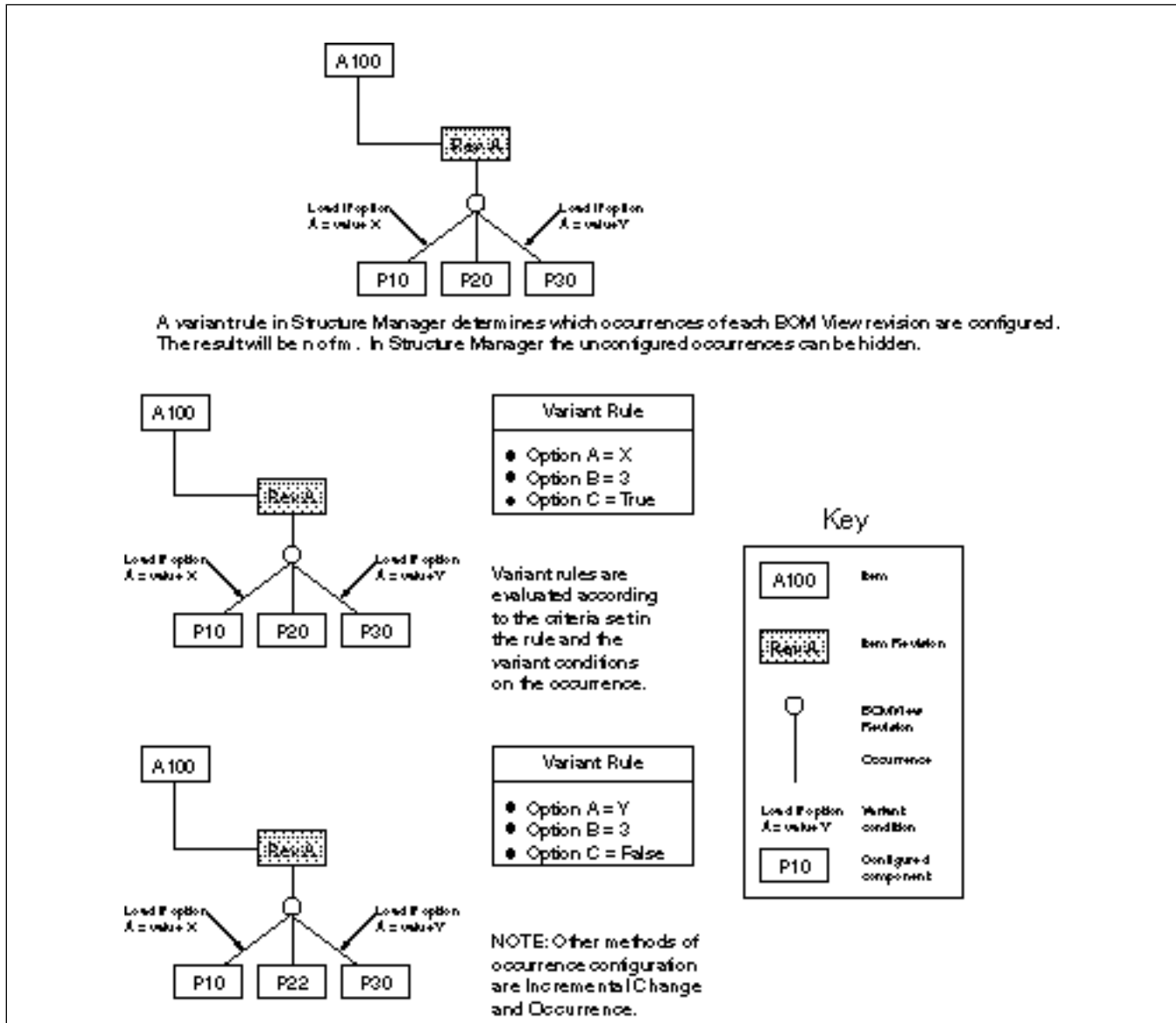
Manufacturers often want to create a range of products based on the same generic platform, offering their customers the greatest choice for the least engineering cost. Traditionally, this configuration has been managed by manufacturing resource planning (MRP) or enterprise resource planning (ERP) systems, often with large duplication of bills of materials. This technique makes maintenance (the implementation of engineering changes) difficult and error prone. A better approach is to create a single generic product structure that can be configured for each different variant of the product offered. The Teamcenter options and variants module supports this capability.

You can define options and the corresponding allowed values and attach them to an item, typically the top-level item in the structure. For example, you could define a **Gearbox** option with allowed values of **Manual** and **Automatic**. You then attach a logical expression, referred to as a *variant condition*, to any occurrences of components that are configurable, for example, the automatic and manual gearboxes. The expression refers to the defined options and can be as complex as necessary.

You can define and attach additional logic to the item, as follows:

- Set error warnings for incompatible option values selected in the variant rule.
- Set default values for options.
- Set derived default values to allow one option value to set another. This is useful when creating option packages.

You choose the desired option values for a configuration and set them in a variant rule. Teamcenter evaluates the variant conditions on the occurrences in the structure against set option values, and components are configured or not configured according to the results of the evaluation. Unconfigured components can be hidden.



Using variant configuration

Modular variants

The approach described previously is the foundation of *classic* variant management. This approach does not impose any restrictions on modularity, which may be desirable. To minimize engineering and production costs, you can create *modules*—self-contained plug-compatible units that can be *reused*. This technique imposes tight constraints on how the product is designed; Teamcenter supports those constraints when you work with modular variants.

When you use modular variants, the options and associated logic are attached to an item referred to as a *module*. (This is not a special item type.) Variant conditions on the components can only refer to options on the parent module, thus making the module independent of any other parts in the structure with regard to variant logic. This approach requires variant data and option values to be propagated up and

down the structure so that end users (customers) can set options on lower level modules when configuring the whole product from the top level.

Platform Designer

Platform Designer allows you to author variants at a conceptual level before design work commences. You can use the variants to drive the engineering design process, as distinct from the traditional approach where variants are created from the finished design.

Use Platform Designer to create placeholders that are subsequently filled with:

- Design solutions representing assemblies or parts, when they are complete.
- Business parts that are associated with the bill of materials, which is typically maintained in an ERP system.

Platform Designer allows you to author variability in one of two ways:

- Create an architecture breakdown, which is linked to the product structure. Platform Designer checks that each element in the architecture breakdown is satisfied before the product structure can be released. The product architecture on which the architecture breakdown is based is typically populated from an external configurator or business system.
- Create generic objects that comprise the generic product breakdown. You can use the generic breakdown as a template for specific product structures. No link is maintained between the generic breakdown and the product structure, so the structure can be revised if necessary.

Creating occurrences

Occurrence creation principles

When you add an item to a product structure, you are creating a specific *occurrence* of that item or item revision in the product structure. It becomes a *component* or node, and is displayed as a line in the product structure.

Teamcenter also supports the creation absolute occurrences from (relative) occurrences. Absolute occurrences contain data that is unique in the context of a certain top line, but not in the context of the entire product structure. For example, the tire pressures on a vehicle may be set differently in the context of the front axle assembly and the rear axle assembly.

Note:

Do not confuse occurrences with absolute occurrences.

The occurrence may be precise or dynamic, depending on whether the BOM view revision (BVR) is precise or dynamic (imprecise) in the parent assembly, as follows:

- If the item revision of the parent assembly has a precise BVR, Teamcenter creates a precise occurrence of the specific revision you added.
- If the item revision of the parent assembly has an imprecise BVR, Teamcenter creates a dynamic occurrence of the item only. The revision is selected by the revision rule you apply.

Publishing occurrence data

Your company may choose to manage business parts and CAD designs separately, as distinct from creating a business part corresponding to each design revision. There are several reasons you may choose this approach:

- Parts and designs have different life cycle rates, for example, you obtain a part from several suppliers, each of which changes the design frequently.
- To maintain different structures of parts and designs, for example, because a flat part structure is preferable for manufacturing.
- To address the situation where many business parts are represented by a single CAD design, for example, colored parts that are identical in other respects.
- To address the situation where one business part is represented by several CAD designs, for example, a flexible part.
- Your existing business data model implements separate part and design objects, for example, because you previously used Teamcenter Enterprise.

Note:

If you have a structure that is based on items and item revisions, you can convert it to separate parts and designs by running the **item_to_part_design** utility.

When you create independent part and design structures, you organize and manage them according to the needs of the users. For example, the design structure may be optimized for component positioning and reuse, while the part structure may be flat for easy identification of components. However, the part and design structures are different views of the same product and must be aligned. For example, some part occurrences may use positioned designs and need transform and shape data from the design for visualization. The action of aligning the structures is referred to as *publishing* occurrence data.

You publish occurrence data between source and destination absolute occurrences of two representations. Teamcenter takes a snapshot of the source occurrence data and creates a publish link to the destination occurrence. For example, you can publish transform and shape information from a source design occurrence to a destination part occurrence. If the source occurrence data was already published, Teamcenter updates the snapshot with the new information.

Understanding properties

Understanding properties

Each line represents a single node or BOM line in the product structure, which contains a specific revision of an item. You may choose to create a product structure from items or item revisions, depending on your business and design practices.

Each such node may have properties that depend on the type of item that the line represents. For example, if the line represents a tire, it may have an attached note that specifies the inflation pressure.

It may display one or more properties according to the item it represents, including item identifier, item revision and item revision master form. It may also display one or more occurrence attributes including find number, release number, quantity, and occurrence notes.

Note:

Some variable properties are only assigned at run time when the product structure is displayed, for example, form attributes and status. Other properties are persistent and are the same each time you display the product structure, for example, occurrence notes and properties such as find numbers.

Defining occurrence properties

If you have administrative privileges, you can create occurrence properties for your site by defining new note types, as described later in this section. Users can then specify the value for any note on a particular BOM line.

Note types are associated with an assembly-component relationship, for example, the torque value for a bolt may be 10 ft lb in one assembly and 15 ft lb in another assembly. The item and item revision may have other properties such as material, weight, or cost that are not specified by the occurrence note. The properties are used as follows:

- **Item properties**
Specify what the item is, for example, a bolt or nail. Units of measure may also be associated with an item and are used when specifying quantities.
- **Item revision properties**
Specify an exact implementation (but not necessarily the implementation) of the item revision, for example, the weight.
- **Occurrence properties**
Specify the use of an item revision in the context of its parent assembly. This is done in an occurrence note.

You can edit the properties of a BOM line on an individual basis. You can also edit multiple properties of several BOM lines simultaneously; when you select several BOM lines, Teamcenter identifies all common

properties that can be changed for the current selection and presents only those properties in the edit dialog box. This method allows you to edit both item properties and occurrence properties.

Teamcenter associates special significance with two occurrence properties:

- **Find number**
Changing the find number causes Teamcenter to reevaluate the order of the BOM lines. If you collapse and re-expand the parent assembly, the BOM is rearranged according to the new find numbers. If you pack or unpack the BOM, the order of the BOM lines remains unchanged and is determined by the find numbers.

Note:

Changing the find number of packed occurrences changes the find number of each individual occurrence.

- **Quantity**
You can define a numeric value to represent a quantity. The permitted values are whole numbers for items that have no units of measure defined or decimal values for items (non-geometric parts) that do have a unit of measure defined. You can also specify **As Required** if you do not want to enter an exact value.
Do not confuse this property with the number of occurrences in the assembly. If you use NX, the number of occurrences in the CAD model corresponds to the number of occurrences in Teamcenter.

Note:

You should only define numeric values for parts such as rivets that are not modeled in separate positions and therefore all share the same transform.

Creating rollup reports

Creating a properties rollup report

Teamcenter allows you to create rollup reports of certain properties, including mass, center of mass, moments of inertia, and products of inertia. A rollup is a recursive calculation for all children of a parent line, as far as the selected root node of the structure.

Calculating mass rollup

To calculate a mass rollup of an assembly, Teamcenter takes the mass information from all the components in the assembly and calculates the total mass of the root assembly and any subassemblies.

Calculating center of mass rollup

To calculate the center of masses of an assembly, Teamcenter:

1. Transforms the center of mass of every component in the assembly to the assembly coordinate system. It takes the center of mass of each component and multiplies it by the transform matrix of the component. It then adds the component's origin to obtain the center of mass of the component in the assembly coordinate system.
2. Adds the center of mass of all the components in the assembly. For each component, it multiplies the mass of the component with each center of mass coordinate of that component. It then separately adds all the **x** coordinates together, all the **y** coordinates together, and all the **z** coordinates together. It divides the total **x** value by the total mass of the assembly to obtain the **x** coordinate for the center of mass of the assembly, then obtains the **y** and **z** coordinates in a similar way.

Calculating moment and product of inertia rollup

To calculate the moment and product of inertia of an assembly, Teamcenter:

1. Creates a tensor matrix for each component that is populated with the moment and product of inertia information that is provided by the CAD system.
2. Transforms each tensor matrix to the assembly coordinate system using the Steiner matrix.
3. Adds together the tensor matrices of each component.
4. Applies the Steiner matrix to the total value to calculate the rolled up tensor with respect to the center of mass of the assembly. You can obtain the values of the moment and product of inertia of the assembly from this value.

Creating occurrence note types

If you have administrator privileges, you can create occurrence note types appropriate for your site. An occurrence note type may represent any specific property. Any user can define a value for a note type on a particular line in the BOM (occurrence).

Occurrence notes may define data that could be different between different installations of the same part. For example, a bolt could be used both on the chassis and the drive shaft. In either case, the properties for length, thread pattern and head size are the same, and would be properties of the bolt item or item revision. But the drive on the drive shaft might require tightening to a higher torque than the chassis bolt. In this case, you could use an occurrence note to hold the torque for each occurrence.

Caution:

Notes are occurrence properties and are not suitable for specifying attributes of the item or item revision, such as material. Use item masters and item revision properties for this purpose.

Creating multiple views

The technique of defining multiple BOM view types has always been available in Teamcenter and continues to be supported for customer implementations. In practice, this technique presents process challenges that may be avoided by using alternative methods. Consequently, Siemens Digital Industries Software recommends customers avoid new implementations of multiple BOM views, although there is no reason for customers with successful deployments to transition to other methods.

Specific process challenges that may be encountered include:

- **Change management**
Revision of released assembly item revisions is required when any of the multiple views change.
- **View specific content**
When multiple views have view specific content, each view must be manually updated for any changes to the common content.
- **Editing efficiency**
When performing paste, drag and drop, duplicate, or other editing operations, it may not be easy for the user to identify the BOM to create or change. For some operations, a **Paste Special** command allows the user to specify the BOM view target of the operation. However, multiple views do not allow the use of shortcut keys or drag and drop operations between the different views. It also restricts the user's ability to perform operations on many objects in a single operation.

Historically, implementations of multiple BOM views were motivated by several business needs. If your company's implementation has these needs, consider the following alternative tools:

- **Domain specific content in the assembly of interest to some, but not all, users of the product structure.**
In general, the hierarchy of the content is the same and much of the content is common. Consider including all content in the structure, but make use of properties or item subtypes to distinguish the special content. When viewing and expanding the structure, closure rules may filter the content if desired.
- **Different views of engineering and manufacturing data.**
Teamcenter provides applications for defining and managing an MBOM that is derived from an EBOM, namely Manufacturing Process Planner and Multi-Structure Manager. It supports accountability management, comparison, and reconciliation of changes throughout the lifecycle. Similarly, if there is a need to model other domains such as logistics, after-sales, or service, these structures can be managed and maintained using multiple related structures.
- **If there are multiple views representing differences between regions or plants producing similar products, you can use Manufacturing Process Planner accommodate those differences with templates and create multiple tailored MBOM structures from a common EBOM source.**
Alternatively, you can express the differences as variability in the structure.

- Assemblies may require full detail in some views and only the top level view in other contexts. You can be accomplished with closure rules, as mentioned previously. Alternatively, if you need the assembly content for visualization, but not for expanding or viewing the detailed content, identify the subassembly as an end item.
- If there are context-specific differences in a reused assembly, consider expressing those differences by performing in-context edits in the structure to create absolute occurrences.

Using BOM view revisions

When you add a component to an assembly, you are creating an occurrence of that item or item revision in the assembly, which is stored on the BVR. This occurrence is displayed as a BOM line. A BVR is a single-level structure that contains occurrences of its immediate children. A multilevel structure is built up from many single line BVRs.

Each assembly revision has its own BOM view revision (BVR). A BVR contains the single level structure of an item revision. When you expand an item revision, you see the BVR in which the occurrence information is stored.

Note:

A BVR stores many occurrences. However, there is a direct correspondence between a single item revision and its BVR for a given view type.

Any modification to a product structure (including changing any of the occurrence attributes or adding a substitute) causes a change to the BVR of the parent assembly.

If no BVR exists in the parent assembly, Teamcenter creates one automatically. When BVRs are automatically created in this way, they may be precise or imprecise according to your specified options; by default, imprecise BVRs are created.

If multiple views are defined for your site, Teamcenter determines the view type to create.

Using representations

Using representations

A representation is a general term for any structure that you can model with one of the following objects.

BOM view

A *BOM view* type represents different, but related structures, for example, the single level structure of a CAD assembly (an engineering BOM or EBOM), a manufacturing structure (a manufacturing BOM or MBOM), and a spares list for a given item. There is no direct relationship between the components in

these different structures, so changes in one structure must be propagated manually by making appropriate structure edits to the others.

Different BOM view types are associated with the same item. When you add a component to a structure, you specify one of the view types created on the item (by default, this is typically the CAD view type). When you expand the structure, the specified view type is expanded. To change the view type in the structure, you edit the BOM.

The multiple BOM view approach for modeling EBOMs and MBOMs is similar to the use of structure contexts, which are described later. You can add items to the MBOM that do not exist in the EBOM, for example, sealant; this is not possible if you use occurrence groups. Multiple views of the same product are associated if you place them in the same item, while those for a structure context are associated if they are in the same collaboration context object. Unlike structure contexts, there is no link between the views.

BOM views are not synchronized to the base structure and are not updated if the base structure changes. Changes can only be identified by manual comparison and reconciliation.

You can create additional view types with the Business Modeler IDE application. You can set a preference to determine the view type CAD integration.

Occurrence groups

An *occurrence group* represent a subset of the components in the top level or root item against which the occurrence group is created. This subset may collect components for a design task, or group components to consume in a manufacturing operation. The base structure or view may have any number of associated occurrence groups, referred to as *derived views* (not BOM views). You can create occurrence groups with the Multi-Structure Manager application.

Unlike BOM views, the components in an occurrence group are loosely tied to the parent item in the base structure from which they were created. This has two advantages:

- Structure changes in the base view are automatically propagated to the derived views. Such changes include the removal and addition of components, changes to occurrence notes. It is not possible to propagate added components as this change requires a decision by the user. Such additions must be made manually to the derived views if appropriate.
- If a variant rule is applied to the base structure, it automatically configures the derived views. Consequently, components may become unconfigured in the derived view if the corresponding components are unconfigured in the base view.

Structure contexts

A *structure context* is a specific configuration of a representation. A structure context is similar to an occurrence group, but contains a configuration context. The configuration context is a persistent object that stores the configuration specified by revision and variant rules. The structure context also contains the root item.

To rearrange the structure of a product, you should:

1. Create a second structure context to contain the new representation.
2. Create a new top-level item as the root of the new representation.
3. Create new items as components of the structure context for the new assembly levels.
4. Assign components from the original base product structure to the appropriate assemblies under the root item of the new structure context.

Unlike an occurrence group, which may only contain a subset of the components in the base structure, a structure context may contain additional items that are not in the base structure. For example, a manufacturing representation may include items such as sealant and welding rods that are not required when designing the product.

A *composition* (described next) is a special type of structure context to which you can assign components from several structure contexts representing different product structures.

Create structure contexts in the Multi-Structure Manager application.

Compositions

A *composition* is a special type of structure context that accepts components assigned to it from other structure contexts. There are two kinds of scenarios in which you may use compositions:

- **Design**
In this scenario, the composition is a study in which components from different products can be brought together. They may then be positioned relative to each other so that a part that impacts all products can be designed or analyzed in context.
- **Manufacture**
In this scenario, the composition is a process structure that includes components consumed from the product structure, and machines or tools from the plant structure for assembly or machining operations.

Compositions are tightly synchronized to the base structure, and any changes to the base structure are propagated to the composition. You can also add items to the structure that do not appear in the base view, for example, raw material, lubricant or weld points.

Overview of configuration methodologies

Teamcenter provides two mechanisms for controlling the configuration of product structures:

- Revisioning of items

An item revision defines the state of an item at a particular time. When the item moves to a new life cycle stage typically with a changed state, Teamcenter generates a new item revision. Changes that necessitate a new item revision include the addition or removal of product structure components and changes to encapsulated data (datasets) and the metadata that describes the item. For changes that do not affect the form, fit, or function of the item, a new item revision is created. For changes that affect form, fit, or function, a new item ID is created and the assembly revision is incremented, if it is released. This approach ensures interchangeability is maintained. This method of change control is widely used for large, complex structures with many layers of nested subassemblies or components.

- Incremental change

An incremental change order collects together a number of structure changes to a component such as addition or removal of components or changes to attachments (data). Effectivity can be applied to incremental changes to configure the associated changes. This method of change control allows several independent structure changes to be made concurrently, including the addition or removal of unrelated components. Those changes can be implemented in any sequence. This method of change control is suitable for large, flat structures without nested subassemblies or components. The revisioning approach is not suitable for this type of structure, as a separate revision would be needed for every permutation of change.

Note:

Incremental change is not supported by CAD integrations.

Integrating with CAD systems

Teamcenter allows a close integration with the NX CAD system, through the optional Teamcenter Integration for NX package. Teamcenter Integration for NX allows you to work in the NX CAD system with Teamcenter running in the background. In this mode, the **Open**, **Search**, and **Where-Used** dialog boxes are available for direct access to Teamcenter functionality.

You can also:

- Synchronize structures between NX and Teamcenter, with the master data held in Teamcenter.
- Synchronize attributes including items, item revisions, item revision master forms, and occurrence notes. (Occurrence notes in Teamcenter map to component attributes in NX.) You may also synchronize part attributes (for example, weight), the owning user, signoffs, and engineering changes. The master data may be held in NX or Teamcenter, and the synchronized data appears in the title block of the drawing.

In addition to NX, you can integrate Teamcenter with most common MCAD systems, including CATIA versions V4 and V5, ProENGINEER, Solid Edge®, and SolidWorks. You can also integrate with ECAD systems such as Mentor Graphics and Cadence.

If the CAD system generates **DirectModel** datasets for JT files, you can visualize designs in the embedded viewer. If you have several CAD systems, Teamcenter allows you to view all the images in a single environment.

Load options allow you to expose configuration rules, including saved variant rules, revision rules, and set dates.

Integrating with ERP systems

When you create a product structure in Teamcenter, you can transfer the data to an ERP (enterprise resource planning) system. If you plan to do this, configure your product structure to suit the needs of the ERP system.

If you have variant structures, you can:

- Send preconfigured variants, including variants items.
- Send generic structures and option logic.

If you configure multiple views, you can send the engineering BOM or manufacturing BOM.

If you configure manufacturing structures, you can send the process structure and consumed items.

Part I: Basic principles

Building and maintaining product structure	2-1
Comparing product structures	2-1
Finding items in the product structure	2-1

2. Building and maintaining product structure

Building and maintaining product structure	2-1
Building product structure	2-1
Cloning the product structure	2-2
Editing the product structure	2-3
Creating precise and imprecise assemblies	2-4
Creating substitute components	2-5
Importing and exporting product structure	2-5
Importing a product structure with Excel	2-6
Packing and unpacking product structure	2-6
Grading	2-7
Copying data between applications	2-9
Making mass updates to the structure	2-10
Using the embedded viewer	2-14
Analyzing clearances	2-15
Clearance analysis principles	2-15
Analyzing clearances	2-16
Understanding the clearance database	2-17
Best practices for managing clearance analysis	2-17

2. Building and maintaining product structure

Building and maintaining product structure

You can create the product structure manually in Teamcenter or you can import it from your CAD system. When you import a structure, Teamcenter keeps the structure synchronized with any changes to the native CAD design. When you have created the structure, you modify it as necessary to reflect any changes to the product design.

Teamcenter also includes several capabilities that allow you to configure and modify product structure including:

- Revisioning
- Effectivity
- Incremental change management
- Variants and modular variants
- Absolute occurrences

When you have completed the product structure, you can create the process and plant structures for the manufacturing processes that will build the product.

Caution:

Certain applications may automatically save changes to a structure that is already open in another application. For example, Manufacturing Process Planner and Multi-Structure Manager automatically save structures that you browse in Structure Manager. To avoid inconsistent data, Siemens Digital Industries Software recommends that you close all other applications when you work in Structure Manager.

Building product structure

You can manually build a product structure from items and items revisions. The item represents a container for the business part and its associated CAD design. You can also build structure from items and item revisions that specifically represent parts and designs using the separate **Part** and **Design** types, which are subtypes of **Item**. A part has an **Is Represented By** relationship with a design.

Parts have many properties, but the following are of interest when building structure:

- Is Design Required

Indicates if the part must have an associated CAD design, which is **true** by default. Certain parts do not have associated designs, for example, glue, paint, and tools.

- **Make or Buy**

(Optional) Indicates if you make the part or buy it from a supplier.

- **Req Pos Design**

A positioned design may not be required for some occurrences of components. For example, a positioned design may be of interest for certain screws in an assembly. The attribute allows you to denote the need for a positioned design in absolute occurrence context. The positioned design is the absolute occurrence of a design in the context of the product.

- **Is Designed in Place**

Some parts may have a small number of occurrences in the product, for example, bumpers or fenders. If so, they may be positioned directly in the product space, that is, designed in place. This attribute denotes that the underlying part revision of the occurrence is already in absolute position. When this part occurrence is visualized, the primary design revision of the occurrence's underlying part revision is visible.

The design has the geometric definition of the part associated with it, for example, CAD or CAE files. A part is associated with a design by the **Is Represented By** relationship. If the part is revised, the design revisions associated with the previous part revision may be carried forward to the new part revision, as configured by the Teamcenter administrator.

When you have created the product structure, you can manually edit or reorganize it when necessary. For example, you can insert or remove levels, move or replace nodes, and split an occurrence. You can also designate global alternates of a selected part.

When a design is complete and approved, it is considered *mature*. If a design is mature, the associated part revision may be released and occurrence data published.

Cloning the product structure

In some cases, you may want to create a new product structure from an existing product structure, for example, if the new product structure is similar to the existing product structure. If so, you can *clone* the existing product structure; that is, create an exact copy. You clone the currently selected structure, which may represent the entire product or only a subassembly. The structure may be precise or imprecise and can be configured if appropriate.

Note:

It is not necessary to open the CAD tool to create the clone.

The user must have write access to all the elements of the structure to successfully create the clone. The clone structure is owned by the same user.

As an alternative to cloning a structure, you can create a product structure template to use as the basis of each new product structure.

The cloning operation may rename items and item revisions, depending on the current naming rules. Datasets are renamed according to the Business Modeler IDE deep copy rules for **SaveAs** actions or CAD integration overrides. If file renaming is selected, the CAD integration renames CAD files and datasets, according to the requirements of the individual CAD system. If file renaming is not selected, the CAD integration does not rename the CAD files.

When you open the clone, you can edit or update any of the items, item revisions, datasets, and attachments without affecting the original structure.

Editing the product structure

You can modify the product structure by adding or removing individual items or item revisions as necessary. Alternatively, Teamcenter includes its own clipboard that allows you to build or edit product structure with the cut, copy, and paste functions.

You can also add or remove levels from the structure, where a level is an item that represents an assembly. You can also move a selected node to another location in the structure, and all assemblies and all items under the affected node also move. Likewise, you can replace one item in a structure with another, and all associated data is preserved.

You can split an occurrence (BOM line) or branch in a structure into more than one branch. The new branch and the original, changed branch have exactly the same data (for example, notes and variant conditions) but may be modified independently. The total quantity after the split equals the original quantity. You cannot split an occurrence into more than two occurrences.

The paste function allows you to insert an item or item revision from the clipboard and create a corresponding component in the selected assembly in the product structure. If you paste an item, Teamcenter creates an item; if you paste an item revision, Teamcenter creates an item revision if the assembly is precise.

Pasting does not remove the item or item revision from the clipboard. You can paste several occurrences of the same item or item revision into the same assembly or a different assembly.

If you copy or cut a component from an assembly and then paste it in elsewhere, the occurrence attributes of the component (for example, quantity and notes) are also copied. However, the find number is not copied as it is automatically generated according to the position in the target assembly.

Optionally, Teamcenter can validate the structure by verifying that you add only appropriate child items and item elements (GDEs). If a user attempts to make an invalid addition, Teamcenter displays an error message.

Your site may also chose to highlight pending changes including attribute value changes and structural edits. Pending changes are marked with redline and strike-through markings until they are committed to

the database. Pending changes can be canceled or applied to the structure. The last set of changes made to the currently selected BOM line can be reverted to its original value or structure. Changes are highlighted only in Structure Manager and not in other structure editors.

Note:

You can also mark up proposed changes without editing the structure. These markups may then be submitted for review and additional proposals before the agreed edits are applied, that is, converted to structure changes and saved in the database. Pending edits and markups cannot be enabled at the same time. Also, pending edits *cannot* be converted to markups.

You can save an edited product structure as a persistent workspace object, route this object for review, and commit the changes after approval. You can also mark up a changed or unchanged product structure with comments, and then save the markups in a persistent object.

Caution:

Users must not edit a structure line that is marked as deleted or cut without first reverting changes or unpredictable results may occur. Similarly, users must not work with the structure in another structure editor application, such as Multi-Structure Manager, until changes are saved or reverted in Structure Manager.

You cannot restructure the product structure by adding levels, removing levels, moving nodes, or splitting occurrences when you are tracking pending changes.

Creating precise and imprecise assemblies

Precise assemblies are fixed structures of specific item revisions. A precise assembly has links (occurrences) to specific item revisions of its components. When a component is modified to a new revision, you manually edit the assembly to remove the old revision of the component and replace it with the new.

Designers may think of a product structure as a *precise* assembly that references particular revisions of components, each revision representing a change to the previous one.

However, when a structure is passed from design to manufacturing, it may be perceived differently. It is a dynamic, *imprecise* product structure that changes over time; the exact components in the product structure at any time depend on the manufacturing plan. Components may be brought into manufacture immediately, when stocks of the old ones run out, or when a particular product is being constructed.

Imprecise assemblies are dynamic and refer to items, not specific item revisions, as their components. Teamcenter applies a revision rule to determine the revision of each component in the product structure when it is loaded.

Creating substitute components

Substitutes are interchangeable with a particular occurrence in an assembly. You may nominate substitutes for manufacturing purposes; for example, if the cheaper or quicker-to-purchase preferred component is unavailable, you may choose to employ a more expensive or harder to obtain substitute.

Note:

The substitute functionality is not intended to address marketing variations.

Any substitute should be approved by the same processes as the primary occurrence. Defining substitutes wherever possible allows production to continue uninterrupted when a primary item is faulty or unavailable because the substitute is already approved for use.

Substitutes are specific to a single occurrence within a given assembly and not for an item. One of the substitutes, the *preferred substitute*, is always displayed in the product structure. The user can change the preferred substitute if necessary. You can define any number of substitutes for a particular line in the product structure. All substitutes of one occurrence share the same occurrence attributes specified for the preferred substitute, for example, find number, quantity, and notes.

Note:

You can only determine if a substitute was actually used for a particular assembly by examining the *as-built* information. You can manage as-built information in Teamcenter with the As-Built Manager application.

Creating substitutes and choosing a preferred substitute require edits to the BVR. Substitutes cannot be configured dynamically with a revision rule.

Adding a substitute modifies the BVR, and you must therefore have write access to the product structure to do this. You can change the preferred substitute during a session (for example, to view the results in NX) but can only save this change if you have write access.

Substitutes for a precise assembly are item revisions, while for an imprecise assembly, they are items whose revision is selected by the revision rule.

Importing and exporting product structure

You can import product structure data that was created in external systems into Teamcenter. When you import files, Teamcenter creates a dataset with named references to a *copy* of the original files located in a Teamcenter volume.

You may decide to import files for several reasons:

- Importing document or image data to attach to a product structure framework.

- Migrating legacy data into Teamcenter.
- Placing product structure data created outside of Teamcenter into the database.

You can also share product structure data with an external system using the PLM XML data exchange format. PLM XML supports the export and import of Teamcenter objects, such as items, datasets, product structures, forms, and folders. It also allows the exchange of system data, such as business rules, organization data, type definitions, lists of values, and saved queries.

You can define transfer mode objects to specify the import or export context by applying closure (traversal) rules, filter rules, and property set rules to the input or output. These rules can be a static set stored in the database or they can be applied only to a specific session. Rules stored in the database can be imported and exported.

Extensible Stylesheet Language Transformation (XSLT) files describe how to transfer the source tree or data structure of an XML document into the result tree for a new XML document. The new document, which may be completely different in structure, can be stored with the transfer mode objects and applied as a post-export or pre-import operation.

Importing a product structure with Excel

You can use Microsoft Excel to import product structure from an external source, for example, a design contractor that does not have Teamcenter. The product structure is defined in an Excel spreadsheet that Teamcenter validates against a predefined control file to ensure all the necessary data is present. If the product structure subsequently changes, you can reimport the same spreadsheet and update the latest working data in Teamcenter. You must have Teamcenter Client for Microsoft Office installed to import or update product structure in this way.

Note:

You can only import imprecise structures in this way. Also, the import process does not configure the structure (for example, by defining revision rules) or set the view, nor does it support absolute occurrences or incremental changes.

Packing and unpacking product structure

Teamcenter employs a technique known as *packing* to group multiple identical components in one level of an assembly. By default, it packs components or assemblies in the same BOM line, if they have the same item revision and find number. For example, a bicycle designer may prefer not to negotiate 50 individual spokes in a wheel assembly if they are identical except for position. If they are packed, the 50 lines are represented by a single line that includes an **x50** property. The packing action only applies to the current session and does not change the product structure in the database.

Teamcenter only allows you to pack components if they satisfy all of the following requirements:

- They have the same item revision.

- They have the same find number.
- None have variant conditions or they all have the same variant condition.

The following restrictions apply to lines in the structure that are packed:

- You may not edit the quantity value.
- If any of the packed lines have occurrence notes, the notes are replaced by the text **Packed Notes**. If none of the packed lines has notes, the notes are left blank. In either case, you cannot edit the notes.
- Item and item revision attributes (for example, part name, weight, or cost) remain visible and you can edit them if you have permission.
- You may still modify the find number. This modification is then applied to each line in the structure, as seen when unpacked.
- If any of the lines has a substitute, the packed line is displayed as a packed substitute.

Grading

Teamcenter can be configured to validate if the design represented by the product structure contains only approved parts. The structure tree then indicates approved parts, unapproved parts, and parts with no data in the system. This process is sometimes called *BOM grading*.

In addition, parts that are approved for one project or locale may not be approved for another project or locale. For example, a part may comply with European standards, but not with U.S. standards. Consequently, you should validate a configured product structure in the appropriate context.

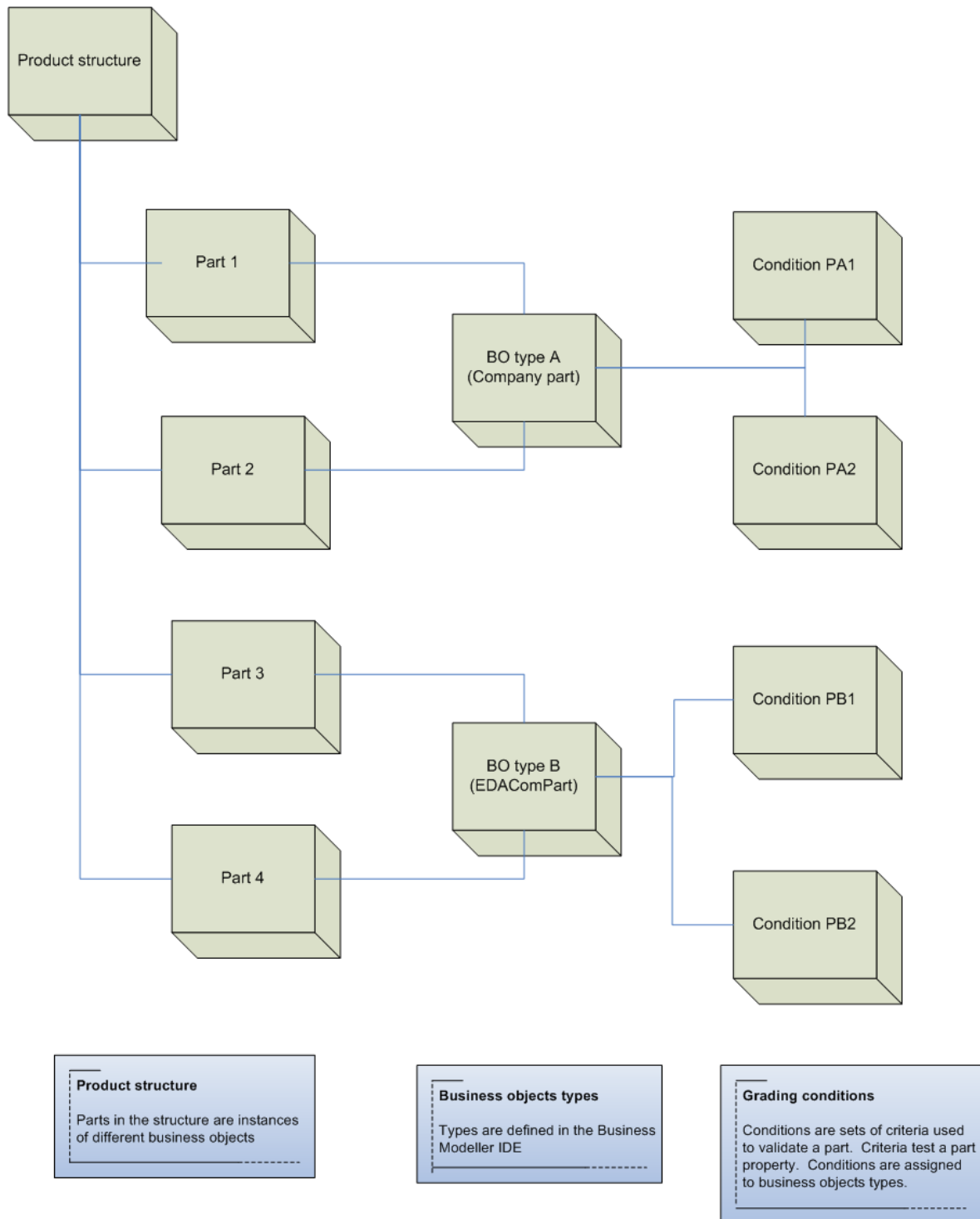
To allow validation or grading:

1. The administrator creates a list of criteria that can be used to validate parts. Each criteria comprises a Business Modeler IDE condition with a list of associated expressions. An expression comprises a business object attribute (for example, **part.obsolete**), an operator, and a value (for example, **false**). For example:

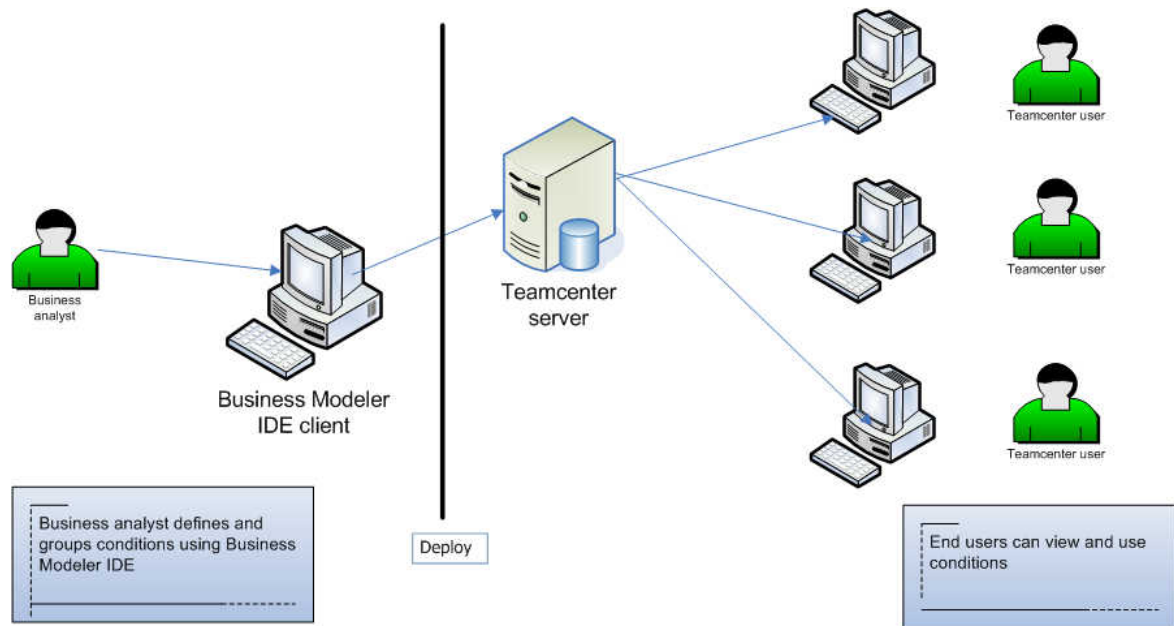
```
Condition1
  (Part.release_status != 'obsolete') AND
  (Part.compliance = pass AND
  part.project_list includes 'ProjectXYZ')
Condition2
  (Part.release_status != 'obsolete') AND
  (Part.price <= '2$') AND
  (part.tolerance <= '5%')
```

You can create a library of basic conditions such as **CheckCompliance** and **CheckReleaseStatus** for use in compound conditions.

- The administrator uses the Business Modeler IDE to associate conditions with parts represented by business object types and subtypes. For example:



The conditions and association rules are then deployed to the Teamcenter database.



3. The administrator groups conditions into context checkers in the Validation Manager application. This allows the product structure to be validated using only relevant conditions. For example, you can create a condition for each compliance standard such as RoHS, JIGA, JIGB, WEEE or REACH, but only validate the product against the compliance standard for the target market. Likewise, certain industries such as military or aerospace may require special validations. Validation Manager uses the **BOMGrading** agent to invoke the checkers.
4. The design engineer validates the configured product structure against the appropriate context checker and reviews the grading report. Validation is also performed against subassemblies, if they exist. Results are saved in the database as a structure context that is associated with the root line of the product structure.
Teamcenter determines the overall pass or fail status of the product structure by assessing the status of each component part. The designer can override pass or fail results for individual parts to achieve an overall pass, depending on your company policy.

Note:

The administrator sets the **CondValidation_NoData** preference to determine whether parts with no data cause an overall pass or fail.

5. The design engineer modifies the product structure and repeats the previous step until optimal grading results are obtained.

Copying data between applications

You can copy and paste product structure (BOM) lines between applications to obtain different views of the same data. For example, you can copy lines in Design Context and paste them into Structure Manager.

If you are working with CATIA data, consider setting the following preferences to the values shown:

- **PortalDesignContextIsInstallationAssemblyMethod** to **transformsComponents**
- **PortalDesignContextFindInstallationTreeWalkIncrement** to **1**
- **PortalDesignContextSupportInstallationAssemblySelection** to **TRUE**

The first two preferences configure Design Context to display the parent assembly responsible for the parent position. That is, it is the lowest node in the tree that still positions the component correctly with respect to the top-level product. For example, if you are working with CATIA V4 data, you only have absolute positioned components and you see the component BOM line itself. If you use true CATIA V5 assemblies, you see the node above which no more 3D transformations occur. You can copy the assembly in the right pane of Design Context and paste them to working assemblies in Structure Manager. This allows you to identify the true product assembly position and any future changes.

Note:

You can only select assemblies in the right pane if the **PortalDesignContextSupportInstallationAssemblySelection** preference is set to **TRUE**. The default value of this preference is **FALSE**.

The **PortalDesignContextIsInstallationAssemblyMethod** preference allows you to specify a **has_bomline_prop** value. Use this value to choose a property when checking for installation assemblies. For example, you can define an occurrence note type called **IA** and establish a convention that a structure line is considered an installation assembly if the **IA** property has a **Yes** value. To use this convention, you must set the following preferences:

- **PortalDesignContextIsInstallationAssemblyMethod** to **has_bomline_prop**
- **PortalDesignContextIsInstallationAssemblyMethod.has_bomline_prop.name** to **IA**
- **PortalDesignContextIsInstallationAssemblyMethod.has_bomline_prop.value** to **Yes**

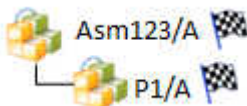
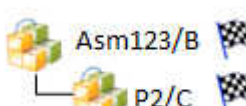
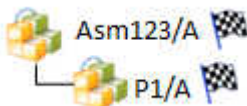
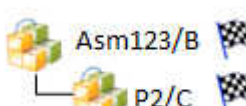
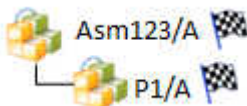
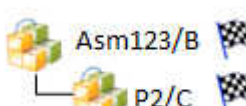
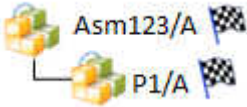
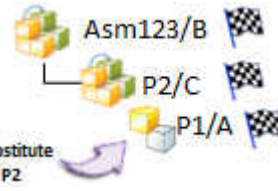
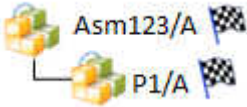
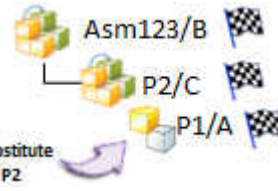
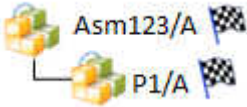
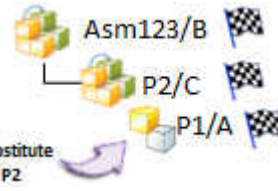
Making mass updates to the structure

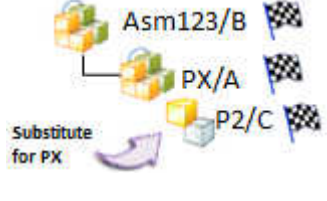
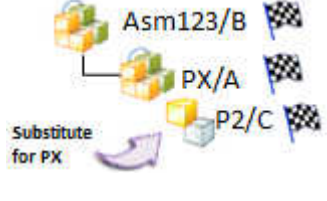
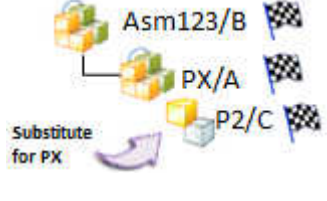
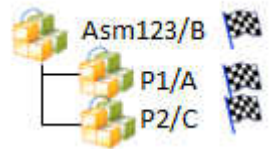
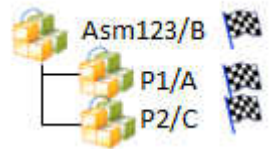
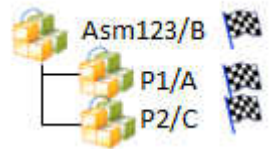
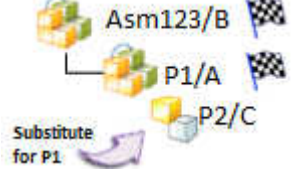
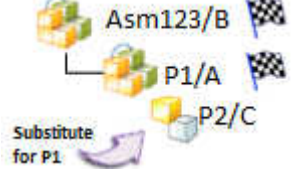
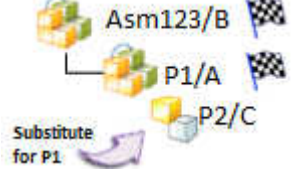
Privileged users can make bulk updates of multiple assemblies with a single action, for example, identifying and automatically replacing every occurrence of an impacted part with the solution item. The update may include multiple types of operations including replace, remove, add, and manual update (revise parts that are released) when making design changes. This automates a manual, time-intensive process that would otherwise take users many hours to complete. Typically, you would replace a part with a preferred substitute or a nonpreferred substitute, although it is not mandatory for the replacing part to have a substitute designation.

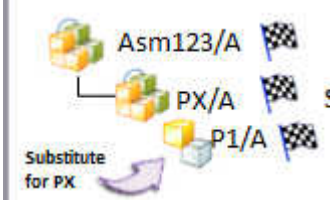
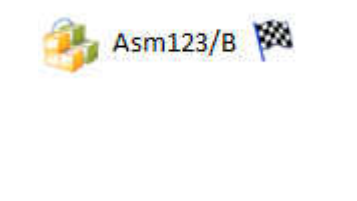
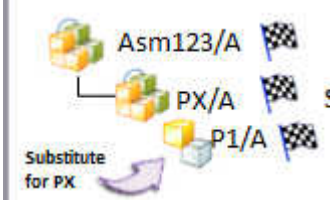
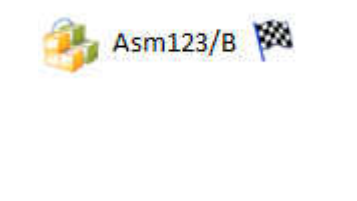
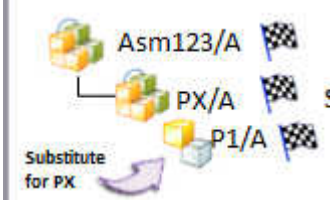
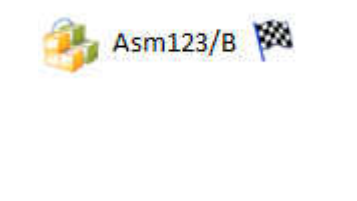
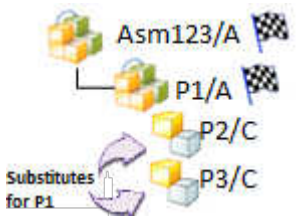
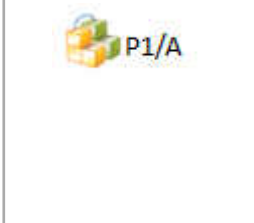
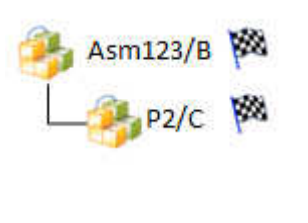
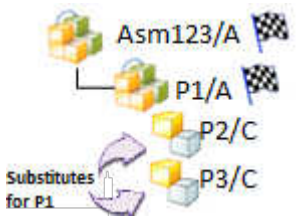
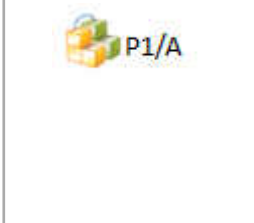
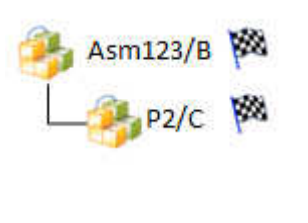
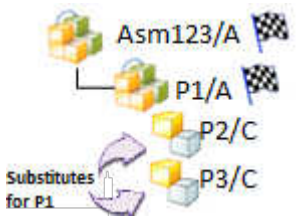
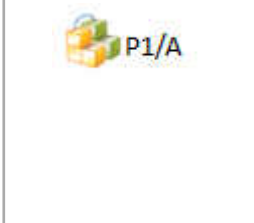
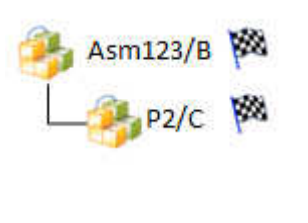
The assemblies may be updated by selecting the item revision of a single part or using a change item to collect affected parts. You then follow one of two processes:

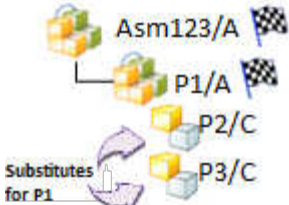
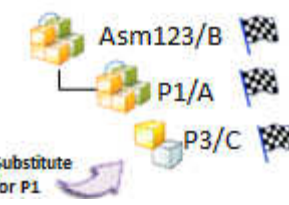
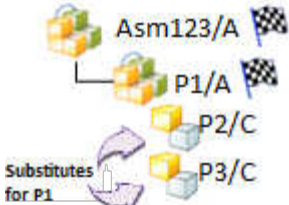
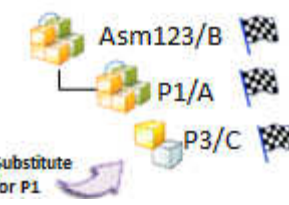
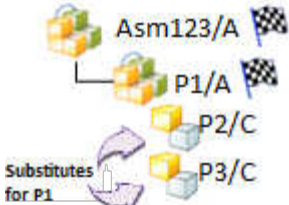
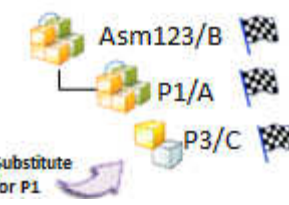
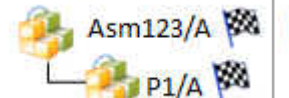
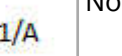
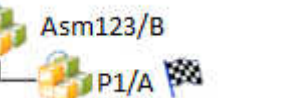
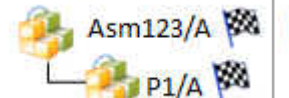
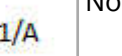
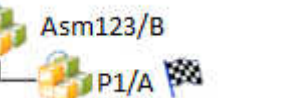
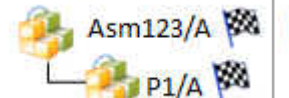
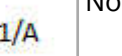
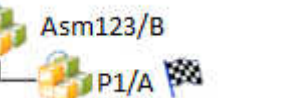
- A single phase manually-initiated process in My Teamcenter. Use of a change item is optional.
- A two phase change request (CR) and change notice (CN) change process in Change Manager. The change processes allow the user to execute, track, and manage all impacted structures when performing a mass replace (add or remove) action. The update is always stored in a change item.

As part of the two-phase process, you also add, modify, and remove markups associated with the change. You cannot work with markups if you use the single phase process.

This operation	Does the following								
Replace Part	Replaces the target part in a structure if the target part has no substitutes or if it is the preferred substitute. • If the target part is an alternate substitute, it is not replaced. • If the target part is a preferred substitute, all the alternate substitutes are retained with the replacement part. For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td></td><td></td></tr></table>	Assembly	Target	Replacement	Result				
Assembly	Target	Replacement	Result						
									
Replace Part and Keep as Substitute	Replaces the target part in a structure if the target part has no substitutes or it is a preferred substitute. If the target part is an alternate substitute, it is not replaced. The target part is added as an alternate substitute after the target part has been replaced. For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td></td><td></td></tr></table>	Assembly	Target	Replacement	Result				
Assembly	Target	Replacement	Result						
									
Replace Substitute	Replaces the target part in a structure if the target part is an alternate substitute. If the target part has no substitutes or the target part is a preferred substitute, it is not replaced.								

This operation	Does the following								
	For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td></td><td></td></tr></table>	Assembly	Target	Replacement	Result				
Assembly	Target	Replacement	Result						
									
Add Part	Adds a part to a structure that contains the target part, regardless of whether or not the target part has substitutes or whether the target part is the preferred or alternate substitute. The part is added to the structure, and you must reposition the component within the structure. For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td></td><td></td></tr></table>	Assembly	Target	Replacement	Result				
Assembly	Target	Replacement	Result						
									
Add Part as Substitute	Adds a part to a structure as an alternate substitute to the target part, regardless of whether or not the target part has substitutes or if the target part is the preferred or alternate substitute. The part is added as an alternate substitute to the target part, and you must reposition the component within the structure. For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td></td><td></td></tr></table>	Assembly	Target	Replacement	Result				
Assembly	Target	Replacement	Result						
									
Remove Part and Substitute	Removes the target part in a structure if the target part has no substitutes or it is a preferred substitute. If the target part is an alternate substitute, it is not removed.								

This operation	Does the following								
	If the target part is a preferred substitute, all the alternate substitutes are removed with the target part. For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td>None</td><td></td></tr></table>	Assembly	Target	Replacement	Result			None	
Assembly	Target	Replacement	Result						
		None							
Remove Part and Keep Substitute	Removes the target part in a structure if the target part has no substitutes or it is a preferred substitute. - If the target part is an alternate substitute, it is not removed. - If the target part is a preferred substitute, the first alternate substitute becomes the preferred substitute when the target part is removed. For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td>None</td><td></td></tr></table>	Assembly	Target	Replacement	Result			None	
Assembly	Target	Replacement	Result						
		None							
Remove Substitute	Removes the target part in a structure if the target part is an alternate substitute. If the target part is not a substitute or is the preferred substitute, it is not removed. For example:								

This operation	Does the following											
	<table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td>None</td><td></td></tr></table>	Assembly	Target	Replacement	Result			None				
Assembly	Target	Replacement	Result									
		None										
Manual Update	Revises structures that contain the target part and does not make any structural changes. It does this regardless of whether or not the target part has substitutes or whether the target part is a preferred or alternate substitute. You must manually make the changes to the revised structures. For example: <table><tr><th>Assembly</th><th>Target</th><th>Replacement</th><th>Result</th></tr><tr><td></td><td></td><td>None</td><td></td></tr></table>				Assembly	Target	Replacement	Result			None	
Assembly	Target	Replacement	Result									
		None										

Using the embedded viewer

Teamcenter provides an embedded viewer that allows you to visualize geometry, make visual comparisons, and analyze clearances and tolerances. A product may be created in NX and, if you manage the product structure in Teamcenter, the viewer allows you to visualize all elements of the structure or only selected elements. In addition, it allows you to create markups during design reviews.

If you have more than one MCAD authoring tool in your environment, you can create industry-standard JT image files in each package and view all of them in the embedded viewer. Likewise, you can create XFATF image files from ECAD designs and visualize them in the viewer.

This viewer is available with several of the applications including Structure Manager and Multi-Structure Manager. It allows you to complete the following actions:

- Generate an outline image of a selected 3D part.
- Translate selected parts relative to the X, Y, and Z axes.
- Rotate selected parts relative to the X, Y, and Z axes.
- Scale selected parts in an exploded view.

- Move or reorientate a group of parts by mapping from one predefined coordinate system to another.
- Align, reset and manipulate parts in an exploded view.
- Analyze clearances and tolerances.
- View 3D cross-sections of parts.
- View and mark up PCB designs.

If you maintain separate part and CAD design objects, you can view the geometry in its correct position in the product structure after publishing the design to the part.

Analyzing clearances

Clearance analysis principles

In a model with n parts, there are approximately $n^2 / 2$ possible clashes between parts. In a car that (when fully configured) comprises 10,000 parts, there are approximately 50 million pairs of parts that may clash. Similarly, a fully configured aircraft comprising 100,000 parts has approximately 5 billion pairs of parts that may theoretically clash. If you were to examine 100 part pairs per second without any intelligent clearance tools, it would take 1.6 years to complete the analysis of the aircraft. In reality, most customers manage clearance data for *unconfigured* vehicles, that is, all variations of a common platform.

You can use the integration between Teamcenter and Teamcenter lifecycle visualization mockup's clearance database to automate many analysis tasks. Geometric (JT) data and optional rules are fed into the clearance calculator that performs the analysis and uploads the results to the clearance database. You can use the stand-alone Teamcenter lifecycle visualization mockup viewer or the embedded viewer in Structure Manager to review clearance issues found. You can click any result pair to examine the issue in more detail. You can also assign issues a priority, status, and owner to part pairs in the results list or in the clearance database.

When clearance violations are identified and enumerated, you can visually analyze part pairs with several techniques:

- Measurements
- Cross sections
- Intersection volumes
- Contact and penetration points
- Bounding boxes of contact penetration points

- Distance lines
- Color coding

You can work with configured or unconfigured product structures. You can also use the valid overlay only option (VOO) option in Design Context to filter out nonbuildable part combinations.

If you work with configured structures, clearance analysis is performed on a limited number of representative models that are already fully configured. No false positives are obtained, but substantial time may be spent managing separate configurations and there is also potential for missed issues.

If you work with configured structures, clearance analysis includes all variants in a single model, for example, **2 door + 4 door** or **v4 + v6**. This technique may result in many false positive clearance results unless variants are effectively managed.

Analyzing clearances

Teamcenter provides an integrated clearance management (ICM) feature that allows you to identify clearance and interference issues, and then examine them in the viewer. You identify a series of target parts and find other relevant data within a given proximity to those parts. You can send any clearance issue to the viewer for visual analysis.

You can initiate clearance analysis in two ways:

- Manually, in real-time analysis mode
You request a clearance analysis as part of your normal working session. Teamcenter retrieves the results from the clearance database and displays them in the third window of Design Context. The results are also stored in a structure context object (SCO) in the database. If you want to retrieve the stored results in the future, you can display the values in the **RDVClearanceAnalysis** form stored in the SCO.
If your system uses cacheless search, you can also perform real-time clearance analysis with Structure Manager.
- In batch (database query) mode
The administrator configures a **cron** job that performs clearance analysis periodically as a background task, typically every night. Clearance issues identified by the background task are stored in the database and you can view them in tabular format in Design Context. As there is a significant delay before you can view the results of a database query analysis, it is not suitable for real-time analysis of changes.

You must have JT files in Teamcenter to represent all the applicable parts. The clearance database accepts any pruned or full product structure provided it is represented by a JT file. Any parts for which JT data is not available are not included in the clearance analysis.

The clearance database allows you to define rules and conditions that determine how changing design decisions impact pairs of parts in your product design. It enables you to establish a part pairs database that defines which specific parts should be checked against each other on a repeated basis.

The rules and conditions allow you to:

- Prevent a part from being analyzed because of its repetition within the model.
- Prevent a subassembly from being analyzed because it is provided by a supplier.
- Focus analysis upon a part or assembly because of its position within the overall model.
- Focus analysis upon parts or assemblies that match a set of conditions that you define.

You can also create interference or clearance zones. Clearance zones allow you to perform analysis based upon the location of parts and assemblies within the model, rather than by attribute information.

Understanding the clearance database

In the clearance database, you define:

- Clearance rules, each of which defines a property (a distance requirement) or a state (exclusion from analysis) for a subset of assembly data.
- Rule conditions that define the scope of data to which the rule applies. You can use any metadata stored with parts or assemblies to define the conditions for a rule. Rules can be applied to a single set of data or to two sets of data.

You define rules and conditions in comma-separated variable (CSV) files. These files can be automatically generated by a batch process, if appropriate.

You can define *patterns* to refine the rules. For example, patterns can:

- Flag all contacts and penetrations.
- Exclude contacts from analysis. Contacts may be defined as values between 0 mm and 0.01 mm.
- Exclude particular parts from analysis, for example, rivets, bolts, welds, and rubber seals.
- Ignore any interferences within a particular subassembly, for example, a subassembly provided by a supplier.
- Require a minimum distance between two functional subsystems. For example, you can specify that no fuel system part may be less than 50 mm from any exhaust system part.

Best practices for managing clearance analysis

Perform clearance analysis on configured or unconfigured structures, using one of the following processes:

- Option 1 – Validate the configured structure and use clearance database rules to manage variants.
 1. Define the configured product structure in Teamcenter.
 2. Export the entire configured product structure to the clearance database.
 3. Create clearance database rules to manage the product variants. You can also manage variant data in other systems.
 4. Perform a clearance calculation on the configured product structure.
 5. Use the clearance database rules engine to filter out any nonbuildable part combinations.

The process gives automated, repeatable analysis. However, significant effort may be required to author and manage the variant rules. Also, there may be redundancies with other variant systems.

- Option 2 – Validate configured product structure and use Teamcenter to extract representative candidate models.
 1. Define the configured product structure in Teamcenter.
 2. Define all product variants within Teamcenter.
 3. Export multiple representative configured product structures to the clearance database used on the defined variability, as required.
 4. Perform clearance analysis on the configured structures.

Using this process, definition of the variability of the product is maintained in a single location. Also, there should be no false positive clearance analysis results because you are managing variants only in Teamcenter. However, it is possible to miss issues, because you are using a limited number of configurations. Also, a significant maintenance overhead and processing time required to manage and analyze multiple configurations.

- Option 3 – Validate variant-aware unconfigured product structures.
 1. Define the unconfigured product in Teamcenter.
 2. Define the product variants in Teamcenter.
 3. Export the unconfigured structure to the clearance database.
 4. Perform clearance analysis on the unconfigured structure, querying for buildable combinations as defined in the Teamcenter valid overlays only (VOO) option.

In this process, all buildable combinations are considered. There is no wasted calculation time because nonbuildable combinations are ignored. Also, a single system (Teamcenter) manages all product and variant information.

3. Comparing product structures

Comparing product structures	3-1
Lowest level comparison mode	3-1
Using the lowest level comparison mode	3-1
Example of lowest level comparison mode	3-2
Multilevel comparison mode	3-3
Comparing product structures in multilevel mode	3-3
Example of multilevel mode comparison	3-3
Single-level comparison mode	3-4
Comparing product structures in single-level mode	3-4
Example of single-level comparison mode	3-5
Using supersedures	3-5
Making graphical comparisons	3-6

3. Comparing product structures

Comparing product structures

When working with product structures, you may want to compare two structures for several reasons including:

- Identifying engineering component changes between different configurations of the same product.
- Testing for consistency between multiple views of the same product.
- Identifying differences between variants of the same product.

According to your needs, you can compare two product structures (BOMs) in one of three modes—lowest level mode, multilevel mode, and single-level mode. Details of each mode follow.

In addition to component changes, the comparison mechanism identifies any components that are moved (transforms). Replacements and substitutions are also considered in the comparison.

Teamcenter compares the *displayed* product structures (BOMs); it compares each level that is visible. Consequently, you can compare different views or configurations by expanding the structure to the desired level. Also, if you collapse assemblies in the product structure, the comparison excludes the content of those assemblies. You can use this feature to limit the scope of a comparison.

To compare different variants, hide unconfigured variants before performing the comparison.

Note:

Display options override any configuration rules. Therefore, to find the differences between two configurations, you must hide unconfigured items.

In addition to identifying differences between the physical parts in the structure, the comparison mechanism identifies differences in features. For example, if the product structures contain item elements (sometimes called general design elements or GDEs) representing features or ports, the comparison also identifies differences between item elements associated with the same item or item revision. Similarly, it identifies differences in ports, signals, connections or allocations in electromechanical structures or their relationships.

Lowest level comparison mode

Using the lowest level comparison mode

Use lowest level mode to compare only the lowest level items in two product structures, while ignoring the intermediate structure. Teamcenter calculates the quantity of each item in the product structures

before comparing them. It ignores find numbers and reports only additions, total quantity changes, and revision changes as differences.

You can use this mode to compare two views, for example, where a design view may have a different structure to a manufacturing view, while containing the same set of piece parts.

Example of lowest level comparison mode

The following figure shows how you can use a lowest-level mode comparison to identify differences in piece parts between two views of the same structure. You are comparing the design view of a car with the manufacturing view. Both views are configured by applying the same configuration and variant rules.

	Seq No	Changes	Qty Chg	Rev Chg
A01000/B-Car (Design)	10	A01000/B		
A0200/A-Body Assembly (Design)	10			
A020/B-Dashboard Assembly (Design)	10			
P020/A-Stereo Radio	10			
P015/A-Screw x2	20	Qty	3->6	
P050/A-Glove Pocket	30			
A095/A-Electric Aerial	20			
A015/A-Screw x1	30	Qty	3->6	
A1600/A-1600 Engine	20			

	Seq No	Changes	Qty Chg	Rev Chg
A01000/B-Car (Manufacturing)	10	A01000/B		
A020/A-Stereo Radio	10			
A015/A-Screw x4	20	Qty	5->8	
A050/A-Glove Pocket	30			
A095/A-Electric Aerial	40			
A015/A-Screw x1	50	Qty	5->8	
J001/A-Jig	60	Added		
A1600/A-1600 Engine	70			

Comparing product structures in lowest level mode

The differences between the two views are:

- The wrong number of screws is placed in the manufacturing view.
- A jig is added to the manufacturing view.

Multilevel comparison mode

Comparing product structures in multilevel mode

Use this mode to identify differences between two configurations of a product, for example, variants. In multilevel mode, if the comparison process matches two subassemblies in the two configurations of the product structures, it proceeds to compare the two subassemblies. If the comparison process does not match the two subassemblies, it stops and does not compare the product structures of the subassemblies. It continues down the product structure until it reaches the bottom or there are no matching subassemblies remaining.

Two subassemblies match if they are the same item, and have the same revision, quantity, and (optionally) find number.

This mode identifies the following changes:

- Additions to either product structure
- Quantity changes
- Revision changes

This mode identifies changes that occur at the same find number. Find number changes are considered as producing a new item and are reported as additions.

Example of multilevel mode comparison

The following figure shows how you can use a multilevel comparison to find the point in a structure where two designs diverge. This example compares revision A of a car with revision B of the same car.

	Seq No	Changes	Qty Chg	Rev Chg
A01000/B-Car (Design)	10	A01000/B		
— A0200/A-Body Assembly (Design)	10			
— A020/B-Dashboard Assembly (Design)	10	Rev		A->B
— P030/A-Blank Plate	10			
— P015/A-Screw x4	20			
— P050/A-Glove Pocket	30			
— E1200/A-1200 Engine	20	Added		

	Seq No	Changes	Qty Chg	Rev Chg
A01000/B-Car (Design)	10	A01000/B		
— A0200/A-Body Assembly (Design)	10			
— A020/B-Dashboard Assembly (Design)	10	Rev		B->A
— P020/A-Stereo Radio	10			
— P015/A-Screw x2	20			
— P050/A-Glove Pocket	30			
— P095/A-Electric Aerial	20	Added		
— P015/A-Screw x1	30	Added		
— A1600/A-1600 Engine	20	Added		

Comparing product structures in multilevel mode

The differences between revision A and revision B are highlighted and are as follows:

- Replace a 1200 engine with a 1600 engine, by applying a variant option.
- Configure a newer design of dashboard, complete with stereo radio, by applying different configuration rules to the two product structures.
- Add exterior items (aerial and a screw) to support the new radio, by applying a variant option.

Single-level comparison mode

Comparing product structures in single-level mode

In single-level mode, you compare only the first level of the product structures. It allows you to identify component changes between the two assemblies.

Two components or assemblies (one in each assembly) match if they are the same item, with the same revision, quantity, and (optionally) find number. Multiple occurrences of the same item with the same find number are rolled up; one occurrence with a quantity of **2** is the same as two occurrences with a quantity of **1** in this case.

This mode identifies the following changes:

- Additions to either product structure
- Quantity changes
- Revision changes

This mode identifies changes that occur at the same find number. Find number changes are considered as producing a new item and are reported as additions.

Example of single-level comparison mode

The following figure shows how a single-level comparison identifies component changes in a product structure. This example compares revision A of a dashboard assembly with revision B of the same assembly.

	Seq No	Changes	Qty Chg	Rev Chg
A01000/B-Car (Design)	10			
└ A0200/A-Body Assembly (Design)	10			
└┬ A020/B-Dashboard Assembly (Design)	10	A020/B		
└┬┬ P030/A-Blank Plate	10	Added		
└┬┬┬ P015/A-Screw x4	20	Qty	4->2	
└┬┬ P050/A-Glove Pocket	30			
└ E1200/A-1200 Engine	20			

	Seq No	Changes	Qty Chg	Rev Chg
A01000/B-Car (Design)	10			
└ A0200/A-Body Assembly (Design)	10			
└┬ A020/B-Dashboard Assembly (Design)	10	A020/A		
└┬┬ P020/A-Stereo Radio	10	Added		
└┬┬┬ P015/A-Screw x2	20	Qty	2->4	
└┬┬ P050/A-Glove Pocket	30			
└┬ P095/A-Electric Aerial	20			
└┬ P015/A-Screw x1	30			
└ E1600/A-1600 Engine	20			

Comparing product structures in single-level mode

The differences between revision A and revision B are highlighted; the blanking plate and its four retaining screws are replaced with a stereo radio and two retaining screws.

Using supersedures

When evaluating an engineering change, it can be useful to know the genealogy or replacement history of a part. A *supersedure* defines the part or parts that replaced a component. Create supersedures when

defining an engineering change so that the genealogy can later be displayed. This feature is useful for evaluating the impact of engineering changes on parts that may still be in service for products with long lifetimes, for example, in aerospace environments. (Associated engineering changes may be required for these earlier, but still active, parts.)

Note:

You can define supersedures with revision changes and not with incremental changes.

Components can be added to and removed from a product structure over its life cycle. Some of these additions and replacements can be related together as a replacement. For example, two *added* parts can replace the form and functionality of one *canceled* component, making a replacement. The parent assemblies of the components must be the **Affected** and **Problem Items** of an engineering change created in the Change Manager application.

You can define a supersedure to record this replacement history. You can then view a graphical representation of such replacement, as a supersedure history, starting from a selected occurrence of a part and continuing to the following superseded or preceding part. This allows you to compare the loaded structure before and after such changes.

Making graphical comparisons

You can compare two structures or two revisions of the same structure in the viewer; Teamcenter highlights components that are present in one structure but not the other in contrasting colors. This capability allows designers to isolate changed parts from common parts and visualize the differences. You can also compare different revisions of piece parts and highlight geometry changes. Visualization (JT) files must be available for the piece parts or assemblies to compare.

Comparisons identify:

- Addition and removal of components.
- Positional changes.
- Geometry changes.

4. Finding items in the product structure

Searching for items in the product structure	4-1
Where-used searches	4-1
Creating where-used searches	4-1
Report types	4-2

4. Finding items in the product structure

Searching for items in the product structure

You can search the product structure to find components in the displayed and expanded structure. This capability is useful when you are not familiar with the structure and are unsure how to navigate to a particular component. You can construct a search expression from any combination of the BOM line properties. You can start the search at the top-level item in the structure or you can restrict the search to a particular assembly in which to look for the component. Once you locate the component, you can make a where-used search to identify every instance of it in the product structure.

In addition to these nonspatial searches, you can search for components located in a particular area or section of the product. If your site maintains spatial data, you can utilize it with cacheless or appearance searches.

Where-used searches

Creating where-used searches

When you assess the impact of engineering change, you must find all the assemblies that contain the changed component to ensure that alterations in one assembly will not affect others. You can run a *where-used* search to find this information. Example of situations in which you may use where-used searches include:

- Finding all assembly items that use a *specified* revision of a part.
- Finding all assemblies that use a *configured* revision of an item.
You can search for assembly items that use a particular revision of a part. You may not know or be concerned about the actual revision, but want to search for items that meet a certain status or effectivity configuration (or both).
- Finding specific assemblies for a specific revision of a part.
You can find specific revisions of assembly items that use specific revisions of the part. Typically, you might search for **Latest Issued** or **Working** revisions.
- Finding those products that use a specified or all revisions.
You can search from the top of the hierarchy to identify all company's products that use a part.

The revision rule setting also affects the results of a where-used search and you can select one of the following:

- All revisions
This reports all item revisions that have an occurrence of the source item revision. Teamcenter displays all combinations of usage that could *possibly* occur. Given a set of revision rules used in practice, not all of these paths may be realized.

- Only the configured revision
You select a revision rule to apply. The search result is filtered to include those revisions that are configured by the selected revision rule. This filter is applied at each level, up to the top level.

Note:

Implementation of a full form, fit, and function definition of each item limits the need for a where-used search. If the form, fit, and function are fully defined, a part may be used anywhere requiring that form, fit, and function; its evolving revisions conform to that form, fit and function and there is no need to consult its usages. The effects of interchangeable changes to the part are isolated from the assemblies that use it.

Report types

A where-used search can produce one of the following reports:

- **Precise Only**
Reports all item revisions having precise occurrences of the selected item revision (the changed component).
- **All Revisions**
Reports any item revision that has a precise or imprecise occurrence of the selected item revision (the changed component).

Note:

This report returns all *possible* combinations of usage. If you apply a set of revision rules used in practice, not all paths may be realized. If you only require combinations that may be realized by the current revision rules, use the configured item revisions report described next.

- **Configured Revisions**
Searches only a configured item revision, as configured by the applicable revision rules. All configured item revisions having precise or imprecise occurrences of the specified item revision (the changed component) are reported at each level, up to the top level. This report also includes any intermediate level, nonconfigured revisions that are referenced by precise occurrences, if the top link of the chain of precise links is configured.

Note:

If you select an item (changed component) as the subject of the search, the **Precise Only** and **All Revisions** reports show the results for each revision of the item.

You can save the report output in a folder or display it in a dialog box; the contents of which you can print or write to a file.

Part II: Configuration

Configuring product structure with revisions	5-1
Configuring product structure with effectivity	5-1
Managing incremental changes	5-1
Configuring product structure with variants	5-1
Configuring structure with branch definitions	5-1

5. Configuring product structure with revisions

Revision configuration principles	5-1
Introduction to revision control	5-1
Revision control principles	5-1
Understanding revisioning terminology	5-2
Defining revision rules	5-3
Revision rule criteria	5-3
Working entry	5-4
Status entry	5-4
Override entry	5-5
Precise entry	5-5
Latest entry	5-6
Date entry	5-6
Unit number entry	5-6
Grouping entries	5-7
Managing user privileges	5-7
Configuring precise and imprecise structures	5-7
What are precise and imprecise structures?	5-7
Changing from precise to imprecise	5-9
Applying revision rules	5-10
Creating baselines	5-10
Creating snapshots	5-11
Creating intermediate data captures (IDCs)	5-11
Best practices for revision configuration	5-13

5. Configuring product structure with revisions

Revision configuration principles

To use revision configuration, you must first implement the appropriate effectivity and release schemes.

Introduction to revision control

Revision control principles

Teamcenter allows you to manage changes to the product structure by revisioning of items. Changes that may necessitate a new item revision include addition or removal of (product structure) components, relationships to other items, and changes to encapsulated data (datasets) as well as to the metadata that describes the item.

You can set the configuration of the structure for a specific time by applying the appropriate *revision rule*. You cannot view multiple configurations at the same time; if you want to see another configuration, you must apply a different revision rule.

A revision rule consists of any number of rule entries, each defining a parameter Teamcenter uses when evaluating the revision to configured.

Teamcenter evaluates the rule entries in precedence order, and thus the order in which you place them in the revision rule significantly affects the result. A typical revision rule might have three rule entries:

- **Latest Working**
- **Latest (Any Status, Configured by Release Date)**
- **Date Today**

If an item has the following revisions, this rule configures revision C:

- A – Production status (Release Date = 1 Jan 07)
- B – Pre-Released status (Release Date = 1 Feb 07)
- C – Working

However, if the rule has the following entries, it configures revision B:

- **Latest (Any Status, Configured by Release Date)**

- **Latest Working**
- **Date Today**

If today's date is 10 January 2007, revision A is configured.

You can create revision rules that allow a structure to be configured in any of the following ways:

- Configure only those parts with a revision at a particular status.
- Define a status hierarchy, for example, **Production, else Pre-Production, else Prototype**.
- Configure the structure at a specific historical date.
- Configure the structure at a specific effective point—date or unit.
- Configure an ad hoc set of revisions, chosen by a user and placed in a folder for that purpose.
- Configure the precise occurrences stored in any precise assemblies (BVRs).
- Configure a snapshot of a configuration.
- Configure a baseline created for a structure.
- Configure working revisions according to the owning user or group.
- Configure according to the date the revision was created.
- Configure according to the alphabetic order of the revision ID.

In addition to revision control, Teamcenter provides several alternative methods of managing changes, including occurrence effectivity and incremental changes.

Understanding revisioning terminology

Term	Meaning
End item	A product, system, or module with respect to which you can configure the structure by effectivity. For example, you can configure the structure of unit number 110 in product X400 , where X400 is the end item.
Imprecise assembly	A single-level assembly that has items (<i>not</i> item revisions) as the components. Teamcenter determines the applicable revision from the revision rule settings.

Term	Meaning
Override list	A mechanism that allows a user to override the revision that would normally be loaded by the revision rule. The user places item revisions in a workspace folder, and the revision rule is overridden by the rule specified in the override list.
Precise assembly	A single-level assembly that has specific item revisions as the components. When Teamcenter applies the revision rule, the precisely specified item revision is configured by a precise entry in a revision rule.
Release status	An object assigned to an item revision after it is successfully released. Item revisions can be configured according to their status. The status may optionally contain effectivity data for revision configuration (<i>not</i> occurrence effectivity configuration).
Revision rule	The parameters set by a user that determine which revision of an item at a particular time. You can also save a revision rule as a workspace object.
Rule entry	A revision rule comprises an ordered list of rule entries. Each type of rule entry is concerned with a particular type of configuration.
Snapshot	You can save a configured product structure as a <i>snapshot</i> . The snapshot folder contains all revisions of the structure, and you can use it to redisplay a saved structure at any time.
Baseline	A dataset with a copy of the currently configured structure attached to it. Baselining configures a completely released structure and thereby guarantees that the models are always the same as when the baseline was created.
Working revision	An unreleased version of the structure. Any user with write privileges can freely change this revision. Teamcenter maintains no record of intermediate states of a working revision.

Defining revision rules

Revision rule criteria

Revision configuration allows you to select the appropriate revision of components in the product structure. A *revision rule* sets the criteria for selecting the revision, as follows:

- Selects working revisions and, optionally, specifies the owning user or group.
- Selects released revisions by specific status, according to a status hierarchy or latest status (by date released).
- Optionally, specifies the date or unit number effectivity of the revisions.

- Selects revisions in a specified override folder.
- Selects the latest revisions according to the revision ID, by alphanumeric/numeric order or creation date. This selection is independent of whether revisions are working or released.

Each of the above criteria is defined by a revision rule entry. A revision rule consists of any number of rule entries, each of which attempts to select a revision according to the associated criteria. For example, it may define what status the revision should have, and which user or group owns the revision. Rule entries are evaluated in precedence order until a revision is successfully configured. Some entries, (for example, **Status**) can be included more than once to define a hierarchy. For example:

```
Working (Owning Group = Project Y)
Has Status (Production, Effective Date)
Has Status (Pre-Production, Effective Date)
```

The order of the rule entries can be modified to change the precedence used when evaluating the revision rule. Certain rule entries can also be grouped so they are evaluated with equal precedence.

Working entry

Use a *working entry* to select working item revisions; that is, those items revisions without any release status. By default, Teamcenter selects the latest working revision of the item according to the date it was created. You can select a more specific revision with one of the following settings:

- **Owning user**
If you specify an owning user within a working entry, Teamcenter configures the latest revision owned by the specified user, if there is such a revision. You can also set the owning user to **Current**, and Teamcenter configures the latest revision owned by the current user.
- **Owning group**
If you specify an owning group within a working entry, Teamcenter configures the latest revision owned by the specified group, if there is such a revision. You can also set the owning group to **Current**, and Teamcenter configures the latest revision owned by the current user's group.

Note:

There may be more than one working entry within a revision rule. For example, a rule may configure the current user's working revision and, if none is found, configure the current group's working revision instead. If a user changes group, it is necessary to reapply the revision rule to configure the appropriate revisions for the new group.

Status entry

Use a status entry to select item revisions that are released with a particular status. The following settings are available for status entries:

- Any release status

Teamcenter configures the latest item revision with a released status, regardless of the actual status.

- **Selected status**
Teamcenter configures the latest item revision with a selected status type. This setting allows you to configure a structure that contains only item revisions with a specified status.
- **Released date**
Teamcenter selects the latest item revision according to the date the revision was released (that is, the date the particular status was added).
- **Effective date**
Teamcenter selects the latest item revision according to effectivity dates defined on the release status.
- **Effective unit number**
Teamcenter selects the latest item revision according to unit numbers defined on the release status.

Override entry

An override entry allows particular item revisions to override item revisions that are normally selected by other criteria. You can copy the overriding item revisions into a folder that you can then reference from the override entry.

Note:

This folder may contain other nested subfolders of item revisions. If any of the subfolders contain more than one revisions of the same item, Teamcenter takes the last revision it encounters. It navigates higher level folders before lower level folders. Typically, you would not put two instances of the same items with different revisions in a particular folder.

Precise entry

You can use a precise entry to select the precisely specified item revisions in a precise product structure. The entry has no effect on imprecise product structures.

Note:

If you place any other entry above the precise entry in a revision rule, it overrides the precise references.

If the revision rule contains only a precise entry, you cannot use it to configure any type of precise structure.

Latest entry

You can specify a *latest* entry to select the latest item revisions regardless of whether they are released. For this entry, Teamcenter does not differentiate between working revisions and revisions with status. The following settings are available for latest entries:

- Creation date
Teamcenter selects the latest item revision according to the date the revision was created.
- Alphanumeric revision ID
Teamcenter selects the latest item revision in alphanumeric order by revision ID. It selects any numeric revision IDs in alphanumeric order by their first character, for example, 1, 10, 2, 21, and so on.
- Numeric revision ID
Teamcenter selects the latest item revision in numeric order by revision ID. It does not configure revisions with non-numeric IDs.

Date entry

Use a date entry to specify a date to configure the structure against. You can only use this entry type with other entries. These types of entry find the latest entry before the specified date:

- Status entry – released date or effective date.
- Latest entry – creation date.

You can set the date in a date entry to **Today** and Teamcenter evaluates the configuration criteria against the current date and time.

You cannot configure working revisions against a date in the past. Teamcenter does not maintain information about the revisions that were in a working state at a particular time in the past.

Note:

If no date entry is present in a revision rule, Teamcenter evaluates the date by default to today's date.

Unit number entry

A unit number entry specifies a unit number to match when configuring item revisions with status using unit number effectivity. You can only use this entry type with other entries. If no unit number entry is present in a revision rule, Teamcenter configures all status entries that configure by effective unit number.

Grouping entries

You can group status entries and working entries for equal precedence. In the case of status entries, you can group two or more different statuses with equal precedence. In the case of working entries, you can group different owners with equal precedence. You can also group entries according to item type.

Managing user privileges

Privileged users have full access to create, modify, and delete all revision rules, while unprivileged users have limited access.

Use access rules to manage user privileges to create, modify, or delete a revision rule. Standard rules for managing the creation of a revision rule provide the following behavior:

- All users can create revision rules.
- Rules created by **infodba** or a member of the **dba** group are visible to all users. Any user may apply these rules to configure the structure.
- Rules created by users who are not **infodba** or a member of the **dba** group are visible only to the creator, **infodba**, and members of the **dba** group. These rules cannot be applied or viewed by members of other groups.

Standard rules for managing the modification or deletion of a revision rule provide the following behavior:

- If a revision rule is created by **infodba** or a member of the **dba** group, users who are not **infodba** or a member of the **dba** group cannot modify or delete it. Only **infodba** or a member of the **dba** can modify or delete it.
- If a revision rule is created by a user who is not **infodba** or a member of the **dba** group, it can be modified or deleted only by **infodba**, a member of the **dba** group, and the specific user who created the rule. Other users cannot modify or delete the rule.

Configuring precise and imprecise structures

What are precise and imprecise structures?

You can create two kinds of assembly or BOM view revisions:

- **Imprecise assemblies**
Imprecise assemblies are dynamic structures that contain links to (*occurrences of*) the items of its components; it does *not* contain links to item revisions.
Imprecise assemblies allow engineers to view a product structure that is configured with only those item revision types they want to view, for example, with revisions that are released for production or

including only the latest working revision of each component. Each user is viewing the same underlying product structure, but is configuring the view to suit their particular needs.

An imprecise product structure is automatically reconfigured when any user releases a part, creates a new part, or performs any other action that affects the view. Therefore, the user need not make a copy of the product structure and manually update it each time.

- Precise assemblies

Precise assemblies are fixed structures of specific item revisions. A precise assembly contains links to (occurrences of) item revisions of its components; it does *not* contain links to items.

When a user modifies any of these components to a new revision, the assembly must be manually updated by removing the old revision of the component and adding the new revision.

You can dynamically configure precise assemblies (that is, treat them as *imprecise*) by applying a revision rule that does not contain a **Precise** rule entry or has the entry at a lower precedence. You can configure these precise references with a revision rule containing a **Precise** rule entry.

Note:

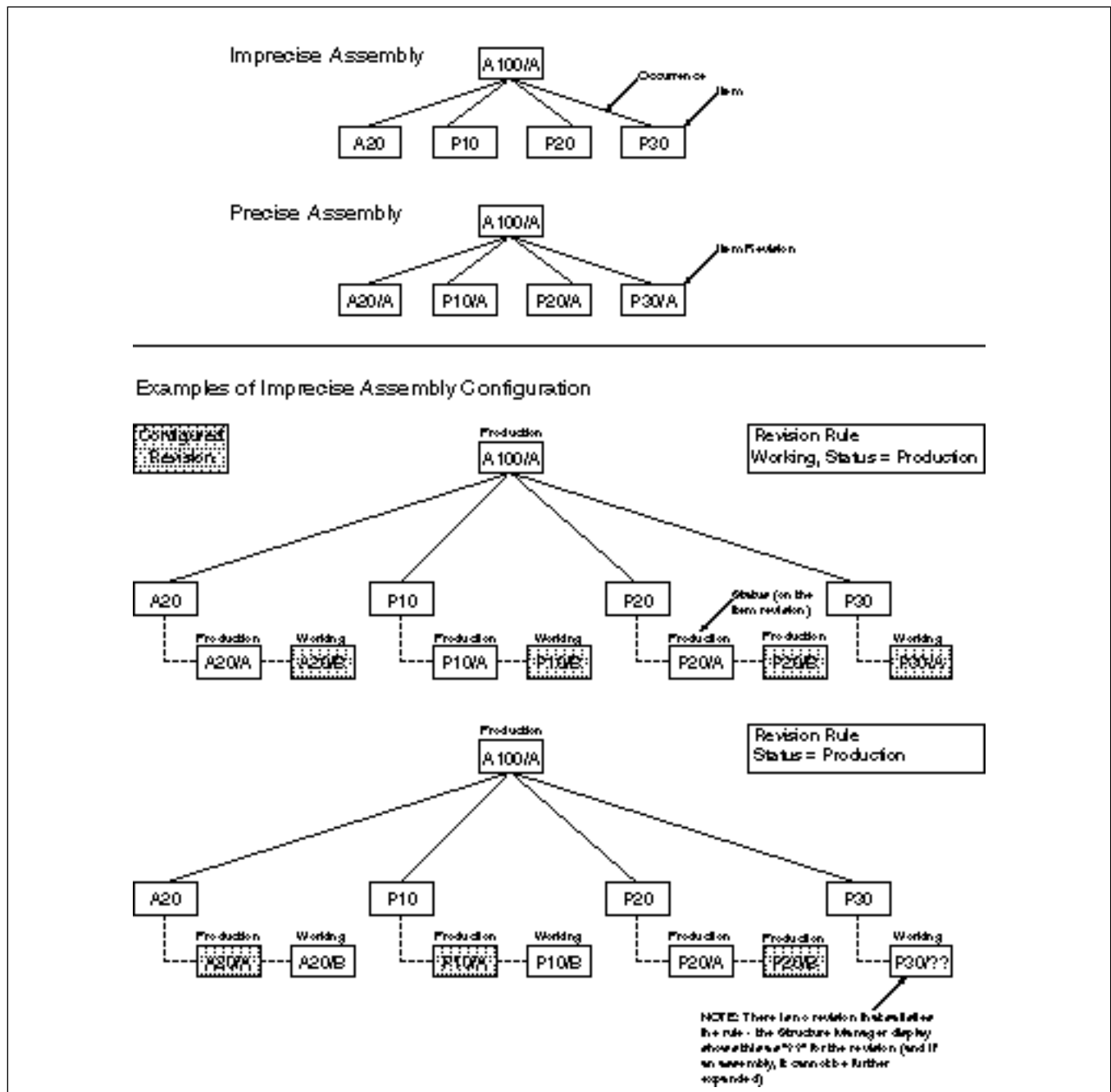
Precise links do not cause additional memory or performance issues relative to imprecise links.

Precise assemblies are useful in situations where you want to control the configuration carefully, for example, during the product design phase or in aerospace manufacturing environments. When the parent assemblies are released and consequently can no longer be modified, any change can have a significant impact on the revisions of related assemblies. If you incorporate a new revision of a component, every affected parent assembly to the top of the product structure must also be revised. However, you can use incremental changes to minimize this effect..

Note:

Precise configurations can result in different revisions of the same item being present in a structure. Some CAD systems, including NX, do not allow more than one revision of a part to be loaded. When this situation occurs, the revision that is loaded is unpredictable.

The following figure shows an example of an imprecise assembly configuration.



Example of imprecise assembly configuration

Changing from precise to imprecise

You can easily change an assembly from precise to imprecise or back again, if it has not been released (releasing the assembly is a change to the structure). You may want to do this when the assembly reaches a different stage in the product life cycle, for example, when it is released from engineering to manufacturing. These considerations depend on your business process, and whether it is necessary to record precise references at any stage.

Applying revision rules

You can use revision rules to configure the product structure for a particular situation. Each revision rule selects the appropriate revision of all the components in the structure for a particular situation. For example, a design engineer may want to see the components specified by the **Latest Working** revision rule, whereas a manufacturing engineer may require the components specified by the **Latest Released** revision rule.

Each revision rule sets the criteria for a single situation and a typical revision rule may contain entries that do one or more of the following:

- Select working revisions, and optionally specify the owning user or group.
- Select released revisions by specific status, according to a status hierarchy. Alternatively, it may select revisions by **Latest** status, according to release date.
- Select the latest revisions according to the revision ID, by alphanumeric/numeric order or creation date. This selection is independent of whether revisions are working or released.
- Select revisions from a specified override folder.
- Optionally, specify the effectivity by date or unit number of the revisions.

Each revision rule may contain one or more rule entries; each entry attempts to select a revision according to the criteria specified in it. For example, an entry may specify the status the revision should have and a user or group that owns the revision.

You can save a revision rule in Structure Manager as a persistent workspace object, then use it in the Multi-Structure Manager application or the NX client. You can also send the object to another user.

Teamcenter evaluates rule entries in precedence until it successfully configures a revision. You may include some entries more than once to define a hierarchy.

You can change the order of the rule entries to alter the precedence of evaluating the entries in a revision rule. You can also group certain rule entries so they are evaluated with equal precedence.

When a user opens a product structure, Teamcenter always enforces a revision rule, typically the user's default revision rule.

Creating baselines

During the development of a product design, you may want to share your working design with other users. To do this, you can create a *baseline* of the work-in-progress (WIP) design. When you request a baseline, Teamcenter creates a new dataset and attaches a copy of the currently configured structure to it. It creates a new revision for each unreleased revision in the structure and releases it with a predefined status, for example, **Baseline**. It then applies a baseline revision rule to configure the base configuration

(for example, **Production** status item revisions); it also includes any new revisions created by the baseline. This method configures a completely released structure and thereby guarantees that the models are the same as when the baseline was created. This approach may be expensive as many new revisions might be created and (with them) copies of the associated data and CAD models.

Optionally, you can implement *smart copy* of baselines by setting the **ITEM_smart_baseline** preference. In this mode, only item revisions that have changed since the last baseline are copied into the new baseline and unchanged item revisions are referenced. Use the smart copy mode if you create large numbers of baselines, as each baseline occupies a significant amount of storage space.

Creating snapshots

A *snapshot* is a workspace folder that stores a reference to all the item revisions contained in a configured structure. You can use a snapshot to save and later redisplay the product structure in the same configuration with the relevant revision rules applied. A snapshot is sufficient to exactly restore those revisions that were configured when the snapshot was taken. However, any revisions that were not released are subject to change, including the associated data and CAD models.

When you request a snapshot, Teamcenter creates a new folder and adds a reference to each item revision in the structure to the folder. This action is analogous to the creation of an override folder, except that its creation and population is automated.

A snapshot is not maintained in its original state because it contains item revisions, not any occurrence information or other data under the item revision. The CAD design the snapshot contains may be modified and the original version no longer appears in the snapshot. If you want an accurate record of the configuration of the structure, you should create a *baseline*.

When you create a snapshot of a product structure that contains precise assemblies, use a revision rule that configures precise references first. If you do not, the snapshot does not match the saved configuration when you reopen it.

Note:

To keep track of snapshots and the structures they refer to, give them names and descriptions that indicate their content. Consider recording the name of the revision rule you used to construct the snapshot in the description field.

Do not confuse snapshots with *product views*, which are saved versions of the data in the assembly viewer in a particular session, including the current items, zoom factor, rotation angle and pan displacements.

Creating intermediate data captures (IDCs)

You can capture the state of any structure or part of a structure for subsequent retrieval and viewing. This capture data does not represent the final released state of the structure, so it is referred to as an *intermediate data capture* (IDC). The configuration rules are saved with the structure allowing its exact

state at the time of capture to be reproduced each time it is retrieved. Creating an IDC does not affect any subsequent changes to the structure or its release by a workflow process.

The captured data is independent of subsequent changes made to the structure or part. For example, you may create an IDC that contains an in-progress assembly that references revision B of a particular piece part. If a user modifies revision B of the part without creating revision C, the IDC is unaffected; if you retrieve revision B, the structure still contains the original revision B, not the modified part.

The IDC is stored as a PLM XML file containing references to specific files in datasets. The files are marked so they are retained even if they are subsequently modified; the copy is edited, not the original. An IDC is independent of a particular application, and you can open it in Multi-Structure Manager, Manufacturing Process Planner, NX, or any other application that interprets IDC PLM XML data.

An example scenario in which you use an IDC follows:

1. Initiate a project to validate how a product is manufactured and create a collaboration context to store the data needed.
2. Collect all occurrences of interest from the product and process structures, and put them in structure contexts in the collaboration context.
3. Set the configuration rule for the product data and save it with the structure context.
4. Create a composition structure, add it to the new structure context, and instantiate the product occurrences into the composition.
5. Add other nodes needed for the project to the structure context such as resources, parts from other products, and documents.
6. Create a document that describes the conclusions of the project and save it with the composition. The document may be updated if the project data changes.
7. Set closure rules for the composition that specify the objects of interest for capture.
8. Create an IDC from the composition that includes only the objects of interest, and save it in Teamcenter.
9. At a later time, you can open the IDC and compare the captured data with the current configuration of the product. If necessary, the captured data can be updated with any changes.

You can also use IDCs to exchange structure data with external systems. For example, you could send an IDC that represents an assembly to the manufacturing execution system (MES). The MES then adds serial numbers of the components and returns the modified IDC to Teamcenter, where it represents the initial as-built state of the assembly. This data can also be shared with a maintenance planning system, such as Teamcenter Service Lifecycle Management (SLM). You can compare and visualize these various BOMs when they are stored in Teamcenter.

An IDC is a PLM XML file that you can manage, share, and release in the same way as other Teamcenter objects.

Note:

You can also manage SLM data with As-Built Manager inside Teamcenter.

Best practices for revision configuration

Some suggestions for managing revision configuration follow:

- Use *imprecise* assemblies if you want to provide many views of the same structure data. Use *precise* assemblies if you want to control the configuration of the structure closely.
- When initially installing product structure management, consider not implementing automatic configuration by revision rules, as new users may prefer to configure product structure manually. Introduce revision rules when users are familiar with the system.
- Create baselines sparingly as each baseline request creates a completely released structure and (potentially) many new item revisions and copies of the associated data and CAD models.
- Create an IDC if you want to capture the exact state of all referenced data, not only the relevant configuration of the product.

6. Configuring product structure with effectivity

Configuring product structure with effectivity	6-1
What effectivity schemes are available?	6-1
Understanding revision effectivity	6-1
Using structure (occurrence) effectivity	6-4
Using nested effectivity	6-6
Best practices when applying effectivity	6-9

6. Configuring product structure with effectivity

Configuring product structure with effectivity

What effectivity schemes are available?

You can configure structures using one of these effectivity schemes:

- **Revision effectivity**
You configure the product structure for a particular date or unit (serial) number by applying a revision rule. You can also specify a range of dates or unit numbers. Teamcenter shows the revision of each item that is in effect for the specified date, unit number, or range. Revision effectivity is frequently used by automotive manufacturers.
- **Occurrence (structure) effectivity**
You configure the product structure in a similar way to revision effectivity, but Teamcenter shows the actual occurrences that are in effect. Occurrence effectivity is frequently used by manufacturers of military and aerospace products.

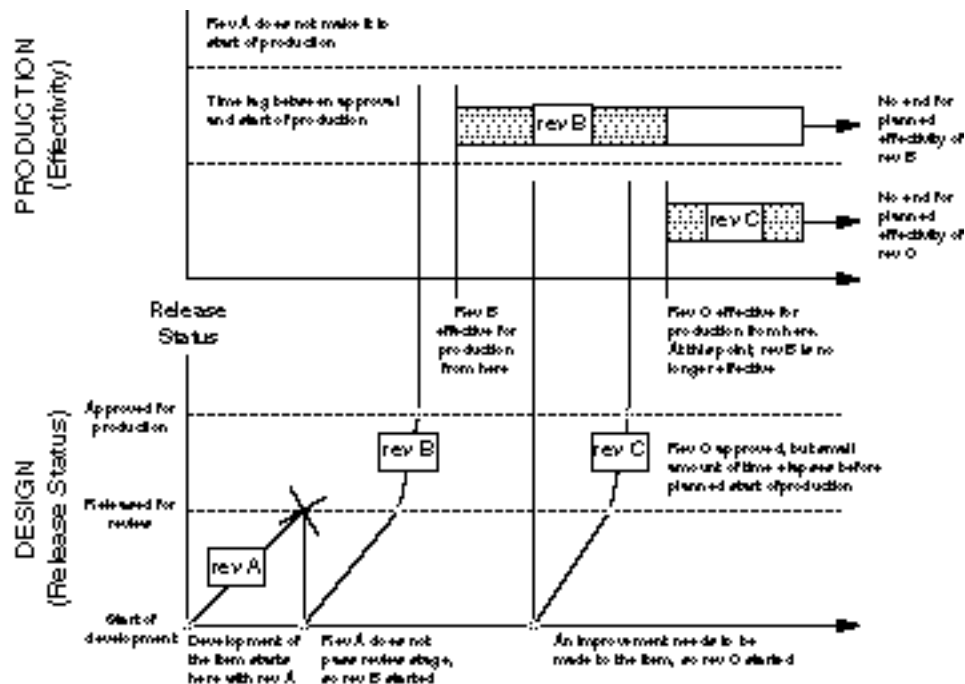
Understanding revision effectivity

Revision effectivity principles

Note:

As an alternative to revision-based effectivity, consider incremental change.

You can use *revision* effectivity to control when various revisions of an item are in effect. The product structure can be configured for a particular date or unit (serial) number by the application of a revision rule. The following example shows an application of revision effectivity.



When you set an effectivity range, consider the following:

- You express effectivity as a start to end range of dates or unit numbers, for example, **1–May-2007 to 31–May-2007**.
- You can configure open-ended effectivity with an infinite end date or unit number (**UP**) or until stock is out (**SO**).
For example, **1–May-2007 to UP** or **1–May-2007 to SO**. **UP** and **SO** are both provided to aid integration with ERP systems but, in Teamcenter, they are functionally the same.
- Effectivity may be discontinuous, for example, **1–May-2005 to 31–May-2007, 1–July-2007 to UP**.
- Effectivity can be shared. A reusable effectivity has an identifier.
- You must qualify unit effectivity by an end item, for example, **Unit 10 to UP, End Item A1000**.

Revision effectivity configuration algorithm

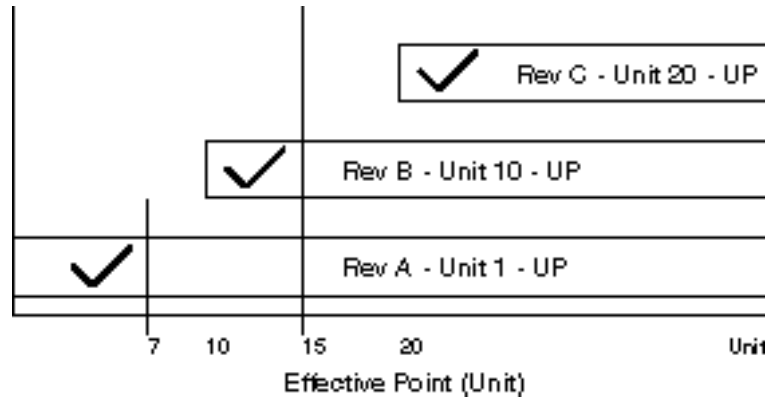
You can use **Has Status** revision rule entries to configure revisions by their effectivities. A revision rule entry is qualified by one of the following:

- Configured by effective date
- Configured by effective unit

For any item, a revision rule must configure only one revision, but revisions may have overlapping effectivities.

Typical effectivity unit configuration

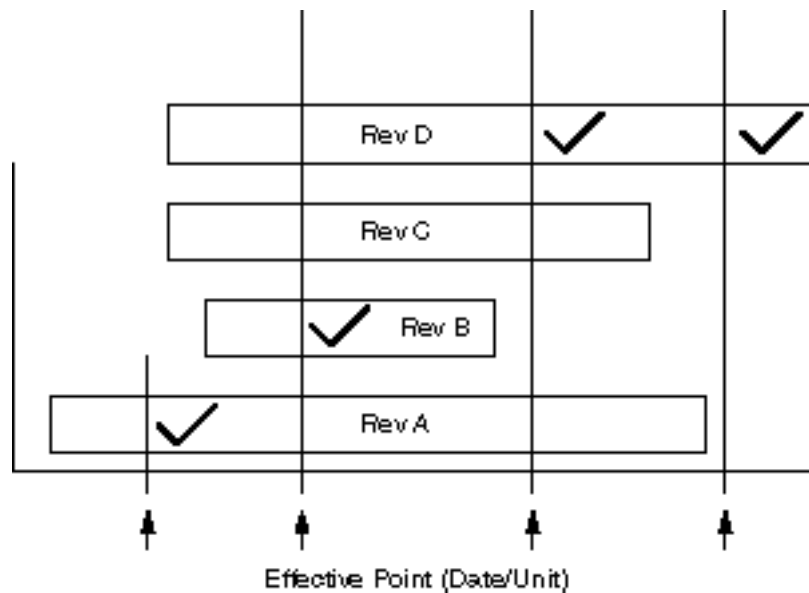
The following is an example of effectivity ranges for three revisions of an item with open-ended unit number effectivity. If the structure is configured at unit 7, revision A is configured. If the structure is configured at unit 15, revision B is configured.



Revision effectivity configuration algorithm with closed range

The following is an example of effectivity ranges for three revisions of an item with closed range effectivity, that is, with an end unit or date set. A check mark (tick) indicates the particular revision that is configured for a given effectivity point.

- When the effective point line crosses only one revision, the revision is configured, as shown for revisions A and D.
The red line corresponds to the effective point (date or unit) at which the structure is being configured.
- If the effective point line crosses more than one revision, the revision with the latest start date or unit is configured, as shown for revision B.
- If two or more revisions have the same start date (for example, revisions C and D), the revision with the latest release date is configured, as shown for revision D.



Using date effectivity

Date effectivity allows you to specify a valid range of dates for a particular item revision. The effectivity definition may be:

- Open
For example, an item revision may be valid from 05 January onward.
- Closed
For example, an item revision may be valid from 01 January to 30 April.

Using unit number effectivity

Unit effectivity allows you to specify a valid range of unit numbers for a particular item revision.

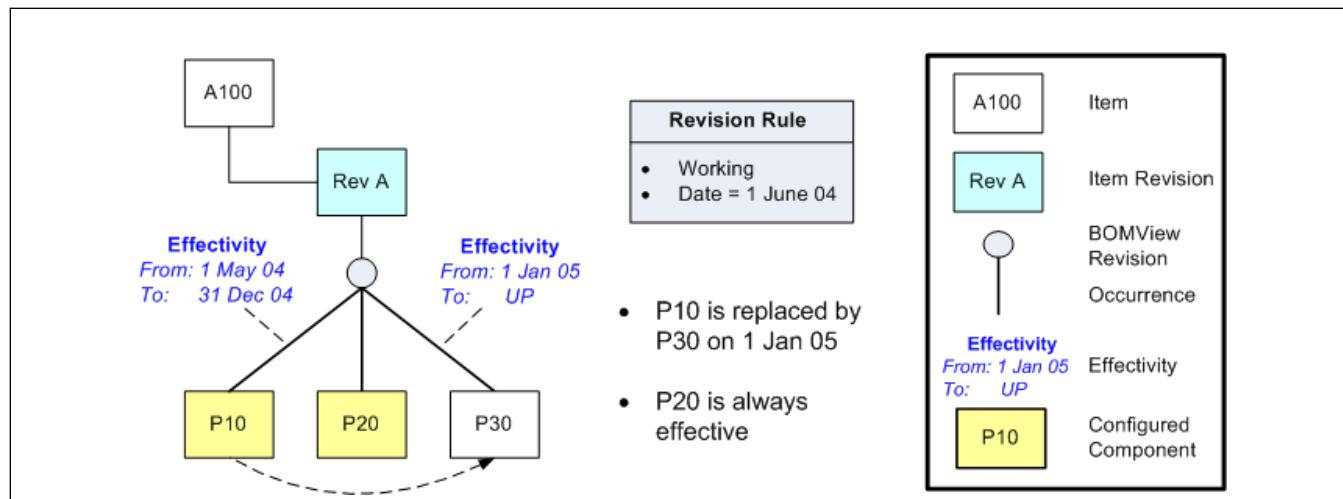
- Unit effectivity is always specified in the context of an *end item* to which the units apply.
- You can specify a discrete, noncontinuous range, if appropriate.

Using structure (occurrence) effectivity

Occurrence effectivity principles

Occurrence effectivity (sometimes called *structure effectivity*) allows you to specify the actual occurrences that are in effect in the structure for a specific date or unit number range. Occurrence effectivity information is stored in an effectivity object in the database. This object has a unique ID and can be referenced by one or more occurrences. It stores the effective date or unit number ranges, the name of the user who created the effectivity, and the date and time that the data was created.

An effectivity object can be shared by several occurrences. If you edit the date range on one such occurrence, Teamcenter applies the change to all occurrences that share the object. In most circumstances, this is appropriate and this action is the reason for sharing the effectivity. If an occurrence does not have an associated effectivity object, it is assumed to be always effective; that is, always loaded regardless of the date set in the revision rule.



Sharing effectivity

When you create an effectivity condition on an occurrence, you change the BOM view revision (BVR). To make this change, you must have write access to the product structure. You can change the effectivity range at a later time, even if the structure (BVR) is already released.

Configuring structures for multiple end items and units

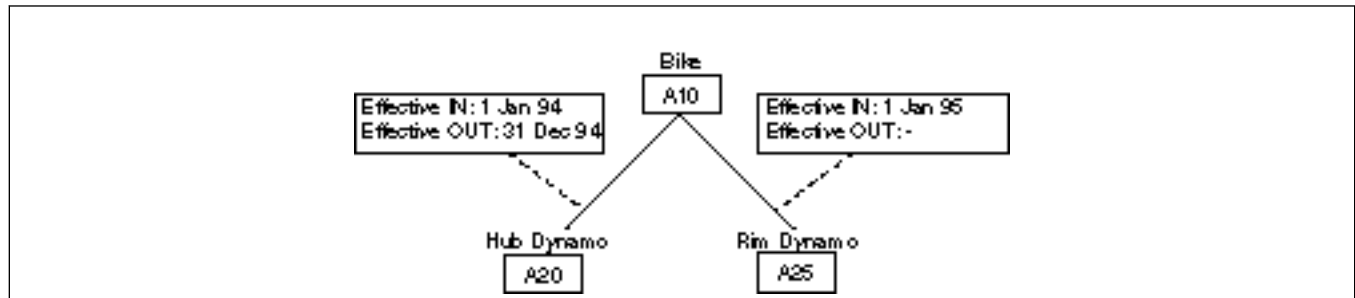
You can configure product structure occurrences of an assembly based on specified multiple end items and the unit effectivity ranges for each of those end items. This allows you to perform impact analysis more easily, as it eliminates the duplicate work required to maintain different product structures and the resulting complicated manual reconciliation.

This use of a combination of multiple end items and range of units for each end item to configure product structure occurrences is referred to as *multi-unit configuration*. To enable this capability, the administrator must set the Business Modeler IDE **Fnd0EnableMultiUnitConfiguration** constant to **true** at each site. Once enabled, you can create **EffectivityGroup** objects, which are a subtype of the **Item** business object type. **EffectivityGroupRevision** objects hold the actual multi-unit configuration information.

Use effectivity groups in Structure Manager to configure the product structure. Occurrences whose effectivity match (even partially) the specified effectivity group are configured.

Mutually exclusive effectivity ranges

Ensure that occurrences intended to be mutually exclusive do not have overlapping ranges. For example, there are two different kinds of dynamo shown in the structure shown in the following figure. It is wrong to configure two dynamos at the same time.



Configuring mutually exclusive effectivity ranges

When specifying effectivity, make clear that this association exists, as it may not always be obvious. One way to do this is to use the same find number; another way is to attach a special effectivity note to each occurrence.

Validating effectivity

Always make a global check to ensure that effectivity ranges are consistent within the whole structure; make sure that ranges on subassemblies or components lower in the structure lie within ranges for assemblies higher up. You may not be aware of the constraint higher up the structure when you initially specify effectivity ranges at lower levels.

This validation is not performed automatically, and you should include it in the workflow process that approves the effectivity ranges. In certain cases (for example, where the structure is shared between different products), this validation may not be appropriate.

Using nested effectivity

A product structure may include a product that has its own effectivity that is separate from that of the product structure. For example, the product structure of a car may include an engine that is obtained from an external supplier. In this case, the effectivity of the engine differs from the effectivity of the car. You can use nested effectivity to change the end item context when you configure the car, as only one effectivity can be set in a revision rule; in this example, you must set the effectivity from car to engine at the engine.

Note:

You can use nested effectivity with date or unit number effectivity.

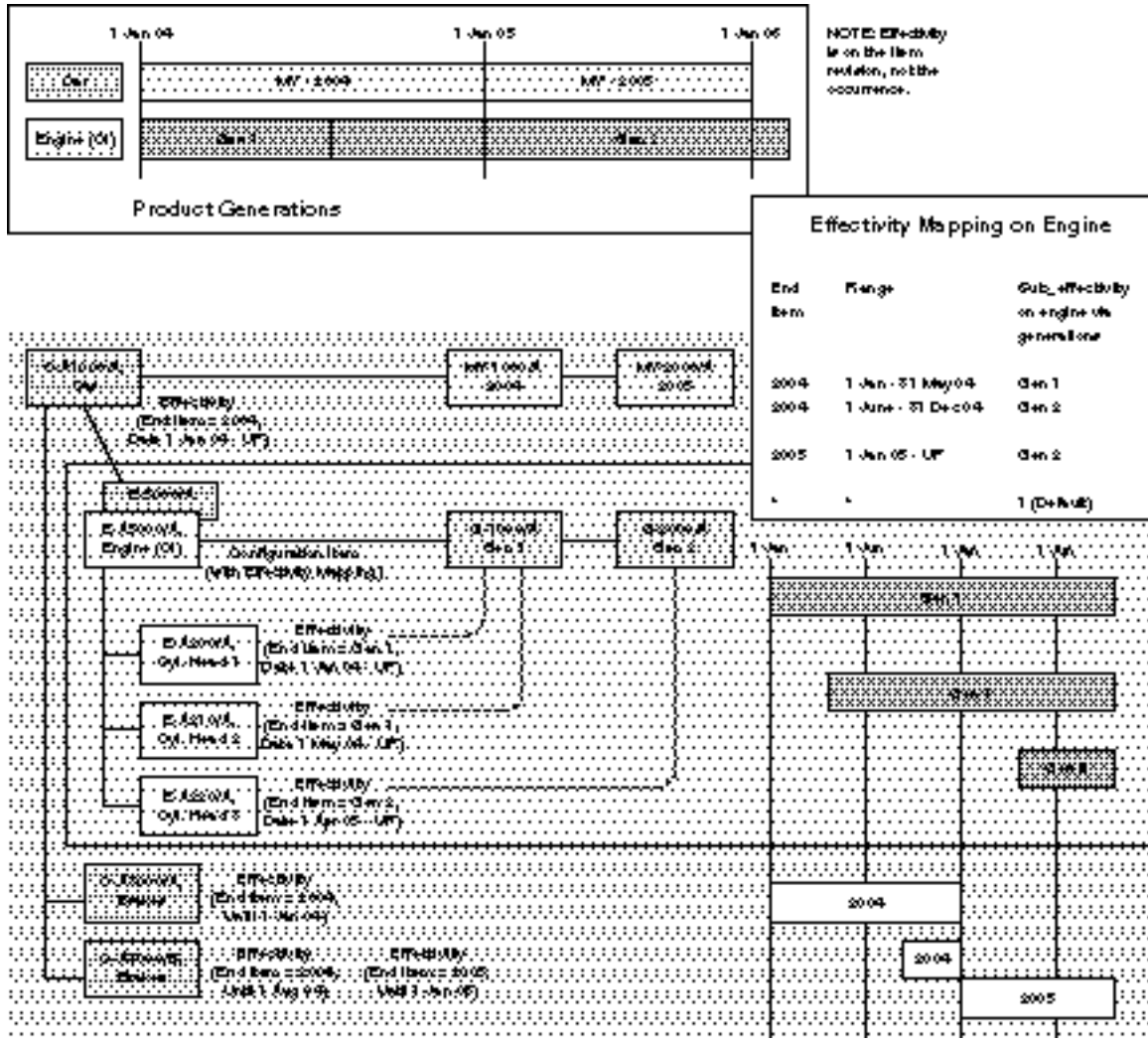
To accommodate variations of effectivity within a product, you create a *configuration item* to attach to each assembly that is configured by a different end item to the top-level item. In the previous example,

you would create a configuration item for the engine and the effectivity of the configuration unit defines the engine assemblies. A configuration item is where the effectivity context of the structure changes; it defines a new end item for the affected substructure.

You can create a mapping table to define the ranges of dates or unit numbers in the top-level product that configure a particular unit number or effective date in the lower level assembly. When you expand the product structure and apply a revision rule, the unit number or date set at the top level is converted to the scheme defined in the mapping specified for the assembly.

Nested effectivity may be suitable in situations where a lower level assembly in a structure is not commercially owned by the manufacturer of the product, but the manufacturer wishes to see the structure of the lower level assembly. For example, the manufacturer of a car may contract out engine design and assembly to a supplier. The mapping is set on the car after the engine is imported, although the mapping is actually attached to the engine. The site that owns the car structure does not need access to the engine's item revision and such access can be limited to the engine supplier. The two sites should be linked by Multi-Site Collaboration.

A specific example of nested effectivity is the *product generations* technique that is used in the automobile industry. Here, cars are developed for a single product year, while their engines have a longer life cycle that may span several model years. The following figure shows an example of product generations, in which **Gen 1** is effective until 31 May 2004 and **Gen 2** is effective from 1 June 2004 through 1 February 2006.



Nested effectivity product generations

In this example, the end item of the components in the engine are not relative to the engine itself, but to an end item such as **Gen 1** or **Gen 2**. In addition, you can specify an effectivity as components may change over the life of a generation. Similarly, the components in the car are configured against an end item for the model year, for example, **2005**.

You can define an effectivity mapping on the engine from the model year end items to the generation end items. By including an effectivity range, you can specify an engine generation as effective for part of a model year, for example, **Gen 2** may be effective in car model year **2005** from June 1 through the remainder of 2005.

Note:

In this example, you define the effectivity on the occurrence, but you can also define it on the item revision.

In summary:

- Effectivity of components in the car is specified relative to the item for the model year (**2004** or **2005**).
- Effectivity of components in the engine is specified relative to the item for the product generation (**Gen 1** or **Gen 2**).

The ranges defined in the mapping on the engine are as follows:

- End item **2004** sets the valid range of **Gen 1** as 1 January – 31 May 2004 and of **Gen 2** as 1 June – 31 December 2004.
- End item **2005** sets the valid range of **Gen 2** as 1 January 2005 onwards.

The resulting configurations are:

Car E1 = MY 2004 Effective Date — 1 Feb 04 Configures Engine Gen 1 at 1 Feb 04 C-A1000/A, Car E-A5000/A, Engine E-A200/A, Cyl Head 1 C-A3000/A, Brakes	Car E1 = MY 2004 Effective Date — 1 Nov 04 Configures Engine Gen 1 at 1 Nov 04 C-A1000/A, Car E-A5000/A, Engine E-A210/A, Cyl Head 2 C-A3000/B, Brakes	Car E1 = MY 2005 Effective Date — 1 Jun 04 Configures Engine Gen 2 at 1 Jun 05 C-A1000/A, Car E-A5000/A, Engine E-A220/A, Cyl Head 3 C-A3000/B, Brakes
--	--	--

Best practices when applying effectivity

Some suggestions for managing effectivity follow:

- Revision effectivity is simpler and easier to manage than occurrence (structure) effectivity. It also allows you to see the status of the product structure at any point in its development.
- Use incremental change if you have many out-of-sequence changes.
You can also use structural effectivity in this situation.
You can also use structural effectivity in this situation.

Also, use incremental change if you are working with large, flat structures with out-of-sequence changes, and do *not* need to view them in NX and other CAD integrations.

- Revision effectivity is simpler and easier to manage than occurrence (structure) effectivity. It works well with NX and other CAD integrations.
- Use incremental change rather than occurrence effectivity if you want to synchronize the added and cancelled components automatically.
- If you configure mixed unit and date effectivity schemes, implement unit effectivity on the higher levels of the product structure and date effectivity on the lower levels.
- There are some limited situations where multiple effectivity statements can contribute to the overall effectivity of the content of the product structure. In these cases, the effectivity statements must be layered on top of each another and applied simultaneously. Typically, these layers represent basic and customer-specific effectivity.

One proven way to achieve this layered effectivity is to use occurrence effectivity and variant conditions in combination. In this situation, you can specify the primary effectivity with occurrence effectivity and achieve the customer qualification with options and variant conditions. The options represent the customer; the variant conditions qualify the customers who impact the validity of a given product structure occurrence.

7. Managing incremental changes

- Managing incremental changes 7-1
 - Incremental changes principles 7-1
 - Tracking changes with incremental change 7-1
 - Configuring incremental changes 7-1
 - Configuring with incremental change 7-3
 - Configuring attachments 7-5
 - Configuring by intents 7-7
 - Configuring against a background configuration 7-9
 - Using incremental change with Change Manager 7-9
 - Using incremental change with revision configuration 7-9
 - Tracking changes to occurrence data 7-10
 - Baselining structures with active incremental changes 7-12
 - Best practices when using incremental change 7-13

7. Managing incremental changes

Managing incremental changes

Incremental changes principles

An incremental change collects a number of structure changes to a component such as addition or removal of components or changes to attachments (data). Effectivity can be applied to incremental changes to configure the associated changes. This method of change control allows several independent structure changes to be made concurrently, including the addition or removal of unrelated components. Those changes can be implemented in any sequence.

Incremental change is an alternative to revision-based effectivity. It is suitable for large, flat structures without nested subassemblies or components. The revisioning approach is not suitable for this type of structure, as a separate revision is required for every permutation of change.

Tracking changes with incremental change

When data is changed in the context of an incremental change, those changes (typically, the addition or removal of occurrences or data) are recorded against that active incremental change. Changes that can be tracked and recorded against an incremental change:

- Addition and removal of components (occurrences) in an assembly.
- Addition and removal of attachments (forms or datasets) attached to an item revision corresponding to the selected component.
- Creation and deletion of forms or datasets, as distinct from attaching the data to an item revision. It applies configuration to the data itself, rather than to its relationship to the item revision.
- Editing of replacements. The replacement retains the occurrence and its associated data, and simply replaces the component item. Replacement of the component item is tracked in the context of the incremental change.

You can track changes in two ways:

- Dynamically with an active incremental change, as the user makes changes.
- Retrospectively, if the changes were already made, but were not tracked against an incremental change. This is a manual process and is consequently time-consuming.

Configuring incremental changes

You can configure incremental changes in one of two ways:

- With status

An incremental change revision is released when Workflow applies a release status. Depending on the release status, effectivity may be applied to the change.

It may be desirable to initially assign a **Preliminary** or **IC in Process** status to the incremental change revision to allow effectivity to be applied. When you apply a final **Released** status, the incremental change can no longer be modified. You need write access to the incremental change, which your administrator configures in the access rule tree for the specific release status.

Note:

Any new component item revisions added by an incremental change must themselves be released before or at the same time as the incremental change itself is released. Otherwise they may not be configured by a production revision rule without a **Working** entry. Ensure that your business process that releases incremental changes checks this requirement is met.

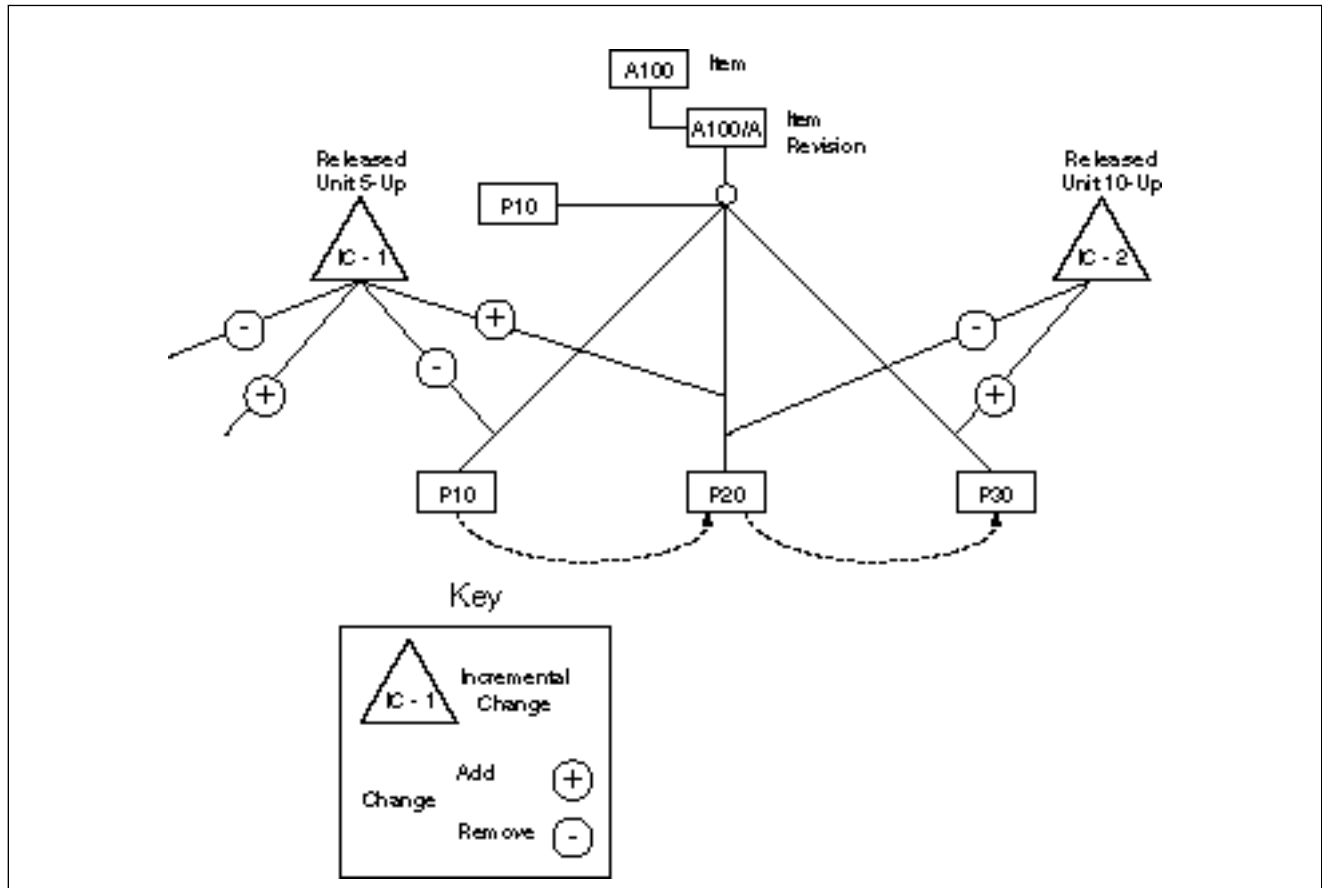
- With revision rules

When you apply a revision rule to the structure, it configures the appropriate incremental changes (revisions) and hence the associated changes. The affected components and attachments are configured accordingly.

You can choose to view all configurations or to hide those components and attachments that are not configured by the current revision rule.

While it is possible to revise incremental changes, this makes their analysis and usability more complex. As a best practice, Siemens Digital Industries Software recommends that you create a new incremental change each time you revise the structure.

The following figure shows how incremental change configures components.



Using incremental change to configure components

Configuring with incremental change

Teamcenter uses the following algorithm to configure a component or attachment:

- If the occurrence has an associated change and the incremental change associated with that change is configured, it applies the change (that is, adds or removes the component or attachment).
- If there is more than one change on an occurrence, it configures a remove change in preference to an addition change.

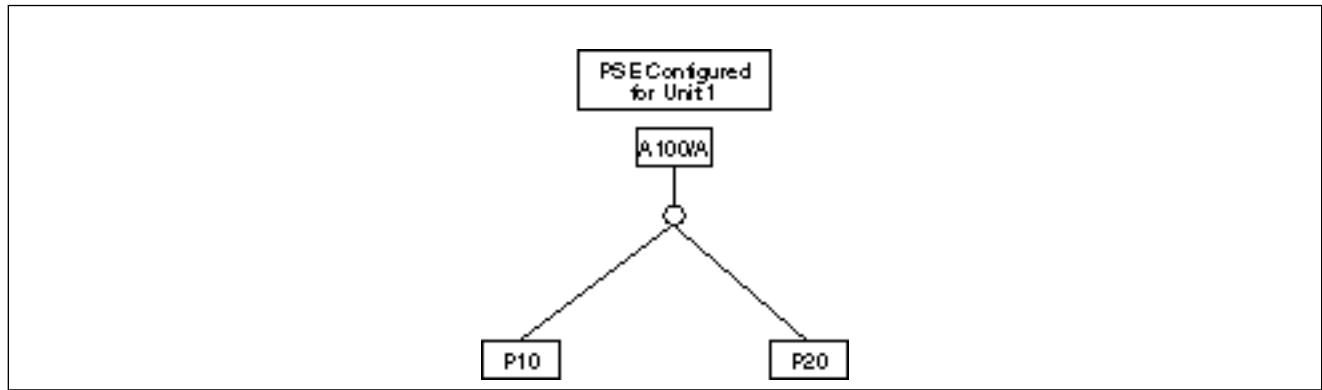
The previous figure shows an example of this process. There are two changes associated with component P20:

- An addition on IC-1, effective from unit 5
- A removal from IC-2, effective from unit 10

For unit 10 and higher, both IC-1 and IC-2 are configured, but the removal takes precedence.

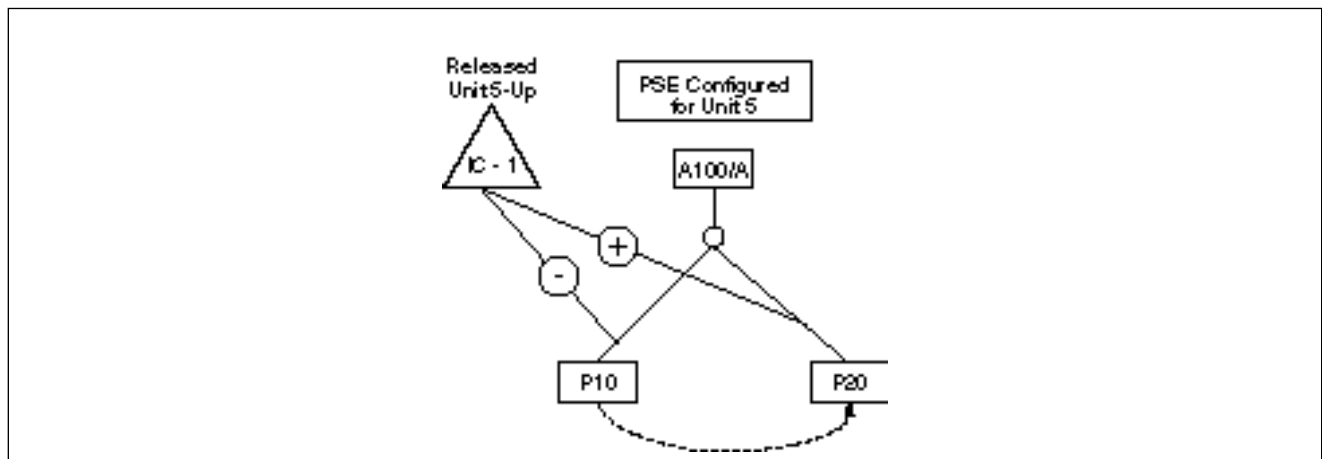
The possible configurations of A100 are as follows:

- Units 1-4: Component P10. There are no incremental changes configured.



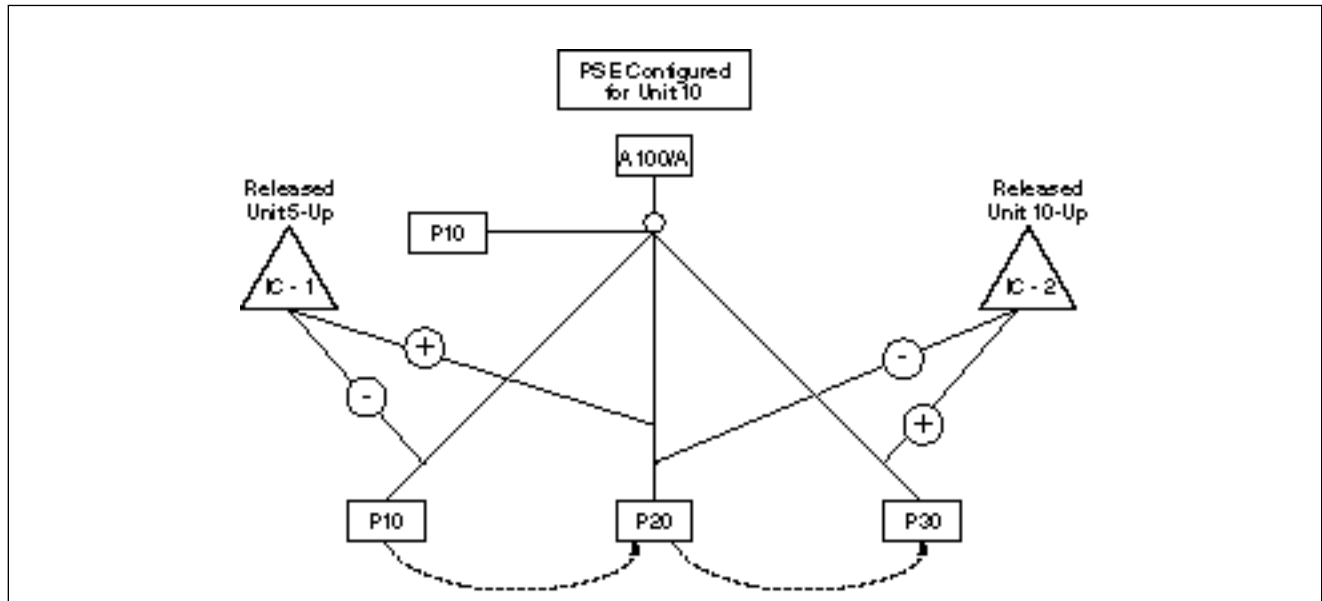
Configuration of assembly A100 at unit 1

- Units 5-9: Component P20. Only IC-1 is configured, hence P10 is removed and P20 added.



Configuration of assembly A100 at unit 5

- Units 10+: Component P30. Both IC-1 and IC-2 are configured, hence P10 and P20 are removed and P30 added.



Configuration of assembly A100 at unit 10

Note:

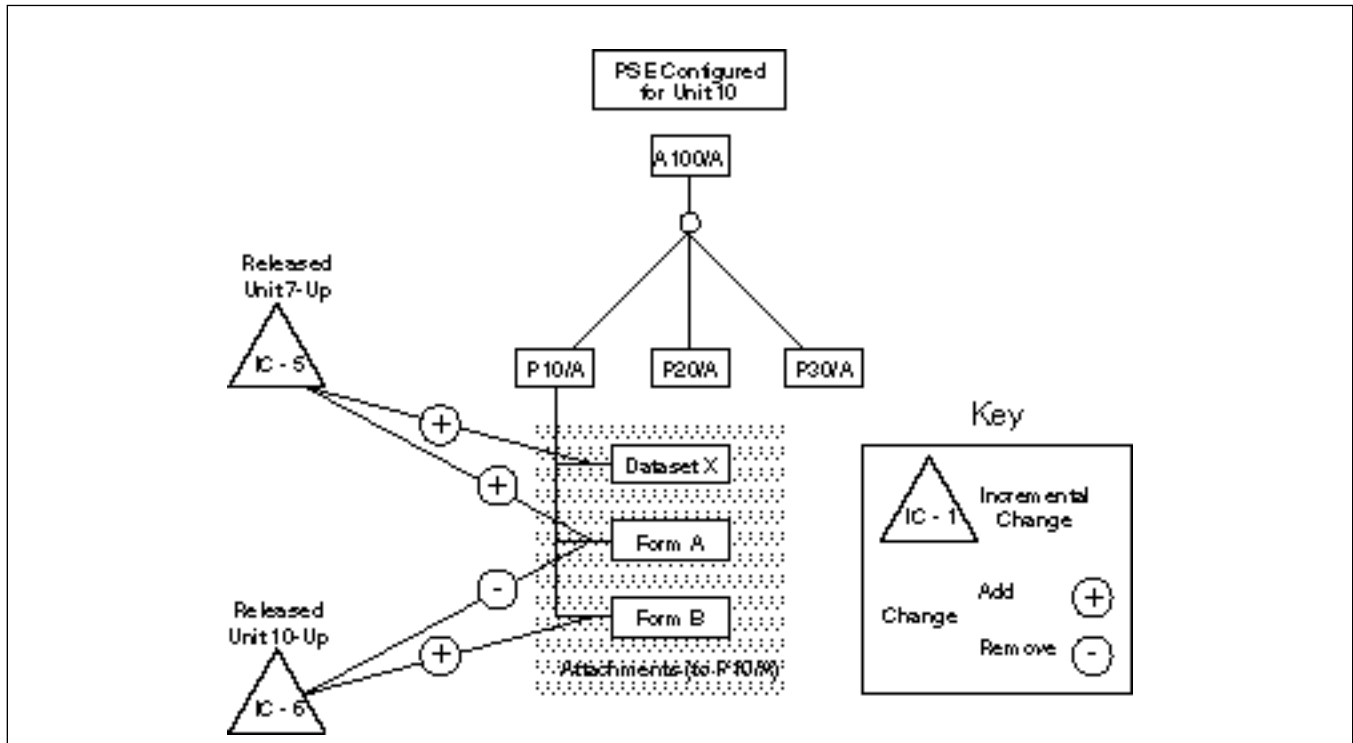
If there are two competing incremental changes referencing a component (both removals), and both are effective from the same unit number, the algorithm configures the incremental change with the latest release date.

Configuring attachments

Using incremental change with attachments

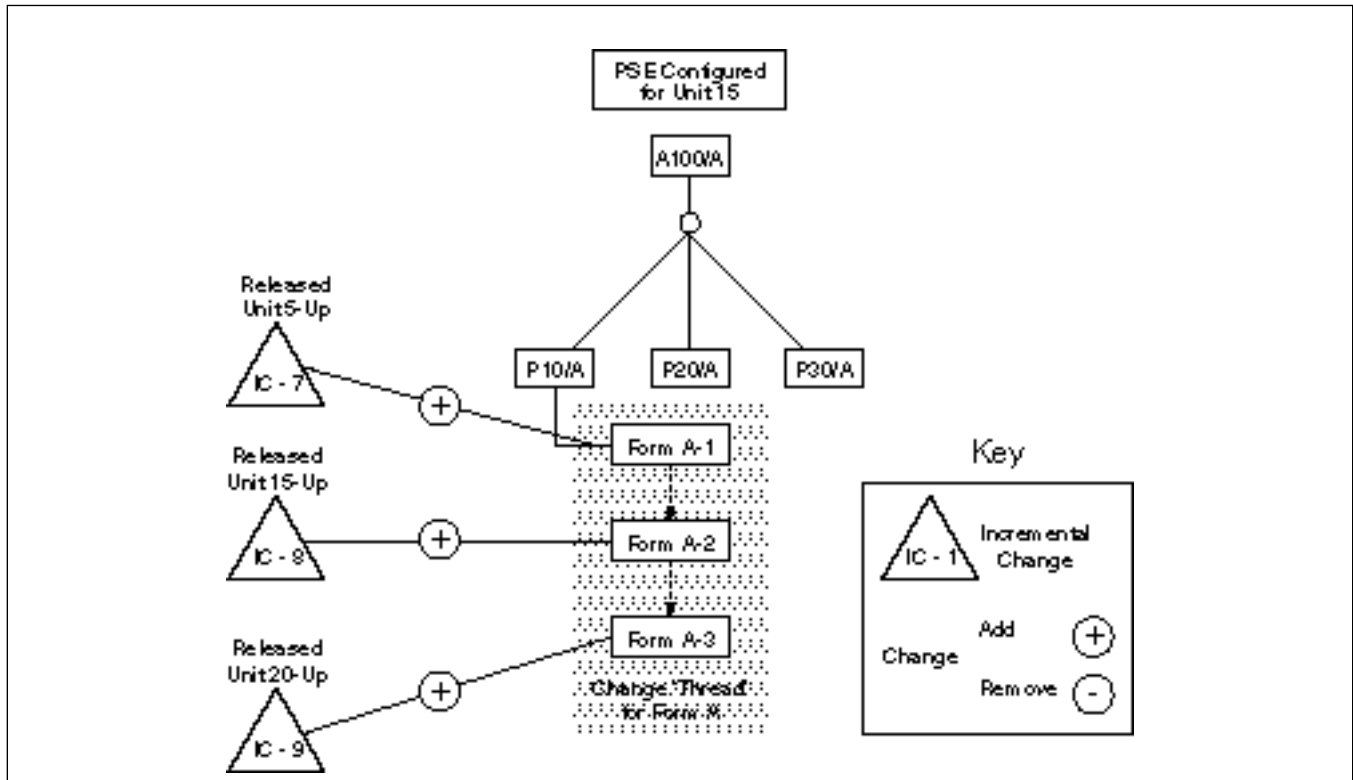
The following figure shows how incremental change configures attachments:

- IC-5 adds attachments to item revision P10/A: dataset X and Form A, effective from unit 7.
- IC-6 replaces form A with form B, effective from unit 10.



Configuration of attachments on item P10

The following figure shows how edits made to attachments are configured when an incremental change is active. In this scenario, a user adds form A-1 on IC-7. Another user edits the same form when tracking it with IC-8. Internally, Teamcenter makes a copy of the form (A-2) and connects the two forms. Another user makes a further change to the form on IC-9, as shown.



Change thread-configuration of edits to attachment form A on item P10

Note that the change on IC-7 added the attachment (form A). The subsequent changes to the form (on changes IC-8 and IC-9) caused **Add** changes to be added to the form itself, rather than to the relationship. In My Teamcenter, only one form is visible—the form that was initially added (A-1). However, if you view attachments in Structure Manager, the form is that configured by incremental change. The possible configurations are as follows:

- Units 5-14 – form A-1.
- Units 15-19 – form A-2. This has the same attributes, but different values.
- Units 20 upwards – form A-3. This has the same attributes, but different values again.

Configuring the creation and deletion of attachments

You can track the creation and deletion of attachments with an incremental change. This is useful if the attachment has effectivity at some point, or becomes no longer effective.

Configuring by intents

An intent represents an alternate solution, for example, a named solution such as **Alternate Solution 1** or a specific configuration for which the incremental change is valid. Intents allow configuration of a structure by criteria other than date or unit number. An intent is a simple object identified by a name. It

can be created by any user and does not require system administrator privileges. The creation mechanism is as follows:

Note:

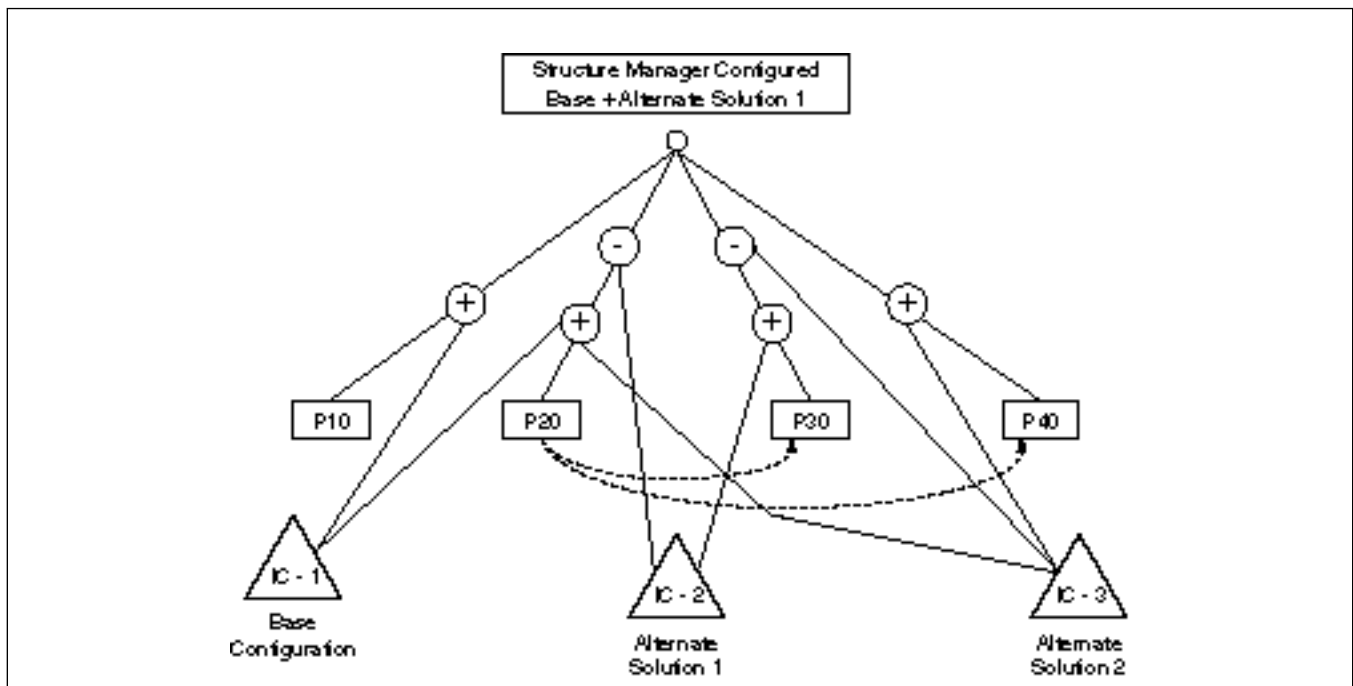
To enable intents, the Teamcenter administrator must set the **EnableIntents** preference to **on**. If this preference is **off** (default), the intents tabs are not visible.

1. Apply one or more intents to an incremental change to indicate that the incremental change is valid for those intents. Intents apply to all revisions of an incremental change. Unlike effectivity, they are not attached to the release status.
2. Set one or more intents to configure by a revision rule. The configuration algorithm selects any incremental changes having an intent that matches the intent set in the revision rule. If an incremental change has no intents, it is always configured.

You can combine intent configuration with effectivity.

Intents are only available for incremental change and cannot be used with item revision configuration.

The following figure illustrates the use of intents for two alternate solutions on a base configuration.



Configuration of components on A100 using intents

The possible configurations in this example are:

- Base configuration – P10, P20

- Alternative solution 1 – P10, P30
- Alternative solution 2 – P10, P40

Configuring against a background configuration

You can review the impact of a specific incremental change that is in process in isolation; this is referred to as a *background configuration*. You can do this by creating a revision rule with an override folder entry at the top, adding the incremental change revisions that you want to configure to a folder, and setting it as the override folder.

The background configuration is set by the remaining rule entries, and may configure only incremental changes with a specific status, for example, **Released**.

Using incremental change with Change Manager

Because incremental changes are engineering changes, you can view them with Change Manager.

Generally, you would release the item revision of a newly created component when you add it to the structure with incremental change. If the component is configured by effectivity, you can add it as a solution item on the incremental change that added it. If you do this, Teamcenter applies the effectivity both to the incremental change and also to the item revision for component added as a solution item (the component itself is configured by conventional revision configuration). If the item revision is configured by release date, rather than effectivity, the revision rule should include an entry for the same status, but configured by release date.

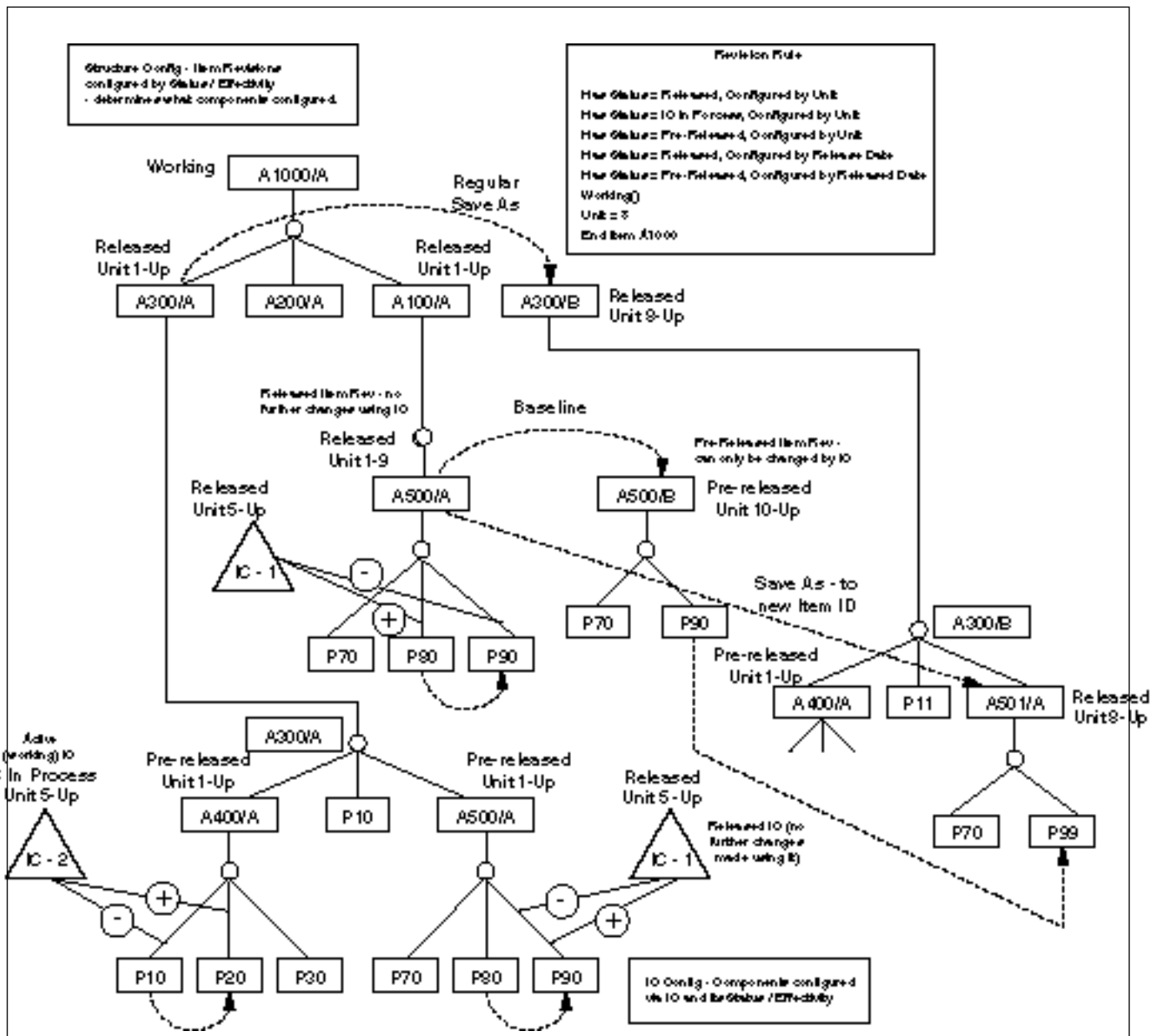
If the component item revision is configured by any other status, it should still be released but in a separate process.

Using incremental change with revision configuration

The following example illustrates how incremental change and revision configuration work together in a baseline. In this scenario, the upper levels of the structure are configured by the revision and the lower levels are configured by incremental change. It is also possible to configure the upper or middle levels of the structure with incremental change to control structure changes, and configure the lower level components by revision; the lower levels may contain CAD design files.

In the example:

- The upper structure (including items A100, A200 & A300) uses standard revision configuration. The revision rule selects the revision of an assembly that is loaded according to release status or effectivity. This in turn determines the components that are configured.
- The lower structure (including items A400 and A500) uses incremental change configuration to determine the loaded components. Any new revisions you create at this level are for incremental change baselines only.



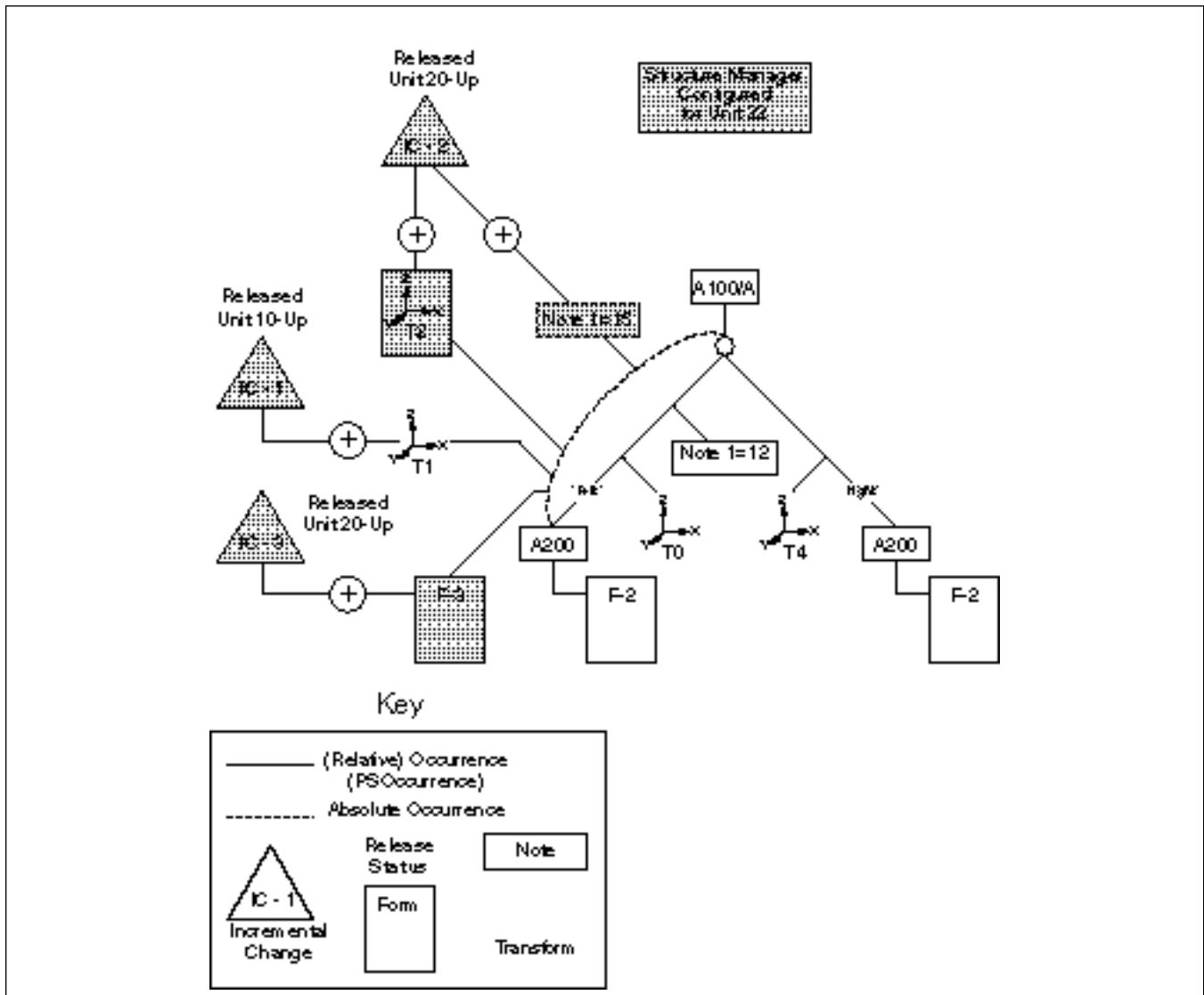
Example of incremental change and revision configuration

Tracking changes to occurrence data

You can track changes to occurrence data with incremental changes, including the addition of attachments to a specific occurrence or changes to occurrence notes and transforms.

You can add an attachment to a specific occurrence in absolute occurrence mode—an *in-context* edit in the context of the immediate parent or a higher level parent. You can track such changes as an incremental change to the specific occurrence. Changes to an attachment can be tracked in a similar way.

The following figure shows that changes to occurrence data (for example, Note 1) on the immediate parent are stored on an absolute occurrence. (The user does not need to be aware of this configuration.)



Tracking occurrence data changes with incremental change

The possible configurations of occurrence data on A200 (left occurrence) in assembly A100 are as follows:

- Units 1-9: N incremental changes configured - Original occurrence data: Transform=T0, Note 1=12, Form F-2.
- Units 10-19: IC1 configured - Transform=T1, Note 1=12, Form F-2.
- Units 20+: IC1, IC2 & IC3 configured - Transform=T2, Note 1=15, Form F-3. Form F-2 is still attached to the right occurrence of A200.

There is no **Remove Change** on the occurrence data, for example, for the transform or Note 1. You simply define a new value on an absolute occurrence that overrides the value on the normal relative occurrence.

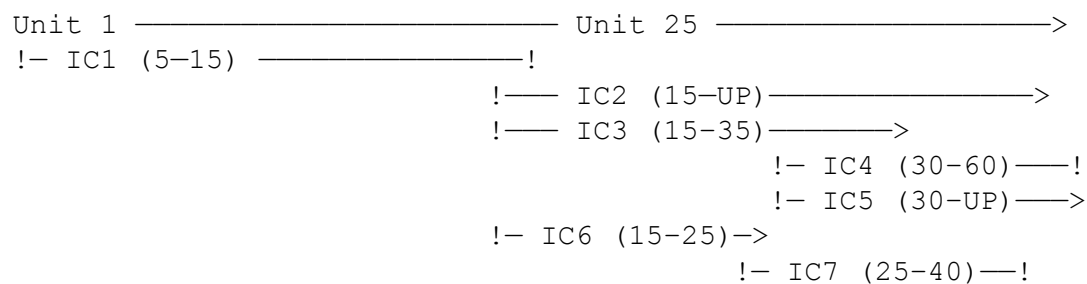
Baselining structures with active incremental changes

Baselines and active incremental changes

By default, incremental changes are dropped when you create a baseline of an item revision. You can optionally configure your system so that active incremental changes are carried forward or rolled up into the new copy of the baselined item revision. Active incremental changes are changes that are currently effective and changes that are effective in the future. This behavior applies whether the structure is configured by unit effectivity or date effectivity.

Baselining an item revision with active changes

The following example shows incremental changes that affect revision 00 of an item. All data owned by the item revision is normally copied forward when you revise it, including occurrences, attachments, edits to attachments, and predecessor links. In the example, the item revision is affected by seven incremental changes, which are numbered IC1 through IC7. Each incremental change has a unit effectivity, for example, **15-UP**.



If you decide to create a baseline of revision 00, Teamcenter creates a new revision 01. You choose the baseline as unit 25 (the *baseline effectivity*) and any incremental changes that are no longer effective are dropped. In the case of currently effective changes, if the qualifying incremental change is released or frozen, changes are rolled up into the new revision. If there are changes that are currently effective (with unreleased incremental changes) or become effective in the future, they are carried forward with their effectivity.

In the example:

- IC1 ceased being effective at unit 15 (that is, before unit 25). It is dropped regardless of its status.
- IC2, IC3, IC6, and IC7 are all currently effective at unit 25. They are carried forward if the incremental change is not frozen or rolled up if it is frozen.
- IC4 and IC5 become effective at unit 30, that is, after unit 25 and therefore in the future. They are carried forward regardless of their status.

If necessary, you can make further effectivity changes to IC2 through IC7 and Teamcenter applies such edits to the configuration of revision 01. However, any such changes you make to the effectivity of IC1 will not affect revision 01.

If you decide to edit the effectivity of a previously past-effective incremental change (for example, IC1) so that it now spans the baseline unit, you must perform the baseline operation again after modifying the effectivity.

Rolling up or carrying over incremental changes

When you create a baseline of a structure that is affected by incremental changes, Teamcenter configures the baseline in the context of a revision rule. By default, it uses the revision rule to determine if each individual object under the item revision is configured by an incremental change. If an affected object is configured by incremental change, it is carried forward into the new item revision, but its incremental change qualification is lost; that is, it is rolled into the baseline configuration. If an affected object is not configured, it is dropped; that is, it is not carried forward into the new item revision.

This scenario is adequate for simpler revision rules, for example, those that use a release date. In this case, any change that was released before the date when you created the baseline is always in effect from the baseline date onward. However, if you use effectivity in configuration, you may encounter changes that are not yet effective but become effective in the future. You may also have changes that are currently effective but have **out** values that terminate their effectivity in future.

In these more complex scenarios, for currently effective incremental changes that are *not* released, you should carry them over to the new baseline. That is, you carry over both the affected objects and their associated incremental change qualification. It is desirable to carry forward the qualification because you can change data such as effectivity on the incremental change that configures in or out affected objects.

For currently effective incremental changes that *are* released, you should roll up changes to the new baseline, that is, carry over only the affected objects. Because the qualifying incremental change is released, you cannot edit the effectivity or other data, so carrying forward the associated incremental change qualification is unnecessary.

For incremental changes that are effective in the future, you should carry over both the affected objects and the incremental change qualification (regardless of status) to the new baseline, so that their change effectivity still applies to the new baseline.

If you have not set the effectivity of an incremental change when you create the baseline, the change is considered active and is carried forward to the new item revision.

Best practices when using incremental change

Some suggestions for managing incremental change follow:

- As a good business practice, make sure the item revision for a newly created component that you added to the structure with an incremental change is released. If the component is configured by

effectivity, consider attaching it as a solution item to the incremental change that added it. If you do this, Teamcenter applies the effectivity from the incremental change to both the **Add** change on the incremental change and the component that is added; the component itself is configured by conventional revision configuration.

If the component item revision is configured by release date rather than effectivity, release it separately and add an entry to the revision rule for the same status but configure it by release date.

- You can track changes to occurrence data, such as notes and transformations, with incremental changes. You can attach this data to a specific occurrence in absolute occurrence mode.
- Use incremental change in preference to structure effectivity if you want to handle a large number of related changes collectively. This is particularly applicable for large, flat structures.
- You can use incremental changes to manage changes associated with *intents*, for example, all the changes necessary for a proposed alternative design solution. Incremental changes are not suitable for tracking changes associated with events, milestones or other time-dependent management techniques. Use a unit number effectivity scheme for time-dependent changes.

8. Configuring product structure with variants

Configuring product structure with variants	8-1
Configuring product structure with variants	8-1
Choosing a variants technique	8-1
Using classic variants	8-3
Using modular variants	8-13
Best practices when designing for modularity	8-26

8. Configuring product structure with variants

Configuring product structure with variants

Configuring product structure with variants

Teamcenter supports three techniques of managing variants:

- Product Configurator variants
- **Classic variants**
- **Modular variants**

Choosing a variants technique

What are the advantages of each variant technique?

The advantages of each technique in a particular application are as follows:

Classic variants:

Use classic variants:

- If you have a nonmodular product structure that you do not want to restructure.
- If you have limited scope for reuse of generic assemblies.

Modular variants

Use modular variants to:

- Enforce modularity to facilitate reuse of lower level assemblies. To do this, you create modules at the top-level item, with variant conditions at all lower levels.
- Instantiate business part numbers for a specific configuration. You can create variant items (VIs) and link them back to the generic modules in which they were generated. Unconfigured occurrences are removed.
- Integrate with expressions in NX. You can create one module item with options that drive the various expressions controlling the geometry of the design. This results in a simpler structure. You can also set the positions of components with mating condition offsets.

- Configure the same part differently in different locations. You can identify the locations by structure paths that are configured with different values for each option.

To use modular variants, you require a modular product structure and parts that are engineered for modularity. This may require a change to your design and business practices.

Caution:

If you migrate from one variants technique to the other, be aware that Teamcenter evaluates the same variant condition differently in modular variants and classic variants. In some circumstances, this may result in different product structure configurations. For example, Teamcenter evaluates classic variant conditions from the left, giving all operators the same precedence. When it evaluates modular variant conditions, it automatically gives **AND** clauses precedence over **OR** clauses.

Modular variants are not available with precise structures. If you want to implement modular variants, create *imprecise* structures.

Understanding variant terminology

Term	Meaning
Derived default	A default value that depends on a certain condition (for example, radio = stereo IF car type = GLX). A derived default is attached to an item revision, but applies globally to a loaded product structure.
Variant (BOM)	A specific BOM that is configured by applying a variant rule.
Variant condition	A condition that an engineering user sets on an occurrence to specify the option values that configure the occurrence (for example, Load IF engine = 1200). More complex condition statements may also be defined.
Variant rule check	A condition that specifies any option values or combinations of values that are not allowed. A variant rule check is attached to an item revision. Also called an <i>error check</i> .

The following terms only apply to classic variant structures:

Constraint	An expression that sets an option value according to the values of other options, that is, derived defaults.
Option default	A specific default value for an option (for example, engine = 1200). A fixed default is attached to an item revision but applies globally to a loaded product structure. Can be used internally to set private from public option values, or to set public option values on the child module.
Option	A parameter of variability. Options have a string type and a name.

Term	Meaning
Variant rule	A collection of option values, typically set by a marketing user, to determine the variant of the BOM to configure (for example, car type = GLS, engine = 1200, gearbox = manual). A saved variant rule is a persistent database object.
The following terms only apply to modular variant structures:	
Error check	A type of constraint in which a message is generated for a given combination of option values. Error checking is supported at three levels—error, warning and information.
External option	An option that is typically placed at the top-level module and sets the value of options that reference it in lower level modules.
Global option	An option that defines a set of standard allowed values in a central module. These values can be used in other modules.
Module	A container for options and constraints that describes the variability of a given item. A module encapsulates option definition to the area of the structure where it is needed.
Multiple configuration	Unique configurations of specific usages of a module. You facilitate multiple configurations with occurrence names and by configuring paths to modules.
Option	A parameter of variability. Options have a type (real, integer, or logical), visibility (public or private), a name, and a default value. You can save several options in a saved option set (SOS).
Presented option	An option in a module that points to a public option in a directly included child module.
Private option	An option that is visible only to the constraints within the module in which it was defined.
Public option	An option that is visible by constraints in a direct parent module and can be presented to a direct parent module. The public interface to the module.
(Variant) Configuration	The values set for all public options in the top-level module that configure a structure.
Variant item (VI)	A specific nonvariant instance of a module.

Using classic variants

Classic variant principles

Variants allow you to create options (for example, **color**) and allowed values of those options (for example, **red** and **blue**) and associate them with an item revision. You usually do this at a top-level

assembly, but you can implement variants anywhere in the structure. You then define a variant condition (for example, **only load IF option color = value red** is specified in the variant rule) on those occurrences that are subject to variant rules.

To configure a particular variant of an assembly or product, set a variant rule (a group of options and values such as **color = red, material = cotton**). This can be stored in the database and retrieved later.

To specify option values or combinations that are not allowed, you can also set default option values (for example, **color = blue**) for the variant rule and create variant rule checks (for example, **error if color = green AND material = cotton**). This functionality supports:

- Options that are a mandatory choice or an accessory.
- Components that are configured by more than one option.
- Variant assemblies that are shared between products.
- A 120% BOM configuration, allowing users to select multiple values for an option.
- Overlay configurations, allowing users to apply multiple saved variant rules (SVRs) to the product structure.

Changes to variant data are controlled by association with item revisions and BOM view revisions.

You can save a variant rule in as a persistent workspace object, then use it in the Multi-Structure Manager application or NX. You can also send the object to another user.

Understanding variant models

Variant model principles

Teamcenter 8.2 introduced a new variant model to enhance the classic variant configuration method. The new model utilizes a string-based variant data structure, whereas the old model utilizes a tree-based structure. It introduces several new storage class attributes on the variant expression class, reducing the number of objects stored in the database for a variant expression. This provides performance improvements when Teamcenter evaluates variant expressions. No changes are necessary to existing variant conditions or named variant expressions, and the syntax you use to develop them is unchanged.

The old and new variant models are not compatible; Teamcenter automatically converts existing variants during upgrade. This does not affect your ability to exchange data with systems that use the old variant model; that is, your system *will* remain compatible with all other versions of Teamcenter when sharing data with Multi-Site Collaboration.

Caution:

Upgrading to the new variant model may take a significant amount of time if you have defined a large number of variant expressions. Siemens Digital Industries Software recommends you run the **upgrade_variants** utility on a test database before upgrading your system. If the test upgrade takes a significant amount of time, consider migrating to the new variant model before you start the upgrade process.

Note:

In previous versions of Teamcenter, you set the **RDVUUseNewVariantModel** global constant to use the new variant model. This constant is now obsolete and should not be set.

The new variant model is also compatible with the modular variants configuration method. However, existing modular variant data is not converted and is unaffected by this enhancement.

When you implement the new variant model, Teamcenter evaluates variant conditions using the new data model, even if they use the old **Variant** object model. The **Variant Rule** and variant condition editor dialog boxes continue to show available **Variant** objects. However, when you save an expression, it is converted to the new data model. Consequently, under normal circumstances, the **Variant** objects and the new object model are always synchronized. However, if you modify the old **Variant** objects with low-level ITK functions, you may encounter synchronization issues. If you open the **Variant Rule** dialog box after changing a **Variant** object, you see the modified **Variant** objects. However, because the new data model objects of all the corresponding variant conditions have not been changed appropriately, they are now out of synchronization. To avoid this problem, Siemens Digital Industries Software recommends that you never edit **Variant** objects. If your site has implemented a custom solution that uses unpublished ITK APIs to edit **Variant** objects, you must also edit the corresponding new data model objects.

Note:

Siemens Digital Industries Software recommends you use 14 or less digits of mantissa when coding numeric variant values.

Compatibility with Multi-Site Collaboration and Global Services

Multi-Site Collaboration and Global Services support the sharing of new variant model data between sites. It also allows sites with dissimilar variant models to share data.

Source site model – target site model	Data compatibility
Old – old	Source and target sites both use the old variant model. Variant data is shared as in previous versions and no conversion is necessary.
Old – new	The source site uses the old variant model; the target site uses the new variant model. Transferred variant data is converted to the new model.
New – old	The source site uses the new variant model; the target site uses the old variant model. Transferred variant data is converted to the old model.
New – new	Source and target sites both use the old variant model. No conversion is necessary.
Mixed – new	The source site has mixed variant data, some in the old format and some in the new format. The target site uses the new format. Data in the old format is converted to the new format for transfer.
Mixed – old	The source site has mixed variant data, some in the old format and some in the new format. The target site uses the old format. Data in the new format is converted to the old format for transfer.
Mixed – mixed	Not supported.

Understanding variant solve precision limits

- Viewing value ranges in normalized rich client grid displays
The rich client displays expressions created in it in the same format in which they are authored. However, if you use an expression created in another application, the rich client may request a *Teamcenter Normal Form* (TNF) normalization. If so, Teamcenter formats value ranges, parentheses nesting levels, and other displayed data according to the needs of the rich client's grid. For example, it may reformat open (**min<val<max**) and closed (**min<=val<=max**) value ranges:

Type	1..10	1..	..10
Integer/Text	>=1 & <=10	>=1	<=10
Double/Date	>=1 & <10	>=1	<10

Consequently, if another client such as NX creates the integer expression **INT < 10**, the rich client displays **..9**, which is equivalent to **INT <= 9**.

- Checking expressions can be satisfied
Teamcenter checks expression can be satisfied with the following mappings, where:

Operator	Meaning
==	Is identical to
=~	Satisfies
!~	Violates

Note:

Date ranges that include the end date using <= represent the time interval until the end of the timestamp specified in the date value. You set the **TC_Fnd0Booleansolve_EffectivityDateRangeFromTo** preference to specify the display format you want to use for effectivity date ranges.

- **LESS THAN** operator

Integer

```
"INT < 4.00000000000000013" == "INT <= 4"
"INT < 4.00000000000000007" == "INT <= 3"
"INT < 4"                      == "INT <= 3"
```

Floating point

```
"DBL < 4.5" !~ "DBL = 4.499999999999993"
"DBL < 4.5" =~ "DBL = 4.499999999999987"
```

Date

```
"Date < 00:00:02+01:00" (2 seconds past midnight MET)
== "Date < 00:00:02.0+01:00" (2.0 seconds past midnight MET)
"Date < 00:00:02.5+01:00" (2500 milliseconds past midnight MET)
== "Date < 00:00:03.0+01:00" (3.0 seconds past midnight MET)
```

- **GREATER THAN OR EQUAL TO** operator

Integer

```
"INT >= 4.00000000000000013" == "INT >= 5"
"INT >= 4.00000000000000007" == "INT >= 4"
"INT >= 4"                      == "INT >= 4"
```

Floating point

```
"DBL >= 4.5" !~ "DBL = 4.499999999999987"
"DBL >= 4.5" =~ "DBL = 4.499999999999993"
```

Date

```
"Date > 00:00:02+01:00" (2 seconds past midnight MET)
  == "Date > 00:00:02.0+01:00" (2.0 seconds past midnight MET)
  "Date > 00:00:02.5+01:00" (2500 milliseconds past midnight MET)
  == "Date > 00:00:02.0+01:00" (2.0 seconds past midnight MET)
```

- **LESS THAN OR EQUAL TO** operator

Integer

```
"INT <= 4" == "INT <= 4"
"INT <= 3.999999999999993" == "INT <= 4"
"INT <= 3.999999999999987" == "INT <= 3"
```

Floating point

```
"DBL <= 4.5" !~ "DBL = 4.5000000000000013"
"DBL <= 4.5" =~ "DBL = 4.5000000000000007"
```

Date

```
"DATE <= 2013-03-29T08:00:00+01:00"
  == "DATE < 2013-03-29T08:00:01.0+01:00"
```

- **GREATER THAN** operator

Integer

```
"INT > 4" == "INT >= 5"
"INT > 3.999999999999993" == "INT >= 5"
"INT > 3.999999999999987" == "INT >= 4"
```

Floating point

```
"DBL > 4.5" !~ "DBL = 4.5000000000000007"
"DBL > 4.5" =~ "DBL = 4.5000000000000013"
```

Date

```
"DATE > 2013-03-29T08:00:00+01:00"
  == "DATE >= 2013-03-29T08:00:01.0+01:00"
```

- **EQUAL** operator (non-empty values)

Integer

```
"INT = 4.0000000000000013" == "FALSE"
"INT = 4.0000000000000007" == "INT = 4"
"INT = 4" == "INT = 4"
"INT = 3.999999999999993" == "INT = 4"
"INT = 3.999999999999987" == "FALSE"
```

Floating point

```
"DBL = 4.5" == "4.5 <= DBL < 4.500000000000001"
```

Date

```
"Date = 00:00:02+01:00" (2 seconds past midnight MET)
== "Date < 00:00:03.0+01:00 & Date >= 00:00:02.0+01:00"
(the time between 2 and 3 seconds past midnight MET)
"Date = 00:00:02.5+01:00" (2500 milliseconds past midnight MET)
== "Date < 00:00:03.0+01:00 & Date >= 00:00:02.0+01:00"
(the same time between 2 and 3 seconds past midnight MET)
```

- **NOT EQUAL** operator (non-empty values)

Integer

```
"INT != 4.0000000000000013" == "TRUE"
"INT != 4.0000000000000007" == "INT != 4"
"INT != 4" == "INT != 4"
```

- Evaluating discretionary logical variant families

Logical families are inherently discretionary. Expressions using logical families evaluate to **TRUE** if (and only if) the criteria match the setting **[DICT]LogicalFamilyName = LogicalFamilyName**. If the criteria contradict **[DICT]LogicalFamilyName = LogicalFamilyName**, the condition evaluates to **FALSE**. It is not important *how* the criteria contradict this setting, for example, because the setting is **[DICT]LogicalFamilyName != LogicalFamilyName** or **[DICT]LogicalFamilyName = ""**.

- Creating free-form logical variant families:

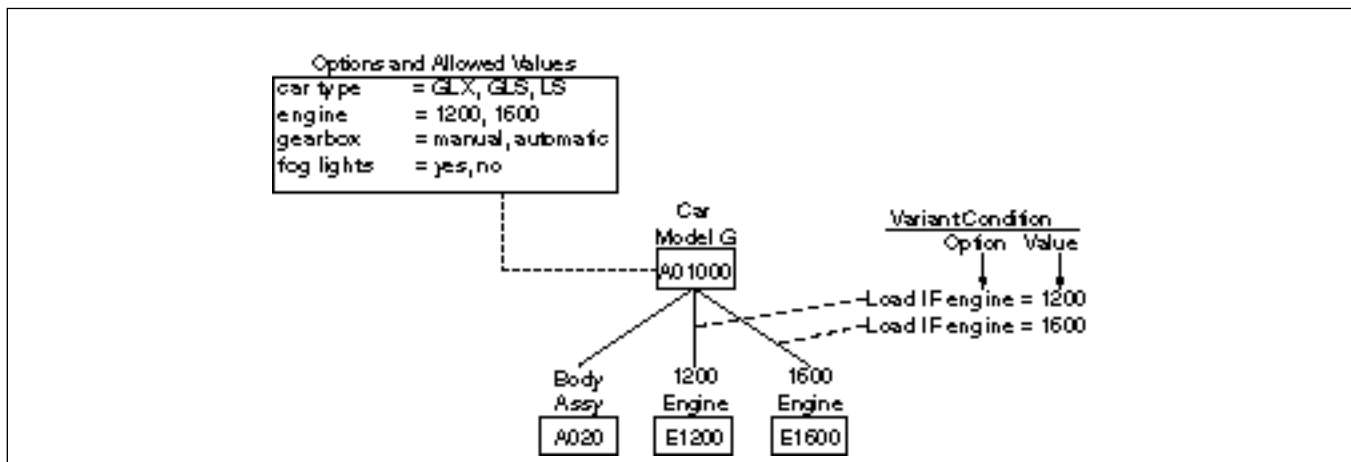
Creating a family of free-form values implies that content authors cannot anticipate the actual value consumers will select to configure their data. Consequently, the variant condition operators **EQUAL** and **NOT EQUAL** are not meaningful. Conversely, **EQUAL** and **NOT EQUAL** are the only meaningful operators for expressions using values in logical families, in which the concepts of family and value mean the same. Teamcenter locks the list of values in logical families to a single value that matches the family name. The rich client blocks the add value operation because the user perceives the value and the family as identical.

- Creating multiselection logical variant families:

Creating a family for multiselection values implies that there is more than one value. However, for logical families, Teamcenter locks the list of values in logical families to a single value that matches the family name. The rich client blocks the add value operation because the user perceives the value and the family as identical. Consequently, Teamcenter does not allow the creation of logical families for multiselection values.

Example of using classic variants

In this basic example of how to use variants, a top-level assembly is identified as **Car Model G**. It includes a body assembly and two choices of engine (**1200** and **1600**), as shown in the following figure. You create variant data on the structure to allow configuration of one or the other engine.



Creating variant data on the structure

Creating basic variant data

To create the necessary basic variant data, do the following:

- Define options
Create an engine option on the item **Car Model G** with allowed values **1200** and **1600**. You can later create other necessary options and allowed values. These options configure components lower in the structure.
- Add variant conditions
Having defined the options that determine different configurations of the car, you now specify a variant condition on each engine component to configure it appropriately. For the **1200 Engine** component, you define a condition that loads this component only if the option engine is set to a value of **1200** (that is, **Load IF engine = 1200**). Similarly, define a condition for the **1600 Engine** (that is, **Load IF engine = 1600**).
You have now created all the static variant data necessary to configure a variant BOM. These steps are typically performed by design engineers or a specialized configuration department.

Note:

A specific option value does not necessarily relate to a single component. Variant conditions including a single option value can cause any number of components at different places in the structure to be configured or not configured.

Setting a variant rule

To configure a particular variant of a structure, you use a variant rule. The variant rule defines the options in the structure for which you can set values. The marketing and sales organizations typically use the variant rule to configure a particular variant of the product. However, such configurations are limited to those that the design engineer allowed when creating the variant data on the structure.

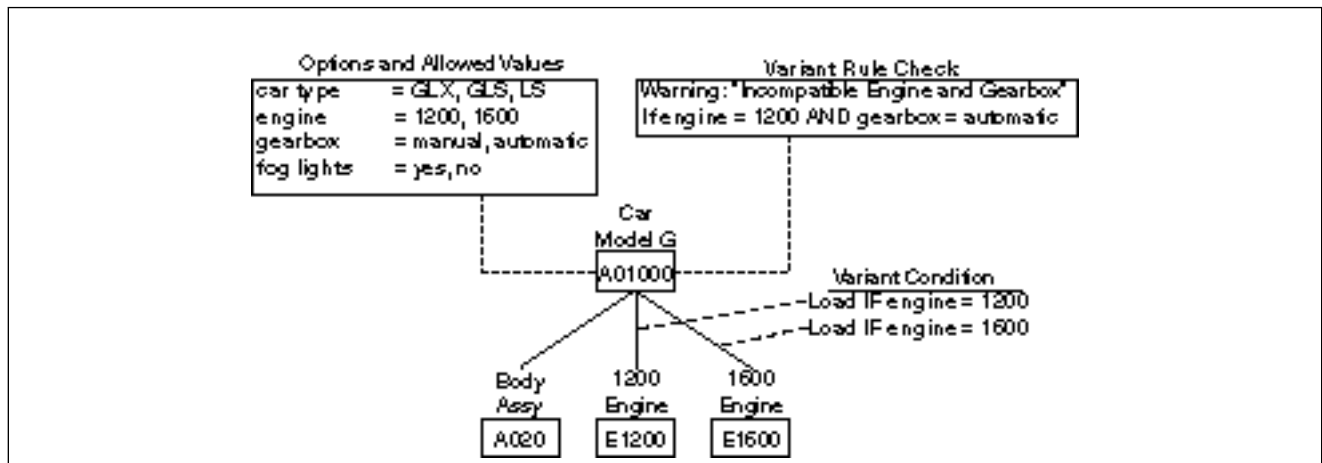
Note:

Some options may already have default or derived values set as part of the basic variant data. The rules define further variant data that can be created on the structure.

There are three ways of setting a valid variant rule that configures an allowed variant of the product. This additional variant data is stored on an item revision.

- Variant rule checks

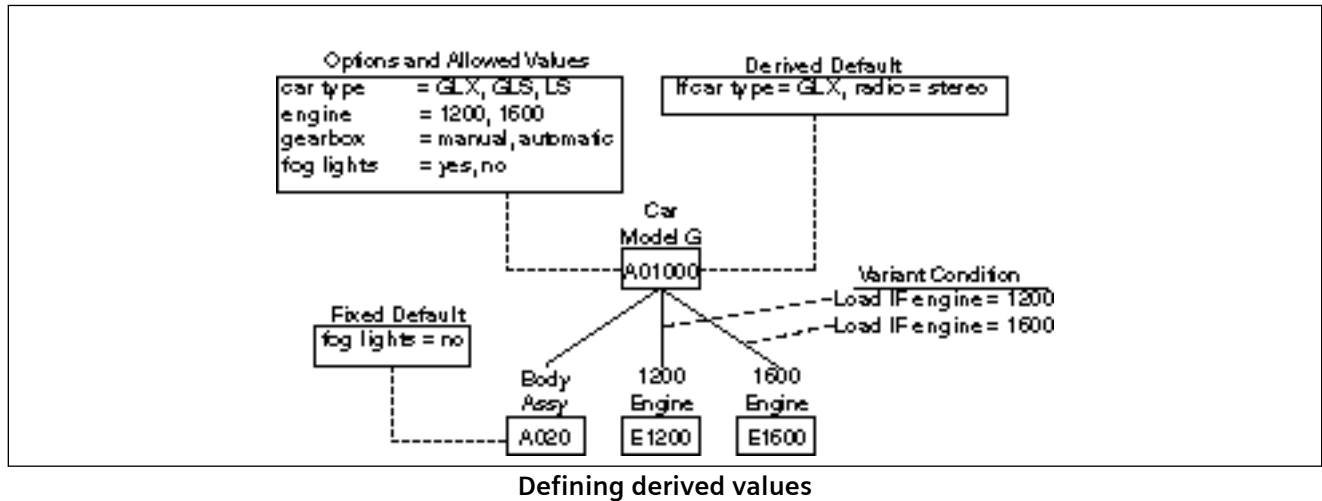
Variant rule checks prevent the designer from defining option values or combinations of option values that are not allowed, as shown in the following figure.



Defining a variant rule check

A variant rule check consists of a condition (for example, **engine = 1200 AND gearbox = automatic**) and a warning message (for example, **Incompatible engine and gearbox**). Both the condition and the warning message are displayed if the condition is met when a user attempts to set values in the variant rule. Teamcenter displays the error when the user tries to specify the condition, rather than allowing the user to set a number of values and then displaying a number of error messages. There are two main uses of checks:

- To prevent incompatible option values. For example, you want to prevent users from configuring a 1200 engine with an automatic gearbox if this combination is not allowed for technical reasons.
- To limit the range of allowed values for a specific option in a particular product. This may be necessary when assemblies are shared between different products in which there are different allowed option value ranges.
- Derived defaults
Derived defaults allow one option value to set any number of other related option values. Options that potentially have derived values are indicated in the variant rule as potentially derived and must be completed last. In the example in the following figure, if the option car type is set to **GLX**, the option **radio** is set to **stereo** by default.



- Fixed defaults
Fixed defaults allow an option to be set to a specific value. In the example in the previous figure, the default value for fog lights is **no**; unless specified, fog lights are not fitted. The fog light components are not shown at this level of the structure.

Storing variant data

Variant data can be stored on the following objects:

- Item revision:
 - Options and allowed values
 - Fixed and derived default values
 - Variant rule checks
- Occurrence:
 - Variant conditions
 - Controlling changes

Changes to variant data can have a dramatic effect on variant configuration. You can control changes with techniques such as access control lists (ACLs), locking, and release procedures.

Any user can set the variant rule to configure a particular variant. Users creating variant data must ensure that the necessary variant rule checks exist so that only valid rules can be set. You can use menu entry suppression to hide the variant rule entry from certain users and groups.

Variant data is owned by a specific revision of an item. As the item evolves, it may be necessary to change the set of allowed values for a particular option or possibly to add further options. Existing values must not be referenced anywhere in the structure or you cannot remove or replace them.

Placing variant data on a structure

When populating a structure with variant data, you must decide where to place options, defaults, and variant rule checks; typically, you attach this data to assembly items. The following types of variant data may be placed on a structure:

Type of variant data	Purpose	Best practice
Marketing variant data	Set by marketing or customers to specify the required variant of the product.	Stored at the product level.
Engineering variant data	Set by design engineers to control detailed variant information that does not concern customers.	Stored at lower levels, attached to specific modules or assemblies.

Note:

Option identification names are not unique in the database; they are unique only to the owning item. You must specify an item when the option is not unique. Options are generally displayed with their owning item.

Using modular variants

Modular variant principles

To minimize engineering and production costs, you can create *modules* in the product—self-contained plug-compatible units that can be *reused*. This technique imposes tight constraints on how the product is designed; Teamcenter supports those constraints when you work with modular variants. When you use modular variants, the options and associated logic are attached to a module item. Variant conditions on the components can only refer to options on the parent module, thus making the module independent of any other parts in the structure with regard to variant logic. This approach requires variant data and option values to be propagated up and down the structure so that end users (customers) can set options on lower level modules when configuring the whole product from the top level.

Manufactured goods are often designed and assembled from modules. For example, consider a company that produces a range of refrigerators and freezers in different sizes and colors. The door assemblies are developed in a department that designs a modular door suitable for use in any refrigerator or freezer. They design a generic door assembly that has all possible components for any use—a sheet steel outer door and two internal covers, one for a freezer and one for a refrigerator. (This is a simplified example; in a real product there would be many more individual components.)

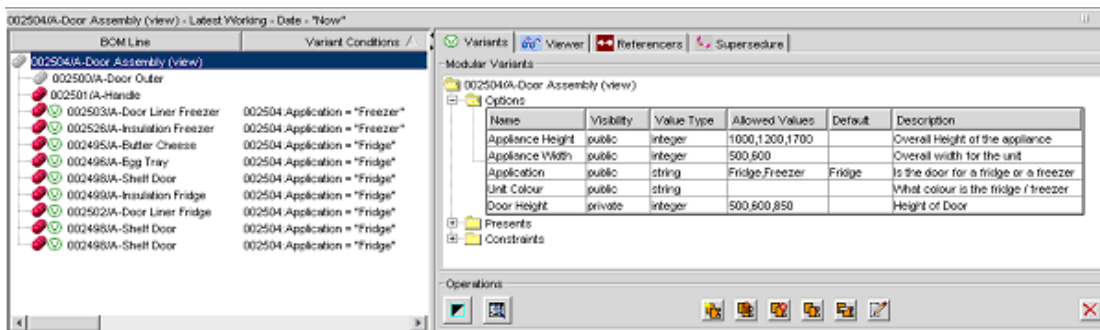
You can then configure the door assembly for a particular use in a refrigerator/freezer by setting various parameters or options that describe it, for example, **door width**, **door height**, **application** (refrigerator or freezer), and **color** (white or stainless steel). This intelligent door assembly is called a *module*.

The door module is completely self-contained and so can be reused in any product by setting its **public interface options**. The public interface options only control the features and components contained in the door module itself, and make no reference to options in higher or lower level modules. The modular variant functionality enforces this linking.

Note:

Teamcenter evaluates options from the top of the structure downwards, so the position of each module is significant.

The structure of the door assembly and its associated options is modelled as follows.



Structure of assembly and options

Configuring a public interface and private options

Public interface options can be set by the *customer* of the module. In the case of the door assembly, the customer is the design department that uses the door module. In the case of the refrigerator freezer module itself, the customer chooses from the public options made available at that level, for example, appliance width, appliance height, freezer at the top or bottom, number of shelves, and so on. Private options are used in constraints.

There may also be some internal options that are not set by the user, but are set according to the values of the public options. For example, the height of the door is set according to the appliance height, allowing for two doors, a 50 mm floor panel, and an LED display at the top. To allow for this, you create a *private* (hidden) option called **Door Height** on the refrigerator freezer module) that is controlled by the public option **Appliance Height** as follows:

$$\text{Door Height} = (\text{Appliance Height} / 2) - 50\text{mm}$$

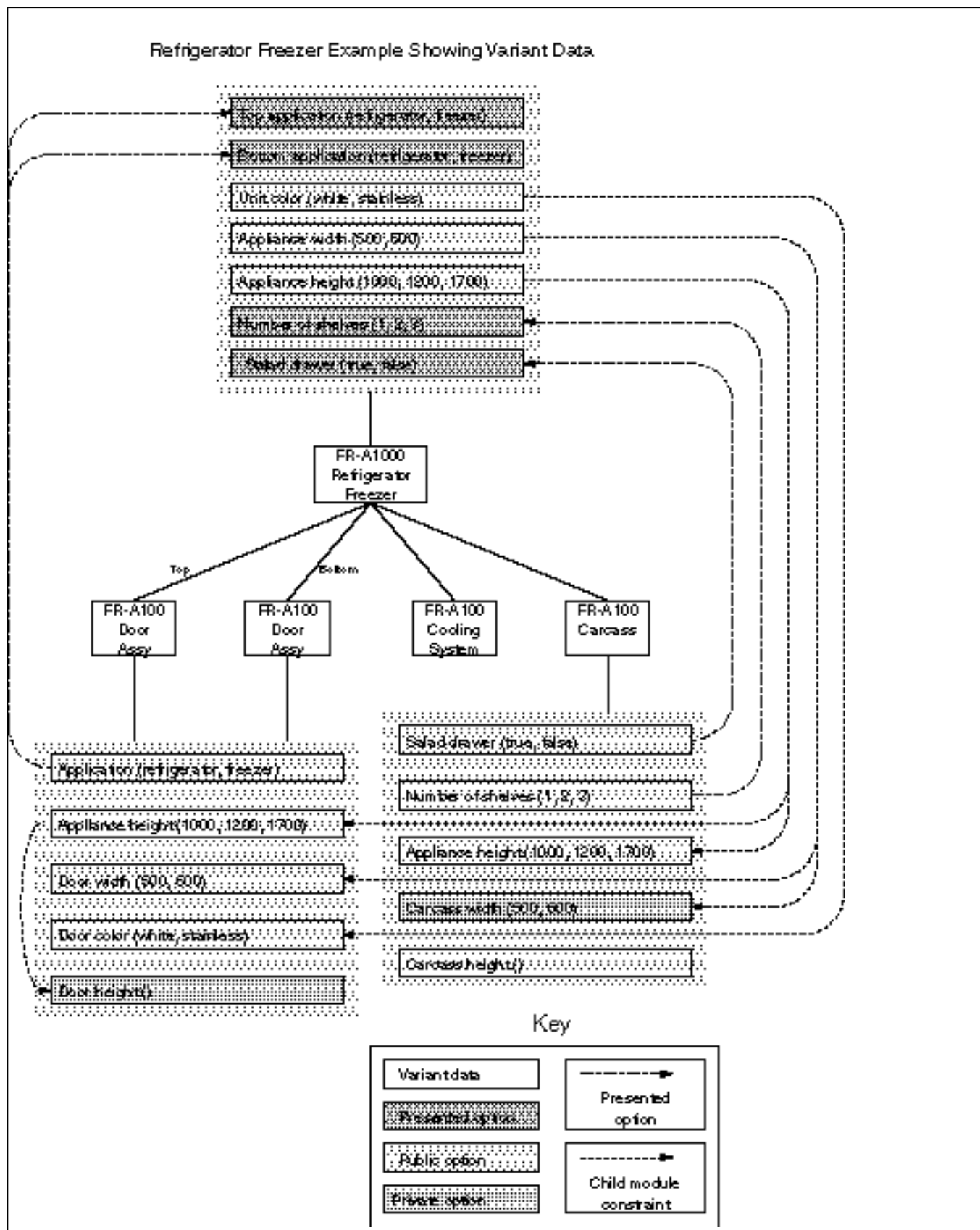
The **Door Height** option on the refrigerator freezer module is used to automatically set the **Door Height** option on the door module. You need not know this relationship, simply enter the overall height of the appliance.

You could use the same principle to define private options for all the assemblies of the refrigerator freezer and bring them together only in the top-level assembly.

Linking options between parent modules and child modules

To configure the refrigerator freezer appliance, you must set the options for each and every assembly. It may take a long time to set all the options, some of which are the same in all submodules. Also, if you consider the width, it would have to be set in the module for the carcass, the door, and the shelves. It may be preferable to set the options at only the top level module for the entire appliance and have this value propagate through the structure. To do this, you must create links between the child modules and the current module (the main refrigerator assembly). You can create these links in two ways, as shown in the following figure:

- Options from the child module are presented up into the parent module where they are visible and can be set.
- Options in the parent module can set options in the child module.



Presenting options

Note:

You cannot set the value of a private option from its parent public option. You can only set its public option in this way.

Presenting options

You can configure the same module differently, depending on where it is located.

Options can be presented to a parent module from a child module to make them visible as though they were part of the public interface for the parent module. For example, the **Number of shelves** option is presented up from the carcass module to the refrigerator freezer module, allowing you to choose the value at the parent level. You could also use the present method if you want to set options for different occurrences of the same module to different values, depending on their location.

You could place the door module in a refrigerator freezer assembly with this method. In this example, the same door module is used twice, once as a freezer door and once as a refrigerator door. The options created previously control the components of each door and the doors have different components due to their different functions. You can name each occurrence of the door appropriately, for example, **top** and **bottom**. You then present the **Application** option (with possible values **refrigerator** and **freezer**) for each door to the parent refrigerator freezer module. This gives two options to set in the refrigerator freezer module, one for each door; you can set these to different values.

Linking a parent module option to options on child modules

To set the width value, you could **present the options**. However, you must set the width option once for each door. As the doors are identical for any given appliance, it would be better to set this value only once. You could create an **Appliance Width** option in the top level module and link this single option to all width options in the child modules. When the **Appliance Width** is set in the refrigerator freezer module, it propagates to the carcass module, to both door modules, the shelves, and so on. The link is referred to as a *child module constraint*.

Linking public and private options

To enforce modularity, it is only possible to link options between a parent and its immediate child modules; you cannot skip a module level. To cascade down or up more than one level, you must repeat the linking process as necessary. Teamcenter enforces the following restrictions on linking:

- You can only present public options up from the child module. You cannot present them down from the parent module.
- You can only link options from the parent module to public options in the child module, although the option in the parent module can be public or private.

Creating variant items

A *variant item* is a specific variant of a completely configured module. For example, it could be a door assembly with: height of 500 mm, width of 600 mm, application of **refrigerator** and color of **white**.

Variant items are physical parts with no variability, and can be allocated an actual manufacturing part number. Conversely, modules cannot be manufactured. Modules with a large number of options and numeric options (with an infinite range unless allowed values are specified) have a correspondingly large number of permutations; not all of these permutations are manufactured. It is useful to be able to reuse permutations that have previously been sold, as significant engineering work may have been invested in creating technical documentation, drawings, and manufacturing data that is attached to the variant items.

The modular variants functionality allows you to create new variant items when they are required. You can also search for existing variant items to reuse in new products. Variant items are built up into a complete structure for a specific configuration corresponding to a product or customer order; in this structure all modules are replaced by specific variant items.

If you are using NX, you can use the option values on variant items to determine the expression values on an NX CAD part.

Caution:

If a module has variant items linked to it, you cannot change any of the variant data without revising the module. Such changes may invalidate existing variant items linked to the module revision. Disallowed changes include:

- Removing or adding an allowed option value.
- Removing or adding an option.
- Setting or editing a variant condition on a BOM line.
- Changing variant logic such as module constraints, defaults, and rule checks.

To make any of these changes, you must first revise the module. Even after revising, you can only add a new value to an existing option. You cannot add a new option because that invalidates any variant item linked to an earlier generic item or item revision.

You may not want to update existing variant items that are linked to previous module revisions. As an alternative, you could create a new variant item, but Teamcenter then contains two different IDs for variant items that have the same configuration and share the same stored option set (SOS) values; this arrangement does not adequately enforce modularity. The new variant item may also contain additional components and, if so, it would not be appropriate to update the existing variant item.

Caution:

Teamcenter does not support the creation of variant items in products that have both modular variants and classic variants, that is, hybrid mode products.

Creating modules for assemblies and piece parts

Modules for assemblies and piece parts

The modular variants functionality allows you to create modules for assemblies and piece parts.

Creating modular assemblies

You can configure the variant components in a modular assembly by applying a variant condition. For example, for the door module mentioned previously, create the following conditions:

On the **Door Liner Fridge**, **Insulation Fridge** and **Egg Tray** components:

```
Load if Application = fridge
```

On the **Door Liner Freezer** and **Insulation Freezer** components:

```
Load if Application = freezer
```

Note:

The options you can use in the variant condition are limited to the public and private options on the parent module. This limitation enforces modularity.

Creating modular piece parts

You can set options on a piece part to use with a CAD model in NX. You can map these options to an associated expression in NX that determines the geometry of the part; for example, you could map the **Width** option to the geometry of the door. When you create a variant item, Teamcenter copies the model (**UGMASTER** dataset), and automatically sets the option value to the associated expression value in NX, for example, **Width = 500**.

If you design the door assembly module of the refrigerator freezer as a modular assembly, having a different component for each width and height of door, this could lead to a very large number of components for each size of door. It is more efficient to design the piece part for the door itself as a module, with options for width and height. The variability is then catered for by the variant items, which are created on demand as required, without needing to modify the parent door assembly module to add components for every new size of door.

Defining options

What option types are provided?

Teamcenter allows you to create several different option types. You define options in standard variants only and string values are available. The following variant types are provided:

- String
- Integer
- Real

Note:

There are no restrictions on the precision of the value you can enter for real types. However, during evaluation, the value may be truncated to the system limitation.

- Logical (true, false)

You can use the `=`, `>`, `<`, `>=`, `=<` and `to` operators with real and integer options, both in the allowed values for an option and in variant conditions and constraints. For example, you can set the door width option to a range of values, such as **400 to 700**. This allows the user to set any value within the range, although there are discrete values allowed for standardization (500 and 600 only).

Defining global options

Global options are a convenient way of defining a fixed set of allowed values. Examples of global options are **Color = white, stainless** and **Appliance Width = 500, 600**. You can also set a range such as **Angle >=0, < 360**. You can then reference these global options in other modules.

Note:

The value configured is *not* the same in each module where the global option is used. Use external options if you require this effect.

The administrator specifies global options by adding the item ID that contains the global option definition to the **PSM_global_option_item_ids** preference. This preference lists the IDs of all items that contain global options, for example:

```
PSM_global_option_item_ids=
000400
000410
000420
```

Caution:

Do not manually modify global options that are defined in this preference.

Note:

Siemens Digital Industries Software recommends that you create the variant options and values after adding the option item to the **PSM_global_option_item_ids** preference when creating and

assigning variability. When creating variant options and values before adding the item ID to the **PSM_global_option_item_ids** preference, Teamcenter does not automatically assign variability and you must assign it manually using the **Variability** tab in Structure Manager.

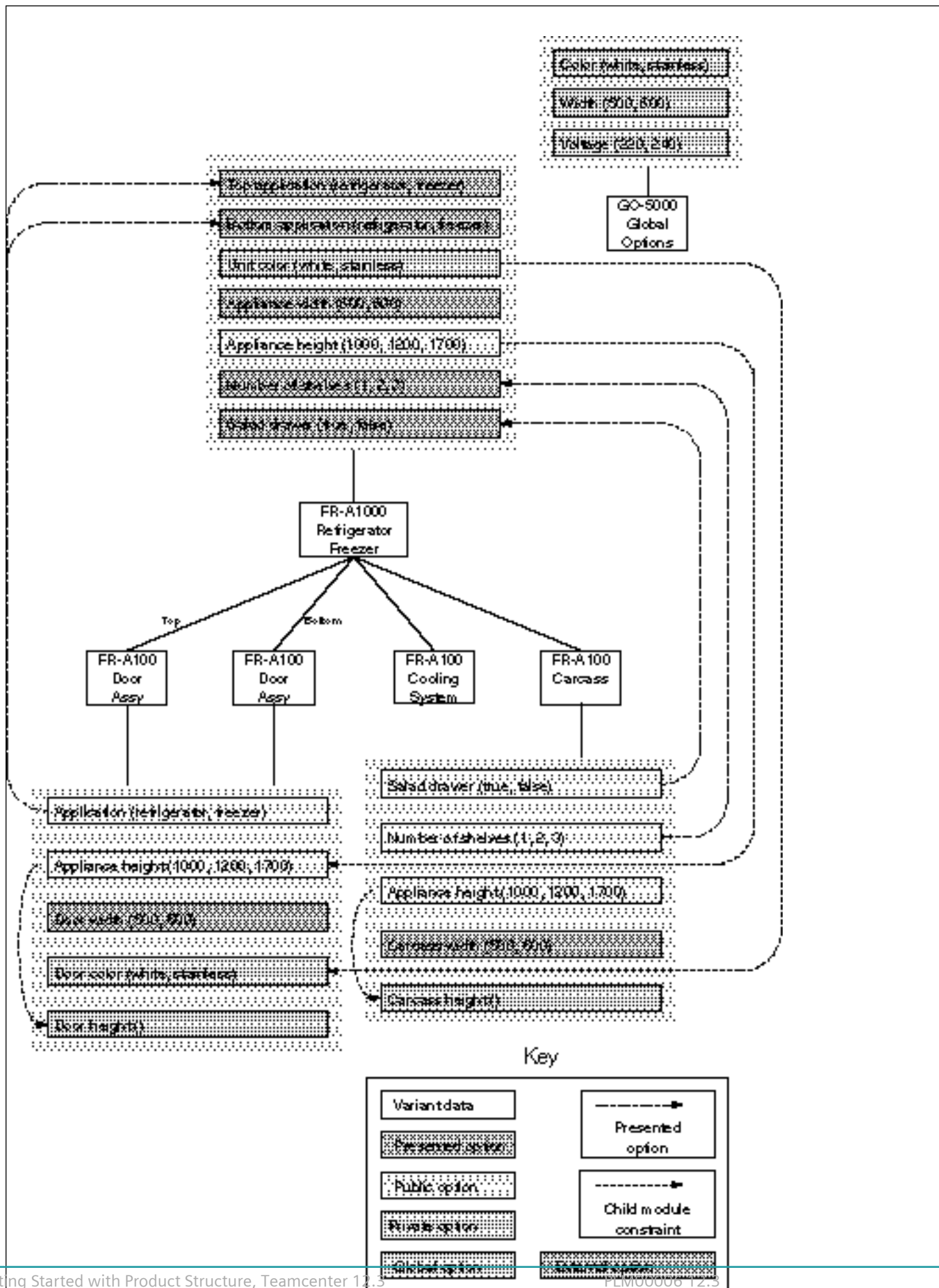
These definitions can then be reused when authoring variant modules. In Structure Manager, you can drop these global options into the module for which they are required.

To add items to the list of IDs, in Structure Manager, select an item and choose **Tools→Variants→Set/Unset Global Option Item**.

Defining external options

An external option is typically defined on the top-level module in the structure. Its allowed values are the same as those on the option that references it. In addition, the value set for the external option on the top-level module in the variant rule is propagated to all options that reference the external option.

The following figure shows an extension of the refrigerator freezer example that illustrates this concept.



Create an item called **GO-5000** to define global options. This item is a standalone module and contains three global options for color, width and voltage. Each such global option is specified with standard allowed values.

The color global option is used in the refrigerator freezer and door assembly modules so that the allowed values are controlled from the color master global option in the **GO-5000** module. To set the actual value configured for the unit color on the door module for some variant of the product, you must create a link between the unit color option in the refrigerator freezer and the option in the door assembly, as shown in the previous figure.

The appliance width external option is created in the refrigerator freezer module. It is used by the door width option in the door assembly module and by the carcass width option in the carcass module. Typically, the external option would reference a global option, but this is not mandatory. In this case, the value set for the appliance width for any variant of the refrigerator freezer is automatically propagated to the options that use it lower in the structure (namely door width and carcass width), without needing to create a link.

Setting option defaults

You can set default values for options in the following ways:

- In the option definition. This is the most common and visible place to set a default.
- Using a module constraint to set a default value.
- Using a module constraint to fix a default value that cannot be overridden by the user.

In the refrigerator freezer example, for some models, the freezer may always be at the bottom and the refrigerator at the top. In this case, the value for the application option would be fixed for each path in the refrigerator freezer module.

Note:

When using a global option definition, you can override a default set in the global definition in the module in which the global option definition is used.

Checking for errors

You may not want to allow all possible permutations of a module. For example, in the refrigerator freezer, the top application and bottom application cannot both be set to the same value of **Refrigerator**. You can use error checks to warn about unsupported combinations, by displaying a message when the user attempts to set option value combinations specified in the error check. You can define one of the following messages to warn of unsupported combinations:

- Error
Teamcenter displays the error message, and the user is not allowed to configure the combination specified.

- **Warning**
Teamcenter displays the warning message, but the user may set the combination specified.
- **Information**
Teamcenter displays the information message, and the user may set the combination specified.

Note:

Create error checks at the appropriate module, typically at the top level. Do not create them on global items.

Configuring a structure

When you have defined the option modules, configure the structure to simulate the choices that the end user or customer is offered. This process ensures that the structure is correctly and completely configured. The end user or customer can only set values for options on the product itself; for example, for the refrigerator freezer; they cannot set options in lower level modules.

Before doing this, you must design all the lower level modules (door assembly, door, carcass and so on) such that they are completely configured by whatever combination of options is set in the refrigerator freezer module. You should do this following the methods previously described, namely:

- Options from the child module should be presented up to the parent module.
- Options in the parent module may set options in the child module.
- Option defaults should be set where appropriate.

Note:

Checking that the refrigerator freezer product is completely authored is a manual process.

Setting expression values

Modular variants support the capability to set an expression value in NX from an associated option value in Teamcenter; this capability is not available in classic variants. Make the association simply by ensuring the expression name in the NX part is the same as the option name on the item module; the **UGMASTER** dataset for the part must be defined in this item module.

You can use options to set any expression. In a piece part, you typically use expressions for geometric parameters such as length or the number of elements in an array of features. In an assembly, you typically use option values to control position, for example, by determining the parameters of a mating condition such as the linear or angular offset.

Best practices when designing for modularity

To make the best use of modular variants, create your basic high-level design with variant modules. This consideration is fundamental and implies that it may not be practical to incorporate variant modules in existing products whose structure is not designed for modularity if large structure changes are required. Your approach to determining modularity depends on your business practices, but an example follows:

- Marketing decides the features to be offered, in collaboration with Engineering, and their approximate cost. Features may be packaged into groups. They may also depend on the region or country where the product is sold. Modules can be defined according to these groups and region or country considerations.
- Engineering decides the *technical* features to offer with the product, deriving this information from the marketing and commercial decisions. It then decides what modules are required to implement these features and designs the product structure accordingly.
- Engineering translates the features agreed with Marketing into variant data on the modules (options and constraints) and components (variant conditions).
- Technical sales staff create variant items as sales orders are received. Teamcenter stores data such as drawings for a particular variant, and this data may be transferred to the manufacturing environment when complete.
- Release the module revision before you create variant items for it.
- Do not exclude any assembly levels from the modules if you want to create variant items.

9. Configuring structure with branch definitions

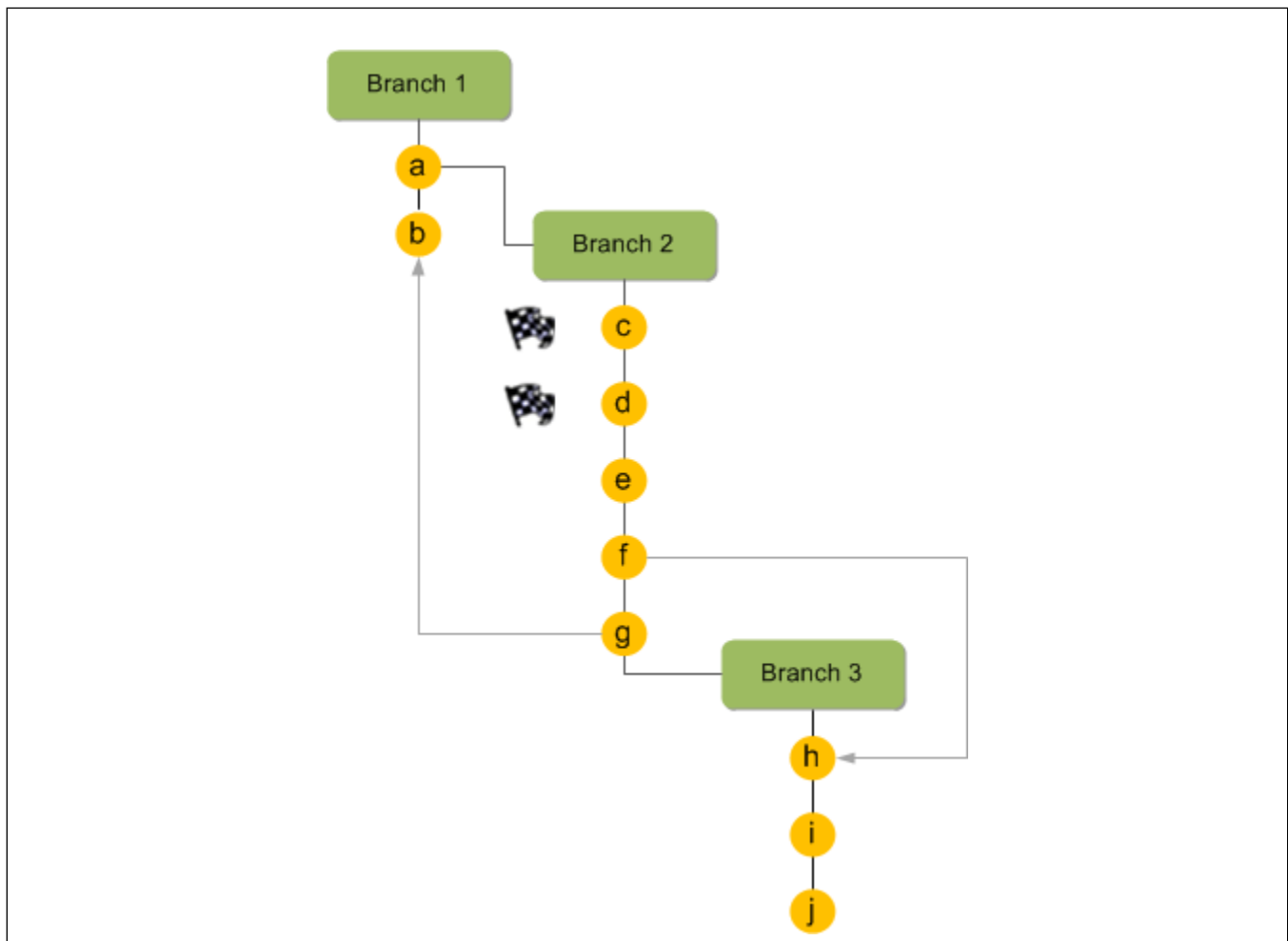
Using branch definitions to configure product structures	9-1
Managing single branches	9-4
Managing branch chains	9-5
Create a branch	9-6

9. Configuring structure with branch definitions

Using branch definitions to configure product structures

Branches provide a mechanism of grouping multiple working item revisions with certain intent. Branch definition is a paradigm similar to that employed by software configuration management systems to define node or release content. A branch definition identifies the contents of a structure in the context of its business purpose.

One use case for creating branches is to track virtual prototypes in a preproduction environment. A virtual prototype represents a specific configuration of a product platform. The variants chosen represent the product intent of the platform and therefore are a repeatable configuration. As the product definition matures, the virtual prototype is used to track the maturity of the product definition, perform analyses, compare data, and construct reports. In practice, a development program can be defined as a progression of virtual prototypes where each iteration can be constructed from only the changes to the previous virtual prototype. Different virtual prototypes may be used for the same platform. They may be used to collaborate towards key milestones or prototype build events.

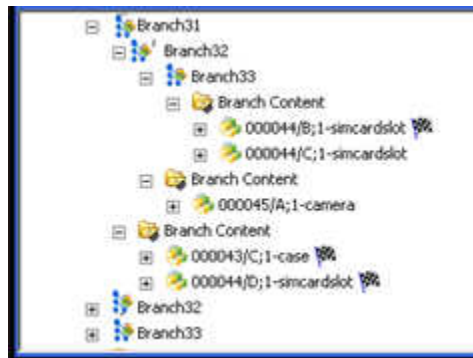


Typical branch configuration

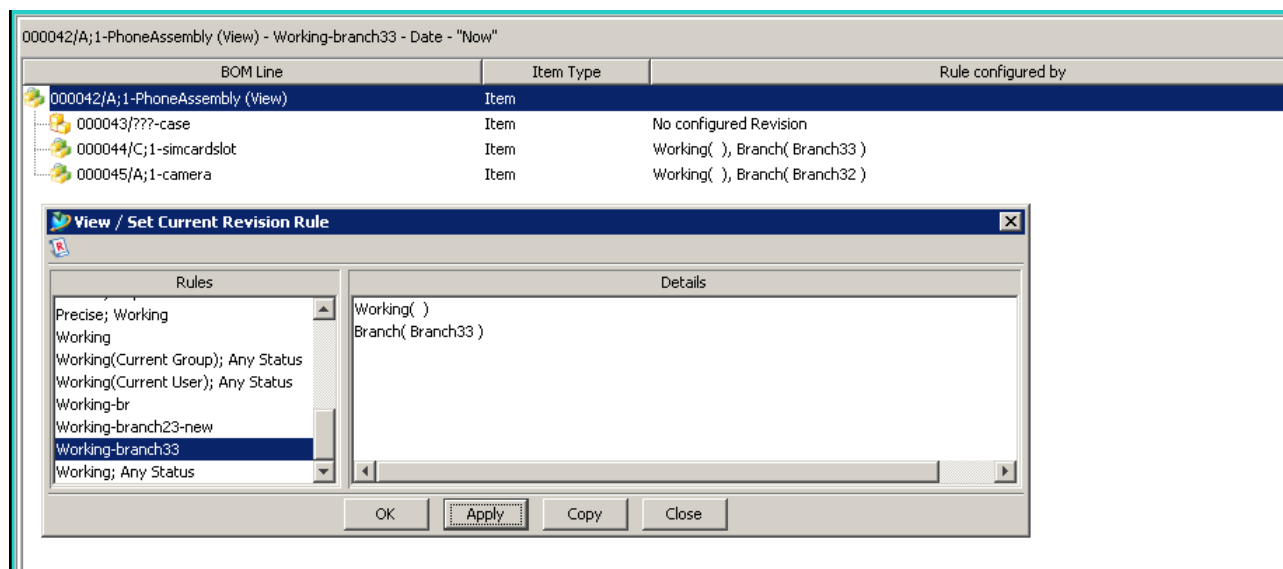
This diagram shows how you can use branches to configure a specific revision with intent-based parameters. The item revisions a through j are split into different branches or intent sets. For each intent set (that is, revisions associated with a certain intent), Teamcenter configures revision candidates by revision rule parameters such as release status, effective date, and unit number. There may not be a valid candidate for any of the relevant intent sets. If there is a valid candidate, Teamcenter considers it, but the precedence and relevance of intents in the intent configuration specification principally determine which revision is configured.

You can specify a branch definition in a revision rule, and Teamcenter configures the structure to the branch definition as well as any effective predecessors. It fetches all effective branches in the hierarchy up to the root. It then considers effective item revisions in each branch for revision configuration. Revisions in the specified rule branch are considered first, and then revisions in any predecessor branches. If none of the revisions of the item participate in the branch hierarchy, it is marked as an unconfigured revision. If revisions of the item participated in the branch hierarchy, but none of the revisions are configured with other revision rule entries, it is marked as an unconfigured revision. Contents of all immediate parents at the same level are considered for evaluation.

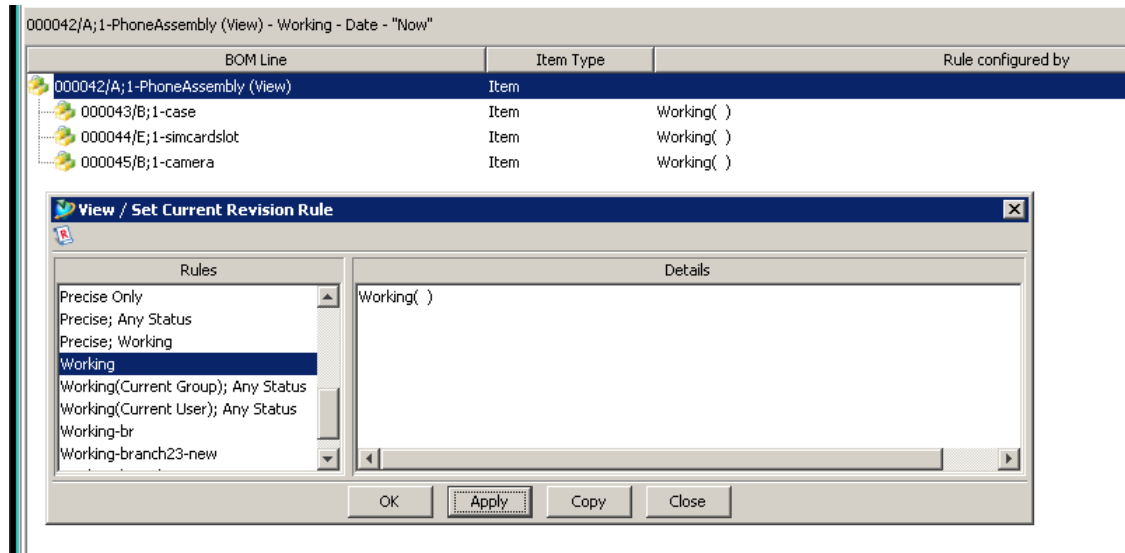
In the following example, a branch chain is created with three branches. Content is pasted into branches 31, 32 and 33.



If you then configure the assembly in Structure Manager with a revision rule that includes a branch definition, the branch content is considered. The following example contains a structure that is configured with a **Working()**, **Branch 33** revision rule. The **simcardslot** item revision is configured with branch 33 content, the camera is configured with branch 32 content, but the case is not configured because the working revision is not available in any of the branches in the branch chain.



In contrast, if you configure the structure with a **Working()** revision rule and do not take into account the branches, the configured structure is as follows.



By default, branch entries are not available for creating revision rules, and this capability must be enabled by setting the **PSEEnableBranchConfiguration** site preference.

You create a branch and then paste in the content of the branch in My Teamcenter. That is, you paste in the item revisions that you want to be configured by this branch. Once created, branches may be added to revision rules that are applied in Structure Manager.

Managing single branches

When managing single branches, you can do the following:

- Control access to branch definitions.
You can restrict the ability to create, modify, or delete branch definitions to identified users with access rules and Command Suppression. Access restrictions on branch definitions are honored during revision configuration, for example, unreadable content in branch definitions is not considered for revision configuration.
- Identify content for branch definitions.
Identify item revisions for branch definitions. Available item revisions are listed in a pseudo folder under the branch definition. Duplicate item revisions in the same branch definition are not permitted.
- Mark branch definitions as active or inactive.
By default, branch definitions are active at the time of creation. You can change the state of a branch definition to inactive at any time. If a branch is in an inactive state, no item revisions can be added to or obsoleted from it. Item revisions in a branch definition are considered during revision configuration, regardless of its state; the state of the branch does not affect any action on any of its item revisions.
- Add an item revision to a branch.
You add an item revision to a branch definition by copying it, and then pasting it with a **Branch Content** relation. After the successful addition of the item revision to the branch, the start date on

the **Branch Content** relation is set to the creation date of the relation. The item revision is effective in the branch definition from this start date onwards or until the end date if one is specified.

- Obsolete item revisions from a branch definition.
To obsolete item revision from a branch definition, set an end date on the **Branch Content** relation. The item revision is effective in the branch definition from the start date to the specified end date.
- Delete branch definitions.
You can only delete a branch definition if it is not referenced by a revision rule or by another branch definition.

Managing branch chains

You can organize branches in a hierarchical way. A branch definition may itself configure 100% content of the structure, or work together with related branch definitions in the chain to configure the full content. A branch definition chain is an ordered set of branch definitions that may configure the structure, with the contents of the later branch definitions having precedence over earlier branch definitions.

For example, a pen assembly has four components and participates in a branch chain, where branch 1 is used by branch 2 and branch 2 is used by branch 3. The results is as follows.

Pen assembly item	Pen assembly item revision	Branch 1	Branch 2	Branch 3
Body	Body;A	x		
	Body;B		x	
	Body;C		x	
Cartridge	Cartridge;A	x		
	Cartridge;B			x
Point	Point;A	x		
	Point;B		x	
	Point;C			
Label	Label;A			x

(x indicates the revision is part of the branch definition.)

When managing branch chains, you can:

- Create or add to a branch chain.

You can link branch definitions to form a hierarchical chain. Any number of branch definitions can be chained in a hierarchical (tree) structure. You can insert a branch definition at any place in a chain. A branch definition is added or inserted into another branch definition by copying it and then pasting it with a **Branch Relation** relation. After successful addition of a branch definition, the start date on the **BranchRelation** relation is set to the creation date of relation. The branch definition is effective in the target branch definition from the start date onward or until the end date if one is specified.

By default, a secondary branch cannot have more than one primary branch, but you can manually override this behavior to allow the multiple parents for a branch. Multiple parents must be siblings in a hierarchy.

- **Obsolete a branch definition from a branch chain.**
To obsolete a branch definition from a branch chain, set an end date on the **BranchRelation** relation. The branch definition is then effective in from the start date to end date. Item revisions under an obsolete branch definition are not considered during structure configuration evaluation whether the cancelled branch is implicitly or explicitly specified in the revision rule.
- **Browse branch chains.**
You can browse the branch definitions used by a specified branch definition. Such branch definitions are listed in a pseudofolder under the branch definition.
- **Break an existing branch chain**
You can break a branch chain, for example, to reorganize it by adding or removing branches using cut and paste commands.

The effective branch that configured in an item revision is identified in the **How Configured** BOM line property.

Create a branch

1. From the navigation pane, click **My Teamcenter**.
2. Select a folder and then choose **File→New→Other**.
My Teamcenter displays the **New Business Object** dialog box.
3. Select **Branch** and click **Next**.
My Teamcenter displays the **Object Create Information-Branch** dialog box.
4. Enter a name and (optionally) a description for the new branch. You should also choose whether to make the branch active once it is created. By default, the contents of the baseline are associated with it using the **Contents** relation, but you can change this if other relations are listed.

Note:

When a branch is inactive, no content can be associated with it. That is, you cannot paste an item revision or another branch on an inactive branch. However, an inactive branch is still

available for structure configuration. You can activate or deactivate a branch at any time by left-clicking it, choosing **Edit Properties**, and then changing the state.

When the properties are shown correctly, click **Finish** and Teamcenter creates the branch in the selected folder.

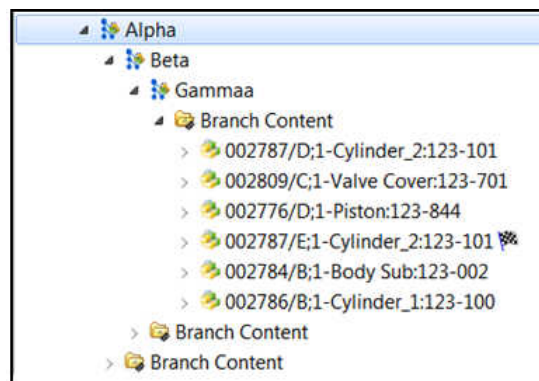
5. (Optional) Set an effective period for the branch by right-clicking it, choosing **Properties on Relation**, and then setting a start date and an end date. By default:

- The start date is the date you pasted the branch under its predecessor.
- The end date is empty, that is, the effectivity of the branch is open ended.

Likewise, you can specify effective periods for any of the item revisions in the branch.

You can perform revision configuration of structures in Structure Manager based on the defined branches. New branch entries are added to the revision rule entry list, allowing you to specify the branch in a revision rule. You can specify only one branch definition in a revision rule.

You can add contents to a branch in My Teamcenter by copying the relevant item revisions and pasting them on the branch, as shown in the following example.



Teamcenter configures each of the item revisions that you paste on the branch when you apply a revision rule in Structure Manager that includes the branch.

You can also create *branch chains* by pasting a branch under another branch. In the following branch chain, branches 03 and 04 are pasted under branch 02 and inherits all the branch content (item revisions) of branch 03 and branch 04. You can still paste unique item revisions under branches 03 and 04 if required.



Part III: Architecture

Using occurrences and absolute occurrences	10-1
Using Platform Designer	10-1
Aligning structure	10-1
Using collaboration contexts	10-1
Manufacturing product structures	10-1
Using allocations	10-1

10. Using occurrences and absolute occurrences

Understanding occurrences and absolute occurrences	10-1
Understanding occurrence terminology	10-1
Using occurrences	10-2
Using occurrence groups	10-2
Using absolute occurrences	10-3
Understanding absolute occurrences	10-3
Tracking changes to absolute occurrences	10-8
Best practices when using absolute occurrences	10-10
Using arrangements	10-10

10. Using occurrences and absolute occurrences

Understanding occurrences and absolute occurrences

When you add a component to an assembly, you create an *occurrence* of the item or item revision in the assembly. You can use *absolute occurrences* to manage data that is unique to a specific use of an item within a product structure; absolute occurrences exist in a specific context in the structure.

Understanding occurrence terminology

Term	Meaning
Absolute occurrence	A relationship between a <i>parent</i> assembly and an item one or more levels lower in the structure. The parent assembly is the <i>context</i> in which the absolute occurrence exists. You can define data on the absolute occurrence that overrides the data stored on the item when you select the context assembly and view the structure. Both relative occurrence data (notes and transforms) and attachments can be overridden with data on absolute occurrences. Each absolute occurrence can have one or more unique attribute values that distinguish it from the other absolute occurrence derived from the same single occurrence.
Arrangement	In NX, an arrangement specifies alternative positions for one or more components in an assembly.
Appearance path node (APN)	See <i>occurrence path</i> .
Instance	A specific node in the product structure relative to the top level. In a multilevel structure, a complete path to the node is associated with the instance.
Occurrence (relative)	(Sometimes called <i>relative occurrence</i> .) A hierarchical structure relationship between the immediate parent assembly and its child component item (in an imprecise assembly) or item revision (in a precise assembly). You can use a find number to identify an occurrence, but this number may not be unique. Data can be stored on the occurrence, including occurrence notes and transforms.
Occurrence group	A collection of nodes relative to the top-level item from any level in the base product structure. An occurrence group is used to create a tightly linked derived view. See also <i>view type</i> .
Occurrence path	An occurrence path maps an occurrence to the corresponding BOM line. It is unique to the context of a specific BOM; different BOMs

Term	Meaning
	cannot contain the same occurrence paths. Also called an <i>appearance path node</i> (APN).

Using occurrences

A product structure consists of nodes related to items (if the structure is *imprecise*) or item revisions (if the structure is *precise*). When the occurrence is of an *item*, Teamcenter configures the structure using a revision rule that selects the specific item revision by date or unit number; if the occurrence is of an *item revision*, this configuration is not necessary. Each item revision has a BVR for the instances that contains the item revision's children; those children are also occurrences of items or item revisions.

An occurrence (relative occurrence) is a persistent object that represents the usage of an item in the context of an immediate parent assembly. It includes unique information about the relative position of the item with respect to its parent assembly. The same part may appear several times in the structure, but each occurrence of the part is unique (called the *instance*). For example, the occurrence of the left front wheel of a vehicle differs from the occurrence of the right front wheel, even though the actual wheels are identical.

For each occurrence, an occurrence path maps the occurrence to the corresponding BOM line. An occurrence path is unique to the context of a specific BOM; different BOMs cannot contain the same occurrence paths. If changes are made to the structure, only removals, substitutions and occurrence note changes are propagated; additions are not propagated. If an assembly level component is removed, all relevant occurrence paths in the occurrence group are removed. If an assembly is added, you must generate occurrence groups for all components.

Using occurrence groups

You can combine multiple occurrences into occurrence groups to collect related parts. An occurrence group may also collect all occurrences in the product that are within a given proximity of a part that you are designing, as the result of a proximity search, if appearances are configured. You can also build occurrence groups of occurrences that are functionally related.

Occurrence groups are implemented by an internal mechanism called *appearance path nodes* (APNs). Although this mechanism is hidden from the user, you should understand it to comprehend the behavior of occurrence groups.

When you copy a component to an occurrence group, Teamcenter creates a persistent APN object that represents a BOM line in the product structure, for example, the right, front wheel. This is independent of the revision of the component in the base product structure. APNs are attached to the occurrence thread, not the occurrence itself, and consequently a revisionless parallel structure of APNs is created in the occurrence groups of the derived views. APNs are always created in the context of a specific top level root item, by default the current top level at the time you create the occurrence group.

You can view the top-level occurrence group under the item in My Teamcenter (not the item revision). However, the occurrence groups attached to it are not visible in their corresponding component items.

You create APNs when you do any of the following:

- Create occurrence groups with Multi-Structure Manager.
- Create an absolute occurrence to override data (for example, transform) in Structure Manager or Manufacturing Process Planner.
- Consume a part in an operation in Manufacturing Process Planner.
- Assign a BOM line in one structure context to a BOM line in another structure context to share an absolute occurrence and its data override. You perform this action in Multi-Structure Manager or Manufacturing Process Planner.

Using absolute occurrences

Understanding absolute occurrences

You can use absolute occurrences to manage data that is unique to a specific use of an item within a product structure. If you have ownership of and write access to a particular BOM view revision, you can define a subset of BOM line properties that other users can edit in the context of the BVR. You can also allow users to attach forms or datasets in context, by configuring business rules that define the types of primary and secondary objects for the relationships.

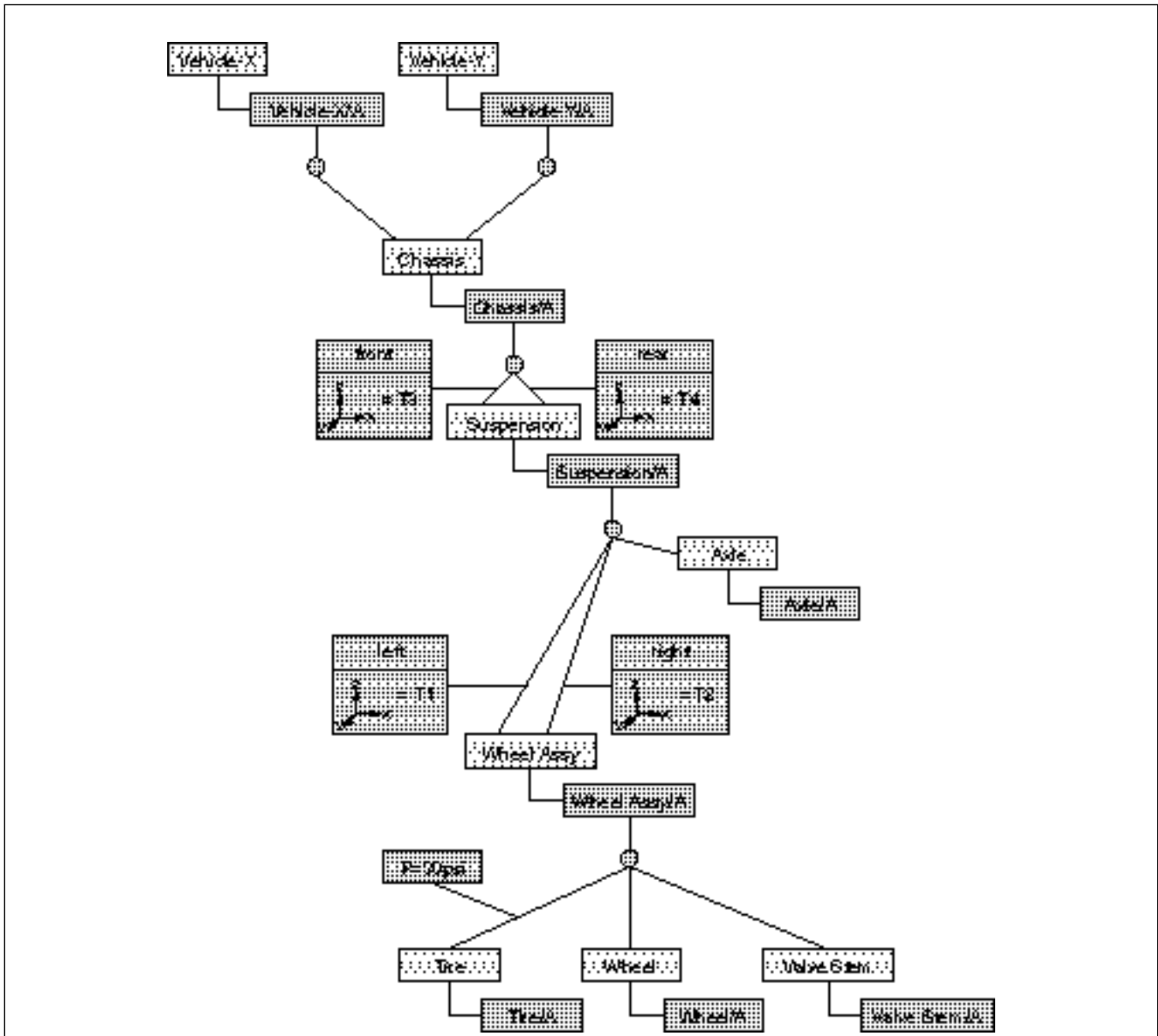
When a user enters in-context values or attachments, Teamcenter *overrides* certain structural data associated with a relative occurrence in the context of a specific higher level assembly. Data you can override includes:

- Occurrence notes
- Occurrence type
- Quantity
- Absolute transform
- Find number
- Variant condition
- Attachment, for example, a JT file for a different rendering
- Component item for a replacement

A BOM line represents an absolute occurrence with respect to the current top line in the product structure. You can set occurrence attribute values on such a BOM line, without those values appearing in the product structure everywhere the BOM line's parent appears. There are cases where a user needs to

store values specific to the BOM line with respect to the top line or some intermediate assembly between the BOM line's parent and the top line. Absolute occurrences are always created in the context of a selected top line; the same absolute occurrence may not be in context for another top line.

The following figure shows an example where two vehicles are built on the same chassis. The chassis contains two occurrences of a suspension system, one for the front of the vehicle and one for the rear; these occurrences are positioned relative to the chassis by transforms T3 and T4, respectively. The suspension assembly contains two occurrences of a wheel assembly, one for the left side of the suspension system and the other for the right side of the suspension system. The wheel assembly occurrences are positioned relative to the suspension assembly by transforms T1 and T2, respectively. The wheel assembly is composed of a wheel, tire, and valve stem. The occurrence of the tire is annotated with a recommended tire pressure of 30 PSI.

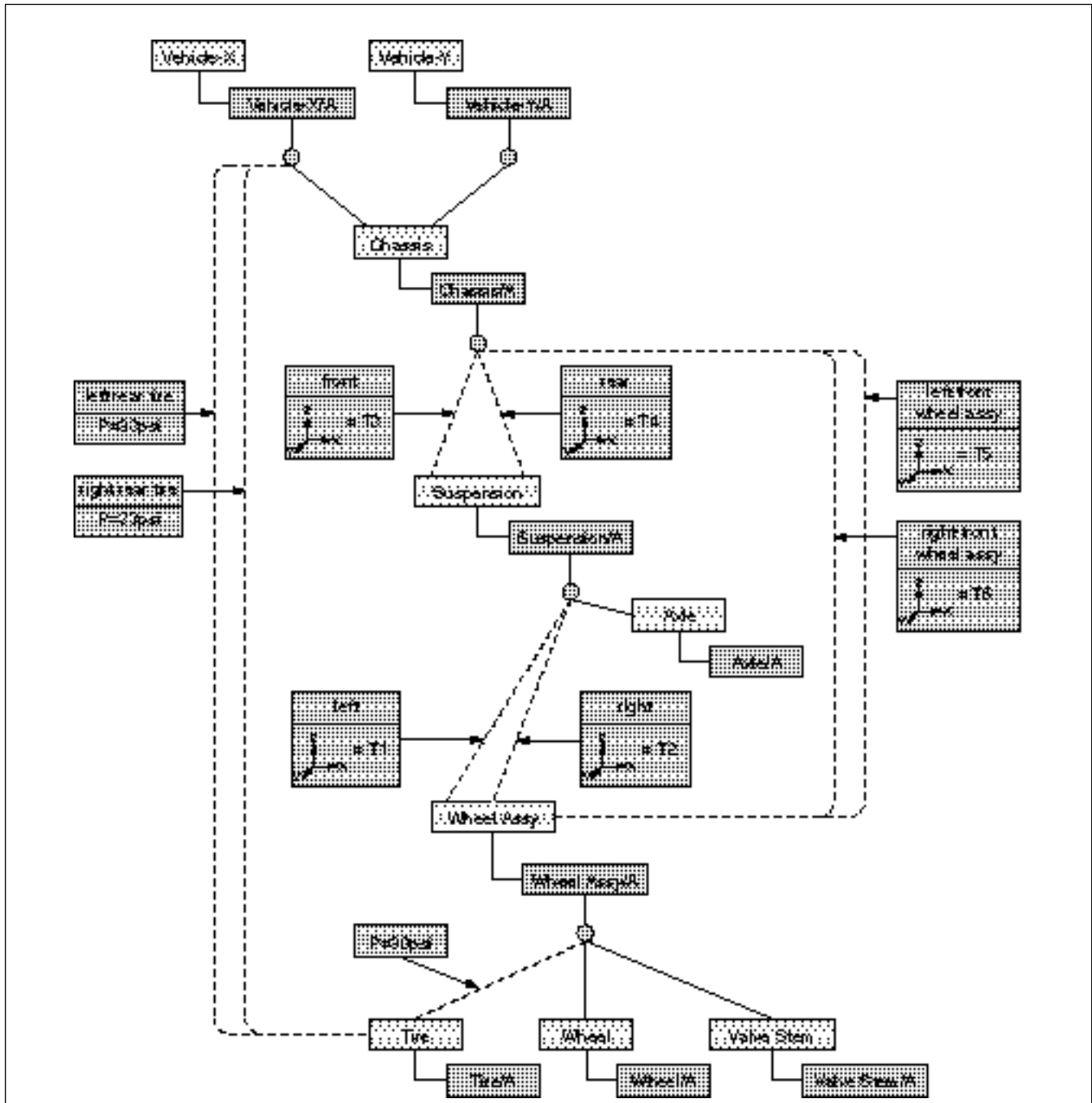


Sample product structure with relative occurrences only

Relative occurrences correlate directly to structure relationships. If the structure relationship has a quantity associated with it, the number of relative occurrences associated with the structure relation equals the quantity.

In the previous figure, when you display the product structure for vehicle Y, every occurrence of the tire shows a recommended tire pressure of 30 PSI; that is, the value set on the occurrence of the tire in the wheel assembly is propagated to all BOM lines for the tire. Positioning information is derived in the same way. The position of the wheel assembly in a vehicle is derived from transformation values specified on the (relative) occurrences in the product structure, for example, the position of the front left wheel is derived by multiplying T1 and T3.

Consider a case when the position of the front wheels must change so they toe-in by 0.5 degrees or the case where the recommended tire pressure for the rear-tires in vehicle X must change to 33 PSI. One way to accomplish this is to modify the structure, but in most situations, this approach is not practical. Instead, you should make context-specific modifications to BOM line properties without changing the product structure. To permit such context-specific modifications, you can create *absolute occurrences*. Creating absolute occurrences allows you to effectively *flatten* the representation of a structure while knowing how to map back to the actual product structure. The following figure illustrates this concept, expanding on the preceding example.



Sample product structure with absolute occurrences

The dashed lines in the figure represent absolute occurrences. In this case, the absolute occurrences specify or override values in a specific context. In the example, they override positioning information and recommended tire pressure. The context of the override and the value is shown in the label on each dashed line.

Specifically, in the context of the vehicle-X, the front tires have a recommended tire pressure of 30 PSI; this is derived from the relative occurrence of the tire in the wheel assembly. However, the two rear tires

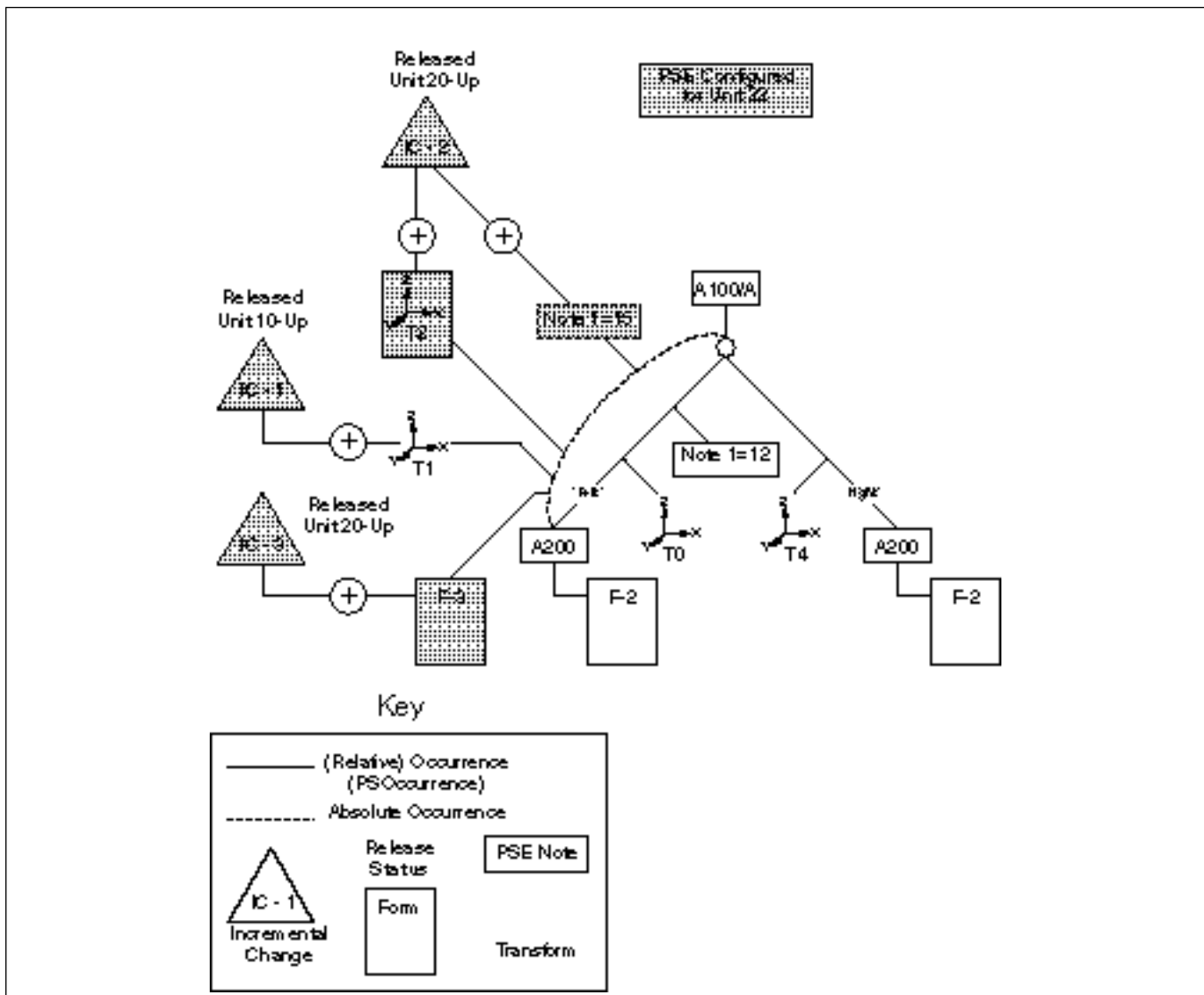
have a recommended tire pressure of 33 PSI; this is explicitly set on the absolute occurrence in the context of the vehicle. The value of 33 PSI on the absolute occurrence overrides the value of 30 PSI that appears on the relative occurrence of the tire in the wheel assembly. Vehicle-Y, however, has a recommended tire pressure of 30 PSI for all four tires.

Positioning information is derived in a similar way. The values specified on the absolute occurrence override the values that are otherwise derived by catenation of the transforms from the relative occurrences. That is, in the second figure, T5 overrides the value that would otherwise be derived from the multiplication of T3 and T1. Because this example specifies the transform overrides in the context of the chassis, both vehicle-X and vehicle-Y use T5.

If you release an intermediate BOM line that represents a subassembly, the line is locked and you do not have write access to it. However, you can still make changes to its absolute occurrences in the context of a parent assembly whose BOM view revision is also released, for example, to attach a dataset with associated JT files.

Tracking changes to absolute occurrences

The following figure shows how you can track changes to absolute occurrences with incremental changes.



Tracking changes to absolute occurrences

The possible configurations of occurrence data on A200 (left occurrence) in assembly A100 are as follows:

- Units 1 through 9: No incremental changes configured. Original occurrence data: Transform=T0, Note 1=12, Form F-2.
- Units 10 through 19: Incremental change 1 configured. Transform=T1, Note 1=12, Form F-2.
- Unit 20 and above: Incremental change 1, incremental change 2 & incremental change 3 configured. Transform=T2, Note 1=15, Form F-3. Form F-2 is still attached to the right occurrence of A200.

There is no **Remove** change on the occurrence data, for example, for the transform or note 1. You simply define a new value that overrides the value on the normal relative occurrence. For example, the new value may define an end unit for an incremental change.

Best practices when using absolute occurrences

Some suggestions for managing absolute occurrences follow:

- Populate absolute occurrences only when necessary. Creating absolute occurrences creates multiple instances of the same object in the database.
- Allow only specialists or *power* users to create absolute occurrences due to their complexity.
- Create only a limited number of contexts for absolute occurrences, for example, the top level item of the structure which may represent the tail number of a particular aircraft. If you have too many contexts, occurrence references are difficult to manage.

Using arrangements

In NX, you can define *arrangements* to specify alternative positions for one or more components in the assembly, and store those alternatives with the assembly. For example, a car door assembly may have different arrangements when the door is open or closed and the position of certain components may be overridden accordingly. Likewise, you can suppress components in a particular arrangement, for example, to hide the hinges. When you manage an assembly with arrangements in Teamcenter, each component in NX has a corresponding occurrence in Teamcenter and each different override or suppression has an equivalent absolute occurrence.

Note:

You cannot create an arrangement for a piece part.

11. Using Platform Designer

What is Platform Designer?	11-1
Purpose of Platform Designer	11-1
Platform Designer principles	11-1
Scenarios for using Platform Designer	11-2
Platform Designer terms	11-2
Set up Platform Designer	11-5
Understanding data storage	11-9
Storing Platform Designer data	11-9
Storing product family data	11-10
Storing product-specific data	11-11
Applying the variability and variance	11-15
Analyzing variability and variance	11-15
Using shared or unshared NVEs	11-15
Creating a full or partial breakdown	11-16
Preventing overlapping NVEs	11-17
Using hierarchical variability	11-17
Choosing variability settings	11-18

11. Using Platform Designer

What is Platform Designer?

Platform Designer provides a mechanism to plan and control the evolution of product structures at your site by building generic structures of your products that contain a high degree of variability. Platform Designer uses architecture breakdowns to create and disseminate design content and product variability.

Purpose of Platform Designer

Platform Designer principles

In a typical organization, only a very few employees understand and control the variant data in the business systems. CAD designers and part engineers are not expected to author the correct variability without guidance, although they must be aware of the variability their design supports. Platform Designer allows configuration experts to define the variability in Teamcenter against a known existing nomenclature with which designers are familiar. This scheme is referred to as the *product architecture* and each company has a unique taxonomy for defining their generic parts or modules. These generic parts are placeholders for designs and parts that may not yet exist. Examples include:

- **Car (0) - Body (5.0) – Doors (5.1) – Left, Front Door(5.1.3)**
- **Printing Press (0) – Front Folder (2.0) – Main Roller (2.2)**
- **Computer (0) – Output Device (1.0) – TFT Screen (1.3)**

Platform Designer helps create these architectures in Teamcenter. Platform Designer allows the variability of a product family to be defined at the concept planning stage before any designs have been started. This is in contrast to classic variants or modular variants, with which the variability is defined directly on the (CAD) product structure.

Variability is defined as a set of global options, each with a set of allowed values (for example, **color = blue, silver, white**) that are associated with the complete product family, for example, **Car Model X, Engine Type Y, or Free Standing Refrigerators**.

Once this base variability is defined for the product family, new products (for example, **Car Model X, Model Year 2012**) can be defined more quickly by using the base product architecture and variability as a template, then copying the subset of generic modules and global option values that apply to the new product. This product-specific architecture (called an *architecture breakdown* in Platform Designer) is now used to specify the variance, that is, the option combinations that will be offered for each generic module, typically at the lowest level modules.

CAD designers and part engineers define design and part solutions that satisfy the variance criteria, then associate them with the appropriate module, specifying the applicable variance. This results in a CAD structure that may have different variant configurations applied, for example, when performing digital

mockup analysis or interference checking. The part solutions are contained in the product-specific architecture structure itself and may have variant configurations applied to display the configured parts, allow cost roll-ups, and so on.

Thus Platform Designer ensures that the CAD and part configurations are consistent as they are based on a common architecture and source of variant data.

In the case of large complex products such as a car, it is likely that the variability is authored in external business systems, and the global options and product architecture are populated via an interface. For smaller products, the variability can be authored in Platform Designer and subsequently exported to the ERP or other business system.

Scenarios for using Platform Designer

Platform Designer supports several basic processes:

- **Defining design solutions**
This allows CAD designers to add their new design items to the CAD structure with the correct variant conditions. The CAD designer is guided in applying the correct variant logic (named variant expression) to use for the variant condition by selecting the appropriate module (architecture breakdown element).
- **Defining part solutions**
This allows part engineers to associate released parts with the appropriate node in the architecture breakdown. When making this association, the part engineer must apply the correct variant logic (named variant expression), depending on what is defined on the selected module (architecture breakdown element).
- **Acting as a bridge between a business system and Teamcenter**
Often, a company has a very large amount of variant data held in external business systems that is required to plan a product. Platform Designer can act as a bridge between these systems and Teamcenter.
- **Authoring product architectures to enable reuse**
Parts are classified functionally (for example, by manufacturing process) in the product-specific architecture breakdown, it is easier to reuse parts by looking at the generic breakdown elements.

Platform Designer terms

The following table describes the objects, terms, and concepts that you use to create and manage architecture structures.

Term	Meaning
Architecture breakdown	One of two types of product-specific architectures. If you choose to use an architecture breakdown to reflect the product-specific architecture, then the product structure relationships in the product-specific

Term	Meaning
	architecture must be identical to those found in the product architecture template. The architecture breakdown maintains a dependence from the product-specific architecture to the template architecture. This is an important characteristic that must be decided during initial definition of a product-specific architecture. The other type choice is a <i>generic breakdown</i> .
Architecture breakdown element (ABE)	The modules of an architecture breakdown into which a product is decomposed, for example, Refrigerator (0)→Housing (1.0)→Door (1.1) .
Design solution	The result of associating a CAD occurrence to an architecture breakdown element (typically the lowest level), for a specified NVE. This action creates a variant condition on the design occurrence according to the NVE selected.
Generic breakdown	One of two types of product-specific architectures. If you choose to use a generic breakdown to reflect the product-specific architecture, the product structure relationships in the product-specific architecture need not be identical to those found in the product architecture template. The generic breakdown does not maintain a dependence from the product-specific architecture to the template architecture. This is an important characteristic that must be decided during initial definition of a product-specific architecture. The type other choice is a <i>architecture breakdown</i> .
Generic breakdown element	The modules of a generic breakdown into which a product context is decomposed, for example, Computer (0)→Output Device (1.0)→TFT Screen (1.3) .
Installation assembly	The node in the CAD structure to which design solutions are added and variant conditions applied, according to the named variant expression selected on the architecture element. An installation assembly may be associated with an architecture element to guide the designer by limiting the choice of named variant expressions from which to choose when adding a design to the product.
Module	A generic object in the product architecture, also referred to as an <i>architecture breakdown element</i> . The lowest level modules on which you can specify the variance and variability, if required. Do not confuse with the term <i>module</i> used with modular variants; this object behaves differently but fulfills a similar role.
Named variant expression (NVE)	An expression of variant logic with option values and Boolean operators that define a set of option combinations that populate variant conditions on the occurrence of the relevant design or part solution. For example, (Number of doors=5 AND Market=USA) . Sometimes referred to as <i>variance</i> .

Term	Meaning
Part solution	The result of associating a part to an architecture breakdown element (typically the lowest level) for a specified NVE. This action creates the variant condition on the occurrence of the part in the architecture element, according to the NVE selected. A part solution may also have information such as the quantity, effectivity, and part number (typically imported from an external system).
Product	The family of products for which variability is defined. It is identified by a market name and may have large commonality of parts.
Product architecture	The general name for the hierarchical structure of architecture breakdown elements that defines the company's coding system.
Product architecture template	Sometimes referred to as a corporate dictionary. This is the hierarchical structure of all architecture breakdown elements that define the company's coding system for a product family. An architecture breakdown element can only exist in one location.
Product context	The root node of a product-specific architecture breakdown with which variant logic (named variant expressions) is associated, for example, Product = Mid Size car, Context = Model Year 2012 . The decision at what level in the product structure to differentiate between product context and product (family) is an individual corporate decision.
Product-specific architecture	The product-specific breakdown created from the product architecture template, with which variant logic (named variant expressions) and CAD assemblies are associated, for example, Car Model X, Model / Year 2012. A product-specific architecture can consist of two kinds of structures: architecture breakdowns or generic breakdowns. Architecture breakdowns enforce the breakdown structure to be the same as the product architecture template from which it is derived. Generic breakdowns are independent of the product architecture template and allow the architecture structure to be modified. An architecture breakdown instance is made of references to the template architecture elements; a generic breakdown is made of instances cloned from the template generic elements.
Variability	An unordered set of option values, with no combinations or logic, for example, Number of doors:4,5 or Market: UK,USA .
Variance	Permutations of option values offered for a module or a complete product-specific architecture. See also the definition of <i>Named Variant Expression</i> .

Set up Platform Designer

To use Platform Designer, configuration experts must first create the necessary data and relations. Installation utilities such as **sync_product_variant_data** and **sync_product_apns** are provided to help automate this process if large volumes of data must be imported to populate the Platform Designer application.

A graphical representation of the necessary steps is shown later. The numbers in the figures correspond to the steps in this procedure.

The first three steps are performed by a corporate standards analyst.

1. Define the master options and values, which are stored on global option items.
2. Create a product to contain the template architecture for the complete family of products.
3. Define the product architecture template (the structured coding system) offered for a family of products. It comprises generic modules that are referred to as architecture breakdown elements (ABEs). This data changes very rarely, typically only if additional generic modules are offered. When there is a high degree of commonality in content between the products a company makes, there may be a single product architecture template. If there is a very large amount of data, it may be authored in a business system.
You can create different product architecture templates so that multiple product specific architectures can be associated to the same product. This may be necessary in large organizations with different historical coding systems across the corporation. Each of these product architectures is defined in Platform Designer as a different architecture *type*.
Platform Designer enforces various rules including:

- A module (ABE) must have a unique identifier within the architecture type, for example, functional or physical.
- A module (ABE) can exist in only one place in the architecture, although it can be repositioned.

The following steps are performed by configuration experts:

4. Create a product for the product-specific architecture that contains only the subset of modules (ABEs) to be offered. The designer must not change data on the product architecture template, because the template applies to all products. The designer creates a new root architecture for the specific product, for example, **Model Year 2005**. At this stage, two decisions are required:
 - Should the product-specific architecture use architecture breakdown elements or generic object elements?
 - What settings should be used to control how the variability and variance are applied?
5. Verify the new architecture includes only the applicable subset of the modules (ABEs) from the product architecture template. The product-specific architecture then presents the user with this

subset of modules (ABEs) for the product in which they are working. The product-specific architecture is overlaid with actual product data relevant to the particular product architecture-specific architecture, namely, option values and combinations, and also design and part solutions.

6. Associate the appropriate subset of options and values with a product. Options are associated with the product, *not* the product context. This allows you to have a corporate dictionary of options and values that is wider than those specific to a single product context.
7. Define options (variability) on the lowest level architecture nodes that will have NVEs applied. Options must be cascaded down through all the intermediate nodes if hierarchical variability is defined.
8. Define the options combinations offered (variance) as *named variant expressions* (NVEs). An NVE is Boolean logic that defines a set of option value combinations, for example, **Option X = 3 AND Option Y > 50 OR Option Z != Blue**. These NVE statements are independent of the product structure (unlike variant data in Structure Manager), allowing interfaces with the business systems.

Note:

You can configure shared NVEs and unshared NVEs.

Shared NVEs are defined at the product level and then assigned to any number of ABEs. Shared NVEs are unique by name within the scope of the product. They are also unique with respect to their Boolean content. If the NVE is changed, it is changed in all ABEs to which the NVE is applied.

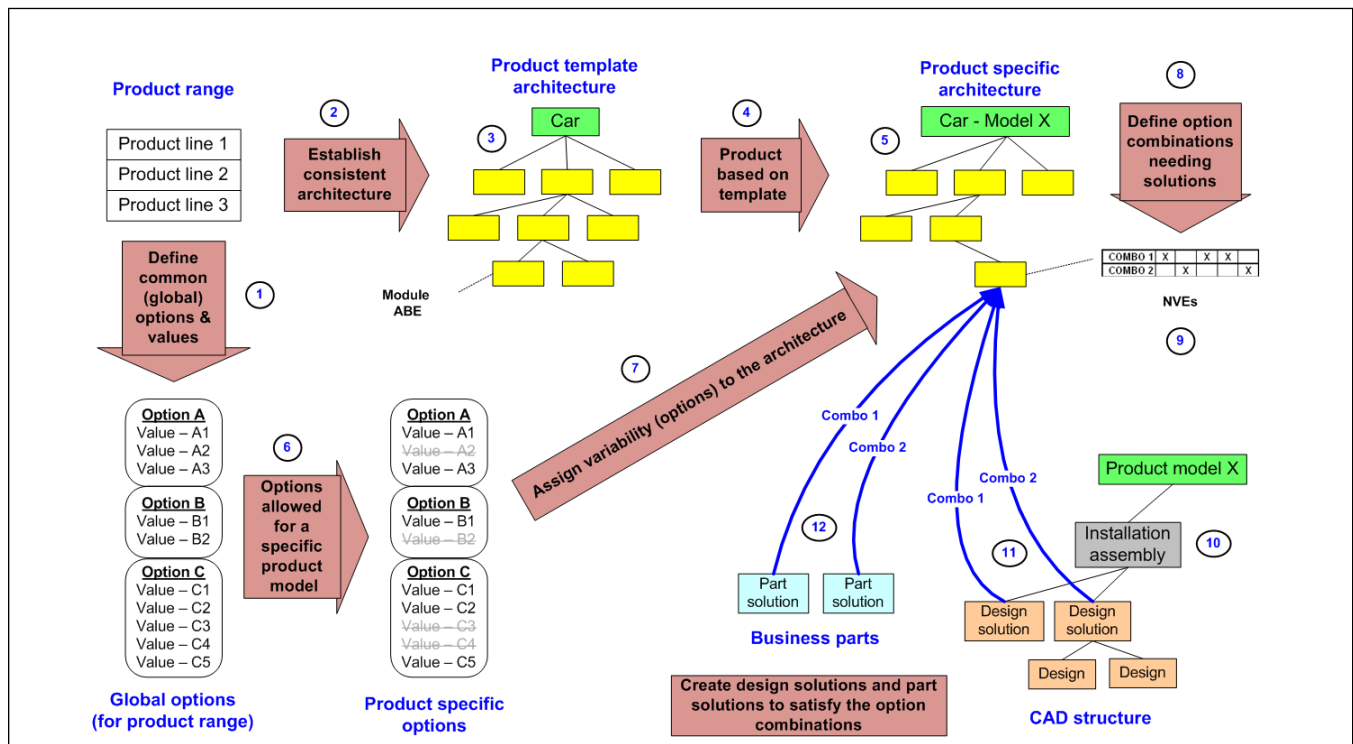
Unshared NVEs are created directly on the ABE and not available to other NVEs. Not sharing NVEs gives greater flexibility, but increases duplication.

9. Associate the NVEs with the corresponding ABEs (shared NVEs). You can relate multiple NVEs to multiple ABEs. This work is done by configuration analysts assigned to a particular program.

Once the data is set up, the following steps are performed daily by designers and part engineers.

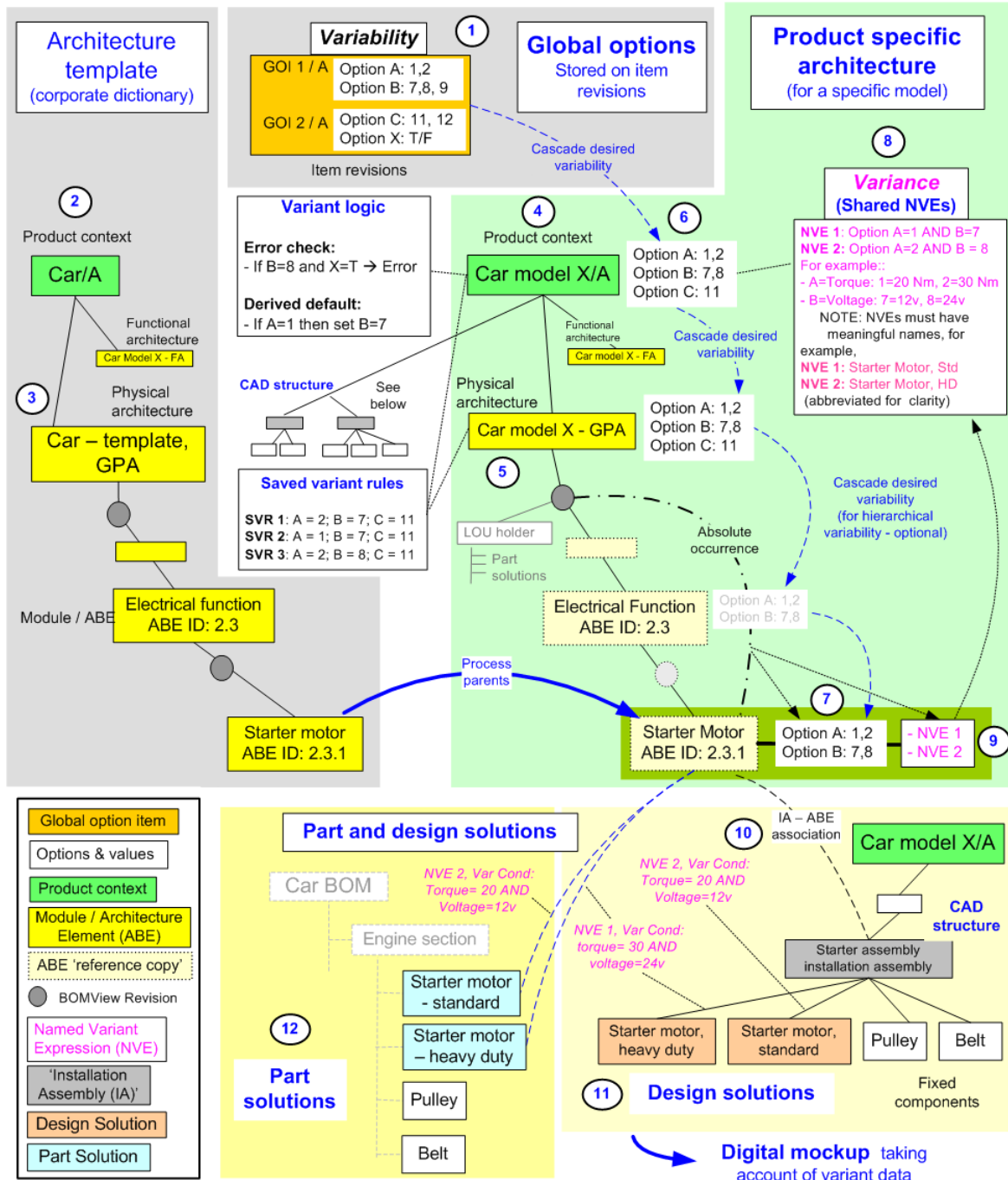
10. Associate the assemblies in the CAD product structure with the ABEs. There is now an implicit relation between the variant logic (defined in Platform Designer outside the CAD structure) and the CAD product structure. This work is done by CAD analysts assigned to the particular program. These assemblies are sometimes called *installation assemblies* (IAs) or pick lists. Typically, a variant condition is only applied to the immediate components of these assemblies. This practice is not enforced by Platform Designer but is recommended.
11. Design engineers create design solutions to represent components (CAD designs) that fulfill the requirements defined in NVEs and associated with the architecture elements.
12. Part engineers create items for part solutions to represent parts (ERP parts) that fulfill the requirements defined in NVEs and are defined on the architecture elements.

The following figure shows the basic process flow of Platform Designer data.



Platform Designer – process flow overview

The following figure shows how the data elements are managed by Platform Designer with program data overlaid. The numbered circles correspond to the steps in the preceding procedure.

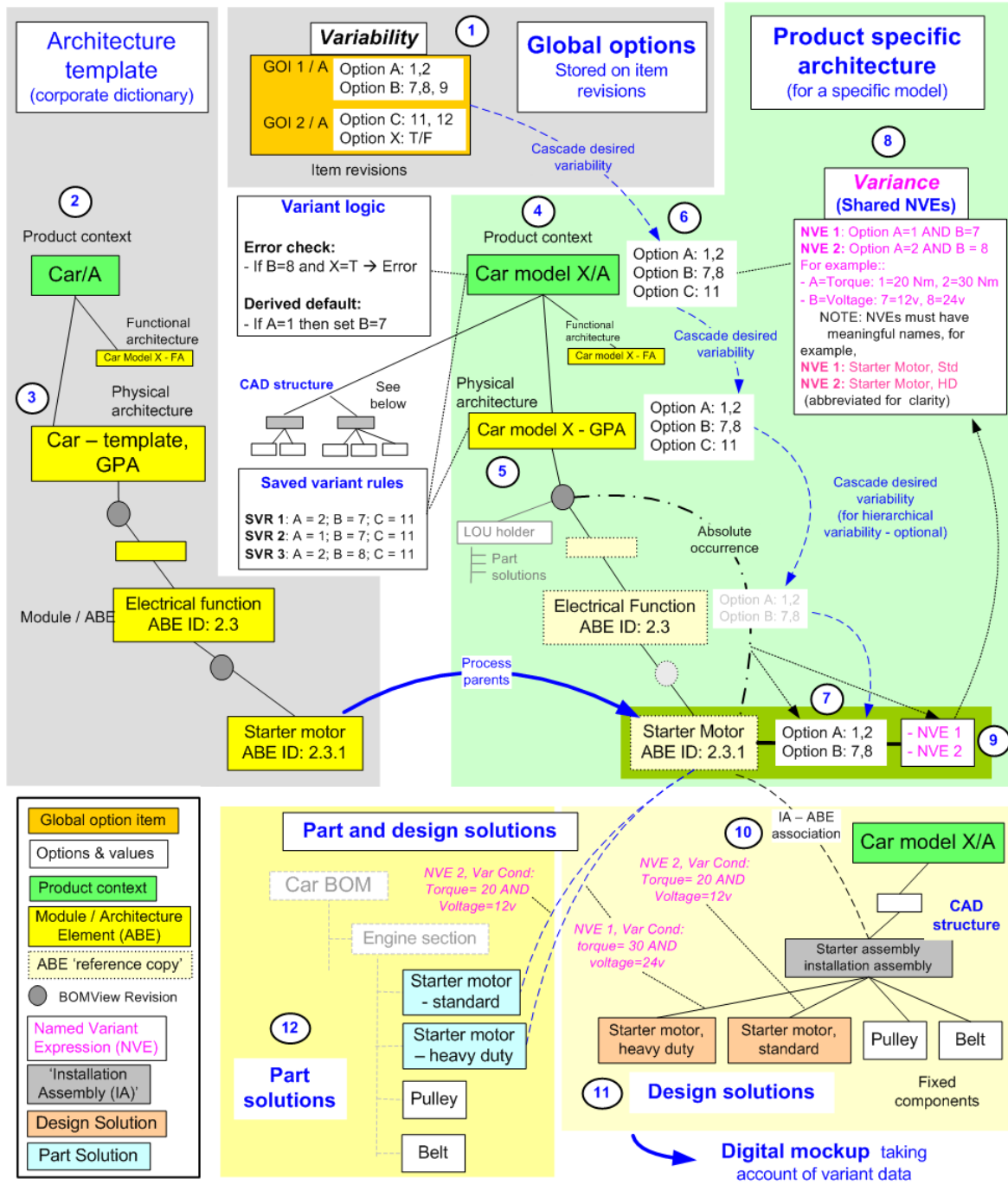


Platform Designer – variant data storage when using part and design locations

Understanding data storage

Storing Platform Designer data

To understand how to populate data in Platform Designer, you must understand where the data is stored. This information is also important if you consider changes to data and want to copy selected data forward to reuse.



Variant data location with part and design solutions

Storing product family data

Product family data is corporate data that can be applied to a family of products (shown with a grey background). This comprises the following types of data:

Architecture template (corporate dictionary) An architecture template defines the total set of generic modules (architecture breakdown elements or ABEs) for a product family. The modules are classified into a hierarchy (architecture) to facilitate navigation to the lowest level modules. The lowest level modules are the significant ones, to which variability, design and part solutions are eventually assigned in the product-specific architecture.

- A product can have different architecture breakdown types, for example, physical and functional. These types typically have the same set of lowest level modules.
- Architecture breakdown element nodes must have a unique ID within the architecture type. The ID is specified by the user. It ID applies across all products in the database and is used to populate the absolute occurrence ID.

Note:

No variant data is attached to architecture nodes in the template. The template is only an organizational structure of modules.

Global options Global options define the total variability for a product family. The options (classic variants) are defined on global option items.

Revise the item to control availability of new option values. Similar options are typically defined under the same global option item, as required for ownership or change control.

Storing product-specific data

Product-specific data refers to data for a specific model, for example, **Car Model X** (shown with green and yellow backgrounds in the previous diagram) and includes variability, variance, design solutions, and part solutions.

Product context The product context is the root node of the CAD structure. The variability (options) associated at this level is a subset of the variability available for the whole product family.

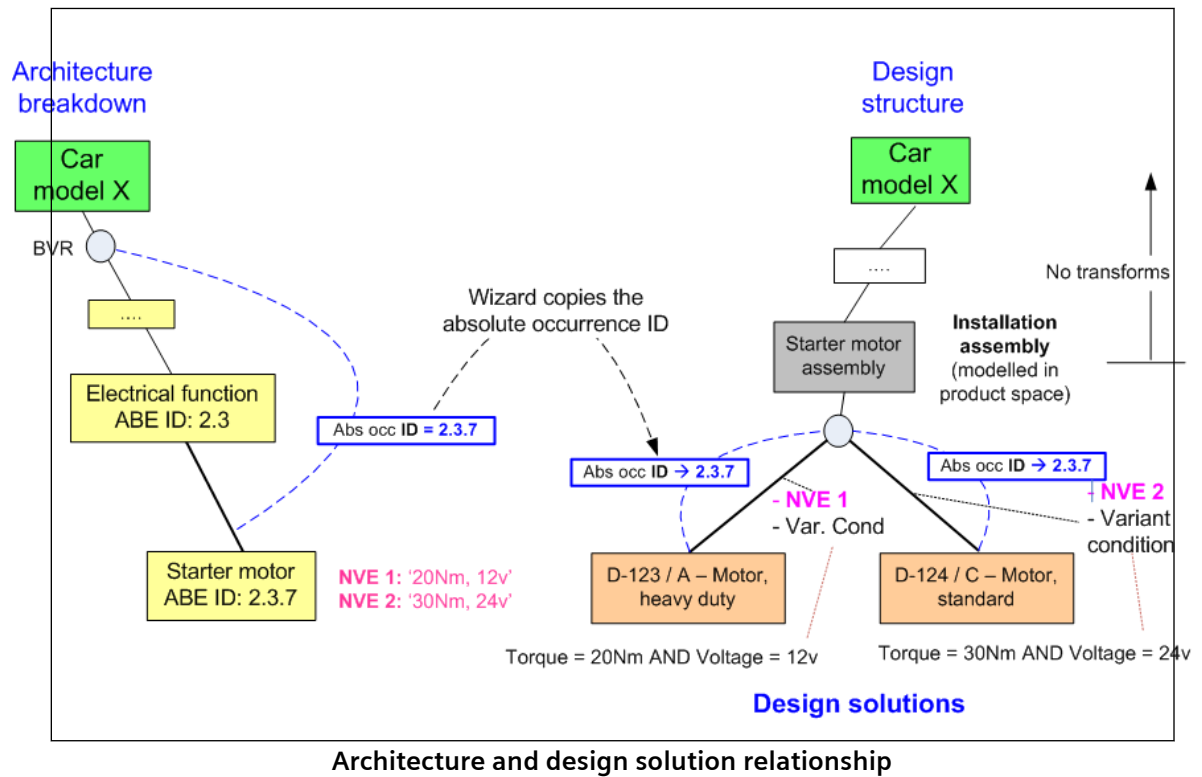
Note:

In general, there is only one product context per product, so the terms are used interchangeably. More than one product context will exist if the product context is revised; this may be the case if you create a new model year, when it is desirable to carry over the same shared architectures.

Product-specific architecture The product-specific architecture defines the subset of modules that are part of a new model. The variability must be cascaded from the product context to the top level architecture (typically, all options on the product).

- The architecture nodes (ABEs) are typically copied from the template, unless generic objects are used. The nodes are not independent objects, but refer back to the original node in the template architecture.
- When variability and variance are applied to nodes in the structure, this data is stored as absolute occurrence override data on the BOM view revision of the top level architecture (a separate item to the top level template architecture).

Variability	You can optionally cascade the options that you defined on the top level architecture down to the lowest level modules, as illustrated in the previous diagram. This depends on whether you enforce hierarchical variability.
Variance (named variant expressions - NVEs)	Variance represents the allowed option combinations that are offered. • Shared NVEs are stored at the product context and are available to all architectures; they are assigned to the ABEs that will have solutions attached. • Unshared NVEs are created locally in the architecture breakdown element. • The options used within an NVE should already have been associated with the ABE to which the NVE applies
Design solutions	Design solutions are occurrences of design items in the CAD structure. There is a link between the absolute occurrence of the associated ABE in the architecture and the installation assembly for the design solutions; they share the same absolute occurrence ID, as shown in the following diagram. • When a design solution is associated with an ABE, the selected NVE populates the variant condition on the design occurrence in the CAD structure. This arrangement allows design solutions to be configured when applying a variant rule Note: It is not possible to see the design solutions associated with an ABE in Platform Designer. You can only view and configure these in Structure Manager. • When viewing the CAD structure in Structure Manager, display the Architecture Element ID column to show the ABE associated with each design occurrence.
Fixed or static components	Fixed or static components are always configured. To model such components, create a shared NVE named, for example, Static , that always evaluates to true . You can take any option and create an NVE with the expression <code>{option X = true OR Option X != true}</code> . You then assign this NVE to those components. All components in the design structure must be associated to a single ABE. The specific ABE is not important, but you should typically use the ABE for which the component most closely performs the function.



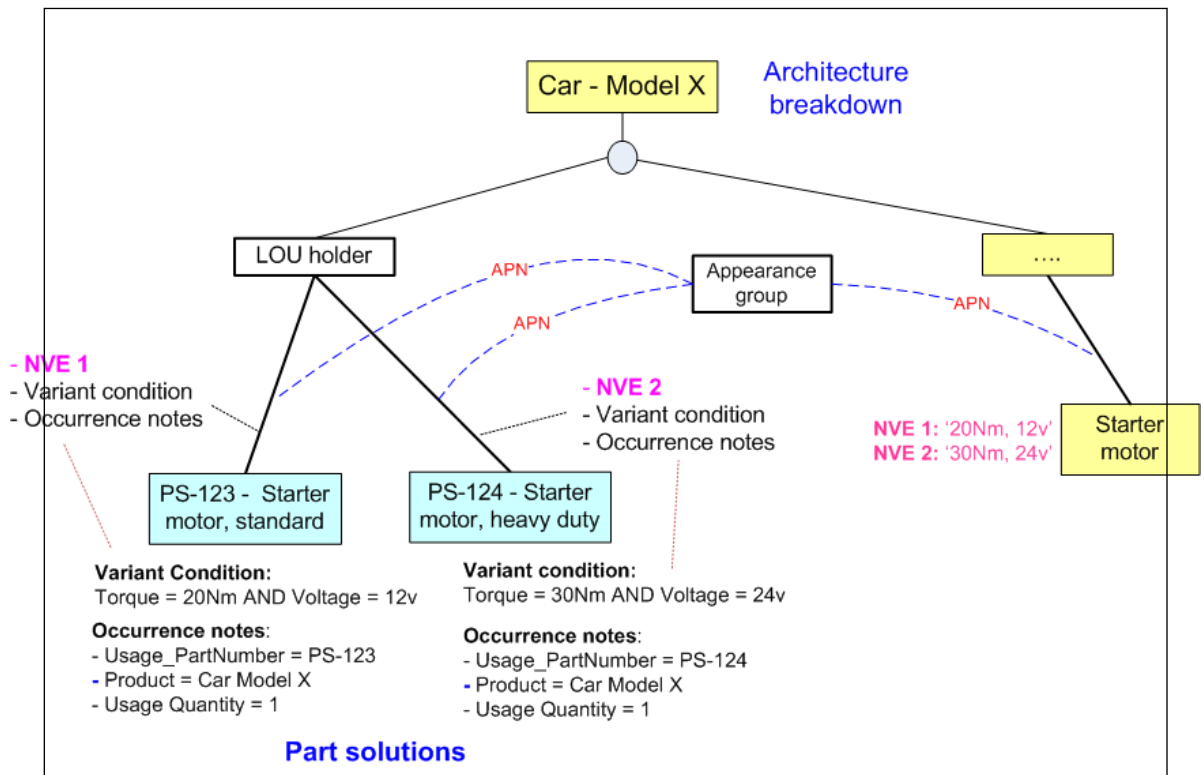
Architecture and design solution relationship

Part solutions

Part solutions are associated directly with the product-specific architecture. You create items for part solutions to meet the option criteria for a specified NVE and associate them with the ABE. Unlike design solutions, the part solutions are displayed directly below the ABE.

- When a part solution is associated with an ABE, the selected NVE populates the variant condition on the part occurrence in the architecture structure. This allows you to configure part solutions in Platform Designer by applying a variant rule.
- Part solutions may not necessarily be joined in any structure, unlike design solutions.
- Part solutions do not have any geometry associated with them. You can view and configure them in the architecture structure in Platform Designer.

A part solution is not an occurrence of the ABE, but they are related through an appearance group, as shown in the following diagram. Part solutions are stored as occurrences of the line of usage (LOU) holder BOM line at the top level of the architecture structure. You should not expand the LOU holder as it contains all the part solutions for the entire product and could include thousands of BOM lines.



Part solutions and line of usage data

If part solutions are in a structure (as happens if they are used with the part CAD alignment functionality), a variant condition is not applied to the occurrence of the part in the part structure. Thus, you cannot configure the part structure with a variant rule. This contrasts with design solutions, for which you can apply a variant condition to the design structure.

Saved variant rules

Saved variant rules (SVRs) can be stored in one of two places, depending on which application generates them.

- If created in Structure Manager for configuring design solutions in the CAD structure, they are saved under the product context item.
- If created in Platform Designer for configuring the part solutions, they are saved under the top level architecture item.

Consequently, you cannot load SVRs to apply them to the architecture if they were created in Structure Manager, and vice versa. To work around this, copy the SVRs stored under the product context item and paste them to the top level architecture item. Alternatively, copy the SVR stored under the top level architecture item to the product context item.

Error checks and derived defaults

You can define error checks and derived defaults in Platform Designer for option values. They are stored on the selected architecture node, typically the top level. They

can also be defined in Structure Manager for the CAD structure, and are stored on the product context.

This variant logic does not propagate from the architecture to the product context or vice versa, as they are separate items. Therefore, variant logic created in Structure Manager is not applicable to or visible in Platform Designer, or vice versa. This is not an issue if you author variant data in an external product configurator.

Applying the variability and variance

Analyzing variability and variance

Before you create a product-specific architecture, you must decide how the variability and variance should be applied. There are several options to select when creating a breakdown that determine the variability settings.

You must set the following options to **true** or **false**:

- Shared NVEs.
- Full breakdown.
- Prevent overlapping NVEs on breakdown elements. (Only available for full breakdown.)
- Enforce hierarchical variability.
- Breakdown based on existing architecture breakdown. (Only available for generic breakdowns.)

Caution:

Once these flags are set, you cannot change them, so it is important to be aware of their effects before creating the breakdown.

Tip:

Once these options are set, it is not easy to identify their values. To see the settings, view the **has_shared_nves**, **has_consistent_nves**, **is_partial_breakdown**, and **has_hierarchical_variability** properties for the top level architecture item (not the item revision).

Using shared or unshared NVEs

If an architecture uses *shared* named variant expressions (NVEs), they are stored at the product level. This set of NVEs is the same for all architectures in the product that use shared NVEs. Consequently, you should only define NVE expressions once.

Multiple architecture elements may share the same NVEs. If a shared NVE is changed on the product, it changes on all associated architecture elements. Shared NVEs must be created with a unique name for the product family. There are no number generators, so you must decide on a naming convention that provides uniqueness.

NVEs can also be created as *unshared*. These NVEs are created on the architecture element itself, for each element to which parts or designs are attached. In this case, there is no reuse of NVEs. However, as NVEs are created locally, the user may change them; this contrasts with shared NVEs, which are typically created by an architect.

Note:

The variant condition in the CAD structure is generated from the NVE, but is not automatically updated when an NVE expression changes. Platform Designer allows you to create an audit report to highlight such cases.

To all the elements to which an NVE is assigned, attempt to delete it, and the system tells you all the architecture elements that reference the NVE.

It is preferable to use shared NVEs to limit the number of NVEs. Many companies put effort into validating that the expressions in all NVEs can resolve to **true**, for example, that the expression **{color=red AND color=blue}** is invalid. This validity check is performed by analyzing all the expressions (NVEs) against all the error checks. These error checks are typically made outside Teamcenter, in which case the NVEs must be exported. This process is simpler when NVEs are shared and stored on the product, because they can be more easily located; if they are unshared and stored across the architecture elements, the query is more complex.

Creating a full or partial breakdown

A *full breakdown* architecture is normally created when the breakdown at the lowest level elements is sufficiently detailed that the assigned NVEs are mutually exclusive, for example, **NVE 1: Headlight = Standard, NVE 2: Headlight = High Spec**. Teamcenter can automatically **check exclusivity**. In addition, Teamcenter can generate an audit report that identifies the elements that do not yet have part or design solutions assigned.

A *partial breakdown* is used for early planning BOMs, when there is not yet sufficient level of detail for a full breakdown. In this case, you can use high-level summarizing architecture elements to define variability and collect solutions. NVEs may be applied, but they are very general and could overlap.

Teamcenter does not support conversion of a partial breakdown to a full breakdown. If you anticipate having to do this, use a full breakdown, but create temporary high level elements and define variability at that level. Later, when more detail is known, you can add lower level architecture elements, the variability cascades down, and the solutions move to the lower levels.

Preventing overlapping NVEs

Preventing overlapping NVEs (using *consistent* NVEs) helps you avoid releasing two sets of parts for the same ABE, when only one is expected. For example, assume you have a spark plug ABE with a variability of **octane-rating:89** and **octane-rating:93**. You have two NVEs on this ABE—NVE1 of **octane-rating > 89** and NVE2 of **octane-rating < 93**; you also have two spark plug parts released to this ABE—P1 for NVE1 (**octane-rating > 89**) and P2 for NVE2 (**octane-rating < 93**). At this point, the data is consistent.

A configuration specialist now changes the variability of this ABE and adds **octane-rating:91**. With the **Prevent overlapping NVEs** option selected, the configuration specialist receives an error that NVE1 and NVE2 would then overlap. Consequently, there is a product configuration (**octane-rating:91**) for which both NVEs evaluate to **true**, causing two sets of spark plugs to be delivered to the assembly line for that engine. Teamcenter checks for NVE consistency when you edit or add NVEs, and also when you change variability. With the **Prevent overlapping NVEs** option selected, Teamcenter guarantees consistency for much more complex conditions than this spark plug example.

Note:

This capability is available only if you use full breakdowns.

Using hierarchical variability

Variability is the set of explicitly allocated values for one or more variant options. Variability is defined on global option items for the enterprise, and determines the total set of values allowed for those options. For example, **GOI – 1000/A** defines **Option X** with values {a, b, c, d, e, f}.

Note:

The options values are stored on the global item revision, as the values can change over time. The revisions can be released, and status and effectivity may be applied. Thus the set of allowed values can be configured for a given date or unit number.

A new product is created that uses **Option X**, but only values {a, c, d, e} are allowed. This restriction is set when options are assigned to the product (context).

If a given architecture is associated with the product, the set of option values may additionally be limited, although this situation typically does not occur.

If hierarchical variability is enforced, the options and values must be cascaded down through all the architecture elements to the lowest level elements on which named variant expressions (NVEs) are defined. This is necessary because, by default, an NVE can only reference options that are defined on the architecture element.

The variability of each element in the breakdown may not extend the variability of its parent element. For example, a lowest level node may have **Option X** assigned with values {a, c, d}, while its parent has values {a, c, d, e}.

If hierarchical variability is not enforced, the variability of any node in the breakdown structure is constrained only by the variability of the top-level architecture. It is not necessary to place options on all intermediate levels, although variability must still be assigned to the lowest level nodes at which NVEs are assigned.

If you enforce hierarchical variability, the options that control a given element at all levels are easy to identify. Teamcenter does not display all options at a given level, just the relevant options. This limited display also helps when you analyze the impact of adding or removing an allowed option in the product. For example, if you want to remove value **a** from **Option X**, you can see the architecture elements (modules) to which **Option X** is assigned.

If hierarchical variability is not enforced, it is possible to assign all options to all lowest level elements to be able to assign NVEs.

Hierarchical variability is independent of whether you prevent overlapping NVEs or create a full or partial breakdown.

Choosing variability settings

To assist in deciding which variability settings best suit your needs, use the following guidelines for creating architecture breakdowns.

For full control and completeness audit reports:

- Full breakdown – on
- Prevent overlapping NVEs on breakdown elements – on

For maximum flexibility in early stages:

- Full breakdown – off
- Prevent overlapping NVEs on breakdown elements – off (obligatory)

For NVE reuse (recommended):

- Shared NVEs – on

To ease usability for the assignment of variability and NVEs at lower levels:

- Enforce hierarchical variability – on

12. Aligning structure

Alignment methods	12-1
Aligning the structure internally	12-1
Aligning structure items internally	12-1
Aligning parts and designs	12-2
Aligning with allocations	12-8
Making alignments in Multi-Structure Manager	12-8
Aligning the structure externally	12-9

12. Aligning structure

Alignment methods

Depending on your product structures, you may want to align related items or item revisions to ensure dependencies are captured and are visible if changes are made. There are two general categories of alignment that you should consider:

- **Internal alignment**
An element in the product structure affects or is equivalent to other elements in the same structure.
- **External alignment**
An element in the product structure is equivalent to an object in an external system, such as a BOM or ERP system.

Aligning the structure internally

Aligning structure items internally

Teamcenter supports several methods of internal alignment; you may apply one or more of them, depending on your product data.

- **Part and design alignment**
You can align any types of items that are appropriate for your business practices, for example, parts with CAD designs, or parts with documents. You can globally associate a part and a design, in which case all revisions of the part and design are automatically associated. The publish link between the items is revision independent. Alternatively, you can choose to evaluate the alignment each time the part or design is revised. This configuration ensures the design can always be correctly visualized wherever it occurs in the structure.
- **Allocations**
Allocations provide a similar ability to publish a link between two items to the part and design alignment feature. They provide more traceability but do not support the copying of transforms and shapes. Allocations can be configured and managed, for example, by applying effectivity under an incremental change.
- **Assign in Multi-Structure Manager**
Use assignments to create a composition that represents a layout for design in context work across several structures or products.
For example, in *Purpose of Multi-Structure Manager*, you create a layout for the bicycle and lock.
- **Assign in Manufacturing Process Planner**
Use assignments in a process to consume parts in operations. Each time you make an assignment in Manufacturing Process Planner, Teamcenter creates a new occurrence in the operation that is corresponds to the selected BOM line. The part and the operation share an absolute occurrence. You

can perform an accountability check on the structure to verify that all parts are consumed by an operation, based on the shared absolute occurrence IDs.

- **Platform Designer**
You can use part and design alignment to complement Platform Designer, if you work with layouts (that is, an ad hoc collection of installation assemblies), rather than installation assemblies themselves. The context for publishing links must be an installation assembly, rather than a layout.
- **BOM alignment**
You can align BOMs of different types by indicating that occurrences in each BOM are functionally equivalent. For example, a manufacturing engineer may link a configured EBOM root with a configured MBOM root. When these structures are linked, all further assignments to the MBOM from the EBOM create logically equivalent MBOM occurrences. Teamcenter allows you validate these occurrence links with an alignment check and keeps the data of aligned occurrences synchronized.

Aligning parts and designs

Aligning parts and designs

A business part typically has a design solution developed for it. The business part and the design solution may have different lifecycles. A part may have several design representations, for example, in the case of flexible parts. Conversely, a design model may apply to multiple parts, for example, colored parts.

Also, users may create independent part and design structures, then organize and manage them separately. When the design structure and the part structure are independent these different views of the same product must be reconciled. The action of aligning or reconciling the structures in this way is referred to as *publishing* occurrence data. For example, some part occurrences may use positioned designs and you need to align each such part occurrence with a design occurrence that is positioned in the context of the product. Alignment allows you to visualize the positioned design from the part structure itself.

You can optionally automate the alignment so that when the user creates a design, Teamcenter automatically creates the corresponding part. To do this, you define the source, target, and relationship of the automated creation process in the Business Modeler IDE, for example, **Part, Design** and **TC_Is_Represented_by**. You can also use custom source and target types. Alternatively, you can configure the creation of a design to initiate a workflow that notifies a part engineer to manually create the corresponding part.

Note:

Automation fails if you do not specify an object naming rule for the target attribute. For example, Teamcenter cannot automatically create a text dataset if you do not specify the name it should give the new dataset. In this situation, consider using property mapping, that is, the text dataset may inherit the object name of the source design object if you configure the **Fnd0InheritFrom** business object constant accordingly.

When you create a business object (for example, an item), there may be certain attributes that cannot be null. If these attributes are not specified, object creation fails. If you are creating a business object manually, the **Finish** button is not enabled if you do not enter all the required values. However, if you are automatically creating the target object, the user is not required to enter target object values; for example, the designer may not be aware of part attributes because the design creation dialog box does not show part attributes. To address this issue, you can do one of the following:

- Provide default values for required and mandatory attributes.
- Write custom code through a post action on the **finalizeCreateInput** operation of the target object. For instance, if you are automating part creation, write a post action on the **finalizeCreateInput** operation of the part.

You publish occurrence data between source and destination occurrences of two representations. Teamcenter takes a snapshot of the source occurrence data and creates a *publish link* to the destination occurrence. For example, you can publish an absolute transform from a source design occurrence to a destination part occurrence, ensuring the shape is correctly positioned in the product. If the source occurrence data was already published, Teamcenter updates the snapshot with the new information.

Depending on your business practices, you can align a design occurrence to more than one part occurrence. Teamcenter may automatically mark one of the part occurrences as the *primary*, based on an assessment of the maturity of the design occurrence and its latest revision. The maturity of the design revision is set with the **Mature Statuses** global constant in the Business Modeler IDE. Users may also manually set a design occurrence as the primary.

Part and design occurrences must be reconciled when the same context is set in both structures. For example, in the design structure, the designer positions a wheel in the **WheelAssembly** assembly; that is, the left front wheel is positioned in the context of **WheelAssembly**. Also, the **WheelAssembly** assembly is itself positioned in the context of the **Car** product; that is, the left front wheel is positioned in the context of **Car**. If you require the positioned design of the part occurrence denoting the left front wheel of the car, you must reconcile the design occurrence of the left front wheel in the **Car** context. Before publishing the occurrence data, you therefore set the context of both structures to **Car**.

When you revise a part, the associated designs are propagated to the new revision.

Note:

If you are automating the creation of a design when a part is created, be aware that the part object has a **Design Required** boolean attribute. If this is set to **false**, the design may not be aligned to a part. Consequently, you should not use this default attribute on the master form to trigger a condition.

Note:

If you visualize a JT file attached to the part structure, Teamcenter first looks for the JT file associated to the part occurrence. However, it does not traverse from the part occurrence to the design occurrence. Consequently, if no JT file exists for the part occurrence, it then looks at the associated primary design revision for a JT file.

Typical structure alignment process

The following example shows a typical process for aligning a product structure comprising parts and designs. It would not apply if you work with items, rather than separate parts and designs.

1. The BOM engineer creates a part.
2. The CAD engineer creates a design for the part, independently of the part.
3. The BOM engineer associates the CAD design as a solution for the part.
4. The BOM engineer adds the part to the part structure. This action creates an occurrence of the *part*.
5. The CAD engineer creates a layout and positions the CAD design. This action creates an occurrence of the *design*.
6. The CAD engineer aligns the design occurrence with the part occurrence by publishing a link between the two occurrences.
7. The BOM engineer validates the part-CAD occurrence alignment occurrence with the part occurrence.
8. The user views the positioned CAD design from the part structure.

Associating a design revision to a part revision globally

You fulfil a business part's need for a design solution by associating a part revision to a design revision in My Teamcenter. This association is *precise*—it is between a revision of a part and a revision of a design. You can associate a part revision with multiple design revisions, in the case of flexible parts such as hoses or pipes. Similarly, a design revision may be valid for several part revisions, for example, for colored parts. The part revision is associated with the design revisions by the **Represented By** relationship.

When you revise a part, you decide whether to carry forward existing associations with design revisions. By default, all associations with **Represented By** relationships are carried forward. Conversely, when you create a new revision of a design, the existing associations with parts are *not* carried forward by default. This allows the designer to determine manually when to make the new revision visible to the BOM engineer.

To allow automatic propagation of a new design revision to the associated part revision, add a **TC_Is_Represented_By,Part Revision,LookLeft** string to the **AutoCopyRel** business object constant on the design revision object with the Business Modeler IDE. However, propagating the design revision to the part revision does not automatically make it the *primary* revision, that is, the revision displayed in the viewer. The primary revision is controlled by the maturity level, as described later.

Note:

Automatic propagation assumes an association between a single design and a single part. If a design revision is associated with several revisions of the same part, the latest mature revision is carried forward. If a design revision is associated with revisions of different parts, nothing is propagated.

Defining the primary design representation

If you associate a part revision with more than one design revision, you must choose a default or primary design representation. This sets a **TC_Primary_Design_Representation** relation on the part revision. It is indicated in the user interface by a check mark against the part revision.

You can customize the action of making a design revision the primary representation, so that Teamcenter checks if the design revision qualifies to be the primary representation. Do this by adding the necessary checks to the **USER_make_design_rev_primary_precond** user exit.

If the primary design representation of a part revision is a piece part, Teamcenter uses the JT file attached to the design revision for visualization in the embedded viewer. If the primary design representation is an assembly, visualization data is captured in a PLM XML file that is attached to the design revision as an assembly dataset by a **Tc_Rendering** relationship. The design revision assembly must be configured using the default revision rule in Structure Manager before capturing the PLM XML data. Any changes to the assembly design revision are only reflected in the PLM XML file if it is set as the primary design representation.

Note:

You can set the **PSM_enable_PLMXML_Assembly** preference to control PLM XML generation. By default, this preference is set to **true**, which generates a PLM XML assembly file and attaches it to the part revision with a rendering relation. If you set the preference to **false**, the PLM XML assembly file is not generated.

When you visualize a part revision, the necessary shape data is derived from the primary design representation that is identified by a **depends on** or **refers to** relationship. Be aware of these relationships if you export the part or design data using Multi-Site Collaboration or PLM XML.

You can use the **PSM_Skip_Unload_PublishedPartAssembly_OnViewer** preference to control the unloading of published part assemblies. After loading a published part structure into the **Graphic** viewer, the following occurs if you uncheck a line that represents a published part.

- If this preference is set to **True**, the part structure becomes invisible in the **Graphic** viewer, but the associated JT data is not unloaded.
- If this preference is set to **False**, the part structure becomes invisible in the **Graphic** viewer and the associated JT data is also unloaded.

Checking design maturity level

As a design changes over time, more than one revision of the design may be associated with a part. One of these revisions must always be the primary, that is, the source of visualization data in the part structure. The primary revision cannot be configured but is determined by a maturity level, for example, if a release status is attached and (if so) its type.

Note:

The name of the release status used in the workflow should already be defined in the Business Modeler IDE. Use the real name of the release status in the workflow, not the display name.

The decision on whether a design revision is considered mature depends on your business processes. You can define the appropriate maturity level by setting the **AutoRevise** constant or **ReviseAndRelateToLatest** action on deep copy rules in the Business Modeler IDE. Alternatively, you can customize the **USER_is_item_rev_mature** user exit.

Teamcenter can automatically change the primary design representation as the design evolves, so that the new design revisions are associated with a part revision. When you associate a design revision to a part revision, Teamcenter compares it with the existing primary design representation by maturity level and latest revision. If one of the design revisions is mature, it is made the primary representation, otherwise the latest is used. For example, **Part1/A** is represented by mature **Design1/A** and you then associate a new working revision **Design1/B** with **Part1/A**. Teamcenter compares the maturity levels of **Design1/A** and **Design1/B**. As **Design1/A** is mature, it remains the primary representation and **Design1/B** is associated with **Part1/A**. If **Design1/B** is subsequently released, it becomes mature and is eligible to become the primary representation. The user can then manually make **Design1/B** the primary representation.

Alternatively, you can use Workflow to initiate the setting of the primary design representation. If the approval of a design revision makes it mature, use the approval status to initiate the **PS-make-mature-design-primary** Workflow handler. This action automatically makes the design revision the primary representation of the part revision that was approved.

Aligning part and design structures

To allow accurate visualization of a part occurrence, Teamcenter must know its positioned design. To achieve this, data is copied from the source occurrence to the target occurrence by a **Publish** action that creates a publish link relationship between the occurrences.

A publish link may associate one source occurrence with several target occurrences. If you are associating parts and designs, the design structure is the source structure and the part structure is the target structure.

You can align any items types that you use in structures. You can align other item types in structures, if appropriate. To configure the source and target types of such alignments, set the following preferences:

- **PUBLISH_AlignableSourceTypes**
Specifies the item types that are valid as the source of a publish link. The default values are **Design** and **Item**.
- **PUBLISH_AlignableTargetTypes**
Specifies the item types that are valid as the target of a publish link. The default values are **Part** and **Item**.

Teamcenter creates the publish link between absolute occurrences of the relevant structure lines. Data published on the target occurrence is stored as an absolute occurrence override. When you publish data, you require write access to the BOM view revision (BVR) of the displayed top-level context item. Ensure the contexts in which you align the occurrences are equivalent. For example, in the case of a car, the position of a wheel in the context of a wheel assembly differs from its value in the context of the entire car.

It may not be practical to load the top-level item of very large structures. If you utilize installation assemblies in your design structure, it is sufficient to load the relevant installation assembly as the top-level assembly for alignment purposes. (Installation assemblies are assemblies above which no transforms are created.) You may have designs aligned from different installation assemblies. In the case of a large part structure, consider breaking it down such that it is not a completely flat structure. The high-level part structure need not mirror the design structure exactly. The context level at which you perform alignment may have designs aligned from different installation assemblies.

The publish link is independent of revisions and persists if the structures are revised, unless one of the associated structure lines is removed. It is preserved if the structure line is substituted or replaced.

For detailed information about aligning structures, see the *Multi-Structure Manager*.

Validating alignment

Teamcenter provides two tools to help assess alignment status:

- **Accountability check**
Allows you to verify that all publish links have valid sources and targets. Incomplete links are reported as having missing sources or targets, indicating incomplete alignment work. Any occurrences of parts that do not require a design are reported as having a missing source. You can use the **Requires Positioned Design** property in one of the structure editor applications to identify such occurrences.
- **Completeness check**

Allows you to identify part occurrences that do not yet have aligned design occurrences. The design occurrences must have a published transform and the part occurrence must have an associated primary design. In the case of flexible parts, a shape must be published. Unlike the accountability check, the completeness check excludes parts that do not require a design, for example, adhesives, lubricants, and consumables. You can only perform a completeness check on structures containing items of the **Part** type.

For detailed information about validating alignments, see *Multi-Structure Manager*.

Deciding order when visualizing a part structure

Teamcenter first looks for the in-context data, such as a PLM XML / JT file, directly associated with the part occurrence. If no in-context data is found, it then looks at the associated primary design revision for the PLM XML / JT file.

The following is the deciding order when visualizing a part structure. Any first match will be returned.

1. PLM XML assembly file attached directly to the part revision with absolute occurrence relation (in-context data).
2. JT file attached to the part revision with absolute occurrence relation (in-context data).
3. PLM XML assembly file from its primary design representation.
4. JT file from its primary design representation.

Aligning with allocations

You can use allocations to capture, locate, and manage dependencies between different *representations* of the product. They provide traceability of dependencies, unlike some other alignment methods such as part-CAD alignment. During the development process, a product may evolve through various representations. For example, during system analysis and engineering, you express the representation of the product in terms of a set of requirements the product must satisfy and a set of functions it is expected to perform.

As the product design matures, you may identify other representations, including the logical structure, the assembly structure and the manufacturing structure. All these representations are related and a change to one representation may necessitate changes to other representations. If you align these representations with allocation maps, the dependencies between elements are identified and the impact of any changes are more obvious.

Create and maintain allocations with Multi-Structure Manager.

Making alignments in Multi-Structure Manager

When you to collect related data into a collaboration context (for example, compositions of structures), you implicitly aligning the data elements. Typically, you align only part of the product structure or

elements of more than one structure in a collaboration context; you omit components that are not of immediate interest.

Aligning the structure externally

Teamcenter structure data can be aligned with data in external business and bill of materials (BOM) releasing systems.

A CAD designer is responsible for creating and maintaining the CAD view of the product data, while the release engineer uses the BOM to identify the actual parts required to manufacture the physical product. The BOM system is often the backbone of the company, as it drives manufacturing, purchasing, costing, logistic, and other related functions. BOM systems normally contain usage information, while the CAD and PLM systems contain part and design data.

The CAD system includes data about detailed part geometry and configured part positions. It allows users to view 3D models in a structure, determine how parts fit or do not fit with parts around them, and identify a part location in a given assembly by coordinates. It does not allow users to make global where-used searches or identify the parts that fit together in a particular assembly.

The BOM system allows users to identify if a part is used on a given product, to determine which parts can be used together, to view every application of a part globally, and to view change history. It does not include information about parts in the context of surrounding parts or location coordinates. In a typical company, business (BOM) data and engineering (CAD) data overlap but are not well aligned. Users of one system must obtain the data they need from the other system. Business users can then quickly visualize and correlate business status and engineering status of development programs. They can then determine completeness of these programs; that is, if you have designed all the parts necessary for all the combinations to build. They can also validate the design against order processing and confirm that all the parts fit for a specific product configuration. Change and release management processes can be connected to the visual representation, making virtual builds easier to maintain.

Teamcenter provides a number of capabilities for aligning with BOM and product master systems. Information on these capabilities may be found in the *Sharing Data* section of the Teamcenter help collection.

13. Using collaboration contexts

Using collaboration contexts	13-1
Purpose of Multi-Structure Manager	13-1
Understanding the collaboration context data model	13-3
Managing occurrences and absolute occurrences	13-4
Using occurrences in collaboration contexts	13-4
Using occurrences	13-4
Using absolute occurrences	13-4
Working with representations	13-5
What are representations?	13-5
Modeling with occurrence groups	13-5
Modeling with structure contexts	13-8
Modeling with compositions	13-8
Building structure contexts	13-9
Structure context principles	13-9
Applying variant rules and selected option sets	13-9
Using absolute occurrences in a structure context	13-10
Managing configuration changes	13-10
Creating compositions	13-11
Transferring data to external applications	13-12

13. Using collaboration contexts

Using collaboration contexts

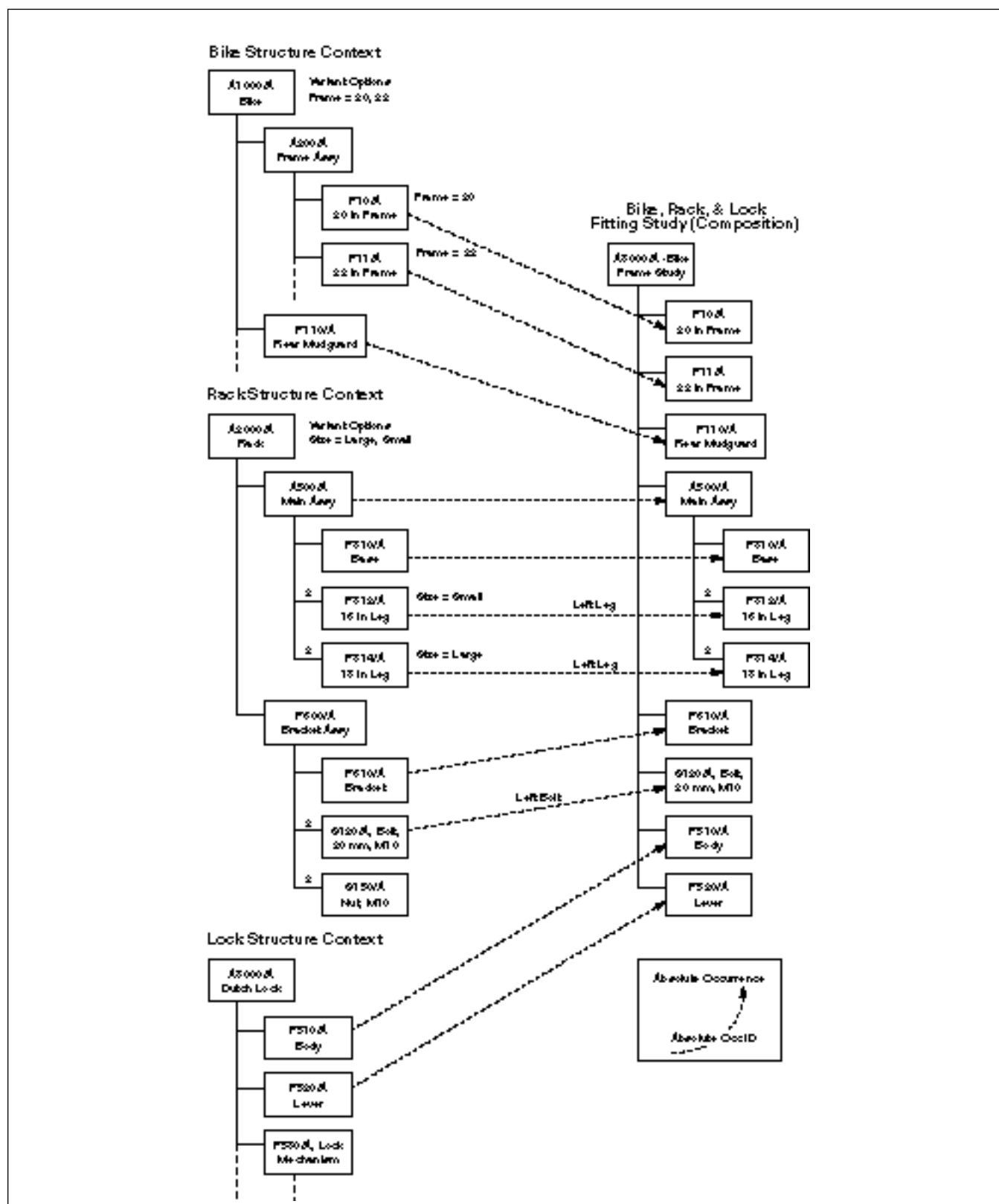
A *collaboration context* is a Teamcenter object that holds a collection of data in structure and configuration contexts. This data allows you to capture multiple different Teamcenter structures in one container. You can open a collaboration context in the Multi-Structure Manager application, in Manufacturing Process Planner, or in Part Planner. You can also use a collaboration context to collect data to share with a third-party application.

Purpose of Multi-Structure Manager

The Multi-Structure Manager application allows you to collect related data into a collaboration context (for example, *compositions* of structures) so you can visualize and work with components of interest in an uncluttered environment. Typically, the data you collect in a collaboration context contains only part of the product structure or elements of more than one structure; you can omit components that are not of immediate interest. The structures may be configured with revision rules and variants, and the structures can be configured differently if necessary.

There are two main scenarios where you could create a collaboration context:

- During the *design process* when you might create a composition with elements from different products. In this case, the composition is a design study in which you can bring together components from different products and position them relative to each other. You can reposition components in the composition without affecting the position in the original structure. Use the embedded viewer to make interference checks, create cross section views and add markups. The following figure shows how a design study may be created in a collaboration context as a composition.



Creating a design study in a collaboration context

- During *manufacturing process design* when you might create a composition of the process structure. This composition would include components consumed from the product structure and machines or tools from the plant structure for assembly and machining operations. You can develop machining operations and position tooling from the plant structure relative to the workpiece (the workpiece is a part consumed from the product). You can also define assembly operations, for example, positioning robots from the plant structure against components assembled in a particular operation. The composition may include occurrence groups representing collections of parts that are already assembled before entering the stage of the manufacturing process the composition represents.

You can also **create intermediate data captures (IDCs)** in a collaboration context.

Understanding the collaboration context data model

The Multi-Structure Manager feature uses the following workspace objects. You can create or view these objects with the My Teamcenter and Multi-Structure Manager applications.

- A *collaboration context* object

The collaboration context object contains one or more structure contexts that may be used to model the representations.

Note:

The collaboration context object is independent of the Multi-Structure Manager application. You can create, view and manipulate a collaboration context object in other applications. Likewise, you can create multiple collaboration context objects in a single session in the Multi-Structure Manager application.

- A *structure context* object

This object is a *representation* and contains the following:

- A *root item*—the top-level item in the structure of the representation.
- Optionally, *occurrence groups* that model the simplified representations and contain only a subset of the components in the structure—those components of interest for the current task.
- Optionally, a *closure rule* for data transfers. This controls the data transferred and specifies the necessary properties and how far to navigate relations to transfer associated objects or properties. The closure rule also specifies whether to transfer only changes on import operations and, if so, the incremental change with which they are associated.

You can create compositions using the default **CompositionContext** and **Process** structure context types. Alternatively, you can create a custom structure context type with the Business Modeler IDE application and declare it as a composition structure context.

- A *configuration context*. This contains a revision rule and (optionally) a variant rule or saved option set to control a specific configuration of the underlying structure.

Managing occurrences and absolute occurrences

Using occurrences in collaboration contexts

The collaboration context functionality supports occurrences (relative occurrences) and absolute occurrences in the same way as other Teamcenter applications.

Using occurrences

When you add an item to a product structure, you are creating a specific *occurrence* of that item or item revision in the product structure. It becomes a *component* and is displayed as a line in the product structure. You can create and manage occurrences in a structure from the Multi-Structure Manager application. If the structure is part of a representation and was created from a larger product structure, any such changes affect that product structure.

Using absolute occurrences

A representation may include components from anywhere in the product, positioned correctly in the product space. To achieve this, the Multi-Structure Manager application uses absolute occurrences to record the complete path, from the top level in the structure (the root item) to the specific component. This occurrence path is sometimes referred to as the appearance path node (APN). Absolute occurrences permit:

- **Data overrides**
You can attach data such as a transform to an absolute occurrence, and use it to override the data on the original structure (that is, the relative occurrence). This allows you to create design studies as composition structure contexts; the products can be positioned correctly relative to each other, without affecting the transforms on the original structure.
- **Component instance associations across structures**
As the absolute occurrence records the complete path, you can easily identify the different instances of the same component, for example, the left front wheel, or a particular bolt on the right, rear wheel. This is important for manufacturing purposes, when there are many instances of the same part, to ensure each component is identified and consumed in the appropriate operation.
- **Component instance identification across structures**
Optionally, you can specify an identifier for an absolute occurrence on the **ID in Context** property. You can then search structure contexts to identify where the component exists, for example, in the composition structure context and the original structure context from which the component was copied. You can then find and identify the corresponding BOM lines.

In certain industries, the absolute occurrence may have a particular purpose, for example, it may correspond to an ATA or CSN number that identifies the component. It may also identify a chapter, section, page and balloon number in a technical publication.

Working with representations

What are representations?

A *representation* is a general term that describes a structure or flat list of components and items. It may represent any product structure data. In a collaboration context, you can model a representation with an occurrence group, structure context, or composition.

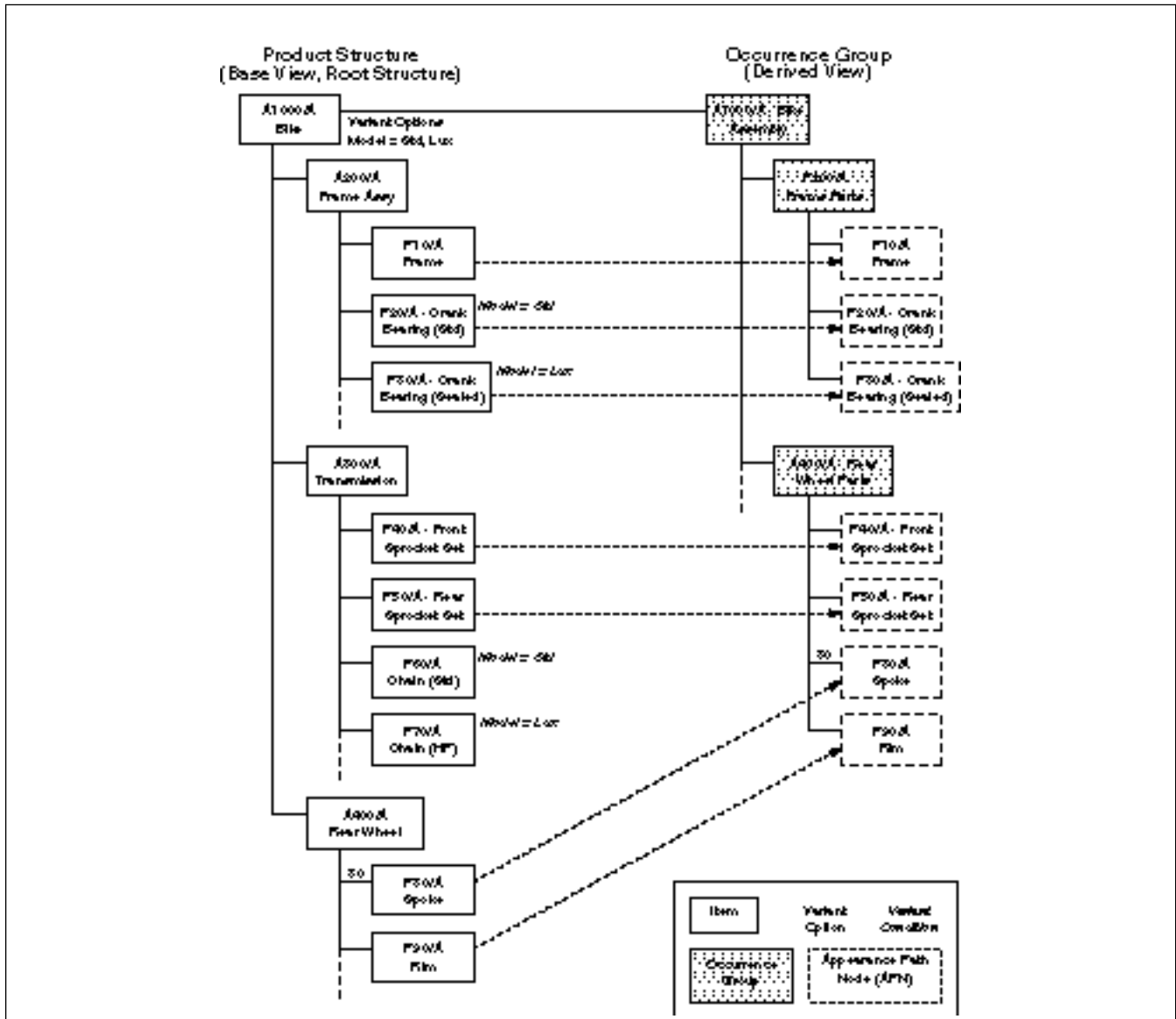
Modeling with occurrence groups

An occurrence group represents a subset of the components in the root item against which the occurrence group is created. This subset may collect components required for a design task, or group components to be consumed in a manufacturing operation. The base structure or view can have any number of associated occurrence groups, referred to as *derived views*.

Note:

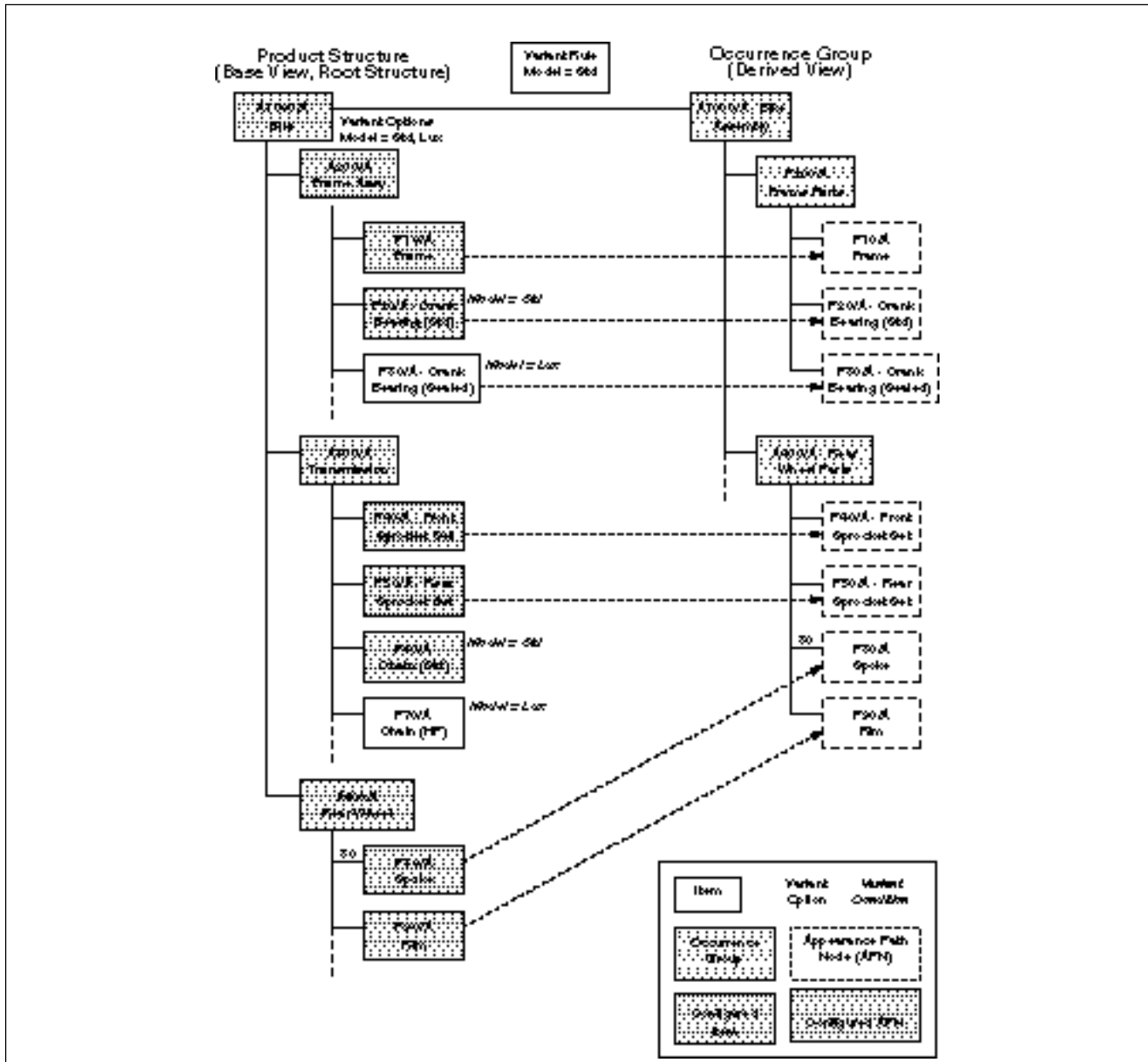
These views and derived views are **not** BOM views.

The following figure shows how an occurrence group may be used to create a derived view from the product structure.



Creating a derived view with occurrence groups

The following figure shows how the same derived view is configured.



Creating a configured derived view with occurrence groups

You can organize occurrence groups themselves into a structure. This can be useful when modeling a sequence of manufacturing assembly operations, in which each occurrence group contains the components consumed in that operation, as well as the occurrence group with the assembled components from the previous operation.

Unlike BOM views, the components in an occurrence group are closely linked to the root item in the base structure from which they were created. Some, but not all, structure changes are propagated from the base view to the derived views, specifically, remove, substitute and occurrence note structure changes. If you add a component to the base structure, you must also add it manually to the derived views.

The advantage of creating occurrence groups for representations, instead of BOM views or compositions, is that when you apply a variant rule to the base structure, it automatically configures the derived views.

Also, if you model a representation with an occurrence group, it can be created from the result of a spatial search.

Note:

If new components are created that enter the area defined for the spatial search (box or proximity), they are *not* automatically included in the occurrence group and you must manually reconfigure the search.

The Design Context application allows you to store the results of a spatial search in a persistent structure context object. Currently, the results of the search may only be reloaded in Design Context, not Multi-Structure Manager.

You can create occurrence groups with the Multi-Structure Manager application.

Modeling with structure contexts

A *structure context* is a specific configuration of a representation. It is similar to an occurrence group, but contains a persistent object, the *configuration context*. This stores the configuration of the structure, namely the revision and variant rules. The structure context also contains the root item.

To create a new representation of the same product:

1. Create a second structure context to hold the new representation.
2. Create a new top-level item as the root of the new representation.
3. Create new items as components of the structure context for the new assembly levels.
4. Assign components from the base product structure in the original structure context to the appropriate assemblies under the root item of the new structure context.

Unlike an occurrence group, which may only contain a subset of the components in the base structure, a structure context may contain additional items that are not in the base structure. In an MBOM, the manufacturing representation may include consumed items such as sealant and welding rods that are needed to manufacture the product and must be specially ordered.

Modeling with compositions

A composition is a special type of structure context that allows components to be added from several structure contexts representing different product structures. There are two kinds of scenarios in which compositions may be used.

Building structure contexts

Structure context principles

You can create a structure context with the Multi-Structure Manager application by copying and pasting components from one structure context or representation to another. Teamcenter creates an APN as well as an absolute occurrence, allowing data overrides that do not affect the original structure context.

A structure context is a collector object that allows you to associate occurrence groups and root structures to define subsets of the source structure. One typical use of a structure context is to reorganize the engineering BOM (EBOM) to create a manufacturing BOM (MBOM).

Alternatively, if you want to create a view of a subset of lines from a single root, but not change the content, use occurrence groups. You can reorganize the lines in an occurrence group, but not change the content. Any changes to the source are reflected in the occurrence group, as are any revision rule changes in the source.

Tip:

Use occurrence groups for reorganization if there is only one view and you do not need to change the content; otherwise, use a structure context.

Note:

Instead of copying and pasting, you can also use the **Assign** action. This allows you to define an occurrence type (for example, **MEConsumed**) for use in Manufacturing Process Planner. This allows propagation of variant configuration from the base structure context to the composition. Only classic variant data is propagated, not modular variant data.

You can also assign an occurrence group that is created for the root item in a structure context to a composition, as well as assigning components directly from the structure under the root item.

A structure context allows you to reorganize components in an existing structure into a different structure by creating an occurrence group. In addition, if the structure context is of the **Composition** type, the root item created for the study or process may have components added from other structures, as well as newly created components. This is not possible with occurrence groups.

Applying variant rules and selected option sets

If you apply a variant rule or selected option set to the original structure context, it does not configure the components in the composition to which they are assigned. (The absolute occurrence mechanism does not support this functionality).

If the components are consumed from occurrence groups, rather than directly from the product structure, the variant configuration configures the operation, as follows.

- When you assign a component from the product structure to the structure context, the component is added to the process structure as an occurrence of the **MEConsumed** type. (This is the default and you can configure another type.)
- When the process structure is configured for components of the **MEConsumed** type, Teamcenter configures the product structure containing the consumed components in a background BOM window by applying the variant rule.
If the assigned component is not configured, the operation in the process structure is not configured. In addition to **MEConsumed**, the **MEOther** occurrence type can be used. If so, the operation is not unconfigured unless all the consumed components are unconfigured.
- The unconfigured component in the base structure becomes unconfigured in the operation.

Using absolute occurrences in a structure context

Use absolute occurrences to create the links between components in a structure context and the composition to which they have been assigned. You can specify an absolute occurrence identifier for a component in the structure context, and this is shared with the component in the composition to which it has been added or assigned. You can then search for the absolute occurrence identifier in the composition and in the structure context from which the component was added, to identify the associated components (the associated components share the same absolute occurrence identifier).

Managing configuration changes

It is necessary for each representation of a product structure in a collaboration context to have an associated configuration context. This includes a revision rule and variant rule or selected option set that controls the specific configuration of the underlying structure. For example, you may create a representation for the structure released on a certain date and for a specific variant. A separate representation (structure context) would be required for the same structure if it is configured for a different date or variant. If you do not specify a configuration context, your default revision rule is used and no variant configuration is made.

When you work on evolving designs, it is important that the representations and compositions reflect changes made to the original structures, without having to recreate the representations. This happens automatically if you choose a **Working** revision rule for the composition. If you create a new working revision, it is configured automatically by a **Working** revision rule, not by a **Released Revs only** revision rule.

Propagation only applies to geometry changes resulting from revision changes to piece parts, not to structure changes resulting from revision changes to assemblies; changes are propagated only when components are added to or removed from the base structure. However, replacement structure changes are propagated.

As mentioned previously, changes to transforms, attachments or geometry that are made in the composition are not propagated back to the original structure. These changes are stored as additional data in the composition structure context and are associated with the absolute occurrence that is attached to the root item of the representation.

Creating compositions

A *composition* is a structure that relates other structures of variable configurations and contains a set of occurrences or filtered occurrences. A composition typically models a scenario or manufacturing process. For example, you may place a product view of an assembly and the corresponding process view in a composition for review and approval purposes.

A composition may contain any of the following:

- An instance of the representation of any top-level item. In this case, the composition includes the complete representation.
- Instances of occurrences from other representations. A new instance is created and associated with an occurrence in the source representation.
- Instances of occurrence groups. An instance of the representation of the top-level item is created and filtered to show only the occurrences that are members of the occurrence group (with its subgroups and members).

Caution:

You should only manage compositions in Multi-Structure Manager, not in Structure Manager or other structure editors.

A composition includes links between its BOM lines and the source or reference structure. To ensure those links are updated when necessary, always open the composition (a structure context) and the associated reference structure in the relevant collaboration context object with the Multi-Structure Manager application.

When you create a composition, you can manage and edit the occurrence groups in it in the same way as any other BOM line, including:

- Attaching data to occurrence groups in the context of the top line composition or any other context. This action creates a persistent absolute occurrence and the occurrence path representing the line in the composition.
- Attaching a 3D markup to them in the viewer.
- Changing their position in any context. If you change the position of a member of the group, the in-context edit must be set to the appropriate context. If no editing context is defined, the relative position is changed.
- Defining item elements (GDEs) and connections as part of the occurrence group.
- Creating connection interfaces on the members of the occurrence group.
- Comparing occurrence groups and their members.

- Creating and retrieving 2D product views (snapshots) in compositions containing instances of occurrence groups.
- Creating, editing, and manipulating graphics.
- Using any member of an occurrence group as the top-level context for creating and editing absolute occurrences.
- Configuring members of an occurrence group to match the configuration of the source view. If a member does not exist in the source, it is unavailable in the composition.

If additions, removals, or changes are made to the source BOM line, the occurrence line in the composition is updated accordingly.

If an occurrence is a member of several occurrence groups in a single hierarchy, it is treated as a single occurrence for all purposes.

Transferring data to external applications

Having created representations, the Multi-Structure Manager application allows you to transfer the data in PLM XML format to and from external applications. These applications allow you to edit the structures or data attached to the components; for example, schematic applications for wire harnesses, pipework and hydraulics, or Tecnomatix for assembly process design.

When you import data into Teamcenter, you can configure the transfer such that only changes are imported. The changed data may be attached to an incremental change object in Teamcenter, allowing the changes made in the external system to be tracked. A closure rule associated with each representation controls the level of data transferred, including the necessary properties and how far relationships are navigated to transfer associated objects or properties.

Teamcenter provides Web services to manage the transfer of the PLM XML data files to and from the external application. It creates a persistent workspace object (known as the application interface or AI object) for each transfer to store information about the data transfer, namely:

- The closure and filter rules.
- If the changes made in the external system are tracked by incremental change.
- The synchronization status—if the data has been transferred and read into the external system, or is waiting for the Web service to complete the transfer.
- The contents of the PLM XML file.

You can create and transfer the PLM XML files manually outside the Web service mechanism. Teamcenter still creates an AI object, but the files are stored in the operating system.

You create an AI object and the object stores the PLM XML file against it in the database. You can create additional AI object types with different closure rules, filters and transfer modes. The input to an AI object for data transfers may be a collaboration context, structure context or an item, but the closure rules are different in each case. Transfer can be initiated by a workflow process, for example, when uploading data to ERP.

14. Manufacturing product structures

Manufacturing product structures	14-1
Using Multi-Structure Manager	14-1
Using Manufacturing Process Planner	14-2

14. Manufacturing product structures

Manufacturing product structures

Teamcenter manufacturing process management includes several applications that allow you to create and manage manufacturing data for the defined product structure. For a detailed description of Teamcenter manufacturing process management, see *Manufacturing Process Planner*.

Using Multi-Structure Manager

Multi-Structure Manager allows a manufacturing assembly planner to rearrange the product structure (which is often referred to as the EBOM or Engineering BOM) so that the components are grouped logically for the assembly process. This grouping process creates the MBOM or Manufacturing BOM. The planner does *not* modify the *as designed* product structure, but rearranges the same components into different groups. The MBOM is a structure of parts and differs from the process structure you create in Manufacturing Process Planner, in which you create a structure of processes and operations referred to as a BOP (Bill of Process). If you are exchanging data with an ERP, you may transfer the MBOM, BOP or both, depending on the business needs.

Multi-Structure Manager allows you to collect components from the base product structure into occurrence groups referred to as *derived views*.

Note:

Do not confuse derived views with BOM view types.

You create a top-level occurrence group that is associated with the top-level item in the product structure. Under the top-level group, you copy from the product structure components that are in the final assembly operation. You then create another occurrence group, as a component of the first occurrence group, containing the components assembled in the previous operation. In this way, you can create a structure of occurrence groups with the lowest levels being the initial assembly operations, representing in process assemblies that are consumed by the parent occurrence group (assembly operation). This structure is repeated until the top-level occurrence group—the final assembly operation.

The relationship between components in the base view and the derived view is as follows:

- Structure changes to the base view such as removal or substitution of components and changes to occurrence notes) are automatically propagated to the derived views. Added components are not propagated as this requires a specific user decision, so such components must be added manually to the derived views if appropriate.
- If a variant rule is applied to the base structure, it automatically configures the derived views. Thus components become unconfigured from the derived view if the corresponding components are unconfigured in the base view.

Using Manufacturing Process Planner

Use Manufacturing Process Planner to assign components from the product structure to the operation in the process structure (BOP) that consumes them. You can also assign occurrence groups representing collections of components to the operation in which they are assembled.

As you consume the parts into the BOP, Teamcenter automatically generates a unique in-context ID for each consumed part. (If an in-context ID already exists in the product structure, it is reused.) This allows you to create a process plan that you can utilize for multiple products with different item IDs but common in-context ID. For example, the piston in a 1.6 liter engine has the same in-context ID as the corresponding piston in a 3.0 liter engine but a different item ID.

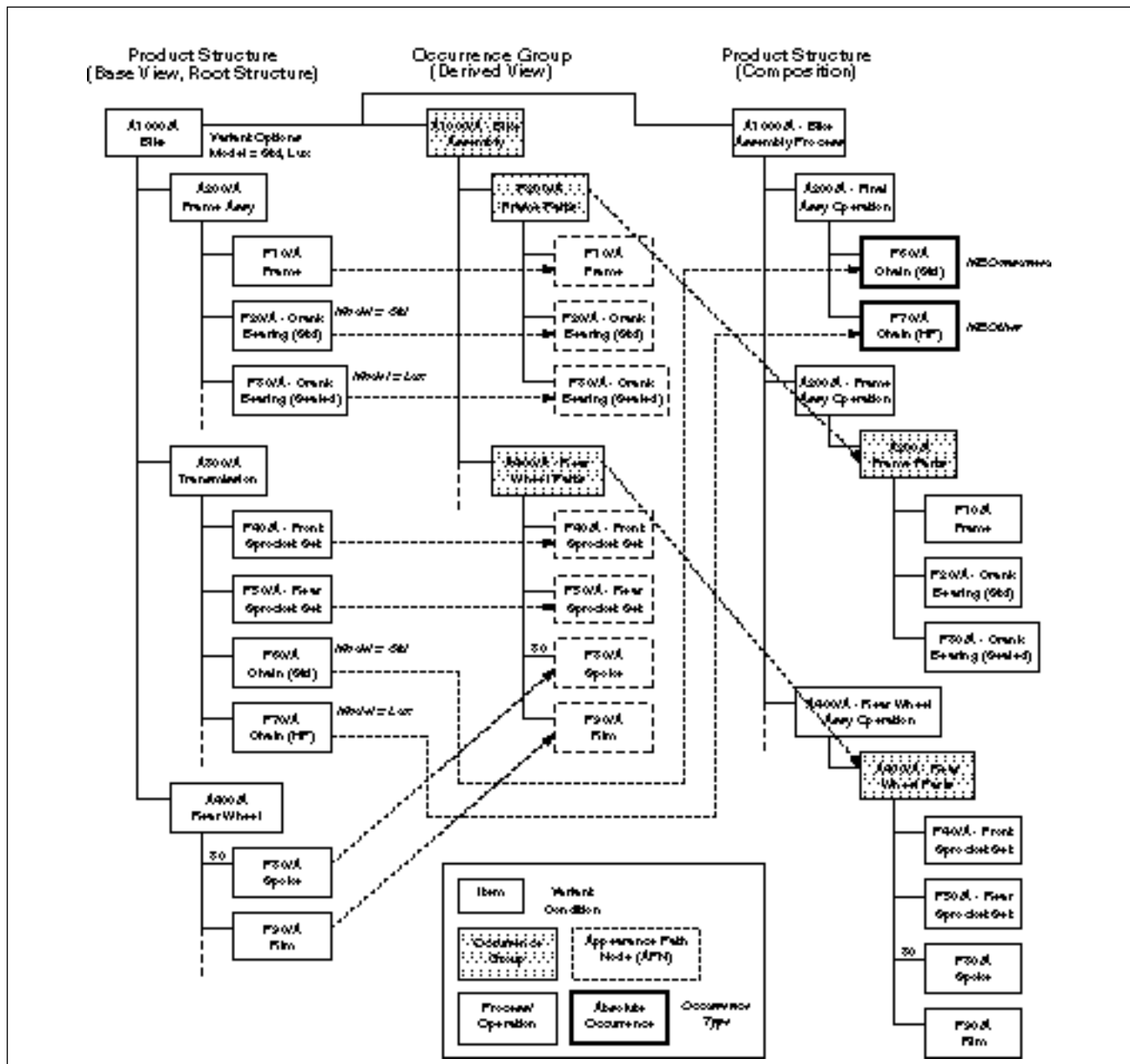
Note:

When it is initially assigned, the link is created from the BOP to the BOM and not vice versa, if the structures are not expanded. When the structures are expanded, you can also navigate back from the MBOM to the BOP.

The in-context ID is equivalent to an automotive catalog number or an aerospace ATA or UIN number. You can use it to represent the item label in a generic company numbering system or industry standard.

When you copy a component from the MBOM to the BOP, Teamcenter copies the transformation matrix from the product assembly to the BOP line. You can reposition the component or assembly according to its actual position in the plant or assembly line at that point. For example, you may model a car in design with the doors shut, but in the assembly process, the doors are opened. To achieve this, you override the transformation matrix in the context of the specific operation. In the next operation, the doors may be closed again. The revision rule of the product structure controls the revisions of the pasted items, not the revision rule of the target BOP or process.

The following figure shows how occurrence groups may be used to configure a manufacturing process operation.



Configuring process operations for a product with occurrence groups

The variant rule applied to the base view configures the derived view, for example, the unconfigured bearing, using the APN mechanism. The variant rule applied to the root structure unconfigures the chain from the composition by the **MEOther** occurrence type. Another occurrence type, **MEConsumed**, unconfigures the whole operation if any of the unconsumed components become unconfigured. A comparison of the structures shows that the chain is not allocated to the derived view. However, a comparison or search of absolute occurrences identifies the chain in the root product structure and the process structure.

When you assign components or occurrence groups to an operation, they are assigned with one of the following occurrence types:

- **MEConsumed**

When you apply a variant rule (in classic variants) or a selected option set (in modular variants) to the product structure, if a component becomes unconfigured, the associated operation in which it is consumed also becomes unconfigured.

If several components are consumed in an operation, and any one of them becomes unconfigured, the operation becomes unconfigured.

Note:

The **controllingOccsForProcessConfiguration** preference defines if occurrence types other than **MEConsumed** behave in this way.

- **MEOther**

This occurrence type behaves similarly to **MEConsumed** except that the component in the operation becomes unconfigured, rather than the operation itself.

- **MEWorkArea** and **MEResource**

When you assign a work area from the plant structure to an operation in the process structure, Teamcenter creates an occurrence of the work area in the operation of the **MEWorkArea** type.

Similarly, you can add a resource (such as a mallet or wrench) from Resource Manager to an operation as an **MEResource**.

- **Link as Required**

This occurrence type is used to link product items that are required to carry out an operation (this relationship is only available for operations). The **Link as Required** relationship does not add an occurrence below the operation, unlike when you assign a product item to an operation as **consumed**. You typically use it to relate components that have variants and can be configured out of the product. In this case, the particular operation is necessary only if the required components are in the in-process assembly when the operation is performed. For example, if you are building a lawn mower with an automatic starter, the operation that assembles the switch to the dashboard has the switch as a required item. If you are building a lawn mower without an automatic starter, this operation is configured out because the switch is configured out of the product structure.

You can assign components to an operation with Manufacturing Process Planner or Multi-Structure Manager. In Manufacturing Process Planner, you can choose an **Assign Consumed** action to assign a component from the product structure to an operation with the **MEConsumed** type. Multi-Structure Manager allows you to assign a component and choose the required occurrence type to assign.

In Manufacturing Process Planner, you can also filter the structure by occurrence type, for example, to hide all occurrences of the **MEResource** type.

15. Using allocations

Understanding allocations	15-1
Understanding allocations	15-1
Choosing allocations or trace links	15-4
Implementing allocations in Teamcenter	15-5
Understanding allocations in Teamcenter	15-5
Making allocations between configured structures	15-5
Making conditional allocations	15-7
Creating allocations	15-8
Revising allocations	15-9
Configuring allocations	15-10
Releasing allocations	15-12
When can I release allocations?	15-12
Releasing an allocation	15-13
Releasing an item with an allocation to a physical component	15-13
Releasing allocation maps	15-13

15. Using allocations

Understanding allocations

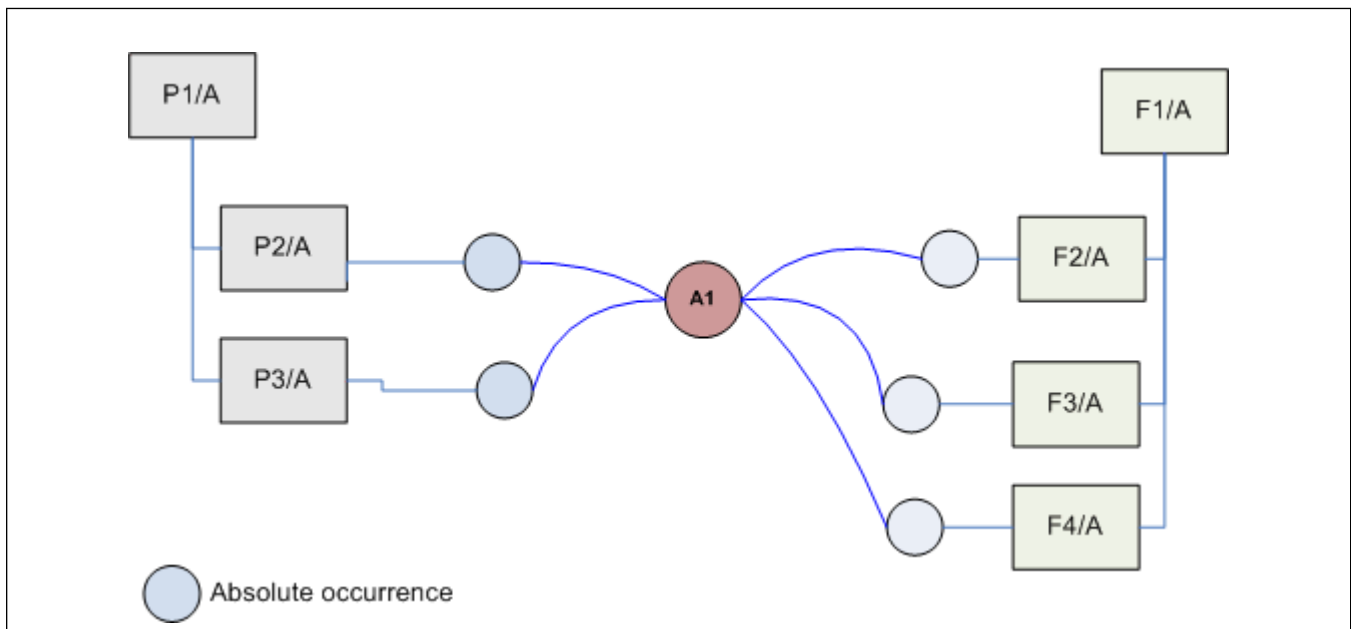
During the development process, a product evolves through various revisions of the product structure and its components. It also evolves through various *representations*. For example, during system analysis and engineering, you express the representation of the product in terms of a set of requirements the product must satisfy and a set of functions it is expected to perform. As the product design matures, you may identify other representations, including the logical structure, the assembly structure and the manufacturing structure. All these representations are related and a change to one representation may necessitate changes to other representations. For example, a change to one of the product requirements may cause the product to perform an additional function. A new physical component is added to perform this function and, in turn, a new manufacturing process is needed to assemble this component into the product. You can capture, locate, and manage these dependencies between representations with allocations.

Understanding allocations

An allocation represents a directional relationship between a specific instance of an item revision in one product structure and one or more item revisions in another structure. For example, you might relate lines in the functional structure to the logical structure or the logical structure to lines in the physical structure. These relationships may span multiple configurations and revisions of both structures. They may change according to the overall configurations or the specific revisions that are configured into the structures.

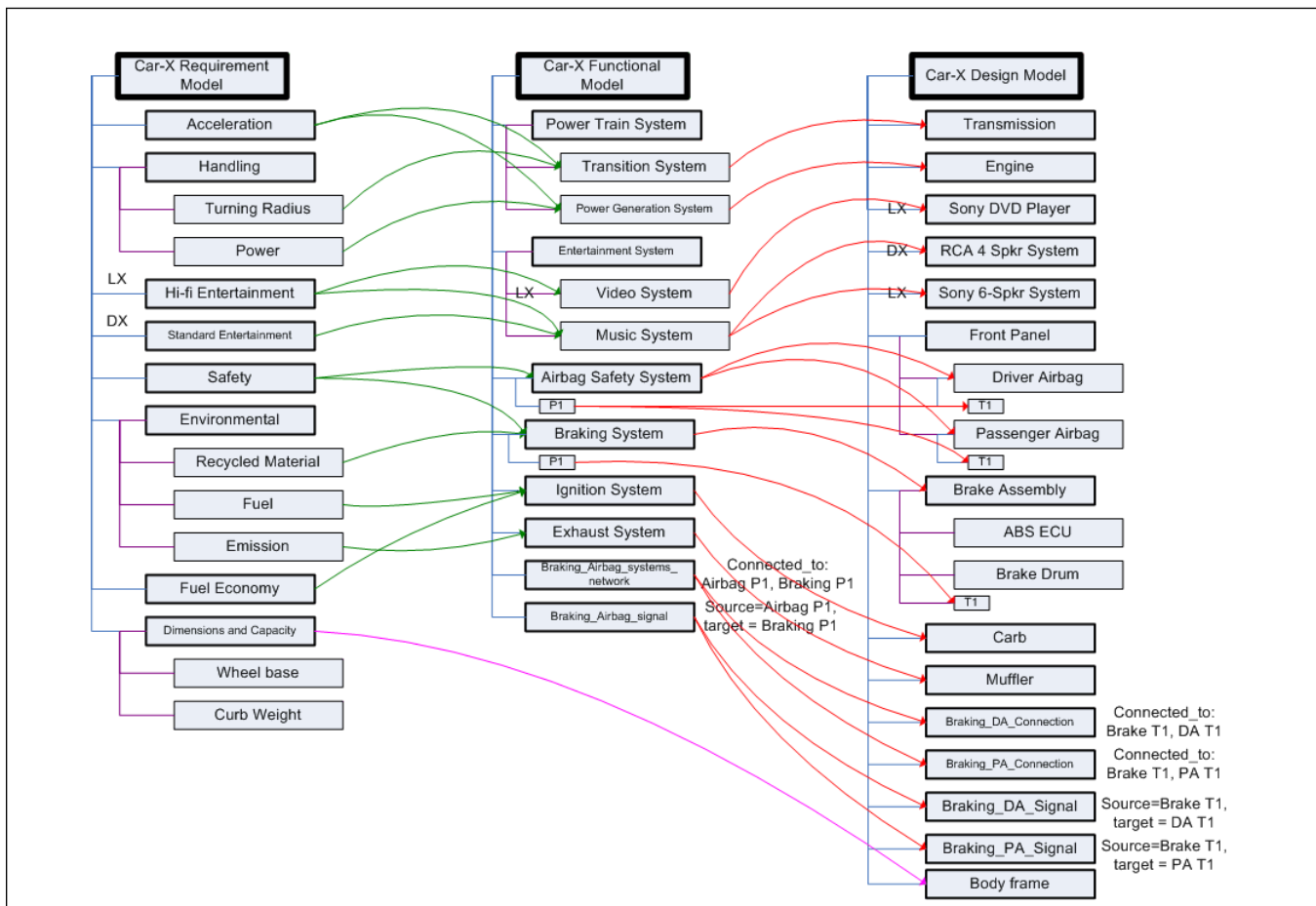
A group of allocations that map together two or more structures is referred to as an *allocation map* or allocation context. An allocation map is a Teamcenter object that can be revised separately from the structures that it maps. However, depending on your business rules, you may want to revise the allocation map at the same time as you revise the associated structures. When you revise an allocation map, you create an allocation map revision in the same way that revising an item creates an item revision. Depending on your business logic, the allocations in the allocation map may be carried forward or dropped from the new allocation map revision; this behavior is determined by preference settings.

You can use allocations to map multiple sources to multiple targets in more than one structure, if the combinations are valid according to your business rules. In the following example, allocation A1 maps two sources (**P2/A** and **P3/A**) to three targets (**F2/A**, **F3/A**, and **F4/A**).



Allocations between multiple sources and targets

You create allocations that map several views or representations of the same structure. For example, you may create three allocation maps that map the requirements, functional and assembly views of a product, as follows.



Allocations between multiple views or representations

To achieve this mapping, you would create three allocation maps in three different contexts:

- **Req-Funct-Map/A** between the requirements view and the functional view.
- **Req-Assm-Map/A** between the requirements view and the assembly view.
- **Funct-AssmMap/A** between the functional view and the assembly view.

Note that you can only work with a maximum of two open structures at any given time. Consequently, you must create these three allocation maps one at a time.

Each of these allocation maps may also be revised independently of the others. The requirements view may be managed in Teamcenter systems engineering and requirements management, while the other views are managed in core Teamcenter.

If you apply configuration rules independently to the source and target structures, you may cause inconsistencies between components configured in the two views. You can apply the same configuration criteria to the source structure, target structure and the allocations mapped between them. To achieve this, you can create a collaboration context and the corresponding configuration

context in the Multi-Structure Manager application. When you place your structures and allocation map in the collaboration context, the specified configuration context configures everything in the collaboration context identically unless you manually override it.

Allocations have a direction, that is, each allocation has a source and a target, for example, **Function 4 is allocated to ECU 1**. An allocation whose source is occurrence A and whose target is occurrence B is not the same as an allocation whose source is occurrence B and whose target is occurrence A.

You can make a *where allocated* check on a selected occurrence to view its relationships in multiple representations. Teamcenter shows the occurrences mapped to the selected occurrence as sources or targets, depending on whether you chose an *allocated from* or an *allocated to* check.

You can optionally implement user exits to determine the allocations between structures are complete and correct according to your company's business rules. For details of how to implement user exits with allocations, see the *Server Customization*.

Choosing allocations or trace links

Teamcenter provides two methods of linking objects in a structure, *allocations* and *trace links*. To help you choose the most appropriate method, the following table lists their advantages.

	Allocation	Trace link
Can additional attributes be added to relationship?	Yes	Yes
Relationship supported with additional data object?	Yes	No
Can additional data object have occurrences?	No	Not applicable
Can be extended in the Business Modeler IDE?	Yes	Yes
Configuration management support?	Yes (revision support for allocation maps; incremental change for allocations)	No
Multi-Site Collaboration support?	No	No
Directional?	Yes	Yes
Maps how many structures?	<i>n</i> (each occurrence must still be a source or target)	Two (typically)
Map <i>n:m</i> objects?	Yes	No
Accountability check support?	No	Yes

Implementing allocations in Teamcenter

Understanding allocations in Teamcenter

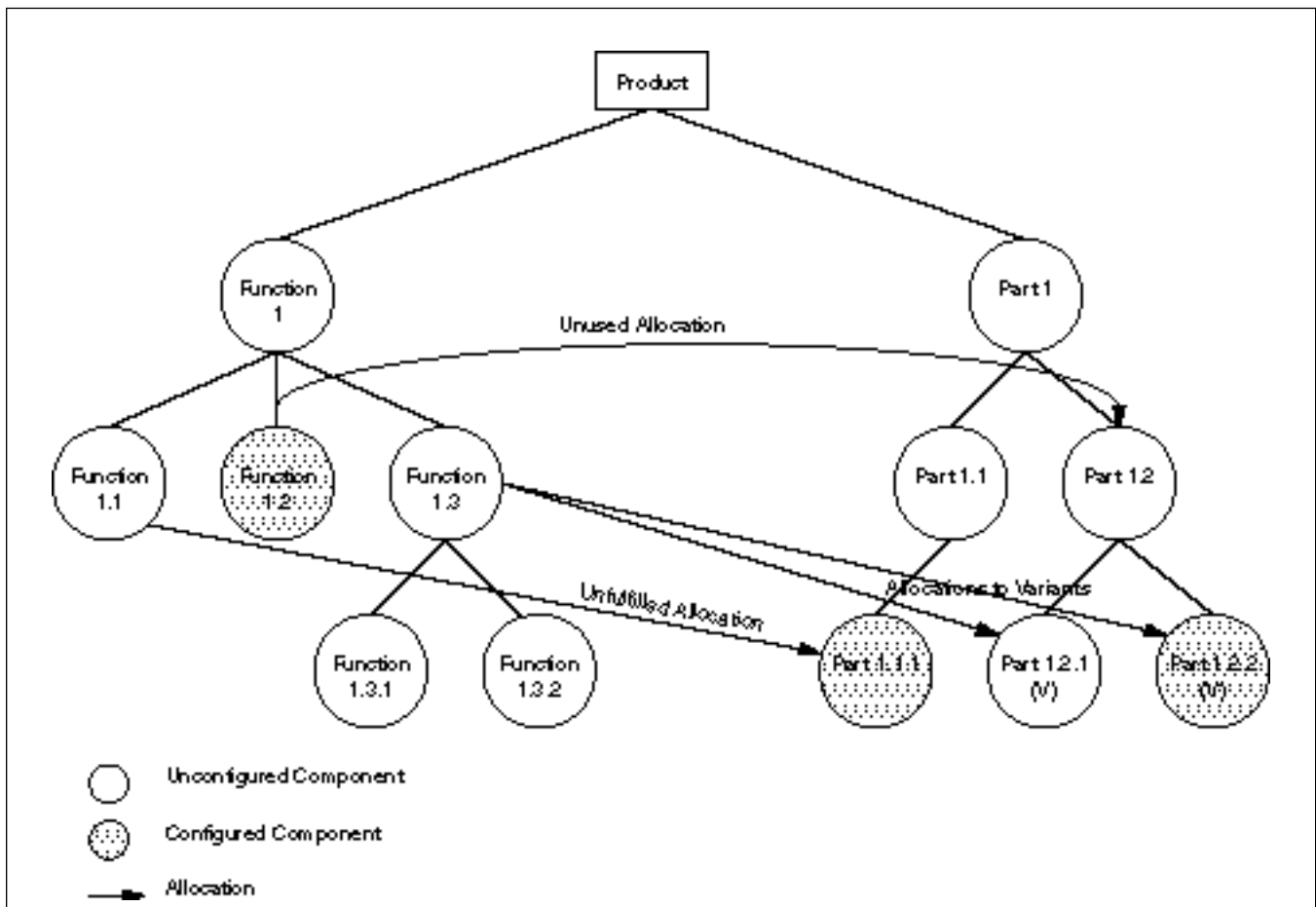
Allocations represent a directional relationship between specific instances of an item revision in one product structure and an item revision in a related structure. For example, you may relate the functional structure to the logical structure or the logical structure to the physical structure. These relationships may span multiple configurations and revisions of both structures. They may change according to the overall configurations or the specific revisions that are configured into the structures.

Making allocations between configured structures

You can use configuration management to determine those components that appear in a target structure. When those components are configured, you can determine the allocations that are possible.

You can manage configurations in Teamcenter by placing options and variants on components in the structures. You can then place allocations between the components in the various structures; these allocations have no conditions associated with them. If a source component is configured in, the associated allocations are configured in. If the target component is not configured in, the allocation is not configured in. You can create multiple allocations from one source component to various target components.

In the following figure, the components shown in white are configured in, while the components shown in gray are configured out.



Configuring allocations

In this example, allocations exist between various components, and there are three basic cases as follows:

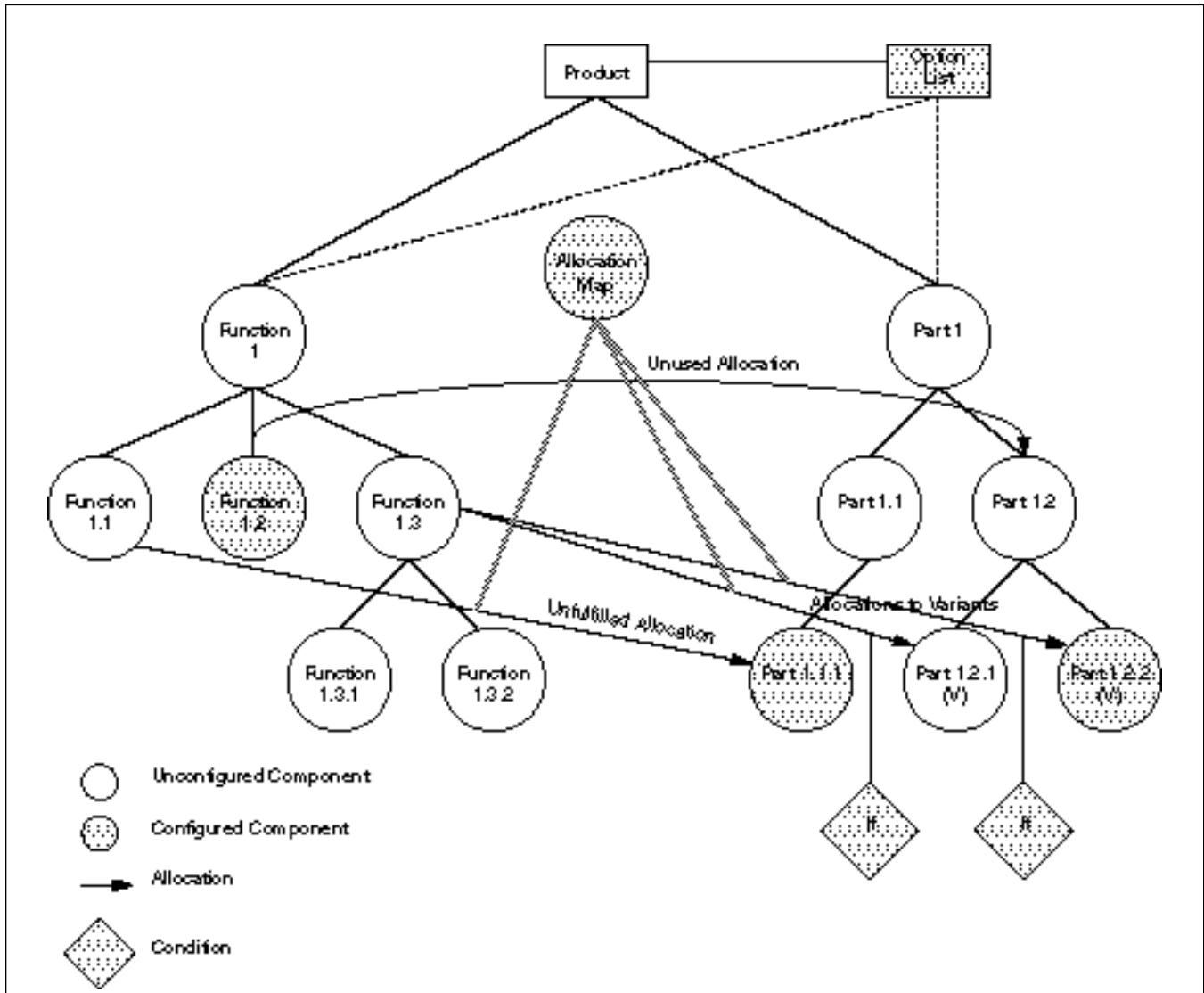
- Allocation from function 1.1 to part 1.1.1
Part 1.1.1 is configured out, so the allocation is configured out or invalid.
- Allocation from function 1.2 to part 1.2
Function 1.2 is configured out, but part 1.2 is configured in. As the source component is configured out, the allocation is configured out as an unused allocation. (The allocation is not evaluated because Teamcenter does not need to look for allocations related to this function.)
- Allocations from function 1.3 to part 1.2.1 and to part 1.2.2
Part 1.2.1 and part 1.2.2 are variants of each other. Only one part is configured in at any time. The allocation to the part that is configured in is valid, and the allocation to the second part is configured out.

Making conditional allocations

You can also use allocations to determine the components included in the target structure. In this case, you can configure the source structure, and also the allocations. The basic configuration rule (if a source component is configured out, the allocation is ignored) is still enforced. From the list of allocations that are configured in, Teamcenter determines the relevant set of physical components. This may be achieved by a conditional relationship, in which you can define certain criteria by which a functional component can be fulfilled by one of many physical components. In this case, the list of parts is not related to a product structure, but is simply a list of parts. When the target parts are part of a product structure, Teamcenter determines the instances of a component to fulfill the functionality that is specified.

If you use conditional relationships, you can define a group of allocations with a set of options that are relevant to the product. You can define various conditions with these options. For example, you could set an option to describe the model of the car. Depending on the model, a particular allocation may be required. As the targets of these relationships are part of a product structure, you can apply configuration rules to the structure. If an allocation is unfulfilled, Teamcenter reports an error that the target structure is not configured properly because there are unfulfilled allocations.

The following figure shows how options and conditions may be used to manage conditional allocations.



Configuring conditional allocations

Creating allocations

You can create allocations at various levels of a structure, but in each case they specify an object of interest—one that performs a particular function. It is possible that every item revision in a structure could contain an allocation. For example, this may be necessary for manufacturing purposes, where all items must be consumed.

Because allocations are independent of the structures they map, you can create and manage them independent of the structures themselves. For example, a functional designer may design the functional model, whereas a mechanical engineer designs the physical model. When both structures are complete, a systems engineer may map the functional model to the physical model. The systems engineer may trade off studies between allocating functions to different components and perform the actual assignments.

Note:

The model is the representation of a particular aspect of the product, not the CAD model. For example, the functional model of a car represents a product containing an air conditioning system, power braking, power steering, and other functionality. In Teamcenter, the model is represented by the product structure.

When you create an allocation, Teamcenter automatically assigns it a unique identifier. The identification scheme is determined by user exits set by the Teamcenter administrator.

Because you can configure structures in Teamcenter, you can create allocations between variable product structures. Allocations are valid only in a particular context and therefore all allocations between two structures are part of a group, and Teamcenter evaluates them as a unit. The allocations tie together various components and revisions of items in the structures.

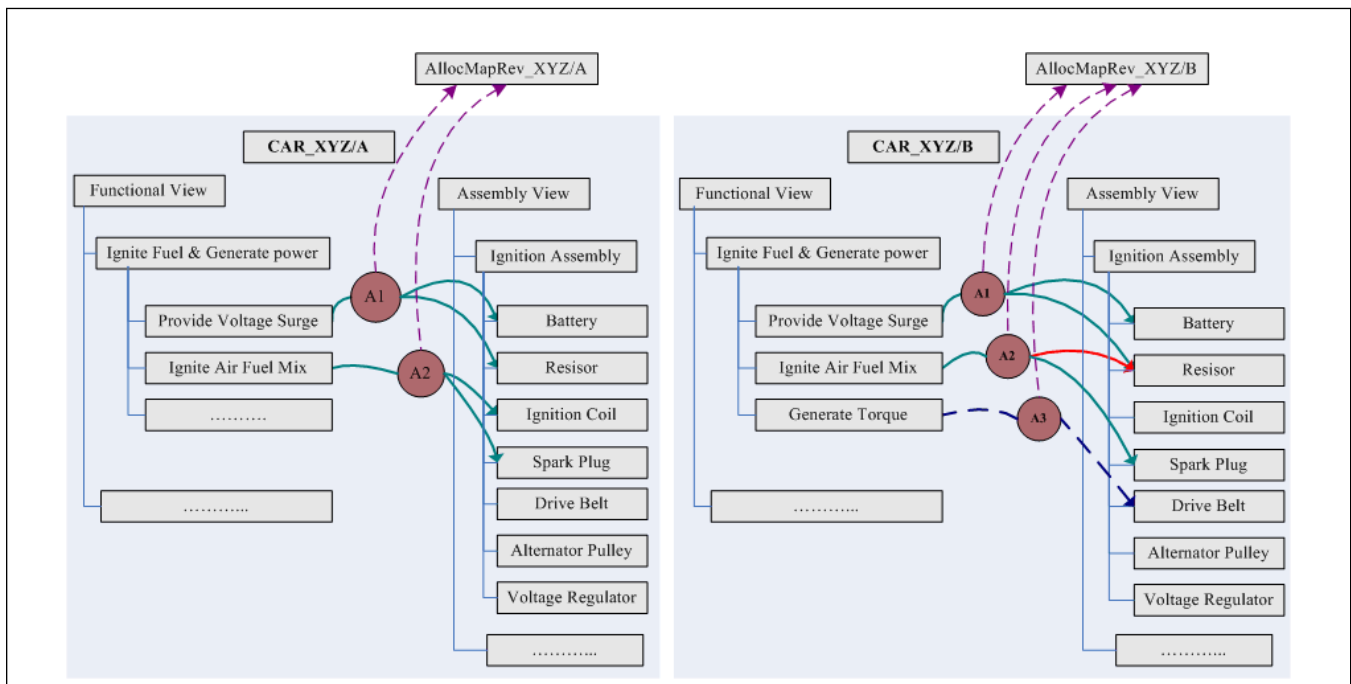
Optionally, a customizer can create subtypes of allocations and allocation maps using the Business Modeler IDE. The Teamcenter administrator can set preferences that determine the allocation subtypes that are valid in a particular allocation map type and, similarly, the allocation maps that are valid between structure type (BOM view types). The administrator also determines whether each allocation subtype is valid for:

- One source and one target.
- One source and multiple targets.
- Multiple sources and one target.
- Multiple sources and multiple targets.

Revising allocations

Allocations do not depend on a particular revision of a item. The allocation exists between the representation of the source item in the source structure and the target item in the target structure. The target item is not a specific revision because all revisions of an item are (by definition) compatible in form, fit, and function. For example, if a source item is revised from revision A to revision B, you can optionally use the same allocations between revision B and the target. To do this, the administrator can configure the system so that, when the allocation map is revised at the same time as the item, all the allocations are carried forward.

The following figure shows allocation **A2** assigned to allocation map revision **XYZ/A**. It has the function **Ignite Air Fuel Mix** in the functional view, which is allocated to the **Ignition Coil** and **Spark Plugs** components in the assembly view.



Revising allocations

When you revise allocation map **AllocMapRev_XYZ/A** to **AllocMapRev_XYZ/B**, Teamcenter optionally copies the entire set of allocations to the new revision. You can then edit the existing allocations, add new allocations or delete selected allocations.

In the example shown, allocation **A2** is edited with respect to the target structure, and is now allocated to the ignition coil and the resistor.

Similarly, revision **B** includes a new allocation between the functional component **Generate Torque** and the physical component **Drive Belt**.

Note:

By default, Teamcenter does not carry forward allocations when you revise an allocation map. To configure this behavior, your administrator sets the necessary preference.

You can delete the source or target of an allocation (or both) and Teamcenter does not delete the allocation itself. If appropriate, you can make a new assignment to replace the deleted source or target.

If appropriate, you can manage changes to allocation with incremental changes. Allocations may be added to or removed from an allocation map in the context of a specified incremental change order.

Configuring allocations

You can define configurations in which an allocation is valid. A *nonconfigurable* allocation represents a relationship between two items in two structures. The allocation is fixed and always exists if the items

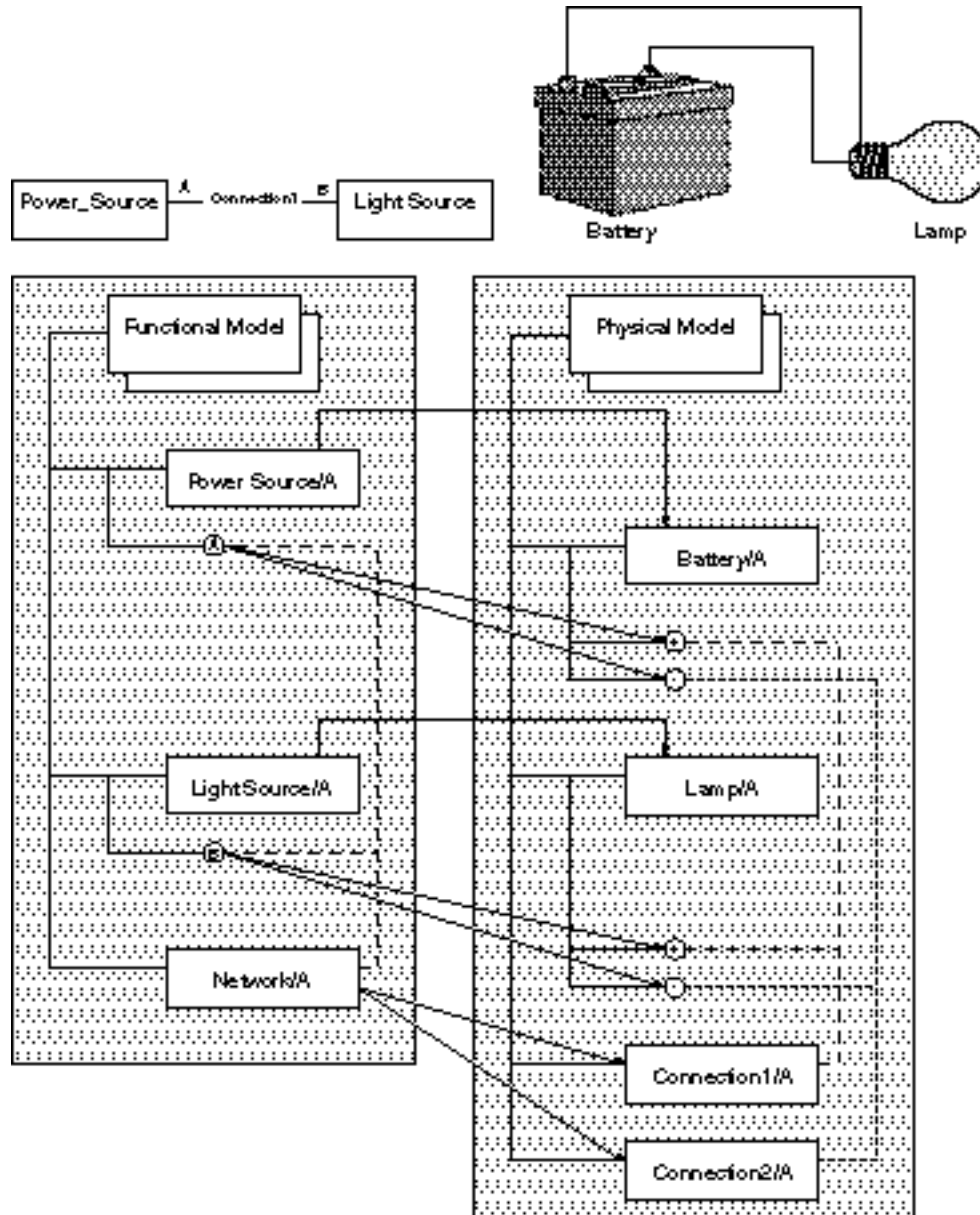
exist in the structures, for example, **Function 1 is allocated to Part 2**. However, if an allocation is configurable, you can attach conditions and configure it in a similar way to items in a structure. By associating conditions with relationships, you can configure various allocations in and out of a product. For example, this allows you to manipulate whether a function is allocated to a certain engine control unit (ECU). For example, you could put a condition on an allocation of the form **If Model==Sport and Color == Green then configure this allocation**. On the second allocation, you could use the condition **If Model == Sport and Color == Blue then configure that allocation**.

Once the option values are set, you can determine the necessary set of allocations. For each of these allocations, if the source is not configured in, the allocation can be discarded. If the destination is not configured in, the allocation can be noted as unallocated or discarded, depending on your site's business practice.

To define the context of a set of allocations, you must define an allocation map, as described previously. The allocation map ties together the two structures that are being allocated and provides the necessary context.

When you create allocations, the structures you work with are typically unconfigured products. You can then create allocations between many items in the structure; multiple allocations can originate from a source item and be allocated to several target items. If the source item is configured out, the allocation is assumed to be configured out with the source item. If the target item is missing, Teamcenter notifies an error as all allocated items must be allocated to a target.

The following figure shows how allocations can be configured in for an electromechanical product.



Configuring allocations

Releasing allocations

When can I release allocations?

You can release allocations independent of their associated structures or items. Similarly, you can release the structures or items without releasing their allocations.

Releasing an allocation

You can release allocations and, therefore, they may be given a status and possibly an effectivity. The release status may configure allocations in one of two ways:

- The release status and effectivity on the allocation configures the allocation from the revision rule.
- Each of the allocations can be associated with an incremental change order that determines effectivity. Teamcenter uses the effectivity on the incremental change order to configure the allocation.

If you add status to an allocation, you can apply various effectivities to this relationship independent of the structures themselves. This provides flexibility when applying several allocations to the structure.

Releasing an item with an allocation to a physical component

Allocations are independent of the structures they allocate. The allocations are modeled in their own environment and must form a valid set. Releasing an item on either side of the allocation has no effect on the allocation itself.

Note:

The allocated target component is identified as a property on the source structure. Because the allocation is independent of source and target release status and the allocation can be created or deleted on a released object, the allocation information (property value) on the source component can change, regardless of its release status.

Teamcenter may allocate the same function structure to multiple physical components. Consequently, the **allocated to** information on the source may differ, depending on the context of allocation.

Releasing allocation maps

Because an allocation map is a workspace object, you can release it. Similarly, you can release allocation map revisions, which is necessary because users typically work with allocation map revisions, not allocation maps. After an allocation map revision is released, you can define additional security rules that depend on the release status, allowing users to control a set of allocations as a whole. For example, you can define a rule that prevents users from modifying a set of allocations once it has been approved.

Index

A

Absolute occurrence, purpose 1-10

Absolute occurrences

- Best practices 4-10
- In structure context 4-10
- Incremental change 4-8
- Tracking changes 4-8
- Using 4-3

Alias identifiers

- Configuring 1-18
- Creating 1-16
- Displaying 1-14

Aligning structure

- Design maturity 4-6
- Example process 4-4
- Externally 4-9
- Global association 4-4
- Primary representation 4-5
- Publish link 4-6

Allocations

- Aligning with 4-8
- Compared with trace links 4-4
- Configuring 4-10
- Implementing 4-5
- Making between configured structures 4-5
- Making conditional allocations 4-7
- Revising 4-9
- Understanding 4-1

Alternate ID

- Search 1-14

Alternate identifiers

- Configuring 1-18
- Creating 1-16
- Displaying 1-14

APNs 4-2

Appearance path node, purpose 4-1

Appearance path nodes 4-2

Appearance path nodes (APNs) 4-4

Architecture breakdown 4-3

Architecture breakdown element 4-3

Arrangement

- Purpose 4-1
- Using 4-10

As-Built Manager, purpose 1-5

Assemblies

- Imprecise 1-9
- Precise 1-9

Assembly, purpose 1-10

Attachments

Creation for incremental change 3-7

Deletion for incremental change 3-7

Incremental change 3-5

B

Background configuration (incremental change) 3-9

Base view 4-1

Baseline, purpose 3-3

Baselines 3-10

Item revision with active changes 3-12

Processing active incremental changes 3-12

Rolling up or carrying over active changes 3-13

Best practices

Absolute occurrences 4-10

Effectivity 3-9

Incremental change 3-13

Modular variants 3-26

Revision configuration 3-13

BOM grading 2-7

BOM view revision 1-41

BOM view revision, purpose 1-10

BOM views 1-41

BOM, purpose 1-10

Branch definitions

Configuring product structures with 3-1

Branches

Managing branch chains 3-5

Managing single branches 3-4

Building structure 2-1

Bulk updates to structure 2-10

BVR, creating 1-41

C

CAD systems, integrating 1-44

Change management 1-28

Change Manager

Incremental change, using with 3-9

Purpose 1-5

Changes, configuration 4-10

Changes, tracking incremental changes 3-1

Check-Out and Edit button 1-19

Classic variants

Creating basic data 3-10

Example 3-9

Explanation 3-3

Placing variant data on structure 3-13

Storing variant data 3-12

Clearance analysis

Best practices 2-17

Clearance database 2-17

- Overview 2-15
- Clearance database** 2-17
- Cloning product structure** 2-2
- Closure rule** 4-3
- Compare command** 1-26
- Comparing product structures** 2-1
 - Lowest level mode 2-1
 - Single-level mode 2-4
- Comparing structure**
 - Multilevel mode 2-3
- Component, purpose** 1-10
- Compositions**
 - Creating 4-11
 - Purpose 1-10
- Conditional allocations** 4-7
- Configuration context** 4-3
- Configuration context, purpose** 1-10
- Configuration control** 1-43
- Configuration items** 3-6
- Configuration rules, incremental change** 3-3
- Configured structures, making allocations** 4-5
- Constraint, purpose** 3-2
- Creating**
 - Alias identifiers 1-16
 - Alternate identifiers 1-16

D

- Data model, collaboration context** 4-3
- Date effectivity** 3-4
- Date entry (revision rules)** 3-6
- Default display identifiers** 1-19
- Default view type**
 - Purpose 1-10
- Derived default, purpose** 3-2
- Derived defaults, variant rules** 3-11
- Derived view** 4-1
- Design Context, purpose** 1-3
- Design solution** 4-3
- Design, purpose** 1-10
- DirectModel dataset, purpose** 1-11
- Display rules**
 - Alias identifier 1-20
 - Alternate identifier 1-20
- Displaying**
 - Alias identifiers 1-14
 - Alternate identifiers 1-14

E

- Editing structure** 2-3
- Effectivity**
 - Best practices 3-9
 - Date 3-4
 - Multiple end items/units 3-5
 - Mutually exclusive ranges 3-6

- Nested 3-6
- Occurrence 3-4
- Ranges 3-6
- Structure 3-4
- Unit number 3-4
- Validating 3-6
- Effectivity, purpose** 1-11
- End item, purpose** 3-2
- Engineering change management** 1-28
- ERP systems, integrating** 1-45
- Error check, purpose** 3-3
- Excel import** 2-6
- Explore dialog box** 1-24
- Explore Selected Components option** 1-24
- Expression values, setting** 3-25
- External option, purpose** 3-3
- External options, defining** 3-22

F

- Find in BOM** 1-26
- Find number, purpose** 1-11
- Fixed defaults, variant rules** 3-12

G

- Generic breakdown** 4-3
- Generic breakdown element** 4-3
- Global alternate, purpose** 1-11
- Global option, purpose** 3-3
- Global options, defining** 3-21
- Grading** 2-7
- Graphical comparisons** 2-6
- Grouping entries (revision rules)** 3-7

I

- IDC** 3-11
- Identifier display rules** 1-20
- Identifiers**
 - Alias 1-15
 - Alternate 1-15
 - Attributes 1-14
- Imprecise assemblies** 1-9
- Imprecise assembly**
 - Configuring 3-7
 - Creating from precise assembly 3-9
 - Purpose 3-2
- Incremental change** 1-44
 - Applying status 3-2
 - Attachments, configuring 3-5
 - Attachments, creation 3-7
 - Attachments, deletion 3-7
 - Background configuration 3-9
 - Baselines 3-12
 - Baselining item revision with active changes 3-12

- Best practices 3-13
- Carrying over current or future changes 3-13
- Change Manager, using with 3-9
- Configuration rules 3-3
- Intents 3-7
- Occurrence data changes 3-10
- Revision configuration, using with 3-9
- Rolling up current or future changes 3-13
- Tracking absolute occurrences 4-8
- Incremental change configuration 1-30**
- Incremental change, tracking 3-1**
- Installation assembly 4-3**
- Instance, purpose 1-11**
- Intents 3-7**
- Intermediate data capture, creating 3-11**
- is_design_required property 2-2**
- is_designed_in_place property 2-2**
- Item ID 1-12**
- Item identifiers, initial 1-14**
- Item name 1-13**
- Item properties 1-37**
- Item revision properties 1-37**
- Item revisions 1-13**
 - Default display identifier 1-19
- Items**
 - Default display identifier 1-19
 - Finding 2-1
 - Working with 1-12

L

- Latest entry (revision rules) 3-6**
- Line, purpose 1-11**

M

- make_or_buy property 2-2**
- Manufacturing Process Management**
 - Multi-Structure Manager 4-1
- Manufacturing Process Planner 4-2**
- Manufacturing Process Planner, purpose 1-4**
- Mass updates to structure 2-10**
- Modular variants 1-34**
 - Best practices 3-26
 - Configuring private options 3-14
 - Configuring public interface 3-14
 - Configuring structure 3-25
 - Creating 3-13
 - Creating modules 3-20
 - Creating variant items 3-18
 - Defining external options 3-22
 - Defining global options 3-21

- Defining options 3-20
- Error checking 3-24
- Linking options 3-15
- Option defaults 3-24
- Presenting options 3-17
- Setting expression values 3-25

Module 4-3

- Module, purpose 3-3**

- Multi-Site Collaboration, purpose 1-4**

Multi-Structure Manager

- Configuration changes 4-10
- Data model 4-3
- Purpose 1-3
- Transferring data to external applications 4-12

- Multiple configuration, purpose 3-3**

Multiple views

- Creating 1-40

N

- Named variant expression 4-4**
- Nested effectivity 3-6**
- Node, purpose 1-11**
- Note type, purpose 1-11**
- Note types 1-39**
- Note, purpose 1-11**
- NVEs**
 - Consistent 4-17
 - Full breakdown 4-16
 - Hierarchical variability 4-17
 - Partial breakdown 4-16
 - Shared 4-15
 - Unshared 4-16

O

- Occurrence configuration 1-29**
- Occurrence effectivity 3-4**
 - Multiple end items/units 3-5
 - Mutually exclusive ranges 3-6
 - Validating 3-6
- Occurrence group, purpose 1-12**
- Occurrence groups 1-42**
 - In representations 4-5
- Occurrence path, purpose 4-1**
- Occurrence properties 1-37**
- Occurrence, purpose 1-11**
- Occurrences**
 - Absolute occurrences 4-3
 - Creating 1-35
 - Tracking with incremental change 3-10
 - Using 4-4

- Option default, purpose 3-2
- Option defaults, defining 3-24
- Option, purpose 3-2
- Options, defining 3-20
- Override entry (revision rules) 3-5
- Override list, purpose 3-3

P

- Part solution 4-4
- Part, purpose 1-12
- Piece part, purpose 1-12
- Platform Designer
 - Choosing variability settings 4-18
 - Consistent NVEs 4-17
 - Full breakdown 4-16
 - Hierarchical variability 4-17
 - Partial breakdown 4-16
 - Purpose 1-4
 - Setting up 4-5
 - Setting variability 4-15
 - Shared NVEs 4-15
 - Unshared NVEs 4-16
- Platform Designer terms 4-2
- Precise assemblies 1-9
- Precise assembly
 - Changing to imprecise 3-9
 - Configuring 3-7
 - Purpose 3-3
- Precise entry (revision rules) 3-5
- Preferences
 - TC_publishable_<business-object-name>_contexts 1-14
- Presented option, purpose 3-3
- Private option, purpose 3-3
- Product 4-4
- Product architecture 4-4
- Product architecture template 4-4
- Product context 4-4
- Product family 4-4
- Product Master Management
 - Purpose 4-9
- Product structure
 - Aligning, example process 4-4
 - Building 2-1
 - Cloning 2-2
 - Comparing 2-1
 - Comparing in lowest level mode 2-1
 - Comparing in multilevel mode 2-3
 - Comparing in single-level mode 2-4
 - Configuring with variants 3-25
 - Editing 2-3
 - Excel import 2-6
 - Exporting 2-5
 - Importing 2-5

- Packing 2-6
- Unpacking 2-6
- Product structure, purpose 1-12
- Product views 3-11
- Product-specific architecture 4-4
- Properties 1-37
 - Creating center of mass rollup report 1-38
 - Creating mass rollup report 1-38
 - Creating moment and product of inertia rollup report 1-39
 - Creating rollup reports 1-38
 - Occurrence properties 1-37
- Properties dialog box 1-19
- Public option, purpose 3-3
- Publish occurrence data 1-36

Q

- Queries, saved 2-1

R

- Release status, purpose 3-3
- Representations 1-41
- req_pos_design property 2-2
- Revision configuration 1-27
 - Best practices 3-13
 - Incremental change, using with 3-9
- Revision control 3-1
- Revision effectivity 3-4
- Revision rule, purpose 3-3
- Revision rules 1-9
 - Applying 3-10
 - Date entry 3-6
 - Defining 3-3
 - Grouping entries 3-7
 - Latest entry 3-6
 - Override entry 3-5
 - Precise entry 3-5
 - Status entry 3-4
 - Unit number entry 3-6
 - User privileges 3-7
 - Using 1-27
 - Working entry 3-4
- Revisions 1-44
- Rollup report
 - Center of mass 1-38
 - Mass 1-38
 - Moment and product of inertia 1-39
 - Properties 1-38
- Rule entry, purpose 3-3
- Rules
 - Applying revision rules 3-10
 - Date entry 3-6
 - Grouping entries 3-7
 - Latest entry 3-6

- Override entry 3-5
- Precise entry 3-5
- Revision 3-3
- Status entry 3-4
- Unit number entry 3-6
- User privileges 3-7
- Working entry 3-4

S

Search

- Alternate ID 1-14

Searches

- Reports 2-2
- Where-used 2-1

Selecting related components 1-24

Sequence, purpose 1-12

Service Manager, purpose 1-6

Shown item relations

- Alias and alternate identifier 1-18

Snapshot

- Creating 3-11
- Purpose 3-3

Status entry (revision rules) 3-4

Storing variant data 3-12

Structure

- Aligning 4-1
- Terms 1-9

Structure applications 1-1

Structure context 1-42

- Absolute occurrences 4-10
- Building 4-9

Structure effectivity 3-4

Structure Manager, purpose 1-1

Structure, configuring 1-27

Substitute components 2-5

Substitute, purpose 1-12

Supersedures 2-5

Systems Engineering, purpose 1-2

T

TC_publishable_<business-object-name>_contexts preference 1-14

Teamcenter Integration for NX, purpose 1-2

Teamcenter manufacturing process management 4-1

Teamcenter Mechatronics Process Management, purpose 1-4

Trace links

- Compared with allocations 4-4

Tracking incremental changes 3-1

U

Unit number entry (revision rules) 3-6

Updates, bulk 2-10

Updates, mass 2-10

User privileges (revision rules) 3-7

V

Variability 4-4

Variability, Platform Designer 4-15

Variance 4-0

Variant (BOM), purpose 3-2

Variant condition, purpose 3-2

Variant configuration 1-33

Variant configuration, purpose 3-3

Variant item, purpose 3-3

Variant items 3-18

Variant model

- Compatibility with Global Services 3-5
- Compatibility with Multi-Site Collaboration 3-5
- New 3-4

Variant rule check, purpose 3-2

Variant rule, purpose 3-3

Variant rules

- Checks 3-11
- Derived defaults 3-11
- Fixed defaults 3-12
- Setting (classic variants) 3-10
- classic variants 3-4

Variants 1-9

- Classic 3-1
- Configuring private options (modular variants) 3-14
- Configuring public interface (modular variants) 3-14
- Configuring structure (modular variants) 3-25
- Creating data (classic variants) 3-10
- Creating modules (modular variants) 3-20
- Creating variant items (modular variants) 3-18
- Defining external options (modular variants) 3-22
- Defining global options (modular variants) 3-21
- Defining options (modular variants) 3-20
- Error checking (modular variants) 3-24
- Example of classic variants 3-9
- Linking options (modular variants) 3-15
- Modular 3-1
- Modular variants 3-13
- Option defaults (modular variants) 3-24
- Placing variant data on structure (classic variants) 3-13
- Presenting options (modular variants) 3-17
- Setting expression values (modular variants) 3-25
- Storing variant data (classic variants) 3-12
- Understanding models 3-4
- Using classic variants 3-3



View type

Purpose 1-12

Viewer 2-14

W

Where-used search 1-25

Reports 2-2

Running 2-1

Workflow, purpose 1-5

Working entry (revision rules) 3-4

Working revision, purpose 3-3

Siemens Digital Industries Software

Headquarters

Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 972 987 3000

Americas

Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 314 264 8499

Europe

Stephenson House
Sir William Siemens Square
Frimley, Camberley
Surrey, GU16 8QD
+44 (0) 1276 413200

Asia-Pacific

Suites 4301-4302, 43/F
AIA Kowloon Tower, Landmark East
100 How Ming Street
Kwun Tong, Kowloon
Hong Kong
+852 2230 3308

About Siemens Digital Industries Software

Siemens Digital Industries Software is a leading global provider of product life cycle management (PLM) software and services with 7 million licensed seats and 71,000 customers worldwide. Headquartered in Plano, Texas, Siemens Digital Industries Software works collaboratively with companies to deliver open solutions that help them turn more ideas into successful products. For more information on Siemens Digital Industries Software products and services, visit www.siemens.com/plm.

This software and related documentation are proprietary and confidential to Siemens.

© 2020 Siemens. A list of relevant **Siemens trademarks** is available. Other trademarks belong to their respective owners.